

The Ledger: A Closed-System Framework for Financial Settlement

Unified Design Specification: Architecture, Lifecycle, and CDM Integration

R. Delloye

June 2026

Preface

A trading business records the same activity many times over. A trade is captured in the trading system, again in the risk system, again in the general ledger, and again in regulatory reports. Each is a separate copy of one economic fact. Because the copies are kept apart, they drift apart, and the work of checking that they still agree—reconciliation—becomes a dominant operational cost and a primary source of error. When two records disagree, someone must decide which is right. Sometimes no one notices in time.

This specification removes the cause rather than managing the symptom. It keeps one record, not many. Every event in scope is written once, as a single kind of entry: the movement of a quantity from one place to another. Nothing is ever overwritten or deleted; entries are only added. A position, a profit-and-loss figure (P&L, the gain or loss on a book), a balance sheet, or a regulatory report is not a second copy to be reconciled against the first. It is a view computed from the one record, as a total is computed from a list of numbers. Among the firm's own records there is nothing to reconcile, because there is only one. (Checking that record against outside authorities such as custodians and depositories remains a separate activity at the system's boundary; it is the internal reconciliation, between the firm's own copies, that this design removes.)

This is what “conservation by construction” means, and why it matters in daily operation. Conservation is the rule that a quantity is never created or lost inside the system: whatever leaves one place arrives somewhere else, exactly. The rule is not a check run afterward and hoped to pass; it is built into the only operation the system permits, so a record that violated it could not be written in the first place. Three consequences follow directly.

- The books cannot silently fail to balance. An entry that did not balance is not a warning to investigate later—it cannot exist.
- Value cannot be created or lost inside the system. Quantities are only moved between places. Gains and losses arise from price changes, which are recorded separately and plainly, never from a quantity quietly appearing or vanishing.
- Any past state can be reproduced exactly. Because the record is only ever added to, the system can be rewound to any earlier moment and will show precisely what was known then. Audit and investigation work from fact, not from reconstruction.

The same single record serves the wider market context. Where the activity must be expressed in the shared industry vocabulary for derivatives—the Common Domain Model (CDM), a standard maintained by the International Swaps and Derivatives Association—the record maps onto it directly, without a separate translation system to keep in step.

A reader wanting the technical argument should continue to the Abstract and to the Roadmap and Reading Guidance, which give the shortest path through the document for each audience.

Abstract

Within its scope, the post-trade activity addressed by this framework reduces to one operation: the atomic move of a quantity between wallets. From this single primitive the Ledger builds a unified system in which financial instruments—equities, bonds, derivatives, swaps, managed accounts, Quantitative Investment Strategies, listed futures—are deterministic contracts that emit sequences of moves. One law governs the system: conservation, which holds by construction. From it follow path-independent profit and loss, complete time travel over an immutable event log, and idempotent lifecycle processing through pure functions, and several classes of internal reconciliation failure become structurally unreachable. Unit state lives in three homes—immutable **ProductTerms**, the shared **UnitStatus**, and per-position **PositionState**; **UnitStatus** is a materialised projection of the event log, not an authoritative mutable store. The scalar balance suffices for buy/sell/exercise workflows; for securities lending it extends to a six-coordinate position vector. Throughout, the design is illustrated in Haskell, with types chosen so that illegal states are unrepresentable. This design specification presents the three-home state model, the **UnitStatus** characterisation, the futures lifecycle, and the managed-account workflow as one self-contained whole.

Contents

Preface	1
Roadmap and Reading Guidance	8
Key Concepts and Terminology	8
 I Foundations: The Closed System and Conservation	 11
1 Purpose, Scope, and Properties	11
1.1 The Problem	11
1.2 The Solution	11
1.3 Three Unreachable Internal Breaks	12
1.4 Architecture	12
1.5 Scope	12
1.6 The Ledger Boundary	13
2 The Closed Ledger System	14
2.1 Wallets and Units	14
2.2 Ownership as Balance	14
2.3 Atomic Moves	14
2.4 Transactions and the Conservation Law	16
2.5 Virtual Wallets	17
2.6 Books and Reference Wallets	18
2.7 The Self-Consistency Principle	19
3 The Unit Store	21
3.1 Units	21
3.2 Unit Identity: OTC versus Listed	21
3.3 Three-Tier Architecture	21
3.4 Registration	22

3.5	Two-Stage Validation	24
3.6	Guarantees	24
3.7	CDM Alignment	24
II	The Three-Home State Model and UnitStatus	25
4	Where Unit State Lives: The Three-Home State Model	25
4.1	The placement rule and the 2×2	26
4.2	Why three homes, and why the fourth cell is empty	26
4.3	The UnitStatus discipline, stated once	28
4.4	The construction	28
4.5	The twelve conditions	37
4.6	Why exactly three maps	37
4.7	The unique design	38
4.8	Invariants made unrepresentable	38
4.9	Reference implementation	39
III	Valuation, Contracts, and Instrument Lifecycles	41
5	Portfolio Valuation and PnL	41
5.1	Quantity versus value	41
5.2	Mark-to-market valuation	41
5.3	State-sufficiency	42
5.4	Path-independent PnL	43
5.5	Price and flow decomposition	43
5.6	Dual valuation	44
6	Smart Contracts as Move Generators	44
6.1	Contract Definition	44
6.2	Composition: Modular over Composite	46
6.3	Equity, Dividend, and Corporate-Action Contracts	46
6.4	European Put—Full Move Schedule	46
6.5	Interest Rate Swap—Full Move Schedule	47
7	Lifecycle Management	49
7.1	The Transition Function	49
7.2	Idempotence	50
7.3	The Single Door	50
7.4	Purity	51
7.5	Time Travel	52
7.6	State Storage and Embedded Defence	53
7.7	Lifecycle Risk Profile	53
7.8	Strategy as Unit: QIS and Leveraged ETFs	54
8	The Futures Lifecycle	54
8.1	Primitives recalled, and where the future’s state lives	54
8.2	The instrument and the worked example	56
8.3	Listing	57
8.4	First trade	58
8.5	Daily variation-margin settlement: the mechanism and the clean case (day 1) . .	60
8.6	A subsequent, intraday trade (day 2)	61

8.7	Settlement under intraday trading (day 2): the anchor	61
8.8	Closing a position to flat	62
8.9	Expiry, final settlement, and the Close	62
8.10	After expiry: state versus derivation	63
8.11	Invariants threaded through the life	64
8.12	Escalations	67
IV	Accounts, Reporting, Settlement, and CDM Integration	67
9	Managed Accounts, Virtual Portfolios, and TRS	68
9.1	The mandate is a unit	68
9.2	Subscription	69
9.3	Trading under mandate	69
9.4	Valuation and NAV	70
9.5	Fee logic	70
9.6	Segregation	71
9.7	CSA margin and collateral	72
9.8	TRS as synthetic managed account	72
9.9	Redemption and wind-down	72
9.10	Account-level balance-sheet substantiation	73
9.11	Worked example	73
9.12	Conformance and flags	74
10	Balance Sheet Substantiation and Dual Valuation	75
10.1	Balance Reconstruction	75
10.2	Substantiation Scope	76
10.3	Dual Valuation	77
10.4	Snapshots and Audit Trail	77
11	Implementation and Operations	78
11.1	Three-Layer Architecture	78
11.2	Balance-Update Algorithm	78
11.3	Constant-Cost Profit and Loss	79
11.4	Fault Tolerance	79
11.5	Corrections as Events	79
11.6	From Moves to Settlement Instructions	80
11.7	ISO 20022 Interface	80
12	The Settlement Layer Interface	80
12.1	The Settlement Projection	81
12.2	Boundary Separation	83
12.3	Transaction Types and Settability	83
12.4	Lifecycle-Originated Settlement	84
12.5	DvP: The Two-Level Guarantee	84
12.6	Gross-to-Net Reconciliation	85
12.7	The Confirmation Return Path	85
13	ISDA CDM Integration	86
13.1	The CDM	86
13.2	Derivation: CDM Maps to the Framework Primitives	87
13.3	The Synonym Layer and Ingestion	87

13.4	The Forgetful Map F	88
13.5	CDM Enumerations as Generator Universe	90
V	Obligations, Generalised Positions, and Securities Lending	90
14	Orchestration and Obligation Liveness	90
14.1	Execution-engine requirements	90
14.2	The executor as an activity	91
14.3	The due-event scheduler	91
14.4	Concurrency: single-writer by construction	92
14.5	Obligation liveness	92
15	Generalised Positions and Securities Borrowing and Lending	97
15.1	Where the Scalar Model Breaks	98
15.2	The Generalised Coordinate Vector	98
15.3	Graceful Degeneration to the Scalar Model	100
15.4	The Lending Resolution: Named Wallets	100
15.5	Analytical Projections and Encumbrance	101
15.6	The SBL Smart Contract	101
15.7	SBL State Machine	102
15.8	Atomic Moves for SBL Lifecycle Events	102
15.9	Title Transfer vs Security Interest	103
15.10	Collateral and Margin	104
15.11	Short Selling and Inventory	104
15.12	Cash Collateral Reinvestment	104
15.13	On-Lending Chains	105
15.14	Locates	105
15.15	Cross-Border Regulatory Treatment	106
15.16	External Reconciliation	107
15.17	CDM Alignment and Regulatory Reporting	107
15.18	SBL-Specific Invariants (P11–P20)	107
VI	Formal Properties, Limitations, and Context	109
16	Invariants, Conservation Laws, and Property-Based Testing	109
16.1	Conservation as Central Invariant and Universal Oracle	109
16.2	The Core Invariants P1–P10	110
16.3	SBL Invariants P11–P20	111
16.4	Obligation Liveness Invariants P21–P23	111
16.5	The Twelve Conditions C1–C12	112
16.6	Which Condition Discharges Which Invariant	112
16.7	Invariants as Specification and Oracle	115
16.8	Property-Based Testing over the CDM Generator Universe	116
16.9	Specification-First Development	116
17	Regulatory Obligations and the Direction of Travel	117

18 Scope and Limitations	118
18.1 Inside the Boundary	118
18.2 Outside the Boundary	118
18.3 What the Framework Does Not Do	118
18.4 Theorems and Modelling Choices	119
18.5 Architecture and Operational Reality	119
19 Conclusion and Open Problems	120
19.1 Summary	120
19.2 The Four Design Goals	120
19.3 Self-Consistency as the Standard	120
19.4 Resolved Design Problems	120
19.5 Open Problems	121
19.6 Flagged Open-Items Register	122
20 Frequently Asked Questions	125
 VII Appendices	 127
A CDM Type Mapping Tables	127
B Property Test Catalogue	127
C Reconciliation Failure Mode Taxonomy	131
D Coordination Between Pricing, Contract State, and Market Data	132
D.1 The coupon	132
D.2 State-aware pricing	132
D.3 Ex-dividend: the external-quote lag	133
D.4 Scope	134
E Glossary	134
F The CDM Product Model—A Developer’s Guide	138
F.1 The Six-Layer Hierarchy	138
F.2 Unit Identity: Trade Is the Unit	139
F.3 Payout Composition and the TransferableProduct Boundary	140
F.4 Tokenised Securities: Double-Counting and the Flat Custodian	141
F.5 Date Handling: Types, Resolution, and Gotchas	142
G Available Inventory Identity: Eight-Scenario Verification	143
H European SBL Worked Example: GMSLA 2010 Title Transfer with Rehy- pothecation	144
H.1 Participants and Setup	145
H.2 Step 1: Loan Initiation (Day 0)	145
H.3 Step 2: Delivery to Hedge Fund and Short Sale (Day 0)	146
H.4 Step 3: Rehypothecation of Bund Collateral (Day 1)	146
H.5 Step 4: Mark-to-Market (Day 3)	147
H.6 Step 5: Manufactured Dividend (Day 7)	147
H.7 Step 6: Partial Recall and Return (Days 10–12)	147
H.8 Step 7: Full Return and Settlement (Day 20)	148

I	US SBL Worked Example: SEC Rule 15c3-3 with Cash Collateral	149
I.1	Participants and Setup	150
I.2	Step 1: Loan Initiation (Day 0)	150
I.3	Step 2: Rebate Mechanics and Cash Reinvestment	151
I.4	Step 3: Rehypothecation Constraints under Rule 15c3-3	151
I.5	Step 4: Daily Mark-to-Market (Day 5)	152
I.6	Step 5: FINRA SLATE Reporting	152
I.7	Step 6: Partial Recall and Return (Day 15)	153
I.8	Step 7: Full Return and Close-Out (Day 25)	154
I.9	Cross-Jurisdictional Comparison	154

Roadmap and Reading Guidance

The title names three pillars, each developed in its own parts. The architecture—one closed system, the atomic move, conservation, and the three-home state model—is Parts I–II (§§1–4). The instrument lifecycle—valuation, contracts, deterministic and optional lifecycle events, and the generalisation to obligations and securities lending—is Parts III and V (§§5–8, §14, §15). CDM integration, with accounts, reporting, and the settlement interface, is Part IV (§§9–13). Part VI (§16, §§17–20) states the formal properties, limitations, and context.

The reading paths below give, for each audience, the shortest route to a working understanding, what to add when more depth is required, and what carries no cost on a first pass.

Audience	Minimal path	Then if needed	May skip on first pass
Regulator	Preface, §1, the conservation core of §2, §17, §18	§12, §19, §20, App. C, App. H, App. I	the Haskell, §4 construction detail, §6, §8 detail, §§14–16 internals
Implementer	§2, §3, §4, §6, §7, §11, §16	§8, §12, §13 + App. F, §14, §15, App. A/B/G	§5 profit-and-loss derivations, §9, §10, §17, worked examples
Quant	§2, §5, §10, App. D	§8, §15	§3, §11, §12, §§13–14 mechanics, §17
Risk manager	§2, §5, §7, §9, §10, §15	§17, App. C	§3, §6 internals, §11, §§13–14, the Haskell

The progression from scalar to vector is deliberate. The scalar wallet balance carries the whole argument through Parts I–IV; the six-coordinate position vector and securities lending wait until Part V, introduced only once obligations are in hand.

Key Concepts and Terminology

This is an orientation list: a one-line gloss of each core term with a pointer to where it is formally defined. Appendix E (Glossary) is the authoritative reference; where this list and the formal definition appear to differ, the formal definition governs.

Wallet

A named account holding quantities of units; a logical partition of position space, not a bank or custody account (§2).

Unit

Any tradable or bookable object: currency, security, derivative, contract (§2).

Atomic move

The single primitive operation: an indivisible transfer of a positive quantity of one unit from a source wallet to a destination (§2).

Conservation law

The total quantity of each unit across all wallets is zero at all times, $\sum_w w(u) = 0$; the system is closed (§2).

Closed system

A system in which quantity is neither created nor destroyed, only moved between wallets (§2).

Event log / move stream

The immutable, append-only log of every atomic move; the sole canonical internal record (§2).

Projection

Any view—balances, P&L, balance sheets, reports—computed from the move stream rather than stored independently (§2).

The three homes

The three places unit state lives: ProductTerms, UnitStatus, and PositionState (§4).

ProductTerms

First home: a unit’s contractual terms, immutable, versioned, and append-only (§4).

UnitStatus

Second home: one shared value per unit, read identically by every holder; a materialised projection of the event log, not an authoritative mutable store (§4).

PositionState

Third home: per-(wallet, unit) economic state holding conserved fields such as accumulated cost and entry net asset value (NAV) (§4).

Position vector

The six-coordinate generalisation of the wallet balance: owned, on-loan, and borrowed quantities together with three collateral coordinates (posted, received, rehypothecated); it degenerates to a scalar for non-lendable instruments (§15).

Smart contract (move generator)

A deterministic program mapping inputs and state to sequences of atomic moves (§6).

Lifecycle event

A state transition on a unit—coupon, reset, margin call, exercise—expressed as a transaction (§7).

Transition function

The pure function carrying a unit’s lifecycle state and an event to the next state and its moves (§7).

Replay

Reconstruction of any historical state by folding the move stream and logged state changes up to a chosen time (§2).

Idempotence

Applying the same lifecycle operation twice has no effect beyond the first application (§7).

Path-independent PnL

Profit and loss $V_{t_1} - V_{t_0}$ depends only on the initial and final states, not on the path between them (§5).

State-sufficiency

Portfolio value depends only on current balances, unit state, and prices, not on transaction history (§5).

Balance-sheet substantiation

Demonstration that each balance-sheet figure is a projection of the same move stream, reconciled against external authorities at the boundary (§10).

Settlement layer

The interface projecting committed ledger activity into external settlement infrastructure (§12).

CDM

ISDA Common Domain Model: the canonical vocabulary for products and lifecycle events, and the input space for property-based testing (§13).

Managed account

An internal book treated as a virtual portfolio with periodic cash settlement of accumulated PnL (§9).

TRS (total return swap)

A contract converting virtual ledger performance into real cash flows (§9).

SBL (securities borrowing and lending)

The lending and borrowing of securities, modelled by the position vector's loan and collateral coordinates (§15).

Obligation liveness

The state of a recorded commitment to perform a move, read by a total discharge predicate: live until a matching committed transaction appears (§14).

Part I

Foundations: The Closed System and Conservation

1 Purpose, Scope, and Properties

1.1 The Problem

Conventional post-trade infrastructure records the same economic reality in many places: trading systems, risk engines, general ledgers, custodian records, regulatory reports. Each carries its own copy of positions, valuations, and histories. Reconciliation—verifying that these independent records agree—is the dominant operational cost and the primary source of breaks. The cause is architectural: a multi-source-of-truth design admits inconsistency by construction.

1.2 The Solution

The post-trade activities within scope use a single primitive: the atomic move of a quantity between wallets. Financial instruments—equities, bonds, derivatives, swaps, managed accounts, quantitative investment strategies (QIS)—are deterministic contracts that emit sequences of moves. Only the moves are stored; everything else is recomputed from them on demand, never kept as a second copy. An immutable move stream records every move with full provenance and is the sole canonical record; every other view—balances, profit and loss, balance sheets, reports—is a projection of it.

The framework satisfies six properties by construction.

1. **Atomicity.** Every transaction is an indivisible operation with complete provenance.
2. **Conservation.** The system is closed: the *quantity* of each unit is neither created nor destroyed. This is double-entry accounting reduced to one equation, $\sum_w w(u) = 0$. Conservation governs quantities per unit, not market values; value changes from price movements are captured by the valuation layer (Section 5).
3. **Determinism.** Given inputs and conditions, execution is reproducible.
4. **State-Sufficiency.** Portfolio value depends only on current state—wallet balances, unit state (including embedded lifecycle state: accrued coupons, exercise status, barrier flags), and the current price vector—not on transaction history. Where unit state lives is fixed by the three-home model (Section 4).
5. **Lifecycle Value Invariance** (restricted form). For contractual lifecycle events involving only deterministic cash flows—coupon and dividend payments, resets, principal redemptions—the total value of all wallets under a given price vector is invariant: the jump in the instrument’s price is exactly offset by the explicit cash move. Lifecycle processing reshuffles value between units and cash; it creates and destroys no PnL. This connects conservation, which governs quantities, to value: across deterministic lifecycle events total value is also invariant.

Qualification. For lifecycle events involving optionality—exercise, expiry, barrier knock-in/out, callable redemption—the invariance holds only for intrinsic value. Time value and embedded option value are extinguished when optionality resolves: exercising an option destroys time value; calling a bond extinguishes the holder’s above-market coupon stream. This is not a conservation violation, because those components reflect probabilistic expectations that cease to exist when uncertainty resolves. Conservation governs quantities,

not market values; value invariance holds exactly for deterministic events and for intrinsic value at option-exercise events.

6. **Time Travel.** The exact ledger state at any historical timestamp is reconstructed by replaying the immutable move stream and unit-state events up to that time. The operation `clone_at( $t$ )` yields the ledger as of time t , replaying activity up to t and ignoring the rest. Combined with pure lifecycle functions, this yields historical truth: the ledger is restored to what was known at time t and re-simulated. “Time travel to what was known at t ” (market data as captured at t) and “time travel to t with today’s corrected data” (restated data) are distinct and are not conflated.

These properties make the ledger a deterministic function of its event history: given the same transactions and market data, the state at any time is uniquely determined.

1.3 Three Unreachable Internal Breaks

When the ledger is the sole system of record for the activities it covers, three of the most damaging reconciliation failures are structurally impossible within that scope:

1. trading-system / general-ledger divergence,
2. PnL / balance-sheet mismatch,
3. inter-entity breaks.

Each is a disagreement between views that are, by construction, projections of one record; disagreement is not representable. The guarantee is scoped: during any migration in which legacy systems run in parallel, it covers only activity recorded in the ledger. Activity remaining in legacy systems retains the legacy failure modes.

1.4 Architecture

A *wallet* holds quantities of *units* (currencies, securities, derivatives, or any contractual obligation). A *move* transfers a quantity of a unit between two wallets. A *transaction* is a finite set of simultaneous moves satisfying conservation. A *smart contract* is a deterministic program mapping inputs and state to sequences of moves. Unit state records each unit’s lifecycle position. The immutable *move stream* records every move with full provenance. All other views are derived projections of it. External authorities—central securities depositories (CSDs), share registrars, regulators—maintain independent records at the boundary.

1.5 Scope

The scope is trading books measured at fair value through profit or loss under International Financial Reporting Standards (IFRS) and US Generally Accepted Accounting Principles (US GAAP).¹ Positions are marked to market using price vectors consistent with IFRS 13 and ASC 820 fair-value guidance. Banking-book activities—amortised-cost loans, held-to-collect debt, hedge accounting for structural positions—are out of scope and extend the same primitives.

The International Swaps and Derivatives Association (ISDA) Common Domain Model (CDM) is the canonical vocabulary: it defines the unit universe $\mathcal{U}$, drives lifecycle state transitions, bounds the input space for property-based testing, and maps ledger transactions to ISO 20022 settlement messages. CDM integration, the product hierarchy, and its layer-by-layer justification are developed in Section 13 and Appendix F.

¹IFRS 9 §§4.1.2, 4.1.4, 5.7.1–5.7.7; ASC 815, ASC 320/321, ASC 820.

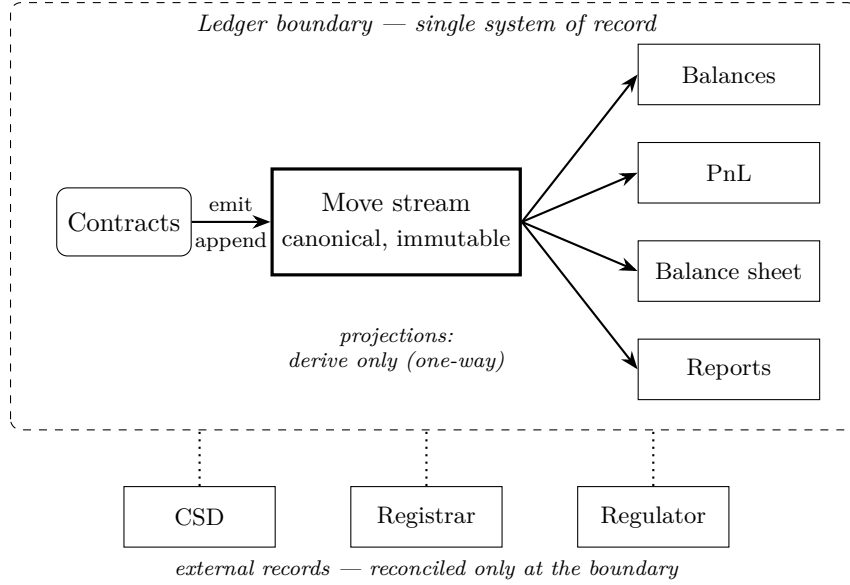


Figure 1: The single source of truth. Contracts emit moves that are appended to one immutable move stream; every internal view—balances, PnL, the balance sheet, and reports—is a one-way *projection* of that stream. The arrows are derivations, not synchronisations: there is no second internal record, hence nothing to reconcile within the boundary. External authorities lie outside the boundary and are reconciled against only at its edge.

1.6 The Ledger Boundary

The framework’s guarantees hold for activity recorded inside the ledger boundary. The boundary is stated here, once.

Inside (governed by the framework’s invariants):

- Positions: wallet balances, the generalised position vector, the conservation law $\sum_w w_t(u) = 0$.
- Moves: atomic transfers of quantity between wallets, with full provenance.
- Lifecycle events: state transitions on units, recorded as transactions with CDM event metadata.
- PnL: portfolio valuation $V_t = \sum_w w_t(u) P_t(u)$ for an externally supplied price vector P_t .
- Time travel: reconstruction of historical state by move-stream and unit-state replay.
- Internal reconciliation: the guarantee that trading-system, general-ledger, and balance-sheet views are projections of one record.

Outside (external systems, not governed by the framework):

- Price formation: P_t is an external input; the framework does not compute, validate, or guarantee executable prices.
- Legal agreements: ISDA Master Agreements, Global Master Securities Lending Agreements (GMSLAs), GMRAs, CSAs, and custody agreements determine legal rights. The ledger records economic positions, not legal ownership or enforceability.
- Settlement infrastructure: CSD connectivity, payment-system access, settlement finality, and real-world delivery-versus-payment (DvP) depend on external infrastructure (DTC, Euroclear, Clearstream, Continuous Linked Settlement (CLS), TARGET2).
- Regulatory gateways: trade, transaction, and prudential reporting (EMIR, MiFIR, Securities Financing Transactions Regulation (SFTR), Dodd-Frank, MiFID II RTS 25, CRR, FRTB) require submission infrastructure beyond the ledger.
- Reference-data authority: instrument master data (International Securities Identification

Numbers (ISINs), contract specifications, corporate-action terms) is consumed from external sources, not created by the ledger.

- Model governance: pricing and risk models and their validation.
- Accounting policy: IFRS 9 classification (FVTPL, FVOCI, amortised cost), hedge-accounting designation, and impairment assessment.

Every “within scope” qualifier elsewhere in this document refers to the inside list above.

2 The Closed Ledger System

The ledger is built from four primitives: *wallet*, *unit*, *move*, and *transaction*. Everything else—balances, valuation, profit and loss, lifecycle state, reports—is a projection of the move stream they generate. This section establishes the primitives, the conservation law that closes the system, and the structural consequence that internally inconsistent states are unreachable. It is the spine on which the remainder of the specification stands.

2.1 Wallets and Units

Definition 2.1 (Wallet). *A wallet w is a named account within the ledger holding quantities of units. It is a logical partition of position space—not a bank account, a custody account, or a legal entity. Let $\mathcal{W}$ denote the set of all wallets and $\mathcal{U}$ the set of all units. A wallet state at time t is a function*

$$w_t : \mathcal{U} \rightarrow \mathbb{R},$$

mapping each unit u to a quantity $w_t(u)$. A negative balance $w_t(u) < 0$ is a short position or obligation.

Units include currencies, securities (identified by ISIN), commodities, derivatives, over-the-counter (OTC) instruments, swaps, loans, and any other contractual obligation. $\mathcal{U}$ is extensible: a new instrument type is a new element of $\mathcal{U}$, not a change to existing infrastructure. The Unit Store (Section 3) manages $\mathcal{U}$ concretely—identification, validation, registration. The mapping from wallets to external accounts (custody, settlement, regulatory) belongs to the settlement layer (Section 12), not to the wallet.

2.2 Ownership as Balance

What a wallet owns is the number standing in its balance, nothing more: there is no separate register of title to keep in step with the holding.

Principle 2.2 (Ownership). *Economic ownership is the wallet balance: to own 100 shares of a stock is for that wallet’s balance in that stock to be 100. Where ownership and possession diverge—nominee structures, securities lending, repo, pledged collateral—the generalised position model (Section 15) extends the scalar balance to a vector distinguishing economic ownership from operational state.*

This eliminates the internal reconciliation problem at its root. Conventionally, ownership is recorded in several systems—trading, back-office, custodian, general ledger—and reconciliation verifies agreement. Here there is one record. The ledger is the canonical internal record of economic positions; external authorities (CSDs; registrars; legal documentation) maintain independent records at the boundary, where reconciliation remains necessary (Section 10).

2.3 Atomic Moves

A move is the one way any balance ever changes: it takes a positive quantity out of one wallet and puts the same quantity into another, in a single indivisible step.

Definition 2.3 (Move). *An atomic move m is an indivisible transfer of a positive quantity q of a unit u from a source wallet w_s to a destination wallet w_d at timestamp t , tagged with a source contract identifier s and metadata (event description, external references). Its semantics are*

$$w_s(u) -= q, \quad w_d(u) += q.$$

The metadata carries the event explanation and external references (ISO 20022 financial-messaging-standard message identifiers, counterparty identifiers); the source s names the smart contract or component that generated the move.

Principle 2.4 (Atomicity). *Every ledger change occurs as a discrete, time-stamped move. There are no implicit adjustments or retrospective calculations.*

The move is the additive operation of the position algebra: a quantity is an element of an additive abelian group, and a move adds the same element to one wallet that it subtracts from another. In Haskell the group is `Qty`, where `<>` is addition, `mempty` the zero position, and `negQty` the additive inverse. Quantities are exact `Integer` minor units, never floating point. Conservation and deterministic replay are arithmetic facts; floating point would forfeit them.

```
-- Qty: the additive abelian group of exact minor units.
-- A negative Qty is a short position; the group must host it, so Qty is unsigned-free.
newtype Qty = Qty Integer deriving (Eq, Ord)

instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)    -- addition
instance Monoid Qty where mempty = Qty 0                    -- the zero position

negQty :: Qty -> Qty                                          -- the additive
    inverse
negQty (Qty n) = Qty (negate n)
-- negQty q <> q == mempty    -- the group's inverse axiom, for every q

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId = UnitId String deriving (Eq, Ord, Show)

-- A move transfers a *positive* magnitude; direction is carried by (from,to).
data Move = Move
    { mFrom :: !WalletId, mTo :: !WalletId, mUnit :: !UnitId
    , mQty :: !Qty, mTime :: !Timestamp, mSource :: !SourceId } deriving (Eq, Show)

-- 'move' is the sole ingest path; it is the parse boundary that rejects a
-- non-positive magnitude (Qty itself stays signed, to hold balances).
move :: WalletId -> WalletId -> UnitId -> Qty -> Timestamp -> SourceId -> Maybe
    Move
move ws wd u q t s | q > mempty = Just (Move ws wd u q t s)
    | otherwise = Nothing

-- The ledger: each wallet maps units to quantities. An unread balance is the
-- group identity, so 'balance' is total; 'adjust' adds a (possibly negative) dq.
type Ledger = Map WalletId (Map UnitId Qty)

balance :: WalletId -> UnitId -> Ledger -> Qty
balance w u = maybe mempty (Map.findWithDefault mempty u) . Map.lookup w

adjust :: WalletId -> UnitId -> Qty -> Ledger -> Ledger
adjust w u dq = Map.insertWith (Map.unionWith (<>)) w (Map.singleton u dq)
```

```

-- applyMove realises  $w_s \leftarrow q$  ;  $w_d \leftarrow q$  as one act.
applyMove :: Move -> Ledger -> Ledger
applyMove (Move ws wd u q _) = adjust wd u q . adjust ws u (negQty q)

```

`Qty`, `negQty`, and the group instances are the cleared definitions of the three-home reference (Section 4, there abbreviated `qneg`); reproduced here as the canonical names the remainder of the specification cites.

2.4 Transactions and the Conservation Law

Definition 2.5 (Transaction). *A transaction τ is the one atomic event recorded in the log: an ordered list of moves $[m_1, \dots, m_n]$ together with the non-balance state delta they ride with—per-wallet bookkeeping (accumulated cost, high-water mark, entry NAV), shared unit-status writes, and a product-terms introduction or append. The moves carry every balance change; the delta carries everything else. When several transactions share a timestamp, the move stream assigns a total order (by sequence number within the timestamp) for deterministic replay. Conservation holds regardless of order; intermediate balances, margin, and lifecycle state depend on it.*

Because every move removes from one wallet exactly what it adds to another, no transaction changes the total quantity of any unit held across the whole system.

Principle 2.6 (Conservation Law). *For any transaction τ and any unit $u \in \mathcal{U}$, the net change in total quantity across all wallets is zero.*

Conservation follows from the move semantics: each move contributes $-q + q = 0$ to the total for its unit, so the sum over the transaction is zero. It is not a runtime check but an algebraic consequence of the additive-group structure—adding an element’s inverse to one wallet and the element to another nets to the group identity. The Haskell makes this exact. `foldMap` over a transaction’s moves is the unique homomorphism from the free monoid of moves into `Qty`: the total is built by mapping each move to its contribution and combining with `<>`. Every move’s contribution is already `negQty q <> q = mempty`, so the fold lands at `mempty` by the monoid laws alone. The empty move list (a registration or a preserving amendment, which carry only a state delta) is `mempty`, so the move-less events are conserved for free. Conservation is therefore a property of the type, not a checked side-condition: there is no separate validation step and no unconserved transaction to reject.

```

-- The ONE atomic event: the conserved economic flow (txMoves, in edge form)
-- PLUS the non-balance state delta it rides with. A balance changes ONLY via a
-- Move -- there is no balance writer in the per-wallet rows -- so "a balance
-- change without a conserving move" is unrepresentable, and conservation needs
-- no validation gate. txIntroduce = Just marks the introduction of txUnit
-- (registration); Nothing operates on an existing unit (trade/settlement).
data Transaction = Transaction
  { txUnit      :: UnitId          -- the unit this event
    concerns
  , txMoves     :: [Move]          -- conserved flow as edges (
    by construction)
  , txRows      :: Map WalletId [SomeWrite] -- non-balance bookkeeping (
    ac/hwm/entryNav)
  , txStatus    :: [StatusWrite]   -- shared UnitStatus writes
  , txIntroduce :: Maybe (TermsVersion, UnitStatus) -- Just => introduce txUnit (
    registration)
  , txAppend    :: Maybe TermsVersion -- Just => append a version (
    amendment)
  } deriving (Show)

-- Conservation, per unit. Each move's contribution to the system-wide total of

```



```

-- its unit is negQty q <> q = mempty; foldMap over the moves is therefore
-- mempty -- conservation by the monoid laws, not by a runtime sum. The empty
-- move list (registration, amendment) folds to mempty too: conserved for free.
unitDelta :: UnitId -> Transaction -> Qty
unitDelta u tx =
    foldMap (\m -> if mUnit m == u
                    then negQty (mQty m) <> mQty m
                    else mempty) (txMoves tx)
-- unitDelta u tau == mempty      -- for every unit u and
-- transaction tau (this is P1's premise)

```

This is the type-level statement of the Conservation Law (Principle 2.6); System Closure (Theorem 2.7) is its consequence over the whole stream.

Theorem 2.7 (System Closure). *If every transaction satisfies the conservation law, then*

$$\sum_{w \in \mathcal{W}} w_t(u) = 0 \quad \forall u \in \mathcal{U}, \forall t.$$

Proof. By induction on transactions. At $t = 0$ all balances are zero, so $Q(u) \equiv \sum_w w_t(u) = 0$. Each transaction preserves $Q(u) = 0$ by the conservation law. Hence $Q(u) = 0$ at all t . $\square$

System Closure (Property P1) is the closed double-entry invariant: units in real wallets (assets) are exactly offset by units in virtual wallets (liabilities). No move creates or destroys units; every move transfers between wallets. The property is consolidated with the other invariants in Section 16.

Remark 2.8 (Conservation as an algebraic identity). Optional reading. The rest of the specification does not depend on it.

Plain reading: *A move debits one wallet and credits another by the same amount, so for every unit the wallet balances sum to zero—for any list of moves, the empty list included. The books close because each move carries its own equal-and-opposite pair, not because a total is checked afterwards.*

Optional categorical lens: *The column-sum $\text{colSum}_u : \text{Balances} \rightarrow \text{Qty}$ is a group homomorphism into the additive abelian group Qty ; precomposed with the monoid homomorphism $\text{balances} = \text{foldMap contribution}$ from the free monoid of moves, the composite is a monoid homomorphism that is identically *mempty*—conservation holds because each move contributes the inverse pair $(\text{qneg } q) \langle \rangle q = \text{mempty}$ by the group law, with no runtime sum compared to zero and no validation gate.*

2.5 Virtual Wallets

Closure requires that every move have both a source and a destination. The framework therefore represents every external counterparty—market maker, exchange, clearinghouse, custodian—as a virtual wallet alongside the real positions.

Definition 2.9 (Real vs Virtual Wallets).

- **Real wallets** ($\mathcal{W}_{\text{real}} \subset \mathcal{W}$): *the firm's own portfolio positions; changes affect financial position and PnL.*
- **Virtual wallets** ($\mathcal{W}_{\text{virtual}} \subset \mathcal{W}$): *external counterparties and market entities; their balances track external positions for reconciliation but do not affect portfolio value.*

A virtual wallet serves three purposes: it gives every move a counterparty, keeping the system closed; its balance records the firm's position with that counterparty for direct comparison against external statements; and the metadata trail through it connects each internal position

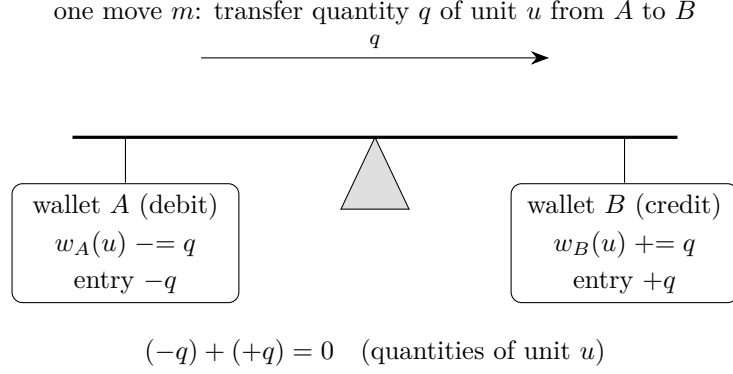


Figure 2: Conservation of quantity, per unit u . A move records two equal and opposite entries—a debit of q in wallet A and a credit of q in wallet B —which sum to zero, so $\sum_w w(u) = 0$ holds for every unit. What is conserved is *quantity*, not market value: prices move and so does value, and across optionality value is not conserved (§1). The beam weighs units, not money.

to its external origin. The external world is thus inside the ledger: within its scope the ledger has no outside. This is the architectural move that makes System Closure (Theorem 2.7) hold.

A share purchase realises the construction. Buying 500 AAPL at \$100 through a broker is one transaction of two moves against the broker’s virtual wallet:

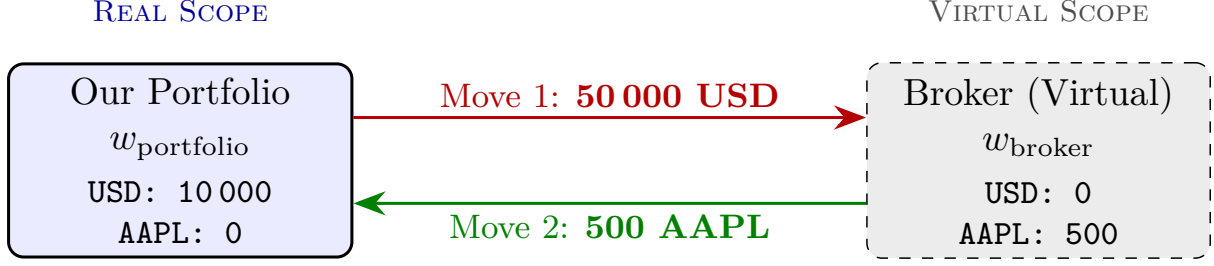
```
-- A trade carries moves and no introduction (txIntroduce = Nothing); the broker's
-- virtual wallet is the counterparty that keeps the system closed.
buyAAPL :: Transaction
buyAAPL = Transaction
  { txUnit  = UnitId "AAPL"
  , txMoves =
    [ Move portfolio      brokerVirtual (UnitId "USD") (Qty 50000) t (SourceId
      "broker")
    , Move brokerVirtual portfolio      (UnitId "AAPL") (Qty 500)   t (SourceId
      "broker") ]
  , txRows      = Map.empty
  , txStatus    = []
  , txIntroduce = Nothing      -- operates on already-registered units
  , txAppend    = Nothing }

-- unitDelta (UnitId "USD") buyAAPL == Qty 0      -- cash conserved
-- unitDelta (UnitId "AAPL") buyAAPL == Qty 0     -- shares conserved
-- after replaying both legs onto the empty ledger:
-- balance portfolio      (UnitId "AAPL") l == Qty 500
-- balance brokerVirtual (UnitId "AAPL") l == Qty (-500) -- totals to 0
```

Initialisation. At inception all wallets start at zero and $Q(u) = 0$ holds trivially. Opening positions are ordinary conservation-preserving moves from a virtual wallet to a real one: seeding 1,000 AAPL is a move of 1,000 AAPL from the custodian’s virtual wallet to the portfolio, paired with its cash leg. No `set_balance` primitive exists, and conservation is never violated, not even at inception. Where the system starts mid-life, the initialisation moves capture the state at migration time t_{init} ; the move stream begins at t_{init} and pre-migration history resides in legacy systems.

2.6 Books and Reference Wallets

Firms group activity into *books* for reporting and governance. A book is associated with one or more real wallets and often a distinguished *reference wallet* holding its positions and cash. This



$$\text{Conservation: } \Delta USD_{total} = -50,000 + 50,000 = 0; \quad \Delta AAPL_{total} = +500 - 500 = 0$$

Figure 3: A share purchase: 500 AAPL at \$100. Cash moves from the real portfolio wallet to the virtual broker wallet; shares move the opposite way. Both moves execute atomically as one transaction; each unit’s total across all wallets is conserved.

grouping is a reporting convention: every definition and smart contract operates at the wallet level. The managed-account contracts (Section 9) act on reference wallets whether classified real or virtual.

2.7 The Self-Consistency Principle

The move stream is the one record the system keeps; every balance, valuation, and report is that same stream read through a different lens, never a second copy stored alongside it.

Principle 2.10 (Self-Consistency). *The ledger’s conservation law, atomic transactions, and canonical event log make internally inconsistent states structurally unreachable, not merely detectable after the fact. Internal reconciliation—comparing two independent records of the same reality within the firm—is a symptom of a multi-source-of-truth architecture. In a self-consistent system the balance sheet is the ledger viewed through an accounting lens; within the ledger there is no second record to reconcile against. External systems—custodians, CSDs, counterparties, regulators—maintain independent records, and reconciliation at the boundary remains necessary.*

Three of the most damaging reconciliation failures are thereby unreachable within scope:

- **Trading system / general ledger divergence.** The trade and its accounting entry are one object—a transaction in the move stream. There is no second system to diverge from.
- **Profit and loss (PnL) / balance sheet mismatch.** $PnL = V_{t_1} - V_{t_0}$ is a theorem (Section 5); both values derive from the same position set and price vector. There is no separate PnL system.
- **Inter-entity breaks.** An inter-entity move is a single atomic event recorded once; within a shared ledger, disagreement is impossible.

The remaining failure modes—custodian breaks, nostro/vostro discrepancies, counterparty disputes—are boundary conditions where the external world is an independent actor. The architecture cannot prevent an external custodian from disagreeing; it makes disagreement detectable (virtual-wallet comparison) and narrowly scoped (ISO 20022 feedback at the boundary).

Settlement timing. The atomic transaction records the *economic intent* of simultaneous settlement. Legs settle asynchronously in practice. Foreign-exchange (FX) currency legs settle in different time zones. CLS provides payment-versus-payment within a window, not instantaneous atomicity. The framework records the trade as an atomic transaction at trade time and

tracks per-leg settlement progress as unit state (Section 4) or move metadata; between trade and settlement the residual is an unsettled receivable in the counterparty's virtual wallet. This represents Herstatt risk; it does not eliminate it. The settlement interface that consumes this state is Section 12.

3 The Unit Store

Section 2 fixed the unit universe $\mathcal{U}$ and the balance function $w_t : \mathcal{U} \rightarrow \mathbb{R}$ but left $\mathcal{U}$ abstract. The **Unit Store** makes it concrete: it decides which units exist, records their identity and static terms, binds each to a smart contract, and validates every unit at registration. The executor, the settlement projection, and the valuation function all read $\mathcal{U}$ from here.

3.1 Units

A unit is any financial instrument that can appear as a wallet balance: cash currencies, listed equities and bonds (by ISIN), listed derivatives (by exchange contract specification), OTC derivatives (each bilateral trade), and structured products (each issuance). A unit has *identity* (how it is distinguished from every other unit), *type*, static *terms*, and a bound *smart contract*. Its evolving state—lifecycle stage and product-specific state—is not stored on the unit entry; it lives in the three-home state model (Section 4).

3.2 Unit Identity: OTC versus Listed

Two positions are in the same unit exactly when they are fungible.

Principle 3.1 (Unit Identity). *Two positions are in the same unit if and only if they are fungible: a holder is economically indifferent between them. Fungible positions share one unit entry and net against each other; non-fungible positions are distinct units with distinct entries.*

Fungibility fixes a different identity key per instrument class.

Instrument class	Identity key	Fungibility
Cash	Currency code	All balances of a currency net
Listed equity	ISIN	All shares of an ISIN net
Listed derivative	Contract spec (exchange, underlier, type, strike, expiry)	All positions in a series net (via CCP novation)
OTC derivative	Full CDM Trade (includes counterparty, Collateral)	Non-fungible: counterparty and CSA make each trade unique
Bond	ISIN	All holdings of an ISIN net
Structured note	ISIN + product terms	Same issuance is fungible
Tokenised equity	Contract address + chain ID	All tokens on a contract net

For OTC instruments the identity is the CDM **Trade** object *including* its **Collateral** field: two trades with identical payoffs under different CSAs (USD versus EUR) are different units, with different discount curves, margin flows, and settlement currencies. For listed instruments the CCP novates itself in as sole counterparty, so all positions in one contract specification are fungible regardless of original counterparty. A tokenised security is a unit distinct from its underlying—NVDA and tokenised-NVDA carry the same exposure under different settlement mechanisms; the double-counting resolution (the custodian nets to zero economic exposure) is in Appendix F.

3.3 Three-Tier Architecture

The store has three tiers, each answering one question.

Tier 1 — Reference Data. *What instruments exist in the market?* Instrument master data—ISIN, exchange, contract specification, issuer, coupon schedule—read from exchanges, vendors, and CSD records. The Ledger consumes reference data; it does not author it. CDM: **TransferableProduct** for securities, **NonTransferableProduct** for derivatives and structured products.

Tier 2 — Product Registry. *What lifecycle logic governs this kind of instrument?*

One smart-contract template per product *type*, not per contract: “European equity index option” maps to one `OptionPayout` template covering every such option regardless of underlier, strike, or expiry. CDM `ProductQualification` classifies `EconomicTerms` into a product type; each type maps to a template (Section 6). An entry is created automatically the first time a Tier-3 registration meets a new product type, and is thereafter immutable.

Tier 3 — Unit Registry. *Which specific units can appear as balances?* The elements of $\mathcal{U}$. For listed instruments the unit is the contract specification; for OTC instruments it is the full CDM Trade, Collateral included. Each entry carries identity, type, the static terms drawn from `EconomicTerms`, the bindings to its Tier-2 template and smart contract, and provenance:

```
UnitEntry:
  unit_id      : UniqueId      -- deterministic, from the CDM object
  unit_type    : UnitType      -- CASH | EQUITY | LISTED_DERIV
                                -- | OTC_DERIV | BOND | STRUCTURED
  -- Identity (varies by type)
  isin         : Option<ISIN>   -- securities, bonds, notes
  contract_spec : Option<ContractSpec> -- listed derivatives
  cdm_trade_ref : Option<TradeRef> -- OTC (includes Collateral)
  -- Bindings
  product_ref   : ProductRegistryRef -- Tier 2 template
  smart_contract : SmartContractRef -- lifecycle logic instance
  -- Static terms (from CDM EconomicTerms)
  currency      : CurrencyCode
  multiplier     : Option<Decimal>
  expiry        : Option<Date>
  -- Provenance
  created_by    : TransactionId -- registering transaction
  created_at    : Timestamp
```

The `unit_id` is derived deterministically and injectively from the CDM object: a hash of the contract-specification fields for listed instruments, the UTI or metadata key for OTC. Distinct CDM objects always yield distinct unit IDs.

Where evolving state lives. The unit entry holds identity and static terms only. The lifecycle stage and the product-specific state—which coupons a bond has paid, the accumulated cost of a futures position—are not entry fields. A unit carries immutable, versioned terms in `ProductTerms[u]` and a shared lifecycle status in `UnitStatus[u]`; per-position state lives in `PositionState[w, u]`. These are the three-home state model (Section 4). The lifecycle stage is read from `UnitStatus`.

Remark 3.2 (`UnitStatus` is a projection, not a mutable store). *UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.*

3.4 Registration

Units enter the registry through three channels.

1. **Cash (pre-registered).** Currency units are permanent entries created at system initialisation: no expiry, no lifecycle events, stage `ACTIVE` permanently, smart contract the identity function.

2. **Listed instruments (reference-data feed).** On a new listing or initialisation against an existing market, the feed supplies the contract specification; the store creates the Tier-1, Tier-2 (if the product type is new), and Tier-3 entries. This is administrative, not a trade: the unit enters $\mathcal{U}$ upon listing, before any position is taken.
3. **OTC instruments (trade execution).** The executed CDM Trade *is* the unit definition; the store creates the Tier-3 entry from it. The unit ID derives from the Trade’s metadata key (counterparty and Collateral included), so identical payoffs under different CSAs receive different IDs. The unit enters $\mathcal{U}$ at execution.

Registration is atomic: a unit is fully validated and added, or rejected with no partial state. Units are never deleted; a unit at end of life reaches a terminal stage in *UnitStatus* (matured, terminated, settled) and the entry remains for audit and time travel, while the executor rejects new moves against a non-**ACTIVE** unit except to settle existing obligations.

Two disciplines bind registration. Registration writes the first value of every per-unit map, and it happens once.

Condition (C7: ProductTerms registration-total). *ProductTerms is registration-total: its first version is written at registration, so `product_terms(u)` is defined for every registered u . (UnitStatus is registration-total by the same write, C5, established in Section 4.)*

Condition (C10: no re-registration). *Re-registering an existing unit id is a hard error, never a silent reset of its state.*

Registration writes *ProductTerms* and *UnitStatus* together, so “registered in *ProductTerms* $\iff$ registered in *UnitStatus*” holds by construction; C10 makes the write once-only, so a re-registration cannot overwrite live state behind the log.

The registering function writes the *singleton* first version of *ProductTerms* and the default *UnitStatus* together, returning the updated ledger, or rejecting an already-registered id with a **Left** (C10). *ProductTerms* is a **NonEmpty** list of versions, so a registered-but-versionless unit is unrepresentable and the in-force-version accessor is total—it needs no **Maybe** for emptiness (C7). The two-map write is a single operation, so the coupling invariant “registered in *ProductTerms* $\iff$ registered in *UnitStatus*” cannot be falsified by any caller; **register** is exported, but *ProductTerms* and *Ledger* are exported abstract (no field setter), so the singleton is the only origin of a unit’s terms and there is no second door.

```
-- ProductTerms abstract: NonEmpty makes "registered but versionless"
-- unrepresentable; currentTerms is then total (no Maybe).          -- C7
newtype ProductTerms = ProductTerms (NonEmpty TermsVersion)

data LedgerError = ReRegistration UnitId | UnknownUnit UnitId      -- C10

-- Registration is ONE move-less Transaction whose txIntroduce carries
-- the unit's first terms version and its default status.
registerTx :: UnitId -> TermsVersion -> UnitStatus -> Transaction
registerTx u tv us = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty, txStatus = []
  , txIntroduce = Just (tv, us), txAppend = Nothing }

-- register is not a separate door: it is applyTx of that transaction, so
-- registration, trade, settlement, and amendment share one apply. The
-- introduce branch of applyTx writes the singleton first ProductTerms
-- version (C7) and the default UnitStatus (C5) together, and rejects an
-- already-registered id with Left (ReRegistration u) (C10). [Section 4]
register :: UnitId -> TermsVersion -> UnitStatus
        -> Ledger -> Either LedgerError Ledger
register u tv us = applyTx (registerTx u tv us)
```

A first registration succeeds; the same id a second time is rejected, leaving the prior ledger and its live state untouched—a hard error, never a silent reset:

```
register uES tvES defaultStatus emptyLedger == Right 10
register uES tvDup defaultStatus 10          == Left (ReRegistration uES)
```

Conservation does not arise at registration: a registration is a move-less transaction, so its edge-sum is `mempty`—vacuously conserved—and it touches no *PositionState* row. A non-vacuous balance, and the conservation that constrains it, appears only when the first position is taken (Section 4).

3.5 Two-Stage Validation

Validity is enforced at registration and at transaction time.

At registration, a unit must satisfy:

1. **Uniqueness**: no existing unit shares its `unit_id` (C10).
2. **Smart-contract binding**: a contract reference exists for its product type.
3. **CDM qualification**: for CDM-native units, `ProductQualification` returns a valid classification for the stated `EconomicTerms`; an unclassifiable product is rejected.
4. **Term consistency**: static terms cohere—expiry in the future at creation, positive multiplier, valid currency code.

At transaction time, the executor checks that every unit a move references (i) exists in the store, (ii) is in a stage compatible with the operation (read from *UnitStatus*), and (iii) has a smart contract invocable for the requested event.

Invalid units cannot enter $\mathcal{U}$, and moves cannot reference non-existent or retired units. With the conservation law (Section 2) and the executor’s purity (Section 7), this completes the correctness chain: the ledger operates only on validated, contract-governed units.

3.6 Guarantees

The Unit Store provides four guarantees to every downstream consumer.

1. **Existence**: every unit a move references exists and is in a valid lifecycle stage.
2. **Immutability of identity and terms**: a `unit_id` always denotes the same instrument with the same static terms. Lifecycle stage and product-specific state evolve (Section 4); identity and terms do not.
3. **Smart-contract availability**: every registered unit has a bound contract the executor can invoke for lifecycle events.
4. **CDM alignment**: every CDM-native unit maps to a CDM object (`Trade`, `NonTransferableProduct`, or `TransferableProduct`) with a successful qualification.

These make referential integrity (P3, Section 16) hold by construction: $m.\text{unit} \in \mathcal{U}$ because the executor consults the store before accepting any move.

3.7 CDM Alignment

Each tier maps to a CDM concept.

Tier	Content	CDM equivalent
1 Reference Data	Instrument master data	<code>TransferableProduct</code> (securities), <code>NonTransferableProduct</code> (derivatives)
2 Product Registry	Smart-contract templates	<code>ProductQualification</code> + <code>EconomicTerms</code>
3 Unit Registry	Position-bearing units ($\mathcal{U}$)	<code>Trade</code> (OTC), contract spec (listed)

CDM has no native catalogue concept: it models individual trades, not registries, and serves both listed and OTC derivatives through one `NonTransferableProduct` type without a dedicated listed-contract-specification type or a model for exchange reference-data feeds (listings, expiry calendars, lot sizes). These are documentation gaps, not design incompatibilities: the store uses CDM objects as its internal representation, and catalogue management sits alongside CDM as a Ledger-specific concern.

Key Invariants and Consequences

The foundations rest on one primitive—the atomic move of a quantity between two wallets—from which the following hold by construction. Each is catalogued, with its executable oracle, in §16.

- **Conservation (P1).** $Q(u) = \sum_w w_t(u) = 0$ for every unit u and time t . Each move debits and credits equal magnitude, so the signed per-unit sum of any transaction is the group identity; a conservation failure is an implementation bug, never a financial event. (§2, Theorem 2.7.)
- **Atomic commitment (P2).** A transaction applies completely or not at all; no partial state is observable, so a lifecycle event commits as one indivisible unit. (§2.)
- **Log monotonicity (P4).** The event stream is append-only and hash-chained; no entry is retroactively rewritten. (§3.)
- **Idempotency (P5, P6).** Re-applying a transaction by `tx_id`, or a lifecycle event to the same unit state, has no incremental effect. (§3.)
- **Determinism and time travel (P8, P9).** Replay of the immutable stream is a pure, total fold, and lifecycle functions are pure; `clone_at(t)` reconstructs balances and unit state identical to the ledger at time t . The ledger is a deterministic function of its event history. (§3.)
- **Consequence—unreachable internal breaks.** Trading-system/general-ledger divergence, PnL/balance-sheet mismatch, and inter-entity breaks are disagreements between projections of one record; for activity inside the boundary they are not representable. (§1.)

Part II

The Three-Home State Model and UnitStatus

4 Where Unit State Lives: The Three-Home State Model

Three kinds of fact describe a unit, and each is kept in its own home. The first is what the contract is — its fixed terms, written once at registration and thereafter only added to, never rewritten. The second is where the contract stands in its life — its lifecycle stage and last settlement price, a single shared value read the same way by everyone who holds it. The third is what a given holder holds — that holder’s own quantity and cost, which differ from one holder to the next. These three facts are the three homes named below.

Unit state lives in three homes, one map each: immutable `ProductTerms[u]` keyed by unit; shared `UnitStatus[u]` keyed by unit; and per-position `PositionState[w, u]` keyed by (wallet, unit). There is no wallet-keyed state sector: every per-wallet economic fact is keyed by a mandate or strategy unit and so lives in `PositionState[w, uMA]`. Each economic fact has exactly one home and one writer, so internal reconciliation failure cannot arise; conservation and deterministic replay then hold by construction, through the sealed single-writer discipline and the purity and

totality of replay. This placement is the minimum basis of the problem (§4.2) and unique under any correctness gate (§4.7).

A *unit* is anything held and conserved (§2): an instrument, and also a mandate or a strategy contract (§4.2). The framework primitives — wallet, unit, holding $h(w, u)$, move, transaction, conservation $\sum_w h(w, u) = 0$ — are those of §2.

4.1 The placement rule and the 2×2

One rule governs every placement: each economic fact has exactly one home and one writer. A fact violates it in exactly two ways. *Keyed wrong*: one home collapses two facts, or one fact duplicates across homes that drift. *Authored wrong*: two authorities keep one fact and their records drift apart. Two questions forestall the two failures, and because there are only two failures there is no third question.

Does the fact depend on the holder, or only on the unit? The scope is one unit’s state, so a unit and a holder of it are the only terms in view. Every economic fact — one entering conservation, valuation, or profit and loss — is then a (holder, unit) fact or a unit fact. What stays keyed by the wallet alone — KYC, permissions, an audit cursor — enters none of these three reckonings; it is identity, not economic state, and lives in a *WalletRegistry* that carries no quantity and joins no conservation law.

Is the fact’s record owned by an outside authority or by the ledger? The criterion is who owns the record’s history, not who sources the number. An externally authored fact is one an outside authority — the exchange, the contract, the reference-data provider — owns and restates, and the ledger preserves version by version. A ledger-authored fact is one the ledger’s own events produce and overwrite. A benchmark level the ledger records and overwrites is ledger-authored; the benchmark identity the provider restates is the authority’s versioned record — the two sort apart though both come from that provider.

The two questions give a 2×2 ; three of its four cells are occupied, and each occupied cell is one home.

	ledger-authored	externally authored
per unit	Status: lifecycle stage, last settlement price, current weights, benchmark level	Terms: multiplier, currency, expiry, strike, ISIN, fee schedule, mandate text, benchmark identity
per (holder, unit)	Position: held quantity, accumulated cost, high-water mark, entry NAV	— <i>empty</i> —

Terms and status share the unit key but cannot be one value, since their authorities of record differ (§4.2); they ride together on the unit key as a co-present pair — two homes whose mutation disciplines differ, each kept in its own map (§4.4).

4.2 Why three homes, and why the fourth cell is empty

Each occupied cell exists for one reason; the empty cell is empty for one; and exactly three maps are forced, no fewer and no more.

Position is keyed by (holder, unit), because two holders of one contract hold different state. A buyer and a seller hold the same future; the buyer’s quantity is +1000, the seller’s −1000, and their accumulated costs differ. Substituting one holder’s position for the other’s changes the value, so any unit-only keying identifies the two and loses the difference. The holder is in the key.

Status is keyed by the unit alone, because one value is read identically by every holder. A future settles at one price on the day; an index stands at one level. Stored per holder, the

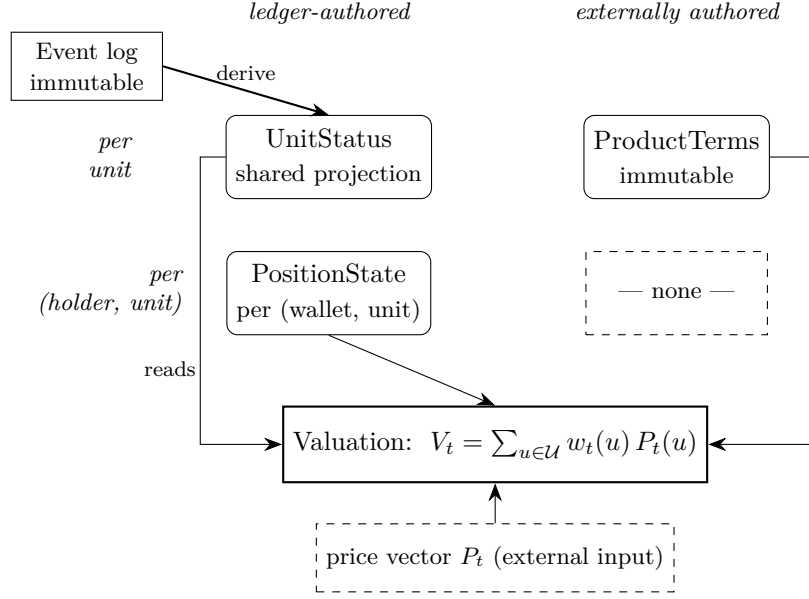


Figure 4: The three state homes and their dataflow. Each economic fact has one home, fixed by the placement above: **UnitStatus** (per unit, ledger-authored), **ProductTerms** (per unit, externally authored), and **PositionState** (per (wallet, unit)); the fourth cell—an externally authored per-position fact—is empty by construction (§4). **UnitStatus** is not an authoritative store but a projection derived from the immutable event log, shown by the incoming arrow. Valuation reads all three homes together with an externally supplied price vector P_t .

number is copied across thousands of rows free to drift — a reconciliation break by construction. Keyed by the unit, it is one value with one writer.

Terms are a home distinct from status, because the two have different authorities of record. Terms are externally authored, status ledger-authored. Co-mingling the authority’s record with the ledger’s own is the single-source-of-truth violation the system exists to prevent. Terms are append-only and versioned for reconstruction; the status cache is overwritten on every settlement while the settling event is appended immutably to the log (§4.3). Merging the two would put two mutation disciplines under one name.

The fourth cell is empty, because no authority issues a fact about one holder’s position. A position is the ledger’s own from origin: held quantity folded from the move stream, high-water mark and entry NAV written by ledger events. A custodian statement or prime-broker position report is a reconciliation input the ledger checks against, not a record it adopts as an authority’s own. So the (holder, unit) key carries only ledger-authored facts, and the externally authored cell is empty by construction.

The W -sector collapses (C12). A wallet-keyed economic-state map — call it the W -sector, $\text{Map}[\text{WalletId}, \text{WalletState}]$ — is the apparent demand of the managed account, whose state is the client’s, not the instrument’s. It does not exist, because the mandate is a unit. The manager issues a mandate unit u_{MA} to the client (§9), conserving like any issued unit:

$$h(w_{\text{manager}}, u_{\text{MA}}) = -1, \quad h(w_{\text{client}}, u_{\text{MA}}) = +1, \quad \sum_w h(w, u_{\text{MA}}) = 0.$$

Every field that looked per-wallet — high-water-mark value, accrued management and performance fee, mandate breach flags, benchmark NAV at this wallet’s inception — is then a fact about the position $(w_{\text{client}}, u_{\text{MA}})$ and lives in *PositionState*. Two mandates on one wallet are two rows, each with its own high-water mark and fees; a wallet-keyed scalar would have merged

them. This is why C12 keys on the mandate unit, not the wallet. A relationship spanning several instruments that does *not* reduce to a single (holder, unit) row would be a (holder, several-units) fact occupying a fourth home and a third map; the count excludes it, and it lies outside this scope.

4.3 The UnitStatus discipline, stated once

UnitStatus is a convenience copy. The ledger keeps it ready to read, but could discard it and rebuild it from the event log without loss. UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is (justified by the idempotent replay of Section 7, §7). Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.

Equivalently, the stored value always equals what replaying the unit’s events produces — the catamorphism property, meaning the fold of the event stream always rebuilds the exact cached value — and no part of the system writes it except through a logged event. In the reference (§4.9) this holds by construction: every *UnitStatus* field is written only by `applyStatus`, folded over the unit’s status writes inside the sole transition `applyTx` (and `register`, which is `applyTx` of a move-less introduction); the `Ledger` constructor is unexported, so no out-of-band `setStatus` exists. Externally sourced inputs — a settlement price, the current benchmark level — enter only as a logged observation event, so the fold stays pure at the boundary. Every later reference to cached unit state in this specification is read in this sense.

4.4 The construction

The homes are built from the smallest pieces, each forced by the one before, and the disciplines C1–C12 (§4.5) are introduced where the construction forces them. Quantities are exact integer minor units, never floating point, so determinism and conservation are arithmetic facts.

Scalars and keys. A balance is a quantity with a zero, associative addition, and a negation — an abelian group. The negation forces the two legs of a transfer to cancel; the zero makes the sum over no wallets conserved with no special case, which discharges the vacuous base of conservation (C9). A price is a number but not a quantity — prices are never summed — so it is a separate type with neither identity nor inverse. A balance is named by two keys, wallet and unit, the holder in the key.

```
{-# LANGUAGE GADTs, DataKinds, KindSignatures, StandaloneDeriving #-}
import           Control.Monad          (foldM)
import           Data.List              (foldl')
import           Data.List.NonEmpty     (NonEmpty (..))
import qualified Data.List.NonEmpty as NE
import           Data.Map.Strict        (Map)
import qualified Data.Map.Strict      as Map

newtype Qty = Qty Integer deriving (Eq, Ord)          -- exact minor units, never
                                                    Float
instance Show Qty      where show (Qty n) = show n
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid Qty    where mempty = Qty 0          -- the zero discharges the
                                                    vacuous base (C9)

qneg :: Qty -> Qty                                     -- group inverse: a transfer'
s legs cancel
```

```

qneg (Qty n) = Qty (negate n)
qmax :: Qty -> Qty -> Qty
qmax (Qty a) (Qty b) = Qty (max a b)

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String deriving (Eq, Ord, Show)

```

ProductTerms — the authority’s record, grown only by appending (C6, C7). “One or more versions, append-only” is a non-empty list, so a registered unit always has a current version and “registered but versionless” is not representable. The type is abstract: its constructor is unexported, so no importer can lay down a fresh one-version value and discard history. The only growth operations are `register`, which writes version one, and `appendVersion`, which appends; no setter exists, so the append-only discipline holds by absence, not by convention (C6). The first version is written at registration, so terms are total on the registered units (C7).

```

data TermsVersion = TermsVersion
  { tvLabel :: String, tvFields :: Map String String } -- opaque payload:
    multiplier, ISIN, ...
  deriving (Eq, Show)

newtype ProductTerms = ProductTerms (NonEmpty TermsVersion) deriving (Show)
  -- exported ABSTRACT: no data constructor, no setter -> append-only holds by
    absence (C6)

currentTerms :: ProductTerms -> TermsVersion -- total: NonEmpty always
  has a last
currentTerms (ProductTerms vs) = NE.last vs
allVersions :: ProductTerms -> NonEmpty TermsVersion
allVersions (ProductTerms vs) = vs
appendVersion :: TermsVersion -> ProductTerms -> ProductTerms -- C6: append,
  never rewrite
appendVersion tv (ProductTerms vs) = ProductTerms (vs <> (tv :| []))

productTerms :: Ledger -> UnitId -> Maybe ProductTerms -- Nothing = unregistered
productTerms l u = Map.lookup u (ledgerPT l)

```

UnitStatus — the shared read cache (C5). Status records the unit’s lifecycle stage together with its last settlement price and any successor, under the discipline of §4.3. The lifecycle is a closed sum, `Registered→Active→Expired→Closed`; `Registered` is the at-registration stage — a unit recorded in and known to the ledger before any settlement, which an OTC option, a bespoke mandate, or an internal strategy reaches exactly as an exchange instrument does. Every *UnitStatus* field changes only through a `StatusWrite` — the closed writer set whose three constructors each name one field — applied by the single total writer `applyStatus`, the projection step of §4.3; no other code overwrites a status field, so each write is last-write-wins and idempotent (P6). Product-declared defaults are written at Unit Store registration (§3), so status is total on the registered units (C5).

```

data Lifecycle = Registered | Active | Expired | Closed deriving (Eq, Show)

data UnitStatus = UnitStatus
  { usLifecycle :: Lifecycle
  , usLastSettle :: Maybe Qty -- None at the registration default (C5)
  , usSupersededBy :: Maybe UnitId -- set only by a Breaking amendment (C8)
  } deriving (Eq, Show)

```

```

defaultStatus :: UnitStatus          -- C5: product-declared default, written at
    registration
defaultStatus = UnitStatus Registered Nothing Nothing

data StatusWrite                     -- the closed status-writer set; each names
    ONE field
    = SetLifecycle    Lifecycle      -- a lifecycle event
    | SetLastSettle   Qty             -- a settle event (a logged observation)
    | SetSupersededBy UnitId         -- a Breaking amendment (C8)
    deriving (Eq, Show)

applyStatus :: StatusWrite -> UnitStatus -> UnitStatus -- the ONLY writer of a
    status field;
applyStatus (SetLifecycle l)    s = s { usLifecycle    = l }      -- total, last-
    write-wins,
applyStatus (SetLastSettle q)   s = s { usLastSettle   = Just q } -- idempotent (
    P6)
applyStatus (SetSupersededBy u) s = s { usSupersededBy = Just u }

unitStatus :: Ledger -> UnitId -> Maybe UnitStatus      -- Nothing = unregistered
unitStatus l u = Map.lookup u (ledgerUS l)

```

PositionState — the per-position home (C1). A position carries more than a balance: its primary field is the held quantity, which conserves; beside it ride non-conserved per-(holder, unit) facts — a high-water mark, an entry NAV — witnessing that the Position home carries facts no zero-sum law binds. A first touch starts a position at 0_P , the all-zero state. Two orthogonal disciplines govern the home (C1): (a) the accessor returns Option, so None (never held) and Some(0_P) (held, now flat) stay distinct — variation-margin settlement, wash-sale lookback, and record-date entitlement each act on the difference; (b) the carrier is monotone — a row, once created, is never removed, and close-out leaves it flat. The two are independent: (a) constrains the return type, (b) the domain. The monotone carrier keeps the key set stable across any checkpoint cut, which is what makes replay a literal fold (P8). The high-water-mark and entry-NAV writers are valuation events specified in the valuation satellite (§5); they are out of scope here and leave those fields at their defaults throughout.

```

data PositionState = PositionState
{ psAc      :: !Qty          -- conserved, additive      <- Settle / Trade
, psBalance :: !Qty          -- conserved, additive      <- Transfer
, psHwm     :: !Qty          -- NOT conserved, monotone  <- FeeCrystallise (
    qmax)
, psEntryNav :: !(Maybe Qty) -- NOT conserved, write-once <- Subscribe
} deriving (Eq, Show)

zeroP :: PositionState          -- flat row on first touch; also the held-and-
    flat baseline
zeroP = PositionState mempty mempty mempty Nothing

position :: Ledger -> WalletId -> UnitId -> Maybe PositionState -- C1 Option
    accessor:
position l w u = Map.lookup (w, u) (ledgerPS l)
    -- Nothing = never held; Just zeroP = held once, now flat. The two stay
    distinct.
    -- Monotone carrier (C1, storage half): applyTx only inserts/updates a row, and
    no
    -- row deleter is exported anywhere -- close-out leaves a flat row, never
    removes it.

```

The three homes, three maps — the sealed Ledger. Terms and status share the unit key but keep separate maps — `ledgerPT` append-only, `ledgerUS` overwrite-in-place — because their mutation disciplines differ. Their co-presence, “ u registered in terms $\iff u$ registered in status,” is preserved by construction, not policed: `register` is the sole introducer and writes both, and every later writer touches them only for an already-registered unit. The position map is the balance map enriched. The `Ledger` constructor and its field selectors are unexported: a `Ledger` is built only by the writers below and read only through the accessors. Were a selector exported, record update through it would install a non-conserving map without naming the constructor, bypassing the single-writer discipline. Coherence is carried by the single introducer; the seal keeps conservation true by construction.

```
data Ledger = Ledger
  { ledgerPT :: Map UnitId          ProductTerms  -- append-only terms, by
    unit
  , ledgerUS :: Map UnitId          UnitStatus     -- overwrite status, by
    unit
  , ledgerPS :: Map (WalletId, UnitId) PositionState -- positions, by (holder,
    unit)
  }
-- exported ABSTRACT: constructor and selectors unexported. A Ledger is built
-- only by the
-- writers below and read only through the accessors -> single-writer discipline
-- ; with no
-- row deleter, the carrier is monotone (C1).

emptyLedger :: Ledger
emptyLedger = Ledger Map.empty Map.empty Map.empty

data LedgerError = ReRegistration UnitId | UnitAlreadyExists UnitId | UnknownUnit
  UnitId
                | DanglingSupersede UnitId -- SetSupersededBy target is not a
  registered unit
deriving (Eq, Show)
```

The Transaction — the one atomic event (C2, C3). One event records, for a single unit, the whole change across the three maps, in two complementary parts. First, the conserved economic flow in *edge* form: a list of moves, each debiting one wallet and crediting another by the same magnitude, so the signed per-unit sum of any move list is the group identity *by construction* — $\text{qneg } q \diamond q = \text{mempty}$ for each leg, summed by the monoid laws, the empty list (a move-less registration or amendment) included, which is the vacuous base (C9). Second, the non-balance state delta that rides alongside the moves: per-wallet bookkeeping (accumulated cost, high-water mark, entry NAV, via the `C11 FieldWrite` table), the shared `UnitStatus` writes, and a `ProductTerms` introduction or append. Conservation is then a property of the type, not a checked side-condition: there is no unconserved `Transaction` to reject, hence no `validate` gate (C2). It applies in full or not at all; a partially applied transaction is not a representable state (C3). One structural field, `txIntroduce`, is the sole asymmetry between registration and every other event: `Just` introduces the unit (writing terms and status together), `Nothing` operates on an existing one — so registration is the move-less instance of this one type, not a separate operation. An event touching several units — a rebalance trading three futures, or a future bought against cash — is a transaction per unit applied as a single fold; per-unit conservation composes to event-level conservation. Induction over the stream lifts the per-event identity to the global invariant P1.

```
-- One value, two notations: txMoves is the per-edge view of the conserved flow;
the
```

```

-- remaining fields are the non-balance bookkeeping that rides with it.
-- (Move, moveDelta, mUnit, mQty are the framework primitives of sec:ledger.)
data Transaction = Transaction
  { txUnit      :: UnitId          -- the unit this event
    concerns
  , txMoves     :: [Move]          -- conserved flow as edges (by
    construction)
  , txRows      :: Map WalletId [SomeWrite] -- non-balance per-wallet
    bookkeeping
  , txStatus    :: [StatusWrite]   -- shared UnitStatus writes
  , txIntroduce :: Maybe (TermsVersion, UnitStatus) -- Just => introduce txUnit (
    registration)
  , txAppend    :: Maybe TermsVersion -- Just => append a version (
    C6, Preserving)
  } deriving (Show)

-- Conservation witnesses. Each move contributes qneg q <> q = mempty, so the
-- foldMap lands
-- at mempty BY THE MONOID LAWS -- not by a sum computed and compared at run time
-- (C2, C9).
unitDelta :: UnitId -> Transaction -> Qty
unitDelta u tx =
  foldMap (\m -> if mUnit m == u then qneg (mQty m) <> mQty m else mempty) (
    txMoves tx)
-- law: unitDelta u tau == mempty for every unit u and transaction tau

netDelta :: Transaction -> Qty          -- the empty/registration
case too
netDelta tx = foldMap (\m -> qneg (mQty m) <> mQty m) (txMoves tx)

-- The canonical move-less instances. Trade/settlement/transfer use the record
-- constructor
-- directly (txIntroduce = Nothing, moves + rows present).
registerTx :: UnitId -> TermsVersion -> UnitStatus -> Transaction -- C5+C7+C10:
introduction
registerTx u tv us = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty
  , txStatus = [], txIntroduce = Just (tv, us), txAppend = Nothing }

appendTx :: UnitId -> TermsVersion -> Transaction -- C6
Preserving amendment
appendTx u tv = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty
  , txStatus = [], txIntroduce = Nothing, txAppend = Just tv }

supersedeTx :: UnitId -> UnitId -> Transaction -- C8 Breaking
: stamp old unit
supersedeTx u uFresh = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty
  , txStatus = [SetSupersededBy uFresh], txIntroduce = Nothing, txAppend = Nothing
  }

```

Per-field canonical writer (C11). Each *PositionState* field is tagged with the closed set of field-writers permitted to mutate it: accumulated cost by settle and trade; high-water mark by fee-crystallisation; entry NAV by subscription. The balance field is not in this table: its sole canonical writer is the Move edge (handler **Transfer**), applied in **applyTx**, so a balance change without a conserving move is unrepresentable. A write by a field-writer outside the permitted

set is a type error at the writer's authorship site; the tag is erased once writes share one delta row, so the guarantee binds at authorship, not at the stored row (C11). These field-writers are a different axis from the event classes of C2: one event class drives one or more field-writers, and the two name-sets are not meant to coincide. The discipline is carried by a GADT (a generalised algebraic data type, in which each constructor fixes its own type index) whose constructors *are* the field-to-writer table.

```

data Handler = Settle | Trade | Transfer | FeeCrystallise | Subscribe deriving (Eq
, Show)

-- The GADT constructors ARE the C11 field->writer table; no other pairing is
representable.
-- The balance field is absent: its sole writer is the Move edge (handler '
Transfer), applied
-- in applyTx -- so "balance written off-ledger" is unrepresentable.
data FieldWrite (h :: Handler) where
  WAc      :: Qty -> FieldWrite 'Settle      -- ac, by settle
  WAcTrade :: Qty -> FieldWrite 'Trade       -- ac, by trade
  WHwm     :: Qty -> FieldWrite 'FeeCrystallise -- hum, by fee crystallisation
  WEntryNav :: Qty -> FieldWrite 'Subscribe  -- entry_nav, by subscribe
deriving instance Show (FieldWrite h)

data SomeWrite where SomeWrite :: FieldWrite h -> SomeWrite -- index erased once
writes share a row
instance Show SomeWrite where show (SomeWrite w) = show w

applyWrite :: FieldWrite h -> PositionState -> PositionState -- total over all
constructors
applyWrite (WAc q)      p = p { psAc      = psAc p <> q }
applyWrite (WAcTrade q) p = p { psAc      = psAc p <> q }
applyWrite (WHwm q)     p = p { psHwm     = qmax (psHwm p) q }
-- monotone
applyWrite (WEntryNav q) p = p { psEntryNav = maybe (Just q) Just (psEntryNav p) }
-- write-once

-- A settle handler authors ONLY 'Settle writes -- the result type IS the C11
checkpoint.
settleHandler :: [(WalletId, Qty)] -> Map WalletId [FieldWrite 'Settle]
settleHandler legs = Map.fromList [ (w, [WAc q]) | (w, q) <- legs ]

erase :: Map WalletId [FieldWrite h] -> Map WalletId [SomeWrite] -- authorship ->
erasure (S3)
erase = fmap (map SomeWrite)

_c11_ok_settle :: Qty -> FieldWrite 'Settle
_c11_ok_settle = WAc
_c11_ok_fee    :: Qty -> FieldWrite 'FeeCrystallise
_c11_ok_fee    = WHwm
-- _c11_bad :: Qty -> FieldWrite 'Settle
-- _c11_bad = WHwm -- TYPE ERROR: WHwm :: FieldWrite 'FeeCrystallise, not '
Settle

```

Registration, settlement, moves, and amendment — one door, applyTx. Every change to the three maps flows through one transition, `applyTx`; the ledger is sealed and there is no second write path. Registration is the move-less transaction whose `txIntroduce` is `Just`: it writes its first terms version and default status as one entry, and refuses a unit already present — re-registration is a hard error, never a silent reset (C10). Because terms and status

are written together, “registered in terms $\iff$ registered in status” holds by construction. **register** is therefore **applyTx** of **registerTx**, not a separate operation. A move’s effect is its per-wallet net delta, written into the balance field and gated on registration; its two legs are exact inverses, so it sums to zero over wallets, and a zero-quantity or self-move nets to zero on every wallet and writes no net change — *held* means named in a move that nets nonzero. Amendment has two tracks, chosen by a product-declared total predicate (C8): a fungibility-preserving amendment is one move-less transaction appending a terms version, leaving positions untouched; a fungibility-breaking amendment spans two units, so it is the composition of two transactions — introduce the successor u_{new} , then stamp **superseded_by**($u_{\text{old}} \rightarrow u_{\text{new}}$) on the predecessor’s status — run atomically through **applyTx** by **amend**, the move of holders being a separate paired-issuance event, never part of the amendment. Ordering the two transactions so the successor is introduced before the predecessor is stamped means the supersede pointer always names a registered unit; a **SetSupersededBy** whose target is absent is rejected as a **DanglingSupersede**, so the pointer can never dangle. Reads are capability-scoped: a cross-overlay read of another holder’s position is forbidden, and strategy exports flow only through **UnitStatus** (C4).

A move carries its conserved flow as edges in **txMoves**; its two legs are exact inverses (**qneg**), so the signed per-unit sum is **mempty** by construction, and a zero-quantity or self-move contributes no net write.

```
-- C5+C7+C10: register is applyTx of the move-less introduction. It is not a
primitive
-- standing outside the fold; registration, trade, settlement and amendment all
flow through
-- the ONE apply.
register :: UnitId -> TermsVersion -> UnitStatus -> Ledger -> Either LedgerError
Ledge
register u tv us = applyTx (registerTx u tv us)

-- The single door. C3 (all-or-nothing) is structural: the WHOLE new Ledger or a
Left, never
-- a partial state. txIntroduce=Just must introduce an ABSENT unit (else
ReRegistration, C10),
-- writing PT and US together (C5/C7); txUnit and every unit a move names must be
registered
-- (else UnknownUnit), so PT<->US co-presence is preserved; rows insert/update,
never delete
-- (C1 monotone). Conservation is already a property of txMoves (edges) -- nothing
to validate.
applyTx :: Transaction -> Ledger -> Either LedgerError Ledger
applyTx tx l0 = do
  l1 <- introduce l0
  () <- requireKnown l1
  () <- requireSupersedeTargets l1
  Right (commit l1)
where
  u = txUnit tx

  introduce l = case txIntroduce tx of
    Nothing -> Right l
    Just (tv, us)
      | u 'Map.member' ledgerPT l -> Left (ReRegistration u) -- C10
      | otherwise -> Right l
        { ledgerPT = Map.insert u (ProductTerms (tv :| [])) (ledgerPT l) -- C7
          , ledgerUS = Map.insert u us (ledgerUS l) -- C5
        }
```

```

requireKnown l =
  case filter (\x -> not (x 'Map.member' ledgerPT l)) (u : map mUnit (txMoves
tx)) of
    (bad : _) -> Left (UnknownUnit bad)
    []         -> Right ()

-- Referential integrity for the supersede pointer: a SetSupersededBy target
must name a
-- registered unit, else the stamp would dangle. A Breaking amendment
introduces the
-- successor first, so its id is registered by the time supersedeTx is applied
(C8).
requireSupersedeTargets l =
  case filter (\x -> not (x 'Map.member' ledgerPT l))
    [ t | SetSupersededBy t <- txStatus tx ] of
    (bad : _) -> Left (DanglingSupersede bad)
    []         -> Right ()

commit l = l
  { ledgerPT = maybe (ledgerPT l)
    (\tv -> Map.adjust (appendVersion tv) u (ledgerPT l))
  -- C6 append
    (txAppend tx)
  , ledgerUS = if null (txStatus tx)
    then ledgerUS l
    else Map.adjust (\s -> foldl' (flip applyStatus) s (txStatus
tx)) u (ledgerUS l)
  , ledgerPS = foldl' (flip applyMoveRow)
    (Map.foldrWithKey applyRow (ledgerPS l) (txRows tx))
    (txMoves tx)
  }

applyRow w writes ps =                                     -- non-balance
bookkeeping
  let key = (w, u)
  cur = Map.findWithDefault zeroP key ps                    -- first touch ->
flat row
  new = foldl' (\acc (SomeWrite fw) -> applyWrite fw acc) cur writes
  in Map.insert key new ps                                   -- insert/update
only

-- A Move writes ONLY psBalance, via the conserving pair from moveDelta (from -= q
, to += q),
-- keyed by the move's own unit -- the sole writer of the balance field.
applyMoveRow :: Move -> Map (WalletId, UnitId) PositionState
  -> Map (WalletId, UnitId) PositionState
applyMoveRow m ps0 = foldl' step ps0 (moveDelta m)
  where
    step ps (w, dq) =
      let key = (w, mUnit m)
      cur = Map.findWithDefault zeroP key ps
      in Map.insert key cur { psBalance = psBalance cur <> dq } ps

-- C8: a product-declared total predicate chooses the track. Either track reaches
the maps
-- ONLY through applyTx -- there is no second write path.
data Fungibility = Preserving | Breaking deriving (Eq, Show)

```

```

type FungibilityPredicate = ProductTerms -> TermsVersion -> Fungibility
data AmendResult = Appended UnitId | Superseded UnitId UnitId deriving (Eq, Show)

amend :: FungibilityPredicate -> UnitId -> TermsVersion -> UnitId -> Ledger
      -> Either LedgerError (AmendResult, Ledger)
amend isFungible u tvNew uFresh l =
  case Map.lookup u (ledgerPT l) of
    Nothing -> Left (UnknownUnit u)
    Just pt -> case isFungible pt tvNew of
      Preserving ->                                -- ONE move-less Transaction;
        positions untouched
        (\l' -> (Appended u, l')) <$> applyTx (appendTx u tvNew) l
      Breaking -- spans two units: a
        composite of TWO Transactions
        | uFresh 'Map.member' ledgerPT l -> Left (UnitAlreadyExists uFresh)
        | otherwise -> do
          l1 <- applyTx (registerTx uFresh tvNew defaultStatus) l -- introduce
            the successor
          l2 <- applyTx (supersedeTx u uFresh)                    l1 -- stamp the
            predecessor
          Right (Superseded u uFresh, l2) -- old terms never rewritten (
C7/P7)

```

Deterministic replay. Replay is the monadic left-fold of the event stream over the empty ledger. It is deterministic because application is a pure, total function of the event and the prior ledger: the same events give the same ledger. It returns the rebuilt ledger on a well-formed stream and an error on an ill-formed one — a repeated registration, or a move or settlement on an unregistered unit — the fold halting at the first refusal. Checkpointing is sound by the monadic left-fold law: the fold over a concatenation splits at any cut. Since registration, settlement, and moves are all events, replay from the empty ledger rebuilds every unit’s terms, status, and positions, so every view — balances, profit and loss, the cached *UnitStatus* of §4.3 — is a projection of the stream. Row retention serves audit, not determinism (P8).

```

-- A Transaction IS the event; applyTx is the pure, total transition. Replay is
  the
-- error-stopping left fold of the stream. Determinism (P8) is the fold-
  homomorphism law,
--   replay (xs <> ys) = replay xs >=> replay ys,
-- so the result is independent of where a checkpoint is cut. On a well-formed
  stream (every
-- event on a registered unit) no Left arises, so replay is total there. (Balances
  and PnL are
-- further projections of the same stream -- netBal, in the balance-sheet section
  .)
replay :: [Transaction] -> Ledger -> Either LedgerError Ledger
replay ds l0 = foldM (\l d -> applyTx d l) l0 ds

```

Remark 4.1 (Projection generalisation (optional)). Plain reading. *Every view — balances, valuation, and UnitStatus itself — is computed by folding over the one event stream; none is a second source of truth, and replaying the events rebuilds each exactly.*

Optional lens. *balances = foldMap contribution is a monoid homomorphism — balances (xs <> ys) = balances xs <> balances ys — into the Balances monoid; UnitStatus is a left fold of its status-write stream and value is a fold of the materialised position state, so every named view is a catamorphism of the log (directly for balances and UnitStatus, by composition with replay for value).*

4.5 The twelve conditions

Each condition renders one illegal state unrepresentable. The labels are stable tags, not a sequence. Each is stated once below and forced at the construction step or instrument named; the four instruments that force them — a future whose state depends on past trades, a managed account, a strategy that trades futures, and an instrument registered but never traded — are worked in §4.4 and §4.2.

	Condition	Forced at
C1	Position accessor returns <code>Option</code> (never-held vs. held-and-flat distinct) <i>and</i> the carrier is monotone (no row deleted); both.	<code>PositionState</code>
C2	Every <code>Transaction</code> carries its conserved flow as edge moves, whose signed per-unit sum is <code>mempty</code> by construction; vacuous base case (empty move list) included.	<code>Transaction</code>
C3	A <code>Transaction</code> spans all three maps for one unit and applies in full or not at all; partial application unrepresentable.	<code>Transaction</code>
C4	Reads are capability-scoped; cross-overlay reads forbidden; strategy exports flow only through <code>UnitStatus</code> .	strategy
C5	<code>UnitStatus</code> is registration-total: product-declared defaults written at registration.	untraded unit
C6	<code>ProductTerms</code> is a non-empty version list, append-only; no version mutated in place.	<code>ProductTerms</code>
C7	<code>ProductTerms</code> is registration-total: first version written at registration.	<code>ProductTerms</code>
C8	Two-track amendment by a total fungibility predicate: <code>Preserving</code> appends a version; <code>Breaking</code> allocates a fresh u and records <code>superseded_by</code> .	amendment
C9	A move-less event has an empty move list, which folds to <code>mempty</code> , so it is conserved vacuously; conservation sums edge legs and never divides by a holder count.	untraded unit
C10	Re-registering an existing unit is a hard error, never a silent reset.	registration
C11	Each <code>PositionState</code> field has a canonical writer set; a writer outside it is a type error at authorship, erased at the delta row.	<code>PositionState</code>
C12	All per-(wallet, mandate/strategy) economic state lives at <code>PositionState[w, u_{MA}]</code> ; the W -sector is empty by schema.	managed account

4.6 Why exactly three maps

Three independent constraints each force one map, and removing any one breaks its constraint. (i) Distinct per-holder state under the same contract forces a (w, u) -keyed map — `PositionState`. (ii) Shared observables read identically by every holder force a u -keyed map distinct from per-holder state — `UnitStatus`. (iii) The two mutation disciplines — append-only versioned terms versus an overwrite-in-place status cache — force a third, append-only map distinct from status — `ProductTerms`. A fourth map, the W -sector, adds a sector whose economic content is empty (C12). Three maps are the minimum basis of the problem, not a compromise.

4.7 The unique design

The adopted design — three maps under C1–C12, design B — is the unique Pareto-optimum on testability, correctness, and simplicity under any correctness gate. Each rejected alternative fails on one forcing point.

- **A** (per-unit state, plus per- (w, u) for futures only) has no sector for shared observables distinct from immutable terms, so settlement prices and terms share one overwrite cell — the two mutation disciplines of §4.6(iii) collapse.
- **C** (a sentinel unit carrying wallet state) has a sentinel with no issuer and no conservation partner, so $\sum_w h(\cdot) = 0$ fails by fiat.
- **D** (an explicit W -sector) adds a sector whose economic content is empty: every candidate datum is (w, u_{MA}) -keyed (C12).
- **E** (state indexed only over the held set) equals B on correctness but has no implementation in available tooling and is far less simple; shippability decides, not elegance.
- **F** (a universe wallet $w_\star$ folding *UnitStatus* into *PositionState* $[w_\star, u]$) re-introduces the per-unit/per- (w, u) split as a runtime predicate on the wallet axis, and conservation must then carve out $w_\star$ by hand — *UnitStatus* is the type-level expression of what F hides as a wallet-id convention.

4.8 Invariants made unrepresentable

Under design B with C1–C12, seven illegal states are placed beyond reach rather than merely tested; each line names the invariant of the hub (§16) it discharges, or, where the guarantee is a contributing mechanism rather than the invariant itself, the hub invariant it underpins. The P-numbers are the canonical hub numbering of §16. The mechanism differs by invariant and is named in each line. “Unrepresentable” is precise: an abstract type whose only constructor is a checked smart constructor keeps the illegal value from reaching the operation that would act on it; a non-empty list makes the versionless state untypable; a per-field tag is a type error at authorship; conservation is carried by the edge form of a move, whose two legs cancel by construction. It is not a claim that every guarantee is a type-level fact — read-scoping, in particular, is enforced at the capability layer wrapping the accessors, not in the data reference.

Invariant	Mechanism	Discharged by
P1 (conservation of quantity)	Conserved flow is carried as Move edges, each naming both legs (qneg $q \diamond q = \text{mempty}$), so the signed per-unit sum of any transaction is mempty by construction; there is no unconserved Transaction to reject, hence no validate gate.	C2 (edge-conserved flow), C3 (atomic transaction)
P8 (snapshot/replay determinism)	Replay is the foldM of the stream, a fold homomorphism: replaying a concatenated log equals replaying each part and composing; the monotone carrier keeps the key set stable across any cut.	C1(b) (monotone carrier)

Invariant	Mechanism	Discharged by
P6 (lifecycle idempotency)	Lifecycle stage is u -keyed in <i>UnitStatus</i> , written by replacement, so re-applying a stage write is the identity; non-conserved fields carry the same algebra (high-water mark by max, entry NAV write-once).	C5 (registration-total status)
Terms immutability (underpins P4)	Append-only non-empty version list; no in-place mutation, no exported setter.	C6 (append-only), C7 (registration-total)
Identity immutability (underpins P4)	Re-registration rejected; a breaking amendment allocates u_{new} and never rewrites u_{old} .	C8 (two-track), C10 (no re-registration)
P7 (virtual/real isolation; read-scoping)	Cross-overlay reads forbidden at the capability layer wrapping the accessors, not in the data reference.	C4 (capability-scoped reads)
Handler-field canon (underpins P9)	Canonical writer set per field; a writer outside it is a type error at authorship, erased once writes share a delta row.	C11 (per-field writer)

No other candidate design places more than three of the core invariants beyond reach.

4.9 Reference implementation

The reference is `reference/Ledger.hs`, woven through §4.4 and collected here. It is total and deterministic, and makes the illegal states unrepresentable rather than merely checked: `Ledger` is sealed with no row deleter (C1 monotone half) and an `Option` accessor (C1 Option half); a `Transaction` carries its conserved flow as `Move` edges, so the signed per-unit sum is `memory` by construction and no `validate` gate is needed (C2, the empty move list giving C9); `ProductTerms` is abstract over a non-empty list grown only by registration and `appendVersion` (C6, C7); the `FieldWrite` GADT is the C11 field-to-writer table; `applyTx` is the single door, returning the whole ledger or an error with no partial state (C3); registration is the move-less `txIntroduce=Just` instance, rejecting re-registration (C10) and writing terms and status together; and `amend` is the two-track C8. Every *UnitStatus* field is written only by `applyStatus`, folded inside `applyTx`, which is the projection-of-log discipline of §4.3 held by construction.

The pieces woven through §4.4 constitute `reference/Ledger.hs` in full, building on the `Move` algebra of §2 (`move`, `moveDelta`, `Timestamp`, `SourceId`); the consolidated, compile-ordered source is collected as the section’s reference. Run on that source, the following evaluation makes the behaviour concrete — the comments are the printed results.

```
expect :: Show e => Either e a -> IO a
expect = either (error . show) pure

main :: IO ()
main = do
```

```

let uES = UnitId "CME-ES"; wB = WalletId "BUYER"; wS = WalletId "SELLER"
10 <- expect $ register uES (TermsVersion "ES-v1" (Map.fromList [("mult","50")]))
    )
                                defaultStatus emptyLedger
print (position    10 wB uES)      -- Nothing : never held (C1 Option half)
print (unitStatus 10    uES)      -- Just (UnitStatus Registered Nothing
    Nothing) (C5)
-- a conserving trade: +1000 to BUYER as a Move EDGE (balance), with accumulated
-- -cost rows
-- authored at 'Settle then erased (C11). (move/Timestamp/SourceId are the sec:
-- ledger types.)
Just mv <- pure (move wS wB uES (Qty 1000) (Timestamp 0) (SourceId "demo"))
let tradeTx = Transaction uES [mv]
                                (erase (settleHandler [(wB, Qty 1000), (wS, Qty (-1000))]))
                                [] Nothing Nothing
print (netDelta tradeTx)          -- 0 : the move legs cancel BY CONSTRUCTION
    (C2)
11 <- expect $ applyTx tradeTx 10
print (psBalance <$> position 11 wB uES) -- Just 1000 : balance, written by the
    Move edge
print (psAc      <$> position 11 wB uES) -- Just 1000 : accumulated cost, via
    the row
-- There is no unconserved Transaction to reject: a one-legged flow is not a
-- Move, so it is
-- not representable in txMoves -- conservation is a property of the type, not a
-- validate gate.
-- close-out: a Move back keeps a flat row (monotone carrier): held-and-flat /=
-- never-held
Just mvC <- pure (move wB wS uES (Qty 1000) (Timestamp 1) (SourceId "demo"))
12 <- expect $ applyTx (Transaction uES [mvC] Map.empty [] Nothing Nothing) 11
print (psBalance <$> position 12 wB uES) -- Just 0 : retained, not Nothing
print (register uES (TermsVersion "dup" Map.empty) defaultStatus 12)
                                -- Left (ReRegistration (UnitId "CME-ES")) (
    (C10)

```

Remark 4.2 (FORMALIS clearance). *The listings above are excerpted verbatim from reference/Ledger.hs, the FORMALIS-cleared reference core; they carry that clearance and introduce no new code. Quantities are exact Integer minor units, a correctness choice that makes determinism and conservation arithmetic facts. Three structural points match the cleared code: the per-unit homes are three coupled maps (ledgerPT, ledgerUS, ledgerPS) with co-presence preserved by construction — matching both the code and §4.6; the lifecycle is the closed sum Registered/Active/Expired/Closed with the last settlement price a separate Maybe field written only through a logged StatusWrite, rather than carried on an Active constructor; and one atomic Transaction carries both the conserved move edges and the non-balance state delta, so conservation holds by construction and there is no separate validate step.*

The testing commitments for this construction — handler mutation score 85–90%, lifecycle-guard 70–80%, core overall $\geq 80\%$, and a state-machine model bounded by a finite generator universe — are carried by the testing agenda (§19, Appendix B), not restated here.

Key Invariants and Consequences

Unit state lives in three homes under one placement rule, and the conditions C1–C12 each render an illegal state unrepresentable rather than merely tested. Each guarantee carries its canonical number in §16.

- **One home, one writer.** Each economic fact has exactly one home and one writer, so internal reconciliation failure cannot arise. The homes are immutable *ProductTerms*[*u*], shared *UnitStatus*[*u*], and per-position *PositionState*[*w, u*]; the externally-authored per-(holder, unit) cell is empty by construction (C12). (§4.)
- **UnitStatus is a projection, not a source of truth.** It holds one value per unit, overwritten in place only as a read cache; the immutable event log is authoritative, and replaying the events to any point rebuilds the exact value that held then. This discipline discharges lifecycle idempotency P6—write-by-replacement at one key—via C5. (§4.3.)
- **Conservation by construction (P1).** A **Transaction** carries its flow as **Move** edges whose signed per-unit sum is the group identity; C2 (edge-conserved flow) and C3 (atomic transaction across all three maps) leave no unconserved transaction to reject. (§4.)
- **Snapshot/replay determinism (P8).** Replay is the fold of the stream; the monotone carrier C1 keeps the key set stable across any checkpoint cut, so the result is independent of where the cut falls. (§4.)
- **Terms and identity immutability (underpin P4).** *ProductTerms* is an append-only non-empty version list (C6, C7); re-registration is a hard error and a breaking amendment allocates a fresh unit, never rewriting the old one (C8, C10). (§4.)
- **Read scoping (P7).** Cross-overlay reads are forbidden at the capability layer wrapping the accessors; strategy exports flow only through *UnitStatus* (C4). (§4.)

Part III

Valuation, Contracts, and Instrument Lifecycles

5 Portfolio Valuation and PnL

Valuation follows from the wallet structure. Ownership is wallet balance; conservation (§2) keeps quantities consistent; valuation multiplies each position by its price and sums.

5.1 Quantity versus value

Conservation guarantees *quantity* balance: the total quantity of each unit across all wallets is zero, an algebraic invariant of the move semantics. It does not guarantee *value* balance. Total portfolio value $V_t = \sum_u w_t(u) P_t(u)$ moves with prices. Prices are external inputs; they change independently of ledger activity. An FX trade conserves EUR and USD quantities separately, yet at any rate other than the trade rate the market value of EUR received differs from the USD paid. Value conservation holds exactly only for deterministic lifecycle events (Property 5). Conservation secures quantity integrity; the valuation layer accounts for value change due to price movement.

5.2 Mark-to-market valuation

Definition 5.1 (Portfolio value). *Let $P_t : \mathcal{U} \rightarrow \mathbb{R}$ be a price function at time t , with reference currency $P_t(\text{USD}) = 1$. A portfolio is a set of wallets W —the real wallets of a book. Its value is the quantity of reference currency that would remain were every position in W unwound at P_t :*

$$V_t(W) = \sum_{w \in W} \sum_{u \in \mathcal{U}} w_t(u) P_t(u).$$

Summing over all wallets gives zero by closure (§2): real and virtual legs cancel, which is what makes the system closed. A meaningful valuation is therefore always of a chosen wallet scope, never the whole ledger. P_t is an external input, not computed by the framework; it requires only a designated reference currency and a price per unit in that currency. Pricing methodology — exchange quote, model output, or dealer quote — and the choice of reference currency and FX rates are fixed by the valuation specification (satellite, out of scope here).

Qualification. $P_t(u)$ presumes orderly, liquid markets. Under halts, closures, or counterparty default it may be stale, absent, or above achievable liquidation value.

Valuation is one total function from a price vector and a ledger to reference-currency cash. The full price methodology and reference-currency machinery live in the valuation satellite; here only the signature and the law it carries.

```
-- Qty, Cash are exact additive groups (Integer minor units, never Float).
-- Price carries NO Monoid: adding two prices is meaningless, so the type
-- forbids the one wrong combination. Qty and Price meet only via markValue.
newtype Qty    = Qty    Integer
newtype Cash   = Cash   Integer    -- a Monoid: Cash a <> Cash b = Cash (a+b)
newtype Price  = Price  Integer

markValue :: Qty -> Price -> Cash      -- markValue (Qty q) (Price p) = Cash (q
    *p)

-- A price vector prices EVERY unit (a total function, not a partial lookup):
-- "a held unit with no price" is unrepresentable; the reference unit prices to 1.
newtype PriceVec = PriceVec { priceOf :: UnitId -> Price }

-- Portfolio value: a total fold over the wallets in scope 'ws' against prices.
-- Summing over ALL wallets is zero by closure, so valuation is of a chosen book.
-- The signature has no time and no history -- value depends ONLY on current
-- balances in scope and prices. State-sufficiency is the signature, not a claim.
value :: PriceVec -> [WalletId] -> Ledger -> Cash
value (PriceVec p) ws l =
    foldMap (\((w, u), ps) -> if w `elem` ws
        then markValue (psBalance ps) (p u) else mempty)
        (Map.toList (ledgerPS l))

-- PnL of one portfolio 'ws' is the difference of two valuations (P10). Each is
-- fixed by its own endpoint alone, so inserting any intermediate snapshot adds
-- (v 'cashSub' v) = mempty: telescoping, hence path-independence.
pnl :: [WalletId] -> PriceVec -> Ledger -> PriceVec -> Ledger -> Cash
pnl ws p0 l0 p1 l1 = value p1 ws l1 'cashSub' value p0 ws l0
```

Three illegal states fall out of the types. A held unit with no price cannot occur: `PriceVec` is a total function, not a partial map. Adding two prices cannot be written: `Price` has no `Monoid`; the only way out of `Price` is `markValue`, which lands in `Cash`. Path-dependence cannot enter: `value` takes a wallet scope, a `Ledger`, and a `PriceVec`, and no transaction history, so the type itself certifies state-sufficiency. The fold is total over a finite balance projection, and `markValue` is exact integer multiplication, so `value` is total and deterministic.

5.3 State-sufficiency

To value the book, the only inputs are where positions stand now and what they are worth now; how they got there does not enter.

Principle 5.2 (State-sufficiency). *Given that every economically relevant state change is recorded, portfolio value at time t depends only on current wallet balances, current unit state,*

and current prices. Transaction history is irrelevant.

Unit state carries all embedded lifecycle state — accrued interest, exercise flags, barrier status. It is a materialised projection of the immutable event log (§3), not a separate source of truth. Computing V_t therefore needs one snapshot of balances and one price vector, not a replay: PnL is $O(n)$ in instruments, not $O(m)$ in moves.

5.4 Path-independent PnL

Profit and loss over a period depends only on its two endpoints. The trading in between, however much of it, leaves the total unchanged.

Theorem 5.3 (Path-independent PnL). *Profit and loss over $[t_0, t_1]$ is*

$$\text{PnL} = V_{t_1} - V_{t_0},$$

a function of the portfolio states at t_0 and t_1 alone, not of intermediate transactions. (Property P10.)

Proof. Each of V_{t_0} , V_{t_1} is fixed by wallet balances and prices at its time, and balances are the cumulative effect of all prior moves. For any intermediate times $t_0 < s_1 < \dots < s_k < t_1$,

$$V_{t_1} - V_{t_0} = (V_{t_1} - V_{s_k}) + (V_{s_k} - V_{s_{k-1}}) + \dots + (V_{s_1} - V_{t_0});$$

the interior values cancel, and no path information survives. $\square$

The mechanism is telescoping; the content is independence of trading activity. A portfolio that executed ten thousand intermediate trades has the same total PnL as one that executed none, given matching final positions and prices. PnL never requires replaying intermediate transactions: `pnl` above takes the two endpoints and nothing between them.

Qualification. The result captures a state change only if it is recorded as a move. An unrecorded fee or an unbooked bilateral agreement falls outside it: PnL is defined as the change in the ledger’s valuation, and the ledger records all economic activity within its scope.

Scope. This is *economic* PnL — the change in fair value. Accounting PnL under IFRS 9 or US GAAP depends on instrument classification (FVTPL, FVOCI, amortised cost), which is outside the ledger’s scope; the framework supplies the position quantities and economic valuations from which it is derived.

5.5 Price and flow decomposition

Proposition 5.4 (PnL attribution).

$$\text{PnL} = \text{PnL}_{\text{price}} + \text{PnL}_{\text{flow}},$$

$$\text{PnL}_{\text{price}} = \sum_i w_{t_0}(i) [P_{t_1}(i) - P_{t_0}(i)], \quad \text{PnL}_{\text{flow}} = \Delta w(\text{USD}) + \sum_i \Delta w(i) P_{t_1}(i).$$

Price PnL is revaluation of the opening holding; flow PnL is the net effect of cash and position moves. Worked example, USD reference, one risk unit AAPL:

Event	USD	AAPL	Price
State at t_0 ($V_{t_0} = 1000 + 10 \cdot 100 = 2000$)	1000	10	100
Buy 5 @ 100	−500	+5	
Dividend	+20		
Fees	−5		
Financing	−8		
State at t_1 ($V_{t_1} = 507 + 15 \cdot 110 = 2157$)	507	15	110

$$\begin{aligned}
\text{PnL} &= V_{t_1} - V_{t_0} = 157, \\
\text{PnL}_{\text{price}} &= 10 (110 - 100) = 100, \\
\text{PnL}_{\text{flow}} &= (507 - 1000) + 5 \cdot 110 = -493 + 550 = 57, \\
100 + 57 &= 157.
\end{aligned}$$

Every cash flow — purchase, dividend, fee, financing — is an explicit USD move; price change revalues existing positions only.

5.6 Dual valuation

The same positions can hold more than one value at once: value is read from a price vector, and more than one price convention can apply.

V_t is a function of an externally supplied price vector, so distinct price conventions value the same positions distinctly. Applying Definition 5.1 under two conventions — mark-to-market and mark-to-mid — yields the dual valuation. Its construction, the multi-exchange futures case, and its balance-sheet use are developed in §10.

6 Smart Contracts as Move Generators

A financial instrument is a deterministic program that emits moves, not an opaque record in a database. A bond emits coupons and a redemption; an option a premium and a conditional payoff; a swap periodic net payments. The logic varies; the output is always one list of moves.

6.1 Contract Definition

Definition 6.1 (Smart Contract). *A smart contract is a deterministic function from inputs, state, and observed conditions to a sequence of atomic moves:*

$$\text{Contract} : (\text{Input}, \text{State}, \text{Conditions}) \rightarrow [\text{Move}].$$

A contract’s sole output is moves. A move transfers a quantity from one wallet to another without creating or destroying it (Section 2), so conservation holds by construction: a contract can move units, never create or destroy them.

Equal inputs, state, and conditions yield equal moves: no randomness, no hidden state, no dependency beyond the passed parameters. The contract is not stored apart from the ledger—its effects *are* ledger entries. Product complexity is isolated in the contract; the ledger sees only moves. A new product type is a new contract. The move stream, balance computation, and reporting are untouched, and its moves are indistinguishable from any other product’s at the ledger level.

A smart contract is the machine-readable, executable form of the legal confirmation. An ISDA derivatives confirmation already states the product, economics, lifecycle events, and settlement obligations in structured terms; the CDM is the formal vocabulary of that structure. Writing the contract from the confirmation is transcription, not engineering: CDM enums map directly onto product types, lifecycle events, and settlement obligations. The contract adds executability and nothing to the information content.

The output type is a **Move** carrying a quantity **Qty**. A quantity is an exact integer in the unit’s smallest denomination (minor units: cents, satoshis, basis-point-notional); never a decimal, never a float. Integer minor units make addition associative and every balance computation exact. Identical inputs therefore produce bit-identical outputs across hardware and compilers. This is the precondition for reproducible time travel and cross-system verification.

Rounding (round-half-to-even, IEEE 754) applies once, at instruction projection—where an exact internal amount becomes a settlement or payment message—never within balance or payment arithmetic.

The types make this exactness structural. A `Qty` is an integer count of minor units, and the contract is a total function emitting a list of moves. The categorical shape is a fold over a finite schedule: the contract is fully determined by the moves it lists, with no hidden step.

```
-- A quantity is an EXACT integer in the unit's smallest denomination
-- (minor units: cents, satoshis, basis-point-notional). Never a decimal float.
newtype Qty = Qty Integer deriving (Eq, Ord, Show)

-- Qty is the additive abelian group of Section 2 (the position algebra).
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b) -- associative, exact
instance Monoid Qty where mempty = Qty 0 -- identity (zero)
qneg :: Qty -> Qty -- inverse
qneg (Qty n) = Qty (negate n)

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId = UnitId String deriving (Eq, Ord, Show)

-- A move: a positive Qty of one unit, one wallet to another (Section 2),
-- tagged with the emitting contract and metadata. Direction carries the sign,
-- so mQty is always > 0.
data Move = Move
  { mFrom :: WalletId, mTo :: WalletId
  , mUnit :: UnitId, mQty :: Qty -- exact minor units, > 0
  , mTime :: Integer, mSource :: String -- timestamp; emitting contract id
  , mMeta :: String -- event description / external refs
  } deriving (Eq, Show)

-- A smart contract is a DETERMINISTIC, TOTAL function emitting a move schedule.
-- The ledger sees only [Move]; product complexity stays inside the contract.
type Contract i s c = (i, s, c) -> [Move]
```

Integer addition is associative and carries no rounding error, so balance and payment arithmetic is exact and identical inputs give bit-identical outputs. Rounding enters *once*, at instruction projection, where converting an exact internal amount to a settlement message can introduce a fraction (a rate, an FX cross); round-half-to-even applies there and nowhere else.

```
-- The SOLE rounding site. Prelude 'round' IS round-half-to-even (IEEE 754).
projectAmount :: Rational -> Qty
projectAmount r = Qty (round r)

-- ghci> Qty 250 <> Qty 12500 -- exact integer addition
-- Qty 12750
-- ghci> qneg (Qty 12500) <> Qty 12500 -- conservation: no rounding error
-- Qty 0
-- ghci> projectAmount (5 % 2) -- 2.5 -> nearest even -> 2
-- Qty 2
-- ghci> projectAmount (7 % 2) -- 3.5 -> nearest even -> 4
-- Qty 4
-- ghci> Move (WalletId "wB") (WalletId "wS") (UnitId "USD")
-- (Qty 250000) 0 "put_ref" "PUT_PREMIUM" -- Move 1 below, typed
```

Remark 6.2 (FORMALIS clearance). The `Qty` group (`(<>)`, `mempty`, `qneg`) is reused verbatim from the FORMALIS-cleared reference `reference/Ledger.hs` and is already cleared. The `Move` record, `Contract` synonym, and `projectAmount` are new: FORMALIS verdict **CLEARED**—all total (`round` is total on `Rational`; `Move` is a plain product; `Contract` is a type synonym)

and deterministic, with rounding confined to the single `projectAmount` boundary so no balance arithmetic can round.

6.2 Composition: Modular over Composite

Principle 6.3 (Modular contracts). *One smart contract per payout or obligation type, not one composite contract per instrument.*

An option payoff depends on strike, spot, and exercise terms; a CSA margin depends on aggregate mark-to-market, threshold, minimum transfer amount, and eligible-collateral currency. These are independent calculations on independent inputs. Separation keeps each a pure function testable in isolation; combination makes the input space the product of both, most of which is irrelevant. A margin contract written once attaches to any collateral wallet—option, swap, or structured note—eliminating duplication.

Modularity requires clear interfaces: the executor—the single component permitted to mutate the ledger (Section 7)—orchestrates the contract invocations for a given lifecycle event. This orchestration is simpler than maintaining a composite script for every product–CSA combination.

6.3 Equity, Dividend, and Corporate-Action Contracts

A stock contract emits moves for trade execution, dividend payment, and corporate actions through one primitive—a set of moves in a transaction—with no separate machinery.

Trade execution. A purchase of n shares at price p is a transaction of two atomic moves: $n \cdot p$ cash from buyer to seller, and n of the stock from seller to buyer.

Dividend. On the ex-dividend date the contract emits cash from an issuer virtual wallet to each holder wallet, proportional to shares held. The unit state records the event (date, amount per share) for idempotency.

Bond accrued interest. Accrued interest is not a separate unit. The bond price function $P_t(u)$ returns the *dirty* price (including accrual), and a between-coupon trade settles as cash-for-bond at the dirty price. The next coupon date pays the full coupon and transitions the bond ex-coupon. Clean versus dirty is a reporting convention derived from unit state (last coupon date, accrual convention), not a separate position; both prices are reported by subtracting accrued interest computed from that state.

Corporate actions. A corporate action is a sequence of state transitions and moves anchored to its dates: announcement, record, ex/effective, payment. At the record date the unit state records the entitlement snapshot; at the ex/effective date the contract emits position-adjustment moves from a corporate-action virtual wallet—a 2-for-1 split doubles each entitled holding while the price reference halves. Each step is a transaction with full metadata (dates, ratios, entitlement rules), so time travel reconstructs both pre- and post-action views. The same multi-date mechanism handles mergers, spin-offs, and rights issues.

Remark 6.4 (Unit state is a projection, not a mutable store). *Unit state above is materialised UnitStatus, the read cache governed in Section 7. UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.*

6.4 European Put—Full Move Schedule

Parameters: strike K , expiry T , underlying S , notional N , premium P , buyer w_B , seller w_S . The contract is completely defined by conditional moves at inception and expiry.

Move 1 (inception, $t = 0$) — unconditional, on contract creation:

```
Move(from: w_B, to: w_S, unit: USD, quantity: P * N,
      timestamp: t_0, source: "put_ref", metadata: "PUT_PREMIUM")
```

Cash settlement. *Move 2 (expiry $t = T$)* executes only if $S_T < K$ (in-the-money); the seller pays the buyer the intrinsic value $(K - S_T) \cdot N$. If $S_T \geq K$, no move.

```
Move(from: w_S, to: w_B, unit: USD, quantity: (K - S_T) * N,
      timestamp: T, source: "put_ref", metadata: "PUT_SETTLEMENT_CASH")
```

Physical delivery. If $S_T < K$, the buyer delivers N of the underlying and receives $K \cdot N$ cash; the two moves are one atomic transaction, requiring the buyer to hold the underlying at expiry. If $S_T \geq K$, no moves.

```
Move(from: w_B, to: w_S, unit: S, quantity: N, ... metadata: "
      PUT_DELIVERY_ASSET")
Move(from: w_S, to: w_B, unit: USD, quantity: K * N, ... metadata: "
      PUT_DELIVERY_CASH")
```

Lot-size constraint. Equity delivery requires whole multiples of the board lot L (reference data in the Unit Store). For lot L ,

$$N_{\text{deliver}} = \left\lfloor \frac{N}{L} \right\rfloor \cdot L, \quad N_{\text{residual}} = N - N_{\text{deliver}}.$$

Whole lots settle as a DvP pair at strike; the residual settles in cash at intrinsic value. All moves are one atomic transaction:

```
Transaction (settlement: txIntroduce = Nothing, carrying the three moves below):
  Move(from: w_B, to: w_S, unit: S, quantity: N_deliver)      -- whole
  lots
  Move(from: w_S, to: w_B, unit: USD, quantity: K * N_deliver) -- strike
  cash
  Move(from: w_S, to: w_B, unit: USD, quantity: (K - S_T) * N_residual) -- cash
  residual
```

Proposition 6.5 (Lot-split conservation). *The underlying nets to zero ($-N_{\text{deliver}} + N_{\text{deliver}} = 0$); cash nets to zero, the seller's outflow $K \cdot N_{\text{deliver}} + (K - S_T) \cdot N_{\text{residual}}$ equalling the buyer's inflow. The economic value transferred is the put payoff $(K - S_T) \cdot N$, independent of the lot split.*

The lot-adjustment is a pure function of (K, N, S_T, L) , tested by the property suite (Appendix B). The settlement projection (Section 12) yields two instructions: a securities delivery (`sese.023`) for N_{deliver} shares and a cash payment for $K \cdot N_{\text{deliver}} + (K - S_T) \cdot N_{\text{residual}}$. The pattern generalises to any physically settled instrument with lot constraints—convertible conversion, warrant exercise, equity-forward delivery—only the payoff formula differs.

The contract is fully defined by its move schedule: a finite list of conditional transfers parameterised by market observables. Pricing complexity (Black–Scholes, Monte Carlo, finite differences) is irrelevant to the ledger, which sees only the emitted moves.

6.5 Interest Rate Swap—Full Move Schedule

Parameters: notional N , fixed rate r_{fixed} , floating reference $r_{\text{float},k}$ (e.g. SOFR) at reset k , reset dates $\{t_1, \dots, t_n\}$, day-count fraction δ_k (CDM date types and conventions fix the dates and δ_k , Section F.5), fixed-payer w_F , floating-payer w_L . For each reset t_k :

$$\text{Payment}_k = N \cdot (r_{\text{float},k} - r_{\text{fixed}}) \cdot \delta_k.$$

```
Move(from: w_F, to: w_L, unit: USD, quantity: Payment_k,
      timestamp: t_k, source: "irs_ref", metadata: "IRS_NET_PAYMENT")
```

Move quantities are positive (Section 2). The contract emits $|\text{Payment}_k|$: $w_F \rightarrow w_L$ when $\text{Payment}_k > 0$, $w_L \rightarrow w_F$ when $\text{Payment}_k < 0$. Direction encodes the sign; quantity is the magnitude. Each reset needs only the current floating rate and the fixed parameters—no historical path. The complete contract is the sequence $\{m_1, \dots, m_n\}$, one move per reset, plus the unit state recording settled resets for idempotency.

Worked example (CDM lifecycle). A 5-year USD IRS, notional \$10M, fixed 3%, quarterly resets.

- *Inception* (CDM `ExecutionEvent`, intent `OPEN`). Unit state initialised: fixed rate, floating index, schedule, reset counter = 0. No cash move (par swap, no upfront premium).
- *Reset at t_1* (CDM `ResetEvent`). SOFR observed at 3.5%; unit state records `float_rate_t1` = 3.5%, counter incremented.
- *Coupon at t_1* (CDM `TransferEvent`). $10\text{M} \times (3.5\% - 3\%) \times 0.25 = 12,500$. `Move(from: w_F, to: w_L, unit: USD, quantity: 12,500)`; period t_1 marked settled.
- *Maturity at t_n* (CDM `TerminationEvent`, intent `TERMINATE`). Final coupon as above; unit state transitions to `TERMINATED`. No further resets or payments.

7 Lifecycle Management

A lifecycle event is a transaction. A coupon payment, a dividend, a margin call, a barrier hit, an exercise, a reset, and a rebalancing step are each a contractual obligation becoming due, realised as value moving between wallets together with a change of contract state. From the ledger’s perspective executing a trade and processing a lifecycle event are the same operation: a finite list of atomic moves, possibly accompanied by unit registrations and state changes, obeying conservation. There is no separate lifecycle engine, data format, or pipeline per product type; every product type uses the move and the state transition.

This identity has four consequences.

1. **Uniformity.** Trade execution, coupon payments, margin flows, conversions, and rebalancing are one primitive—a transaction carrying moves and optional state updates.
2. **Conservation.** Every transaction satisfies the conservation law of §2; lifecycle events preserve closed-system integrity by construction.
3. **Auditability.** The evidence of a lifecycle event is the transaction: dated moves with origin and metadata, plus explicit state changes.
4. **Composability.** Lifecycle events sequence and simulate exactly as trades do. “Execute a trade, process a coupon, settle a margin call” is three transactions applied in order, each conserving.

7.1 The Transition Function

A lifecycle event is computed from the present, not from history. It reads the unit, that unit’s current state, and the observed market data, and returns the moves to make together with the next state. It commits nothing.

A lifecycle event is a pure function

$$f : (\text{unit}, \text{state}_t(u), \text{market_data}) \rightarrow (\text{moves}, \text{state}')$$

It mutates nothing, reads no hidden global, and returns a description: a list of wallet-to-wallet moves and the next state. The function is local to the unit and the event—it requires no knowledge of other units, wallets, or pending events. `applyTx` commits its output atomically.

Lifecycle is therefore state-transition logic plus a collection of moves, defined without assuming a pricing model exists. This generality is required: the ledger carries convertible bonds, margin loans with bespoke collateral rules, and rule-based strategies whose economics live in state transitions rather than closed-form prices. Each is handled at the lifecycle level on the same footing as a plain-vanilla instrument, because the framework demands only the transition function above.

In code `f` is `handle`: it reads the ledger (which supplies `state_t(u)`), takes the event (which carries the unit and any market datum), and returns the proposal as a `Transaction`—the moves and the state delta they ride with—never a changed ledger.

```
-- A lifecycle event is a pure function of the current state: it reads and
-- proposes, it commits nothing. The proposal is a Transaction (sec.4) -- the one
-- atomic event: the moves plus the non-balance state delta they ride with -- not
-- a changed Ledger.
handle :: Event -> Ledger -> Either LedgerError Transaction

-- The concrete Event type is given per product family: the three-home model
-- (sec.4) and the futures lifecycle (sec.8). The Transaction it builds carries
-- * txMoves : the conserved flow in edge form, each leg debiting one wallet and
--               crediting another by the same magnitude, so the signed per-unit
--               sum is empty BY CONSTRUCTION (conservation, C2/C9);
-- * the state delta : the status / stage writes the event causes (txStatus,
--                       txRows, txIntroduce, txAppend).
```

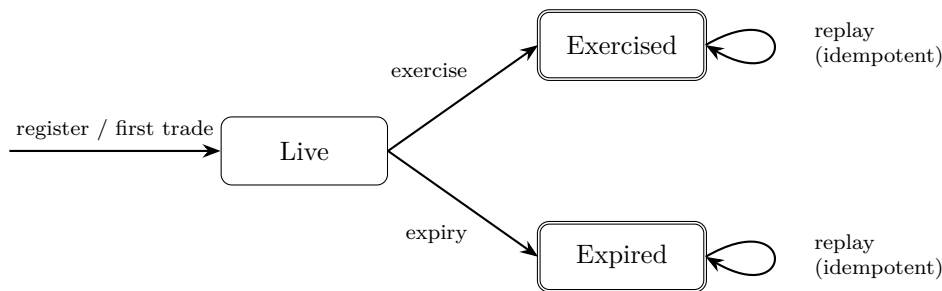


Figure 5: Lifecycle as a state machine, for a generic instrument: from *Live*, exercise leads to *Exercised* and expiry to *Expired*. The terminal states are absorbing; their self-loops depict the idempotence of replay—re-applying a settled event produces no further effect (§7). There is no single global machine: the transitions are determined by **ProductTerms** per unit-type. The concrete futures machine REGISTERED → ACTIVE → EXPIRED → CLOSED is one instance (§8).

```

-- Market data enters only inside the Event, as a logged observation (e.g. a
-- settlement price), so handle stays pure -- a fresh replay reproduces the same
-- moves bit for bit.

```

7.2 Idempotence

A lifecycle event that has already taken effect does nothing when applied again: the second application reads a state that already records the first, so it proposes no moves and no change of state.

Principle 7.1 (Idempotence). *Applying the same lifecycle event twice to the same state has no additional economic effect.*

Idempotence holds because moves are derived from $(\text{state}_t(u), \text{event})$, not from history. The state records which events have occurred; if it already shows the coupon at date T paid, f produces no moves and no state change. Paying a coupon twice at the same date pays it once.

7.3 The Single Door

One function changes the ledger; nothing else does. A lifecycle function only proposes a transaction, and that proposal becomes part of the ledger only when this one function commits it.

Principle 7.2 (Single door). *`applyTx` is the only function permitted to change the ledger.*

The Transaction returned by a lifecycle function is a proposal; it takes effect only when `applyTx` commits it. Conservation is not checked at this boundary—it is a property of the Transaction type, since the signed edge-sum of the moves is `mempty` by construction (§2, C2/C9), so there is no unconserved Transaction to reject. `applyTx` requires that the unit be registered, inserts or updates rows and never deletes one, and commits the whole Transaction atomically to the immutable event log of §2. Product guards—sufficient source balance, stage legality—live in `handle`, which builds the proposal.

Confining mutation to one door has three consequences. Conservation holds in exactly one place, the Transaction type, so a defective contract cannot corrupt the ledger: there is no unconserved proposal for `applyTx` to admit, and a balance moves only via a conserving `Move`. Lifecycle functions stay pure—they compute a proposal and cannot alter state, so they are testable, composable, and replaceable. The audit trail is complete: every state change passes

through `applyTx` and is logged. The attack surface for state corruption is bounded to this single door, whose properties P1–P10 are stated and tested in §16. The role is that of an operating-system kernel: all privileged operations pass through one narrow, auditable interface.

```
-- handle PROPOSES a Transaction and commits nothing; applyTx (sec.4) is the SOLE
-- door that returns a changed Ledger. The step stops at the first error (the bind
-- ):
step :: Event -> Ledger -> Either LedgerError Ledger
step ev l = handle ev l >>= \tx -> applyTx tx l

-- There is no separate validate gate. Conservation is a property of
-- the Transaction type -- the signed edge-sum of txMoves is empty BY
-- CONSTRUCTION
-- (C2/C9) -- so an unconserved move is unrepresentable, not rejected at run time.
-- applyTx (sec.4) is the single mutation boundary: it requires the unit be
-- registered, inserts/updates rows and NEVER deletes one (monotone carrier, C1),
-- and returns the whole new Ledger or a Left, never a partial state (atomicity,
-- C3). Product guards -- sufficient source balance, stage legality -- live in
-- handle (sec.8, G1–G5).
-- applyTx :: Transaction -> Ledger -> Either LedgerError Ledger -- sec.4

-- Time travel is this same step FOLDED over the event stream (foldM threads the
-- ledger, stopping at the first error); equivalently, once the proposals are
-- built, it is foldM applyTx over the transactions (sec.4). Replaying to any t
-- rebuilds the exact ledger as of t -- this is clone_at t.
replay :: [Event] -> Ledger -> Either LedgerError Ledger
replay evs l0 = foldM (flip step) l0 evs
```

Replay is a *fold homomorphism*—meaning replaying two event batches joined, `replay (xs <> ys)`, equals replaying each in turn, `replay xs >=> replay ys`, where `f >=> g` runs `f`, feeds its result to `g`, and stops at the first error. The reconstructed ledger is therefore independent of where a checkpoint is cut: checkpoint-independence (§16, P8) is a consequence of the law, not a separate test.

The event log is therefore canonical. Every state change passes through the single door and is logged, and replay rebuilds any state from those events alone. Any ledger, current or historical, is reproducible by replaying its log; the projections of §4, `UnitStatus` among them, are rebuilt this way.

Remark 7.3 (Replay law (optional)). Plain reading. *The ledger state is fixed by the sequence of events alone. Applying events A then B then C gives the same state whether or not you checkpoint after B—order matters, where you cut does not.*

Optional lens. `replay = foldM applyTx` is a monoid homomorphism from the free monoid of events (`[Transaction]`, `<>`, `[]`) into the monoid of state-transitions—the endo-Kleisli arrows `Ledger -> Either LedgerError Ledger` under (`>=>`) with identity `Right`—so `replay (xs <> ys) = replay xs >=> replay ys` holds exactly; checkpoint-independence is a consequence of this law, and functional determinism follows from `applyTx` being pure and total.

7.4 Purity

Principle 7.4 (Purity). *For fixed inputs—view, parameters, market data—a lifecycle function produces identical outputs, independent of when or where it runs.*

Purity is a design requirement, not a coding convention. A lifecycle function has no hidden dependency: no runtime database lookup, no configuration read, no ambient state. Every input affecting the output is passed explicitly.

Purity alone does not give reproducibility; reproducibility also requires a deterministic market-data oracle—a fixed specification of price source, snapshot time, and fallback chain.

The market data used by each invocation is captured and stored at execution time as a versioned snapshot. Replays read the stored snapshot, not a live feed. Vendor corrections are separate data versions: a replay “as known at t ” uses the snapshot from t ; a replay “with corrected data” uses the restated snapshot. Each is deterministic given its inputs. Market data is governed by the valuation and market-data satellite specifications; this section depends only on the determinism the oracle provides.

Purity reduces testing to three mechanical forms. Unit tests assert “given view and inputs, expect these moves and state changes.” Property tests, such as idempotence (Principle 7.1), run the same function repeatedly on cloned views. Regression tests construct the appropriate historical view and feed the stored snapshot.

7.5 Time Travel

Time travel reconstructs exact ledger state at any historical timestamp t : not merely today’s totals replayed, but what each position was, what it referenced, and how its value evolved. Four situations make this non-trivial; any implementation must reconstruct each.

1. **Futures and margin PnL.** The margining story is reconstructed step by step: the sequence of variation-margin calls and the prices at which each was computed, not only the final cash total.
2. **Corporate actions and share denominators.** A 2-for-1 split turns 100 pre-split shares at a pre-split price into 200 post-split shares at roughly half the price; economic value is preserved but the unit changes. Replaying trade tickets into a current share definition mis-describes the t_0 position. Correct time travel reconstructs both views and explains the jump as a corporate action, not a reference-table edit.
3. **Exercise and novation.** On exercise the derivative is replaced by a cash or physical position; on novation the original contract is extinguished and a new one created with a different counterparty. The reconstruction must show why the derivative is present at t_0 and absent at t_1 , not only the cash that moved.
4. **Basket composition changes.** A basket of 40% A, 30% B, 30% C is redefined when B is acquired, substituting the acquirer D at a merger-dictated ratio. Time travel before the change shows the original basket; after, the updated basket with the conversion explained. A basket read from today’s static file cannot distinguish the two.

These are routine features of real portfolios. Unit state makes their reconstruction possible: each event—split, exercise, novation, redefinition—is recorded as an explicit state change on the relevant unit, so a view cloned at any historical date carries the correct state, and re-applying f to that state with the stored market data reproduces exactly the moves generated then. Without explicit unit state, time travel is limited to balance replay—enough for cash totals, insufficient to explain what the portfolio meant.

Cloning. Two operations realise reconstruction.

- **clone()** deep-copies the current view—units, states, and moves to the current time—for simulation or testing without affecting the original.
- **clone_at(t)** replays the move stream and all state changes up to t and returns the ledger as of t ; later moves are ignored.

Any lifecycle function re-run against a clone reproduces the same pending transaction given the same market data. Backtesting and scenario analysis reduce to: take a historical state, apply f , inspect the moves and state deltas. Time travel is a design property, not a debugging aid: any state is reachable again by replaying the same transactions, and any lifecycle logic re-applied yields identical outcomes (§16, P8).

7.6 State Storage and Embedded Defence

Unit state is not stored as a private dictionary on the unit. It lives in the three-home state model of §4, where the per-unit lifecycle projection is `UnitStatus`.

Remark 7.5. *`UnitStatus` holds one value per unit, shared—read identically by every holder. Its value changes over time, but `UnitStatus` is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.*

A lifecycle function never writes `UnitStatus` off-log; it returns a Transaction, `applyTx` logs the event, and the projection is rebuilt from the log. This is why time travel is exact: `clone_at(t)` rebuilds the projection from the events up to t .

Attaching every lifecycle state change to the unit gives an embedded explanation layer. For any instrument u , its current state $\text{state}_t(u)$ and recorded changes reconstruct its lifecycle history—coupons paid, dividends accrued, barriers triggered, margin calls made. Three consequences follow.

1. **Valuation justification.** Current valuation is explained directly from the state—remaining cash flows, exercised versus unexercised rights, triggered versus untriggered features—without chasing flags across systems.
2. **Client notification.** Coupon advices and margin-call letters are generated mechanically from the state-change stream, since every economically relevant event is an explicit transition.
3. **Traceability.** Any amount or flag in the current state traces back to the exact transaction that created it.

7.7 Lifecycle Risk Profile

Every lifecycle function category is either a pure function or a deterministic projection; `applyTx` is the sole mutation point, and no function bypasses it. The table maps each category to its purity guarantee and its property-test coverage (§16).

Function gory	Cate-	Description	Purity antee	Guar-	Property Coverage	Test
Contract tion	evalua-	Maps (input, state, con- ditions) to moves	Pure (§6)		P1 (conservation); totality (no unhan- dled error)	
State transition		Maps (unit, state, event) to (moves, new state)	Pure (Princi- ple 7.4)		P5, P6 (idem- potency); valid transitions	
Transaction com- mit (<code>applyTx</code>)	com-	Sole door: registration check, monotone atomic commit	Deterministic; sole mutator		P1 (conservation by construction), P2 (atomic commit- ment)	
View / snapshot		Read-only ledger acces- sor	Pure (read-only)		P8 (snapshot / time- travel consistency)	

Function gory	Cate-	Description	Purity	Guar-	Property	Test
PnL projection		$V_t = \sum w_t(u) \cdot P_t(u)$	Pure from prices)	(derived state +	P10 independence)	(path- independence)
Balance computa- tion		$w_t(u)$ from stream replay	Pure (projection of log)		P1 (conservation), P3 (referential in- tegrity)	

7.8 Strategy as Unit: QIS and Leveraged ETFs

A quantitative investment strategy or leveraged ETF is itself a unit. Its state carries the last rebalance timestamp, the current weights or holdings, configuration (target leverage, rebalancing frequency, reset rules), and path-dependent flags (triggered barriers, suspension status). This is strategy-as-unit; the collapse of its holdings sector is governed by condition C12 (§4).

The lifecycle function computes, at each rebalance or leverage reset, the trades required to hold the target allocation—moves between the strategy wallet and market wallets—and an updated state recording new weights, reference level, and performance lock-in. Subscriptions and redemptions are lifecycle events: a subscription creates units, triggers the underlying rebalance, and updates composition and leverage; a redemption destroys units and unwinds correspondingly.

Time travel follows the general rule. A view cloned at t reconstructs the strategy unit’s state and all past rebalances; re-applying the rebalance logic with the stored market data reproduces the same transactions. Client-level exposure, leverage, and performance at t are explained directly from the reconstructed state and its moves.

8 The Futures Lifecycle

A listed future exercises every primitive at once. An equity trade is one transaction — cash one way, shares the other. A futures trade reads the same, “buy 10 at 100,” yet moves no cash at inception; its economic effect unfolds over days through variation margin, partial closes, and settlement. The future is therefore the canonical lifecycle: recorded, traded, settled daily, expired, and flattened to zero, each step one atomic `FutStateDelta` over the move and conservation primitives of §2 and the three-home state model of §4. The `Fut*` types form a futures-engine kernel that *projects onto* the core `Transaction` — exactly as the CDM and settlement boundary types do, and under the same naming discipline; it is not a second core. Conservation is shown at each step, not asserted. The centrepiece is variation-margin settlement: one event that writes the shared settlement price once and fans out over the current holders, resetting each accumulated cost and moving cash.

8.1 Primitives recalled, and where the future’s state lives

A *wallet* w holds a signed quantity $h(w, u)$ of a *unit* u ; a *move* transfers a positive quantity of one unit from a source wallet to a destination ($w_s(u) \mathrel{-}= q$, $w_d(u) \mathrel{+}= q$); *conservation* is $\sum_w h(w, u) = 0$ for every u , an algebraic consequence of the move pattern, not a runtime check. Write $\text{net_qty}(w, u) := h(w, u)$.

State attaches to three homes (§4):

Map	Key	Discipline
<i>ProductTerms</i>	u	immutable, append-only, versioned; registration-total
<i>UnitStatus</i>	u	materialised projection of the log; registration-total
<i>PositionState</i>	(w, u)	monotone carrier; Option accessor

Principle 8.1 (*UnitStatus* is a projection, not a source of truth). *UnitStatus* holds one value per unit, shared—read identically by every holder. Its value changes over time, but *UnitStatus* is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.

For the future, the fields divide by who reads them and how often they change:

Field	Home	Reason
multiplier, currency, expiry, clearinghouse, exchange, product_id	<i>ProductTerms</i> $[u_F]$	immutable; the CME and ICE contracts are distinct units.
lifecycle_stage, last_settlement_price, last_settlement_date	<i>UnitStatus</i> $[u_F]$	one settlement value per contract, read identically by every holder.
accumulated_cost (ac)	<i>PositionState</i> $[w, u_F]$	two wallets holding the same contract carry different ac.

Accumulated cost **ac** is a *conserved* field: $\sum_w \mathbf{ac}(w, u) = 0$ for every u , enforced one level up at the event handler (C2), since a refinement type on a sum of exact integers is free in no production language.

The atomic delta. One event proposes one **FutStateDelta** for one unit across the three maps: at most one shared **FutStage** write, a map of per-holder position deltas ($\Delta \text{net_qty}$, Δac), and a map of per-holder cash legs. It applies in full or not at all (C3); a partially applied delta is not representable. A delta reaches application only through **futValidate**, which discharges conservation — $\sum_w \Delta f(w, u) = 0$ for each conserved field — and returns the abstract witness **FutValidDelta**. An unconserved delta cannot be applied.

The futures kernel is a boundary, not a core. **FutStateDelta** and its validated form **FutValidDelta** are the futures engine’s own constructions, named apart from the core **Transaction** on purpose. Each validated futures delta *projects onto* one core **Transaction** — its conserving moves (the edge-sum of the per-holder legs) together with the non-balance rows (the **ac** resets and the shared stage write) — recorded through the single core door **applyTx**. The core therefore keeps no separate validate gate and remains sealed; the local **futValidate** discharges conservation *within* the futures kernel before it projects out, the same way the CDM and settlement boundary types validate at their own edge.

The single dimension bridge. Contracts, money, and price are distinct types (**Qty**, **Cash**, **Price**). The only multiplication in the lifecycle,

$$\text{futMark}(\text{net_qty}, S, m) = \text{net_qty} \cdot S \cdot m : \text{Cash},$$

converts contracts at a price into money. **Price** carries no addition: two prices are never summed, nor moved between wallets. The load-bearing identity $\text{VM} = \text{net_qty} \cdot S \cdot m + \mathbf{ac}$ typechecks only because both summands are **Cash**. **Price** and **Cash** share one fixed minor-unit scale, chosen so $\text{net_qty} \cdot S \cdot m$ lands in **Cash**’s unit. The worked example carries **Price** in whole index points and **Cash** in whole dollars for readability; a production scale carries **Price** in scaled

minor units — a quarter-point for ES, a cent for a bond quoted in 32nds — so sub-point ticks are exact. The arithmetic is integer throughout and never divides, so the scale is free to fix and no rounding breaks conservation at any scale: the minor-unit scale is exact-integer.

The scalar layer — the three dimensions, futMark, and the PosQty parse boundary of §8.4 — is the file's foundation.

```
-- Three dimensions kept apart. Qty and Cash are additive groups; Price is not
-- (you never sum two prices, nor move one between wallets) -- so Price has no
-- Monoid instance, and that absence makes "a price summed into a balance"
-- unspellable rather than merely unintended.
newtype Qty    = Qty    Integer deriving (Eq, Ord, Show)
newtype Cash   = Cash   Integer deriving (Eq, Ord, Show)
newtype Price  = Price  Integer deriving (Eq, Ord, Show)  -- deliberately NO Monoid
newtype Day    = Day    Int     deriving (Eq, Ord, Show)

instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid Qty where mempty = Qty 0
instance Semigroup Cash where Cash a <> Cash b = Cash (a + b)
instance Monoid Cash where mempty = Cash 0

qneg    :: Qty -> Qty ; qneg    (Qty n) = Qty (negate n)
cashNeg :: Cash -> Cash; cashNeg (Cash n) = Cash (negate n)

-- The single dimension bridge: contracts at a price, scaled by the multiplier,
-- become money. The only multiplication in the lifecycle, and the only place the
-- dimensions cross. VM = futMark ... <> ac typechecks only because both are Cash.
futMark :: Qty -> Price -> Integer -> Cash
futMark (Qty q) (Price s) mult = Cash (q * s * mult)

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String deriving (Eq, Ord, Show)

-- The positive-quantity parse boundary (G5). PosQty is abstract: mkPosQty is its
-- sole constructor, so q <= 0 is unrepresentable downstream of the boundary.
newtype PosQty = PosQty Qty deriving (Eq, Ord, Show)

mkPosQty :: Qty -> Maybe PosQty
mkPosQty q@(Qty n) | n > 0      = Just (PosQty q)
                  | otherwise = Nothing

unPosQty :: PosQty -> Qty
unPosQty (PosQty q) = q
```

Evaluations. futMark (Qty 10) (Price 100) 50 = Cash 50000; futMark (Qty (-10)) (Price 102) 50 <> Cash 51000 = Cash 0 (a Cash sum); Price 100 <> Price 102 is a type error — Price has no <>. Price and Cash share one fixed minor-unit scale; the example carries whole index points and whole dollars for readability.

8.2 The instrument and the worked example

Definition 8.2 (The worked instrument). *One cash-settled listed future $u_F = ES-FUT$, with $m = 50$, currency USD, expiry Day 3, clearinghouse CME-CH, exchange CME. Wallets A, B, C are traders; CH is the clearinghouse virtual wallet, the variation-margin settlement hub whose cash leg is the residual that balances the holders' legs.*

CH stands in as a settlement hub, not as a *novating* central counterparty: faithful novation would interpose CH on the *contract* leg of every trade ($A+q/CH-q$ and $CH+q/B-q$), so

the two cleared sides never face each other. Clearing infrastructure is out of scope; here CH carries only the variation-margin cash residual. Two rules govern every figure:

- **Trade.** For a signed-quantity change Δ at price p , each leg sets $\text{ac} += -\Delta \cdot p \cdot m$.
- **Settlement.** At price S , the per-holder variation margin is $\text{VM}(w) = \text{net_qty}(w) \cdot S \cdot m + \text{ac}(w)$ (cash, routed through CH); then $\text{ac}(w) \leftarrow -\text{net_qty}(w) \cdot S \cdot m$.

The whole life, with net_qty , ac , and per-step variation-margin cash for (A, B, C) :

Step	Event	net_qty (A,B,C)	ac (A,B,C)	VM cash (A,B,C)
Listing	u_F registered; stage REGISTERED, no mark; no rows	(0, 0, 0)	(-, -, -)	-
T1	A buys 10 from B @100	(10, -10, 0)	(-50000, +50000, 0)	-
Settle d1	$S_1 = 102$; shared price $\leftarrow 102$	(10, -10, 0)	(-51000, +51000, 0)	(+1000, -1000, 0)
T2	C buys 4 from A @103	(6, -10, 4)	(-30400, +51000, -20600)	-
Settle d2	$S_2 = 101$; shared price $\leftarrow 101$	(6, -10, 4)	(-30300, +50500, -20200)	(-100, +500, -400)
T3	B buys 4 from C @101 (C $\rightarrow$ flat)	(6, -6, 0)	(-30300, +30300, 0)	-
Expiry	stage $\rightarrow$ EXPIRED; final $S = 105$	(6, -6, 0)	(-31500, +31500, 0)	(+1200, -1200, 0)
Close	units returned to CH; positions $\rightarrow 0$	(0, 0, 0)	(0, 0, 0)	-

The subsections below walk this table in event order. Each event is one `FutStateDelta` showing the three conservation sums $\sum \Delta \text{net_qty}$, $\sum \Delta \text{ac}$, $\sum \text{VM}$. The example is the argument; the prose names the mechanism.

8.3 Listing

Registration writes `ProductTerms[uF]` (the six immutable terms) and `UnitStatus[uF]` together, establishing “ $u \in \text{terms} \iff u \in \text{status}$ ” by construction. The status initialises to stage REGISTERED with no settlement mark. No `PositionState` row exists: `futPosition(w, uF) = None` for every w .

REGISTERED denotes a unit recorded in and known to the ledger before any settlement, not admission to an exchange; an OTC option or an internal strategy reaches it identically. The two unreachable status states — a never-traded unit carrying a price, an expired unit lacking a final mark — are made unspellable by fusing the mark onto the stage:

`FutStage = FutRegistered | FutActive(Maybe Settlement) | FutExpired Settlement.`

`FutRegistered` carries no mark; `FutExpired` always carries one. `last_settlement_price` and `last_settlement_date` are projections of the `Settlement` carried by the stage, not independent fields. Conservation holds vacuously: no holders, so every sum is the empty sum, zero (C9). Re-registering an existing unit id is a hard error, never a silent reset (C10).

The shared status type and its registration follow.

```
-- FutTerms: the six immutable contract terms, one per unit, co-registered with
-- FutStatus. ptMultiplier drives mark value and the settlement fan-out. The Fut*
-- prefix marks this engine as a futures-boundary construction, distinct from the
-- core types: it PROJECTS onto the core Transaction (see "The atomic delta").
```

```
data FutTerms = FutTerms
  { ptMultiplier    :: !Integer
  , ptCurrency      :: !String
  , ptExpiry        :: !Day
  , ptClearinghouse :: !String
  , ptExchange      :: !String
  , ptProductId     :: !String
  } deriving (Eq, Show)
```

```
-- The mark is FUSED onto the stage, so the two unreachable states
```

```

-- (FutRegistered, Just p) and (FutExpired, Nothing) are unspellable.
-- FutRegistered carries no mark; FutExpired always carries one; FutActive may or
  may not.
data Settlement = Settlement { settlePrice :: !Price, settleDate :: !Day }
  deriving (Eq, Show)

data FutStage
  = FutRegistered          -- recorded, never traded: no holders, no mark
  | FutActive (Maybe Settlement) -- trading; Nothing = not yet settled, Just s =
    marked
  | FutExpired Settlement   -- final mark ALWAYS present; absorbing
  deriving (Eq, Show)

newtype FutStatus = FutStatus { fusStage :: FutStage } deriving (Eq, Show)

-- last_settlement_price/_date are PROJECTIONS of the mark the stage carries,
-- total over all three stages -- not independent fields.
settlement :: FutStage -> Maybe Settlement
settlement FutRegistered = Nothing
settlement (FutActive m) = m
settlement (FutExpired s) = Just s

settlementPrice :: FutStatus -> Maybe Price
settlementPrice = fmap settlePrice . settlement . fusStage
settlementDate  :: FutStatus -> Maybe Day
settlementDate  = fmap settleDate . settlement . fusStage

-- The coarse rank REGISTERED < ACTIVE < EXPIRED: the one fact futApplyDelta needs
-- to reject a regressing stage. It never regresses.
stageRank :: FutStage -> Int
stageRank FutRegistered = 0
stageRank (FutActive _) = 1
stageRank (FutExpired _) = 2

-- Rank alone cannot separate FutExpired 105 from FutExpired 110, so a strict new<
  cur
-- guard would re-admit a second Expire. isExpired is the precise absorbing test.
isExpired :: FutStage -> Bool
isExpired (FutExpired _) = True
isExpired _              = False

futRegisteredStatus :: FutStatus
futRegisteredStatus = FutStatus FutRegistered

```

Evaluations. `settlementPrice (FutStatus FutRegistered) = Nothing`; `settlementPrice (FutStatus (FutExpired (Settlement (Price 105) (Day 3)))) = Just (Price 105)`.
 The states `(FutRegistered, Just p)` and `(FutExpired, Nothing)` cannot be written.

8.4 First trade

A buys 10 contracts from B at 100. The trade emits one `FutStateDelta`:

- **Shared.** REGISTERED $\rightarrow$ ACTIVE with no mark (`FutActive Nothing`); the first trade activates the unit. Any existing mark is preserved — a trade never wipes the shared price.
- **Per-position.** The contract quantity moves from seller to buyer: $\Delta_{\text{net_qty}}(A) = +10$,

$\Delta\text{net_qty}(B) = -10$. Each leg sets $\text{ac} += -\Delta \cdot p \cdot m$:

$$\Delta\text{ac}(A) = -(+10) \cdot 100 \cdot 50 = -50000, \quad \Delta\text{ac}(B) = -(-10) \cdot 100 \cdot 50 = +50000.$$

- **Cash.** None: entering a futures position pays no notional.

Conservation: $\sum \Delta\text{net_qty} = 0$, $\sum \Delta\text{ac} = 0$, $\sum \text{VM} = 0$. State after T1: $\text{net_qty} = (10, -10, 0)$, $\text{ac} = (-50000, +50000, 0)$. A and B cross from None to Some on first touch; C is still None.

Positive quantity. A trade leg carries a strictly positive quantity $q > 0$, parsed at the boundary (`mkPosQty`); $q \leq 0$ is rejected (`NonPositiveQty`) and never recorded. $q = 0$ would promote `REGISTERED`→`ACTIVE` and create Some-flat rows for two never-held wallets, collapsing the never-held/held-flat distinction (§8.8); $q < 0$ would swap buyer and seller, violating the move primitive’s positivity. The sign of the position change is supplied by the leg — buyer $+q$, seller $-q$ — not by q .

The per-event delta, its conserved image, and the conservation witness follow.

```
-- FutPos per (wallet, unit). ac is per (w,u): two wallets holding the same
-- contract carry different ac -- the field that makes intraday margin correct.
data FutPos = FutPos { fpNetQty :: !Qty, fpAccumCost :: !Cash }
  deriving (Eq, Show)

futZeroP :: FutPos
  -- flat row created on first touch; held-and-flat
  baseline
futZeroP = FutPos mempty mempty

-- Conservation is a monoid identity. Totalling a delta is a monoid homomorphism
-- -- a map that respects (<>) and mempty -- into Conserved; "conserving" means
  the
-- image is mempty. The zero-holder case is the empty fold, mempty (C9); we SUM
-- deltas, never apportion, so no divide-by-holder-count bug can arise.
data Conserved = Conserved !Qty !Cash !Cash deriving (Eq, Show) -- (sum net, sum
  ac, sum cash)
instance Semigroup Conserved where
  Conserved a b c <> Conserved a' b' c' = Conserved (a<>a') (b<>b') (c<>c')
instance Monoid Conserved where mempty = Conserved mempty mempty mempty

data FutRowDelta = FutRowDelta { rdNetQty :: !Qty, rdAc :: !Cash } deriving (Eq,
  Show)

-- One event's proposed change: at most one shared FutStage write, per-holder
  additive
-- row deltas, per-holder cash legs. Applied in full or not at all (C3).
data FutStateDelta = FutStateDelta
  { fsdUnit :: UnitId, fsdStage :: Maybe FutStage
  , fsdRows :: Map WalletId FutRowDelta, fsdCash :: Map WalletId Cash } deriving (
  Show)

futNetDelta :: FutStateDelta -> Conserved
futNetDelta sd =
  foldMap (\(FutRowDelta nq ac) -> Conserved nq ac mempty) (fsdRows sd)
  <> foldMap (\c -> Conserved mempty mempty c) (fsdCash sd)

-- The C2 witness, ABSTRACT: futValidate is the only constructor, so an
  unconserved
-- delta cannot reach futApplyDelta. This is the futures engine's OWN conservation
```

```

-- discharge, distinct from the core (which has no validate gate); the validated
-- delta projects onto one conserving core Transaction. Conservation is a value-
  level
-- fact (a sum of exact integers), so this is a parse boundary, not a free type
  fact.
newtype FutValidDelta = FutValidDelta FutStateDelta

futValidate :: FutStateDelta -> Either FutLedgerError FutValidDelta
futValidate sd | net == mempty = Right (FutValidDelta sd)
               | otherwise     = Left (FutNotConserved (fsdUnit sd) net)
  where net = futNetDelta sd

-- A trade activates without wiping any existing mark, and sets each leg's
-- ac += -(delta signed_qty) * p * m (buyer +q, seller -q). No cash: entry pays no
  notional.
activateTrade :: FutStage -> FutStage
activateTrade cur = FutActive (settlement cur)

tradeDelta :: UnitId -> FutStage -> Integer
              -> WalletId -> WalletId -> PosQty -> Price -> FutStateDelta
tradeDelta u cur m buyer seller pq p = FutStateDelta
  { fsdUnit = u, fsdStage = Just (activateTrade cur)
  , fsdRows = Map.fromList
    [ (buyer , FutRowDelta q      (cashNeg (futMark q p m)))
    , (seller, FutRowDelta (qneg q) (futMark q p m)) ]
  , fsdCash = Map.empty }
  where q = unPosQty pq

```

8.5 Daily variation-margin settlement: the mechanism and the clean case (day 1)

Settlement at price S , multiplier m , over the current holders. For each holder the reset target is $\text{target} = -\text{net_qty} \cdot S \cdot m$, the row delta is $\Delta\text{ac} = \text{target} - \text{ac}$, and the cash leg is

$$\text{VM}(w) = -\Delta\text{ac}(w) = \text{net_qty}(w) \cdot S \cdot m + \text{ac}(w)$$

The cash leg is the mirror of the change in accumulated cost. Therefore $\sum_w \text{VM} = -\sum_w \Delta\text{ac}$, and variation-margin zero-sum is the *same* fact as ac conservation, not a separate reconciliation. The reset discards the entry prices: with $\text{ac} \leftarrow -\text{net_qty} \cdot S \cdot m$, the next day's margin needs only S and the position — each daily settlement economically closes and reopens the position at the new price (the resettable-forward property).

Settlement is one atomic `FutStateDelta` touching both layers:

- **Shared**, one write: `UnitStatus[uF]` acquires the mark (S, d) . The *coarse rank* `REGISTERED < ACTIVE < EXPIRED` is unchanged — it stays `ACTIVE`; the *embedded mark* updates, the stage becoming `FutActive (Just (Settlement S d))`. The settlement price enters only as the logged `FSettleVM` event; `futReplay` rebuilds the mark, and the fold stays pure at the boundary — the shared value is a projection of the log, never an off-log write.
- **Per-position**, a fan-out over the current holders: each row's ac is reset to target, and each holder gets the cash leg $\text{VM}(w)$.

The cash legs are emitted over the holders; the clearinghouse leg is the residual that balances them. Since the holder legs already sum to zero, CH's leg is zero throughout this example and no CH cash row materialises. This is the special case of a *closed* holder set $\{A, B, C\}$, where

$\sum_w \text{net_qty}(w) = 0$ at every step. In the Ledger's intended use — an internal firm system of record facing an external clearinghouse — the firm is generally *not* flat: when $\sum_w \text{net_qty}(w) \neq 0$ over the firm's holders, the CH leg is the firm's nonzero net variation margin, the boundary figure reconciled against the clearing broker's daily statement.

Day 1: $S_1 = 102$. Holders A and B; C has no row and is untouched — the contract settles vacuously over C, the shared price still updating (C9). With pre-settle $\text{ac} = (-50000, +50000)$ and $\text{net_qty} = (10, -10)$:

$$\text{VM}(A) = 10 \cdot 102 \cdot 50 - 50000 = +1000, \quad \text{VM}(B) = -10 \cdot 102 \cdot 50 + 50000 = -1000.$$

Reset: $\text{ac}(A) \leftarrow -51000$, $\text{ac}(B) \leftarrow +51000$. Conservation: $\sum \Delta \text{net_qty} = 0$, $\sum \Delta \text{ac} = 0$, $\sum \text{VM} = 0$. State: $\text{net_qty} = (10, -10, 0)$, $\text{ac} = (-51000, +51000, 0)$.

The fan-out, shared by settlement and expiry, is one pure function of the current holders.

```
-- Shared by FSettleVM and FExpire. For each holder at price S, multiplier m:
--   delta ac = (-net*S*m) - ac      VM = -(delta ac) = net*S*m + ac
-- so VM = -(delta ac) exactly: cash conservation IS ac conservation, not a
--   separate
-- reconciliation. A flat holder gives delta ac = 0, VM = 0; no holders -> empty
--   fan-out.
settlementFanout :: Price -> Integer -> [(WalletId, FutPos)]
                -> (Map WalletId FutRowDelta, Map WalletId Cash)
settlementFanout s m holders =
  ( Map.fromList [ (w, FutRowDelta mempty (deltaAc ps)) | (w, ps) <- holders ]
  , Map.fromList [ (w, cashNeg (deltaAc ps))               | (w, ps) <- holders ] )
  where deltaAc ps = cashNeg (futMark (fpNetQty ps) s m) <> cashNeg (fpAccumCost ps)
```

8.6 A subsequent, intraday trade (day 2)

C buys 4 contracts from A at 103, after the day-1 settle and before the day-2 settle — A trades intraday. One `FutStateDelta`, stage already `FutActive` (mark preserved):

$$\Delta \text{net_qty}(C) = +4, \quad \Delta \text{net_qty}(A) = -4; \quad \Delta \text{ac}(C) = -20600, \quad \Delta \text{ac}(A) = +20600.$$

Conservation: $\sum \Delta \text{net_qty} = 0$, $\sum \Delta \text{ac} = 0$, $\sum \text{VM} = 0$. State after T2: $\text{net_qty} = (6, -10, 4)$, $\text{ac} = (-30400, +51000, -20600)$. C crosses from None to Some on first touch.

8.7 Settlement under intraday trading (day 2): the anchor

The day-2 settle at $S_2 = 101$ runs the same fan-out over holders A, B, C:

$$\begin{aligned} \text{VM}(A) &= 6 \cdot 101 \cdot 50 - 30400 = -100, \\ \text{VM}(B) &= -10 \cdot 101 \cdot 50 + 51000 = +500, \\ \text{VM}(C) &= 4 \cdot 101 \cdot 50 - 20600 = -400. \end{aligned}$$

Conservation: $\sum \Delta \text{net_qty} = 0$, $\sum \text{VM} = -100 + 500 - 400 = 0$. Reset $\text{ac} \leftarrow (-30300, +50500, -20200)$ with $\sum \Delta \text{ac} = 0$. State: $\text{net_qty} = (6, -10, 4)$, $\text{ac} = (-30300, +50500, -20200)$.

Principle 8.3 (The intraday-margin subtlety forces stored per-position ac (C11)). *A's day-2 variation margin is -100 , not the naive $\text{net_qty} \cdot (S_2 - S_1) \cdot m = 6 \cdot (101 - 102) \cdot 50 = -300$. A sold 4 contracts at 103 intraday, one point above the prior mark 102, gaining $4 \cdot (+1) \cdot 50 = +200$, which offsets the -300 mark loss on the held position. Only the stored per-wallet ac , which*

absorbed the intraday trade, yields the correct -100 . A price-derived model gives the wrong cash. Accumulated cost is therefore per-position stored state with a single canonical writer (C11), not a consequence read off the shared price.

The settlement answers, stated once:

1. *Settlement is a state update split by layer.* The shared part — `last_settlement_price`, `last_settlement_date` — is one write on `UnitStatus[uF]`, one value per contract. The per-position part — the `ac` reset and the cash leg — is a fan-out over the current holders on `PositionState[w, uF]`. One atomic `FutStateDelta` touches both.
2. *One atomic event that fans out, not a derived consequence of the price.* The cash leg is real daily money moving between longs and shorts through CH; every cash move is a recorded, conservation-bearing event, so a per-holder pass is unavoidable. The `ac` reset rides in the same atomic delta. The fan-out is forced by the cash, not chosen for the bookkeeping.
3. *Price only in shared state, consequence only in per-wallet state.* The price is shared; its consequence — the `ac` reset and the variation-margin cash — lives only in `PositionState[w, uF]` and the move stream.

Remark 8.4 (Direction reversal needs no special case). *The accumulated-cost rule handles a long-to-short reversal in one trade with no branch. From $\text{net_qty} = +5$, $\text{ac} = -500$ (entered at 100, $m = 1$), a single sell of 8 at 105 sets $\Delta\text{ac} = -(-8) \cdot 105 \cdot 1 = +840$, so $\text{ac} = -500 + 840 = +340$ and $\text{net_qty} = -3$. The economic value at 105 is $340 + (-3) \cdot 105 = 25$, exactly the gain from closing the long 5 at \$5 per contract; the new short 3 marks forward from here with no extra bookkeeping. The same arithmetic that accumulates a position unwinds and reverses it.*

8.8 Closing a position to flat

B buys 4 contracts from C at 101; C goes flat. One `FutStateDelta`:

$$\Delta\text{net_qty}(B) = +4, \Delta\text{net_qty}(C) = -4; \quad \Delta\text{ac}(B) = -20200, \Delta\text{ac}(C) = +20200.$$

Conservation: $\sum \Delta\text{net_qty} = 0$, $\sum \Delta\text{ac} = 0$, $\sum \text{VM} = 0$. State: $\text{net_qty} = (6, -6, 0)$, $\text{ac} = (-30300, +30300, 0)$. C is now flat. Its row is *retained*, Some at $(0, 0)$ — not deleted, not None. The two readings are distinct and both load-bearing: None means never held; Some-flat means held and now flat, read after close-out for tax reporting, wash-sale lookback, and trade reconstruction (C1(a)). The carrier is monotone: once a row is created it is never removed (C1(b)); no row deleter exists, because the ledger is abstract.

8.9 Expiry, final settlement, and the Close

Expiry is the settlement fan-out of §8.5 composed with a stage write to `EXPIRED`; like a settle, it acts only on an `ACTIVE` unit, never a never-traded one (§16). The final mark $S = 105$ fans out over the remaining holders A and B (C is flat, its row touched to no effect):

$$\text{VM}(A) = 6 \cdot 105 \cdot 50 - 30300 = +1200, \quad \text{VM}(B) = -6 \cdot 105 \cdot 50 + 30300 = -1200.$$

Conservation: $\sum \Delta\text{net_qty} = 0$, $\sum \text{VM} = 0$; $\text{ac} \leftarrow (-31500, +31500, 0)$ with $\sum \Delta\text{ac} = 0$. The coarse rank advances to `EXPIRED`, the stage becoming `FutExpired` (`Settlement 105 (Day 3)`); the final mark is always present.

Cash settlement and the Close. No unit is delivered; the economic result is the cumulative variation margin already moved. The *Close* step extinguishes every holder’s position against the clearinghouse, flattening `net_qty` and `ac` by an additive negation of each row, with no further cash. A held +6 and B held −6; closing them returns the contracts to CH:

$$\Delta\text{net_qty}(A) = -6, \Delta\text{net_qty}(B) = +6; \quad \Delta\text{ac}(A) = +31500, \Delta\text{ac}(B) = -31500.$$

The clearinghouse legs are the residual of the holder legs — zero, since the holder legs balance — so no CH row materialises. Conservation: $\sum \Delta\text{net_qty} = 0$, $\sum \Delta\text{ac} = 0$, $\sum \text{VM} = 0$. State after Close: `net_qty` = (0, 0, 0), `ac` = (0, 0, 0); rows retained at zero (monotone carrier). Close carries no stage write, so it is the one event still admissible on an **EXPIRED** unit.

Physical settlement. The variant differs only at the Close. Instead of flattening against CH for cash, the contract converts to delivery: the deliverable unit and the cash payment move together as one atomic transaction (delivery versus payment; the settlement interface is §12), the future’s `net_qty` goes to zero, and the holder’s wallet gains the deliverable. The cash leg is struck at the *final settlement price* S , not the trade price: expiry has already marked every position to S through variation margin, so delivery exchanges the deliverable for `net_qty` · S · m and moves no further mark-to-market — pricing the delivery at any other figure would double-count the variation margin already settled. Per-unit conservation holds on each unit separately — the future, the deliverable, the cash — and the cross-unit delivery is one all-or-nothing transaction. The worked example is cash-settled; the figures above are unchanged by this variant.

Theorem 8.5 (Closing identity: cash ties to economic PnL). *Cumulative variation margin per wallet equals the wallet’s economic profit and loss. Here $A = +1000 - 100 + 1200 = +2100$, $B = -1000 + 500 - 1200 = -1700$, $C = -400$, $CH = 0$, summing to 0; and $A = 4 \cdot (103 - 100) \cdot 50 + 6 \cdot (105 - 100) \cdot 50 = +2100$, $B = 4 \cdot (100 - 101) \cdot 50 + 6 \cdot (100 - 105) \cdot 50 = -1700$, $C = 4 \cdot (101 - 103) \cdot 50 = -400$.*

The terminal flatten delta has no stage write and no cash leg.

```
-- Terminal flatten: NO stage write (stays EXPIRED, so it does not breach the
-- absorbing rule), NO cash; every row driven to zero by additive negation. It
-- conserves because net and ac each already sum to zero, so their negations do
-- too.
closeDelta :: UnitId -> [(WalletId, FutPos)] -> FutStateDelta
closeDelta u holders = FutStateDelta
  { fsdUnit = u, fsdStage = Nothing
  , fsdRows = Map.fromList
    [ (w, FutRowDelta (qneg (fpNetQty ps)) (cashNeg (fpAccumCost ps)))
    | (w, ps) <- holders ]
  , fsdCash = Map.empty }
```

8.10 After expiry: state versus derivation

After the Close, every row is retained at zero: $\text{PositionState}[w, u_F] = \text{Some flat}$ for A, B, C. $\text{UnitStatus}[u_F]$ holds stage **EXPIRED** and final mark 105. $\text{ProductTerms}[u_F]$ is unchanged — immutable.

`first_touch_date` is *not* stored: it is derived from the event log on demand. Caching it in PositionState would make a replay disagree with itself under a back-dated correction — the cached value reflecting the pre-correction order while the fold reflects the corrected one. The rule is general: what the fold over the log determines is derived, not stored; only what the fold cannot reconstruct from prior events is state.

8.11 Invariants threaded through the life

Invariant 8.6 (Per-event conservation, C2). *Every `FutStateDelta` satisfies $\sum_w \Delta \text{net_qty} = 0$, $\sum_w \Delta \text{ac} = 0$, $\sum_w \text{VM} = 0$, discharged by `futValidate` before application. Induction over the stream lifts the per-event identity to the global invariant (P1). The vacuous case — a unit with no holders — is the empty sum, zero (C9); conservation sums deltas and never divides by a holder count.*

Invariant 8.7 (Deterministic replay as a fold, C1(b), P8). *`futReplay` is a Kleisli fold (an error-stopping fold whose combining step halts at the first failure) of the event stream over the ledger: $\text{futReplay}(xs \triangleleft ys) = \text{futReplay}xs \gg \text{futReplay}ys$ — replaying a concatenation equals composing the replays. Checkpoint independence is a consequence of this law, not a test; the monotone carrier keeps the key set stable across cuts. The stream is assumed unique — de-duplicated at the ingestion boundary by an event key — because the two event kinds differ in idempotency: a duplicated `FSettleVM` at a fixed mark is inert ($\Delta \text{ac} = 0$, $\text{VM} = 0$), whereas a duplicated `FTrade` accumulates a second position delta. Duplicate suppression is a boundary obligation, not a ledger function.*

Invariant 8.8 (Monotone, absorbing stage). *The coarse rank `REGISTERED` < `ACTIVE` < `EXPIRED` never regresses; a proposed stage of strictly lower rank is rejected (`StageRegression`). Settle and expire act only on an `ACTIVE` unit; both reject a `REGISTERED` (never-traded) unit (`NotActive`), which has no holders and no mark slot — so a never-traded unit is never silently promoted, and `EXPIRED` is reachable only from `ACTIVE`: the chain is `REGISTERED` → `ACTIVE` → `EXPIRED` with no skips. `EXPIRED` is absorbing: the rank guard alone is too weak — a second expiry has equal rank, and $2 < 2$ is false — so an explicit absorbing test rejects every stage-writing delta on an expired unit; `FClose`, which carries no stage write, is the one event still admissible. Re-settling at the same price before expiry is a no-op: with $\text{ac} = -\text{net_qty} \cdot S \cdot m$ already, $\Delta \text{ac} = 0$ and $\text{VM} = 0$.*

The event dispatcher, the abstract ledger, and the replay fold realise these invariants: the stage and positivity guards live once in `futHandle`; the monotone carrier and absorbing rule are the absence of any deleter and the `isExpired` test in `futApplyDelta`.

```
-- The four event classes. FTrade carries a RAW Qty (the unparsed input of the
  free
-- monoid of events); futHandle parses it once into PosQty. FClose is the one
  event
-- with no stage write, so it alone is admissible on an EXPIRED unit.
data FutEvent
  = FTrade    UnitId WalletId WalletId Qty Price    -- buyer, seller, qty, price
  | FSettleVM UnitId Price Day                      -- settlement price, date
  | FExpire   UnitId Price Day                      -- final price, date
  | FClose    UnitId                                     -- terminal flatten to zero
  deriving (Eq, Show)

futEventUnit :: FutEvent -> UnitId
futEventUnit (FTrade u _ _ _) = u
futEventUnit (FSettleVM u _ _) = u
futEventUnit (FExpire u _ _)   = u
futEventUnit (FClose u)        = u

-- The dispatcher: a pure function of (event, current ledger), hence deterministic
.
-- The stage/positivity guards live here once: self-trade (G3); EXPIRED absorbing
-- (G2); positivity parsed (G5); settle/expire reject REGISTERED, never silently
-- promoting a never-traded unit (G4); close only after expiry (G1).
futHandle :: FutEvent -> FutLedger -> Either FutLedgerError FutStateDelta
```



```

futHandle ev l = case Map.lookup u (flUnits l) of
  Nothing      -> Left (FutUnknownUnit u)
  Just (terms, us) ->
    let m = ptMultiplier terms; cur = fusStage us in case ev of
      FTrade _ buyer seller q p
        | buyer == seller -> Left (SelfTrade u buyer)           -- G3
        | isExpired cur   -> Left (UnitExpired u)               -- G2
        | otherwise       -> case mkPosQty q of                  -- G5: parse once
          Nothing -> Left (NonPositiveQty u q)
          Just pq  -> Right (tradeDelta u cur m buyer seller pq p)
      FSettleVM _ s d -> case cur of
        FutExpired _ -> Left (UnitExpired u)                   -- G2
        FutRegistered -> Left (NotActive u)                     -- G4
        FutActive _   -> let (rows, cash) = settlementFanout s m (futHoldersOf u l)
      )
        in Right (FutStateDelta u (Just (FutActive (Just (
Settlement s d)))) rows cash)
      FExpire _ s d -> case cur of
        FutExpired _ -> Left (UnitExpired u)                   -- G2
        FutRegistered -> Left (NotActive u)                     -- G4 (symmetric)
      )
        FutActive _   -> let (rows, cash) = settlementFanout s m (futHoldersOf u l)
      )
        in Right (FutStateDelta u (Just (FutExpired (Settlement s
d)))) rows cash)
      FClose _ -> case cur of
        FutExpired _ -> Right (closeDelta u (futHoldersOf u l)) -- G1
        _             -> Left (NotExpired u)
    where u = futEventUnit ev

-- The ledger, ABSTRACT: no field setter or row/status/cash deleter is exported,
-- so
-- futApplyDelta (insert/update only) is the sole mutator and that absence IS the
-- monotone-carrier discipline (C1). Terms and status are FUSED into one map, so
-- the
-- desynchronised state (one map has u, the other does not) is unrepresentable.
data FutLedger = FutLedger
  { flUnits :: Map UnitId (FutTerms, FutStatus)
  , flPos   :: Map (WalletId, UnitId) FutPos
  , flCash  :: Map WalletId Cash }

data FutLedgerError
  = FutReRegistration UnitId | FutUnknownUnit UnitId
  | StageRegression UnitId FutStage FutStage | FutNotConserved UnitId Conserved
  | UnitExpired UnitId | NotActive UnitId | NotExpired UnitId
  | SelfTrade UnitId WalletId | NonPositiveQty UnitId Qty
  deriving (Eq, Show)

futEmptyLedger :: FutLedger
futEmptyLedger = FutLedger Map.empty Map.empty Map.empty

-- Co-registers terms AND status as one tuple. Re-registration is a hard error (
C10).
futRegister :: UnitId -> FutTerms -> FutLedger -> Either FutLedgerError FutLedger
futRegister u terms l
  | u `Map.member` flUnits l = Left (FutReRegistration u)
  | otherwise = Right l { flUnits = Map.insert u (terms, futRegisteredStatus) (
flUnits l) }

```

```

-- Apply a validated delta: total, atomic, monotone. EXPIRED rejects ANY stage
  write
-- (absorbing); a bare new<=cur would wrongly reject the ACTIVE->ACTIVE daily
-- re-settle, so the precise rule is isExpired plus a rank-regression guard.
  FClose's
-- fsdStage = Nothing bypasses both. Rows and cash are inserted/updated, never
  removed.
futApplyDelta :: FutValidDelta -> FutLedger -> Either FutLedgerError FutLedger
futApplyDelta (FutValidDelta sd) l
  | not (u `Map.member` flUnits l) = Left (FutUnknownUnit u)
  | otherwise = case fsdStage sd of
    Nothing -> Right written
    Just new
      | isExpired cur -> Left (UnitExpired u)
      | stageRank new < stageRank cur -> Left (StageRegression u cur new)
      | otherwise -> Right written
        { flUnits = Map.adjust (\(t,_) -> (t, FutStatus new)) u (flUnits l) }
where
  u = fsdUnit sd
  cur = maybe FutRegistered (fusStage . snd) (Map.lookup u (flUnits l))
  written = l
    { flPos = Map.foldrWithKey applyRow (flPos l) (fsdRows sd)
    , flCash = Map.foldrWithKey (\w c -> Map.insertWith (<)> w c)
      (flCash l) (fsdCash sd) }
  applyRow w (FutRowDelta dnq dac) mp =
    let key = (w, u); p0 = Map.findWithDefault futZeroP key mp -- first touch
    -> flat
      in Map.insert key p0 { fpNetQty = fpNetQty p0 <> dnq
        , fpAccumCost = fpAccumCost p0 <> dac } mp

-- One event end to end: read state -> discharge conservation -> apply. Each
  futStep
-- projects onto one core Transaction recorded through the core applyTx; futStep
  is
-- the futures engine's narration of that projection, not a second core door.
futStep :: FutEvent -> FutLedger -> Either FutLedgerError FutLedger
futStep ev l = futHandle ev l >>= futValidate >>= \vd -> futApplyDelta vd l

-- futReplay is a Kleisli fold -- a fold whose step composes in the Either monad.
  Its
-- homomorphism law futReplay (xs <> ys) = futReplay xs >=> futReplay ys makes
-- checkpoint independence a CONSEQUENCE, not a test. The stream is assumed
-- de-duplicated at ingestion: a duplicate FSettleVM at a fixed mark is inert, but
  a
-- duplicate FTrade accumulates a second delta.
futReplay :: [FutEvent] -> FutLedger -> Either FutLedgerError FutLedger
futReplay evs l0 = foldM (flip futStep) l0 evs

-- Total accessors. futPosition is the C1 Option accessor: Nothing = never held,
-- Just futZeroP = held and now flat -- never collapsed.
futProductTerms :: FutLedger -> UnitId -> Maybe FutTerms
futProductTerms l u = fst <$> Map.lookup u (flUnits l)
futUnitStatus :: FutLedger -> UnitId -> Maybe FutStatus
futUnitStatus l u = snd <$> Map.lookup u (flUnits l)
futPosition :: FutLedger -> WalletId -> UnitId -> Maybe FutPos
futPosition l w u = Map.lookup (w, u) (flPos l)
futCashOf :: FutLedger -> WalletId -> Cash

```

```

futCashOf l w = Map.findWithDefault mempty w (flCash l)
futHoldersOf :: UnitId -> FutLedger -> [(WalletId, FutPos)]
futHoldersOf u l = [ (w, ps) | ((w, u'), ps) <- Map.toList (flPos l), u' == u ]

```

8.12 Escalations

Both items are carried to the consolidated register in §19.

E1 — Fan-out cost at scale. A settlement writes one *PositionState* row and emits one cash leg per open holder, every day. Across $\sim 10^6$ contracts each with up to thousands of holders, daily settlement is $O(\text{open positions})$ writes and cash legs — a large daily fan-out and write amplification. This is intrinsic to per-wallet variation margin. Mitigations (batching, snapshotting) trade against the conservation-per-event and single-writer disciplines and must be shown to preserve C2 and C11. The cost stands. *Owner: the performance/risk agent.*

E2 — The derived-consequence alternative, declined. Storing only the shared `last_settlement_price` and computing each wallet’s mark lazily saves nothing on the dominant cost — the cash leg is per-wallet and unavoidable (E1) — cannot produce the correct variation margin for an intraday trader without reconstructing `ac` anyway (§8.7), and would split the single-writer discipline for `ac` (C11). The fan-out is forced by the cash, not chosen for the bookkeeping.

Key Invariants and Consequences

Valuation and lifecycle processing are projections of state; the following hold once conservation is granted. Each is catalogued in §16.

- **State-sufficiency.** Portfolio value depends only on current state—wallet balances, unit state (including embedded lifecycle state), and the current price vector—not on transaction history. Valuation is the linear functional $V_t = \sum_u w_t(u) P_t(u)$. (§5.)
- **PnL path-independence (P10).** $\text{PnL} = V_{t_1} - V_{t_0}$, independent of the intermediate transactions. Dual valuation applies two price vectors to one state. (§5, Theorem 5.3.)
- **Lifecycle value invariance.** Across deterministic lifecycle events—coupons, dividends, resets, redemptions—total value under a given price vector is invariant: the instrument’s price jump is offset by the explicit cash move. Under optionality the invariance holds for intrinsic value only. (§1; §7.)
- **Contracts as move generators.** A smart contract is a deterministic program emitting sequences of moves; because every lifecycle effect is a move, conservation P1 is preserved with no separate validation. (§6.)
- **Lifecycle idempotency (P6).** Applying the same lifecycle event to the same unit state produces no additional moves. (§8, Invariant 16.2.)
- **Lifecycle purity (P9).** For fixed inputs, lifecycle functions produce identical outputs and no effect beyond the return, so valuation and time travel are deterministic. (§8.)

Part IV

Accounts, Reporting, Settlement, and CDM Integration

9 Managed Accounts, Virtual Portfolios, and TRS

A managed account is not a primitive. It is the composition of four that already exist (§2, §4): a wallet partition that holds positions and produces PnL; a mandate unit u_{MA} issued by the manager and held by the client, conserved by the issuance law; a deterministic reset contract that observes performance, crystallises one net cash flow, and resets its baseline; and the three-home state model of §4, which homes the mandate's terms, the shared observables, and the per-client economic state. A total return swap is the same composition with the wallet partition placed in a virtual ledger and the reset contract's settlement crossing into the real one. Both reduce to a single conservative cash move whose magnitude is a function of observed performance and a recorded baseline; the worked example (§9.11) carries one account through subscription, trade, NAV, crystallisation against a high-water mark, and redemption, with $\sum_w w(u) = 0$ at every step.

9.1 The mandate is a unit

A managed account is opened by issuing the mandate unit u_{MA} from the manager w_M to the client w_C , giving $w_M(u_{MA}) = -1$, $w_C(u_{MA}) = +1$, $\sum_w w(u_{MA}) = 0$. This is a real two-named-wallet issuance with a real issuer and holder, not a self-reflexive sentinel; conservation holds by the issuance law (P1).

A **Move** is one conserved transfer of a unit between two named wallets: from $- = q$, to $+ = q$. Conservation (P1) is by construction, not by check — the per-wallet deltas are group negatives, so they sum to the identity. The mandate issuance is the canonical instance.

```
data Move = Move
  { mFrom :: WalletId, mTo :: WalletId, mUnit :: UnitId, mQty :: Qty } -- q > 0

-- The per-wallet deltas a move induces; their sum is the conservation witness.
moveDelta :: Move -> [(WalletId, Qty)]
moveDelta (Move s d _ q) = [(s, qneg q), (d, q)] -- qneg q <> q = mempty :
  conserved

-- Mandate issuance: a real two-named-wallet transfer of  $u_{MA}$ , manager -> client,
-- q = 1. A real issuer and holder, not a self-reflexive sentinel.
issueMandate :: WalletId -> WalletId -> UnitId -> Move
issueMandate manager client uMA = Move manager client uMA (Qty 1)
```

Invariant 9.1 (Mandate is a singleton bilateral contract). *While the mandate is live,*

$$\text{support } (w \mapsto w(u_{MA})) = \{(w_M, -1), (w_C, +1)\}.$$

Conservation alone does not establish Invariant 9.1: P1 admits $(-5, +5)$ or a three-way split. Identity is pinned by four existing primitives together: Invariant 9.1 for the active phase; the **Option** accessor distinguishing **None** (pre-issuance) from **Some(zero)** (terminated, retained); a **UnitStatus** lifecycle guard admitting issuance only from an unissued status, so a terminated u_{MA} cannot be resurrected; and C8 routing a re-mandate to a fresh u'_{MA} stamped **SetSupersededBy**.

Principle 9.2 (Mandate is non-valued). *u_{MA} is excluded from the domain of valuation: $P_t(u_{MA})$ is undefined, not zero. A price on u_{MA} would add $+1 \cdot P_t(u_{MA})$ to the client's*

per-wallet NAV and double-count the underlying exposure; the term cancels only in the global sum.

What is recorded where.

Datum	Home
Mandate text, fee schedule, benchmark identity, position limits, HWM/hurdle methodology, crystallisation frequency	<i>ProductTerms</i> [u_{MA}] (immutable, versioned, append-only)
Lifecycle stage, superseded_by	<i>UnitStatus</i> [u_{MA}] (shared)
HWM value, entry NAV, accrued fees, mandate-breach flags, subscription/redemption cursor, benchmark NAV at this wallet’s inception	<i>PositionState</i> [w_C, u_{MA}]
Current benchmark level (externally sourced, captured as a logged observation event)	<i>UnitStatus</i> [u_{bench}]

The benchmark level is a shared cached observable, governed by the same discipline as every *UnitStatus* value (§4): *UnitStatus* holds one value per unit, shared—read identically by every holder. Its value changes over time, but *UnitStatus* is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.

The per-client economic scalars are homed in *PositionState*, yet each is a fold of the event log: whether they are stored or re-derived is escalation E1 (§19).

9.2 Subscription

Capital enters as a cash move from the client’s external funding wallet w_{ext} , a virtual wallet at the ledger boundary, into w_C , tagged **SUBSCRIPTION** and indexed by the subscription/redemption cursor in *PositionState*[w_C, u_{MA}]. The tag is load-bearing: it lets the net external flow over any interval be reconstructed as a projection over the stream (§9.5). At subscription the entry NAV and the initial high-water mark are set to the subscribed capital base; the high-water mark before subscription is **None** (unrepresentable), never 0 — a 0 mark would charge a performance fee on the entire contributed capital at the first crystallisation. The benchmark NAV at inception is recorded from *UnitStatus*[u_{bench}] for performance reporting.

9.3 Trading under mandate

A trade is an ordinary two-legged transaction against a virtual market wallet w_{mkt} , admitted only if it satisfies the mandate guards.

Definition 9.3 (Mandate guard). *A quantitative mandate constraint is a precondition g on the move-generation function. The contract evaluates g against the current state and the recorded price snapshot; if g fails, no moves are emitted and the state is unchanged.*

Guard determinism is price-relative. “Same wallet state + same trade $\Rightarrow$ same decision” is false for value-based limits — leverage, concentration-by-value, currency caps — because holding positions fixed and doubling a price flips the same trade from admit to reject. Determinism is recovered only when the guard is a pure function of (*ProductTerms*, state, trade, P_t) with P_t an explicit, snapshot-pinned input. Two totality obligations follow. A missing price ($P_t(u) = \text{None}$) makes the guard fail closed, a typed reject, never a silent admit. A passive breach — a limit

crossed by price appreciation with no admitted trade — lies outside the domain of a move-time guard and is detected only by a periodic valuation sweep. The pinned-snapshot requirement is escalation E2 (§19).

9.4 Valuation and NAV

The account NAV is the wallet valuation with the mandate excluded (Principle 9.2):

$$\text{NAV}_t = \sum_{u \neq u_{\text{MA}}} w_{C,t}(u) P_t(u).$$

By state-sufficiency, NAV_t depends only on current balances, current unit state, and current prices, not on history. Performance against the benchmark is computable from the inception benchmark NAV and the current level in $\text{UnitStatus}[u_{\text{bench}}]$; it feeds reporting, and feeds the fee only when the mandate’s methodology names a benchmark hurdle.

9.5 Fee logic

Performance is net of capital flows. Over a reset interval $[t_{k-1}, t_k]$ the performance is the gross NAV change less the net external flow:

$$\text{Perf}_k = (\text{NAV}_{t_k} - \text{NAV}_{t_{k-1}}) - \text{NetFlow}_{[t_{k-1}, t_k]}, \quad \text{NetFlow} = \sum_{\substack{m \text{ tagged sub/redemption} \\ t_{k-1} < m.t \leq t_k}} \pm m.q.$$

The gross change conflates performance with capital flows: by the PnL decomposition a mid-period subscription is a flow, and crystallising it as performance would charge a fee on the client’s own contributed capital. NetFlow is a projection over the already-tagged cursor moves (§9.2); the correction adds no new coordinate.

Fee is not PnL settlement. A single whole-Perf $\rightarrow$ beneficiary move is the internal-desk case, where a book is swept to Treasury. A client managed account pays the manager only a fee fraction; the gain stays in the client book. The fee handler is total over sign($\text{NAV} - \text{HWM}$):

$$\begin{aligned} \text{management fee } f_k^m &= r_m \cdot \Delta t_k \cdot \text{NAV}_k^{\text{base}} && (\text{accrues on AUM regardless of sign}), \\ \text{performance fee } f_k^p &= r_p \cdot \max(0, \text{NAV}_k^{\text{net}} - \text{HWM}_{k-1}) && (\text{floored at zero; no clawback}), \\ \text{HWM}_k &= \max(\text{HWM}_{k-1}, \text{NAV}_k^{\text{net}} - f_k^p) && (\text{monotone ratchet; C11 writer } \text{fee_crystallise}), \end{aligned}$$

where $\text{NAV}_k^{\text{net}} = \text{NAV}_{t_k} - f_k^m$. On a loss period the performance fee is 0 and the high-water mark is unchanged, while the management fee still accrues.

Crystallisation is double-entry. A fee is realised as a real cash move $w_C \rightarrow w_M$, so it is conserved and carries its own contra-entry; a non-conserved stored scalar would be single-entry and sit outside P1. The handler emits both legs atomically (C3): a management-fee move and a performance-fee move, $w_C \rightarrow w_M$ in USD.

A signed amount x crystallises into *at most one* conserved cash move: magnitude $|x|$, direction by sign(x), no move at $x = 0$. Totality is the **Maybe** — the $x = 0$ case is **Nothing**, never an illegal $q = 0$ move. The fee handler emits two such moves $w_C \rightarrow w_M$ atomically (C3) and ratchets the high-water mark through the one canonical writer `WHwm :: FieldWrite 'FeeCrystallise` (reused verbatim from the three-home model, §4): the phantom index — meaning the write names its handler in its type — makes a settle or trade handler bumping the HWM a compile error, so the mark has exactly one author. `applyWrite (WHwm _)` ratchets by `qmax`, so $\text{HWM}_k = \max(\text{HWM}_{k-1}, \text{NAV}_k^{\text{net}} - f_k^p)$ holds by construction, not by a runtime branch.

```

-- Crystallise(magnitude, from, to): |x|, direction by sign(x), no move at x = 0.
crystallise :: Qty -> WalletId -> WalletId -> Maybe Move
crystallise (Qty x) from to
  | x > 0      = Just (Move from to usd (Qty x))          -- from -> to
  | x < 0      = Just (Move to   from usd (Qty (negate x))) -- direction flips
  | otherwise  = Nothing                                   -- no move at zero

data FeeEvent = FeeEvent
  { fcMgmt :: Qty, fcPerf :: Qty, fcNavNet :: Qty } -- f_m, f_p, NAV net of f_m

-- Two cash moves w_C -> w_M (mgmt, perf), atomic (C3), + the HWM ratchet write.
-- f_m, f_p >= 0; on a loss period f_p = 0, so crystallise returns Nothing and the
-- performance move is absent (no zero-quantity move).
crystalliseFees :: WalletId -> WalletId -> FeeEvent
  -> ([Move], FieldWrite 'FeeCrystallise)
crystalliseFees wC wM fc =
  ( [ m | Just m <- [ crystallise (fcMgmt fc) wC wM
                               , crystallise (fcPerf fc) wC wM ] ]
    , WHwm (fcNavNet fc <> qneg (fcPerf fc)) ) -- HWM_k = max(HWM_{k-1}, NAV^net - f_p)

```

At the worked example's Step E, `crystalliseFees wC wM (FeeEvent (Qty 5750) (Qty 28850) (Qty 1144250))` yields the two legs `Move wC wM USD (Qty 5750)` and `Move wC wM USD (Qty 28850)` together with `WHwm (Qty 1115400)`; applied to a row with $HWM_{k-1} = 1,000,000$ the `qmax` ratchets the mark to 1,115,400. On a loss period $f_k^p = 0$ leaves one leg and the mark unchanged.

Theorem 9.4 (Fee zero-sum per crystallisation). $\sum_w \Delta_{fee}(USD) = 0$ at each crystallisation: the client pays exactly what the manager receives, creating and destroying no value.

Proof. Each leg is $w_C - = q$; $w_M + = q$; summing over wallets gives $-q + q = 0$ per leg, hence 0 over the transaction. $\square$

Quantity, baseline, residual. Every settlement leg here is a `crystallise` call (above): magnitude $|x|$, direction by $\text{sign}(x)$, no move at $x = 0$. The reset baseline is the post-settlement capital base $B_k = \text{NAV}_{t_k} - (f_k^m + f_k^p) + \text{NetFlow}$; reading the pre-settlement NAV would claw the distributed performance back next period. The rounding residual $\text{Perf}_k - \text{round}(\text{Perf}_k)$ is retained in w_C as a conserved un-crystallised remainder, never dropped; it is the reconciling item between economic performance and moved cash. Whether B_k , HWM_k , and the accrued fee are stored or re-derived is escalation E1 (§19). Crystallising unrealised mark-to-market can drive the cash wallet negative; conservation alone cannot distinguish a funded obligation from an insolvent overdraft, a liveness gap recorded as escalation E5 (§19).

9.6 Segregation

Theorem 9.5 (Segregation). *Client partitions are mutually isolated if and only if conservation is combined with locality and capability scoping. A cross-client move is an illegal state ruled out by C4 under locality, not by conservation.*

Conservation alone is not segregation. A move $m = (w_A, w_B, \text{USD}, q)$ between two different clients' partitions is perfectly conservative ($\sum_w \Delta = 0$), so P1 does not forbid it. Two independent facts are at work: conservation (CONS, $\sum_w \Delta = 0$) and locality (LOC, $\text{supp}(\Delta) \subseteq \{w_s, w_d\}$). Locality gives “no effect beyond the named wallets”; isolation of distinct clients additionally requires an authorisation predicate — C4 capability scoping, forbidding a handler from naming or reading across a (w, u_{MA}) boundary it has no capability for. Segregation is therefore a theorem under $\text{CONS} \wedge \text{LOC} \wedge \text{C4}$, supplying the logical-segregation component

of CASS 6 / MiFID II Art. 16(8). Legal segregation — distinct custodial accounts, trust or nominee structure — is outside scope: the ledger evidences it, it does not establish it. That C4 is asserted in prose rather than typed on the accessor signature is escalation E4 (§19).

9.7 CSA margin and collateral

The CSA margin contract is a wallet-level smart contract attached to a per-counterparty collateral wallet. It reads the aggregate mark-to-market of the trades the CSA governs in the real ledger, computes the required collateral (threshold, minimum transfer amount, eligible-collateral and haircut rules), and emits margin-call moves to or from the collateral wallet. Margin eligibility and haircut are contractual data in *ProductTerms*; the exposure is the observed valuation.

For a synthetic managed account the netting set is the single real unit u_{TRS} , never the simulated constituents in the virtual ledger: reading those would breach the isolation invariant (§9.8) and over-margin unmargined positions. The realm tag and C4 realm-scoped reads enforce this. The margin exposure $N_k \cdot \text{TR}_k$ is itself an observation of the virtual book lifted across the realm boundary as a number; that this observation is unattested and unversioned is escalation E2 (§19).

9.8 TRS as synthetic managed account

A total return swap connects a virtual ledger $\mathcal{L}_v$ to real counterparty wallets in the real ledger $\mathcal{L}_r$ through periodic cash settlement. Ledger isolation is invariant P7: no move has one endpoint in $\mathcal{L}_v$ and the other in $\mathcal{L}_r$. A minimal virtual ledger is not a second engine but a realm $\in \{\text{real}, \text{virtual}\}$ tag enforcing P7.

At reset t_k the TRS contract in $\mathcal{L}_r$ observes the virtual book's performance and emits one real move of $|\text{Payment}_k|$, $\text{Payment}_k = N_k \cdot \text{TR}_k - N_k \cdot r_k \cdot \Delta t_k$, direction by sign, $w_{\text{payer}} \rightarrow w_{\text{receiver}}$ in USD, tagged `TRS_NET_SETTLEMENT`. The total return $\text{TR}_k = (V_{t_k}^v - V_{t_{k-1}}^v) / V_{t_{k-1}}^v$ is partial: it requires $V_{t_{k-1}}^v > 0$, a precondition that is a typed failure, not a division by zero.

Theorem 9.6 (Equivalence to the funded managed account). *The TRS settlement and the funded managed-account fee/PnL settlement are the same ledger operation: one net conservative cash move whose magnitude is a deterministic function of an observed performance and a recorded baseline. The funded book plays the role of $\mathcal{L}_v$; the periodic settlement is the TRS payment.*

Proof sketch. Both reduce to `Crystallise(magnitude, from, to)` with magnitude = observed performance over the interval and direction by sign; both reset a baseline to the post-settlement value; neither moves a position across a boundary. The settlement primitive is one; the products differ, and that difference is a CDM (reporting) distinction, not a ledger one: a CDM `TotalReturnSwap` is two-legged ($N \text{TR}$ and $N r \Delta t$), so the report derives from the retained CDM `BusinessEvent`, never from the netted move (§9.12). $\square$

Price consistency. $\mathcal{L}_v$ valuation and $\mathcal{L}_r$ settlement use one price vector P_t . Because P7 forbids a cross-ledger move, a price divergence produces unexplained PnL with no internal reconciliation path — the one place the no-internal-break guarantee can be silently voided. Both realms read the price from the same logged event, so a fresh replay reconstructs one price for both; what remains is attestation of that logged price, escalation E2, the Nazarov attestation finding (§19).

9.9 Redemption and wind-down

Wind-down strikes a final NAV, settles the final fee, returns capital, and retires the mandate. Positions are liquidated against the market wallet; the cash is paid to the client's external wallet;

the mandate unit is returned to the manager. All closed-out rows are retained at zero under the monotone carrier (C1), preserving the audit trail, the final high-water mark, and the quantities needed for tax reconstruction.

```

Move(from: w_C,      to: w_mkt, unit: <asset>, quantity: <held>)      -- liquidate
Move(from: w_mkt,    to: w_C,   unit: USD,    quantity: <proceeds>)
Move(from: w_C,      to: w_ext, unit: USD,    quantity: <final NAV>) -- capital-
                        out
Move(from: w_C,      to: w_M,   unit: u_MA,   quantity: 1)           -- retire
                        mandate

```

9.10 Account-level balance-sheet substantiation

Each account's balance at any date is a deterministic projection of the move stream filtered to its wallets; client statements, desk PnL, and regulatory position reports differ only in the filter. Within the ledger there is no separate account-level quantity record to reconcile against. The move stream substantiates quantities and provenance; valuations depend on the external price vector, and disclosures and legal status require external evidence. Net settlement is operational netting: presenting a balance sheet net requires, in addition, a legally enforceable right of set-off and the intent to settle net (IAS 32.42 / ASC 210-20), a determination outside the ledger. The accounts' assets = liabilities + equity reconciliation holds at the quantity level by P1 (the trial balance balances by construction); the valued and classified presentation does not follow from P1 alone.

9.11 Worked example

One managed account w_C under mandate u_{MA} issued by manager w_M ; virtual market wallet w_{mkt} ; client external wallet w_{ext} . USD reference. Terms: management fee 2%/yr, performance fee 20% over a high-water mark (no hurdle), quarterly ($\Delta t = 0.25$). Conventions: management fee on end-of-period NAV; performance fee on NAV net of the management fee, above the mark; the mandate is non-valued. $\sum_w w(u) = 0$ holds for every unit after every move.

Step	Moves (src $\rightarrow$ dst : qty unit)	NAV
A. Mandate issuance	$w_M \rightarrow w_C : 1 u_{MA}$	—
B. Subscription	$w_{ext} \rightarrow w_C : 1,000,000 \text{ USD}$	1,000,000
C. Trade (buy 5,000 AAPL @100)	$w_C \rightarrow w_{mkt} : 500,000 \text{ USD}; w_{mkt} \rightarrow w_C : 5,000 \text{ AAPL}$	1,000,000
D. Q1 NAV (AAPL $\rightarrow$ 130)	(price move; no ledger move)	1,150,000
E. Fee crystallisation	$w_C \rightarrow w_M : 5,750 \text{ USD (mgmt)}; w_C \rightarrow w_M : 28,850 \text{ USD (perf)}$	1,115,400
F. Wind-down	$w_C \rightarrow w_{mkt} : 5,000 \text{ AAPL}; w_{mkt} \rightarrow w_C : 650,000 \text{ USD}; w_C \rightarrow w_{ext} : 1,115,400 \text{ USD}; w_C \rightarrow w_M : 1 u_{MA}$	0

Fee arithmetic (Step E). No capital flow occurred inside the period, so $\text{Perf} = (1,150,000 - 1,000,000) - 0 = 150,000$. Management fee = $1,150,000 \times 0.02 \times 0.25 = 5,750$. NAV net of management = 1,144,250. Performance fee = $0.20 \times (1,144,250 - 1,000,000) = 28,850$. NAV after both fees = 1,115,400. New high-water mark = $\max(1,000,000, 1,115,400) = 1,115,400$; reset baseline $B_1 = 1,115,400$. Fee zero-sum (Theorem 9.4): $\sum_w \Delta_{\text{fee}}(\text{USD}) = -34,600 (w_C) + 34,600 (w_M) = 0$.

Closing identity (all wallets). Final non-zero balances: $w_M = +34,600$ (fee revenue), $w_{\text{mkt}} = -150,000$ (the gross gain paid out by the market), $w_{\text{ext}} = +115,400$ (client net). Their sum is $34,600 - 150,000 + 115,400 = 0$. The client received principal 1,000,000 plus net gain 115,400; the gross gain 150,000 equals net 115,400 plus fees 34,600. Per-unit conservation holds across the chain: u_{MA} is issued $(+1, -1)$ then returned $(0, 0)$; USD nets to 0 at w_C and globally after every move; AAPL is acquired and liquidated with w_{mkt} as the mirror. Every position row is retained at zero under the monotone carrier.

9.12 Conformance and flags

No conformance is asserted; each item is demonstrated against a cited source or flagged for the responsible owner. The CDM strata named below (**LegalAgreement**, **Trade**, **TotalReturnSwap**, **TradeState**, **BusinessEvent**) are identified at the type level only; pinning them to a named CDM release is owed to the CDM owner and is flagged, not asserted.

- **CDM mandate representation.** u_{MA} maps to the CDM **LegalAgreement** stratum (the ISDA Create-digitised investment-management agreement: parties, fee schedule, benchmark identity, methodology, limits), not the **Trade**/product stratum. A discretionary mandate is a legal agreement, not a tradable contract.
- **CDM TRS representation.** A CDM **TotalReturnSwap** is two-legged; the report is the retained **BusinessEvent** (performance leg and financing leg $N r \Delta t$), never the netted settlement move. The ledger “same mechanism” identity (Theorem 9.6) is a settlement-primitive identity, flagged as not a CDM/reporting identity.
- **SFTR/EMIR reporting surface.** u_{MA} issuance is not reportable: a discretionary mandate is the MiFID II Annex I § A(4) portfolio-management service, not a Section C instrument, not an SFT, not a derivative. The reportable artefact is never u_{MA} but the underlying book trade or u_{TRS} , each carrying its own UTI/UPI/LEI. Reportability is a per-relationship, regime-cited attribute on the wrapped unit’s *ProductTerms*, not inferable from the coarse unit-type key. Governance of that attribute, together with the dual-sided EMIR/CFTC/SFTR reporting that needs validated ISO-17442 counterparty LEIs and classification—neither quantity facts nor with any home in the move stream—is escalation E3 (§19).
- **IFRS classification.** Economic NAV and economic PnL are derived here; accounting classification of fair-value changes (FVTPL/FVOCI/amortised cost) and net presentation (IAS 32.42) are outside scope and require external evidence. Accrued fees, modelled as conserved moves rather than stored scalars, carry their own contra-entry and are auditable as double-entry.
- **CDM TradeState alignment.** That the CDM **TradeState**-per-**Trade** view and *PositionState* $[w, u]$ stay replay-consistent (both projections of one minted event) is asserted, not yet verified; it is the outstanding F6 deliverable (a Rosetta mapping rerun against the three-map schema).

Escalations E1–E5 and the Nazarov attestation finding are cited at their points of origin above and consolidated in the flagged register (§19). They are not failures of the workflow, which is sound as quantity algebra — every step is atomic $q > 0$ moves with $\sum_w w(u) = 0$ verified, and the worked example closes to the penny — but the points at which the performance and reporting layer outruns the framework’s current treatment of recorded observations and external identity.

10 Balance Sheet Substantiation and Dual Valuation

The balance sheet is a projection of the move stream, not a separate record. Each move carries CDM-native metadata linking its balance to the originating product, business event, and counterparty, so every line item traces to its business event along a standardised evidence chain. Substantiation—the demonstration to an external auditor that a balance is complete, accurate, and properly valued—therefore reduces to replaying the log. Because the balance sheet and the ledger are the same stream read two ways, the two cannot disagree.

10.1 Balance Reconstruction

The move stream is an immutable, chronological log. The balance of wallet w in unit u at time t is its opening balance plus the signed sum of every move on u up to t :

$$w_t(u) = w_0(u) + \sum_{\text{moves } m: m.\text{unit}=u, m.\text{time} \leq t} \begin{cases} +m.\text{quantity} & m.\text{destination} = w \\ -m.\text{quantity} & m.\text{source} = w. \end{cases}$$

The result is deterministic and reproducible from the raw stream alone. An auditor verifies any balance at any date t by replaying the stream to t and applying this formula; no independent balance record exists to reconcile against.

The reconstruction is a projection of the move stream: map each move to the signed balance it contributes, then combine all contributions. The combining operation and its unit (the all-zero balance) make the per-(wallet, unit) balance map a *monoid*, and the projection balances is foldMap over that monoid.

```
-- Scalars and keys are those of Section 2: Qty is the additive abelian
-- group of exact minor units; a Move carries a positive quantity of one
-- unit from a source wallet to a destination wallet.
newtype Qty      = Qty Integer      deriving (Eq, Show)
newtype WalletId = WalletId String  deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String  deriving (Eq, Ord, Show)

instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid   Qty where mempty          = Qty 0
qneg :: Qty -> Qty
qneg (Qty n) = Qty (negate n)

data Move = Move { from, to :: WalletId, unit :: UnitId, qty :: Qty }

-- The projection target: one signed total per (wallet, unit). The monoid
-- is POINTWISE addition -- combine cell by cell, all-zero map as unit. This
-- is NOT the library default for Map, which is left-biased union and would
-- silently drop the second contribution to a cell. Choosing the right
-- monoid is the whole correctness content of this type.
newtype Balances = Balances (Map (WalletId, UnitId) Qty)

instance Semigroup Balances where
  Balances a <> Balances b = Balances (Map.unionWith (<>) a b)
instance Monoid Balances where
  mempty = Balances Map.empty

-- One move debits its source and credits its destination: the two-cell
-- signed contribution the displayed formula writes with + / -.
contribution :: Move -> Balances
contribution m = Balances (Map.fromListWith (<>)
  [ ((to m, unit m), qty m)
  , ((from m, unit m), qneg (qty m)) ])
```

```

-- foldMap: map every move to its contribution, then combine them all with
-- one associative operation; the empty stream maps to the all-zero balance.
-- Folding the chronological prefix up to t gives w_t; the opening balance
-- is just the moves before the window, so there is no separate opening term
-- and no separate balance record to reconcile against.
balances :: [Move] -> Balances
balances = foldMap contribution

netBal :: WalletId -> UnitId -> [Move] -> Qty
netBal w u ms = let Balances b = balances ms
                 in Map.findWithDefault mempty (w, u) b

```

Remark 10.1 (Balance-as-fold lens, optional). *Plain reading: balances are not stored and maintained; they are recomputed by folding the move stream. Concatenating two runs of events and folding gives the same result as folding each and adding—so balances of any sub-period compose into the whole.*

Optional. *balances = foldMap contribution : [Move] -> Balances is a monoid homomorphism (balances (xs <> ys) = balances xs <> balances ys) into the Balances monoid; this single law gives deterministic replay, $O(k)$ incremental snapshots, and conservation together.*

Remark 10.2. *balances is a monoid homomorphism from the free monoid of move lists (lists under concatenation, empty list as unit) into the monoid of balance maps—meaning it carries concatenation to combination,*

$$\text{balances } (xs ++ ys) = \text{balances } xs \text{ <> } \text{balances } ys.$$

*This one law is the substantiation guarantee read three ways. With **ys** empty and the operands reassociated, the grouping of independent batches cannot change the result, so replay is order-stable and deterministic. With **xs** the moves up to a snapshot and **ys** the moves since, the snapshot balance combined with later moves equals the balance of the whole stream—the $O(k)$ snapshot query of Section 10.4 is correct by this law, not by separate argument. And summing any unit’s column over all wallets cancels each $+q$ against its $-q$ to **mempty**: that is the conservation law P1 (Section 2), here a consequence of **contribution** emitting equal and opposite legs, not an assertion to enforce.*

10.2 Substantiation Scope

Position quantities and transaction provenance are substantiated by construction: the position set, the flows between wallets, and unit state at any historical date all derive from the move stream by the reconstruction above. This eliminates the manual comparison of independent records that substantiation otherwise requires.

Valuation and disclosure are not so substantiated. Fair value hierarchy classification under IFRS 13 (Level 1/2/3, with sensitivity analysis for Level 3 inputs) and the disclosures of IAS 1 paragraphs 117–124 (accounting policies, significant estimates and judgments) depend on inputs outside the move stream: external price feeds, valuation-model documentation, management judgment, and audit assessment. Legal enforceability depends on external documentation. The design claim is therefore exact: the ledger substantiates quantities and transaction provenance by construction; valuations, disclosures, and legal status require supplementary evidence.

Unit state read during substantiation comes from UnitStatus, a materialised projection of the same log. UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held

then. Nothing is lost by overwriting, and there is no other way to change the value. A value exists from registration onward.

The internal reconciliation failure modes the construction renders structurally unreachable, the boundary modes that remain detectable, and securities lending's by-design treatment are enumerated once in Appendix C.

10.3 Dual Valuation

No single price serves settlement and risk management at once: settlement requires the price at which an external party will transact, risk management a single coherent price across related positions. Both views read the same position set; only the price vector differs.

Definition 10.3 (Dual Valuation). *Each unit $u \in \mathcal{U}$ carries two price functions:*

- $P_t^{\text{MtMk}}(u)$, mark-to-market, from observable market data—used for settlement, margin, external reporting, and custodian reconciliation;
- $P_t^{\text{MtMd}}(u)$, mark-to-model, from internal valuation models—used for risk management, portfolio optimisation, and performance attribution.

Their differential is $\Delta_t(u) = P_t^{\text{MtMd}}(u) - P_t^{\text{MtMk}}(u)$.

Multi-exchange futures. Nikkei 225 futures settle on three exchanges at different cut-off times:

$$\begin{aligned} P_t^{\text{MtMd}}(\text{NKY}) &= 35,500 && \text{(theoretical fair value)} \\ P_t^{\text{MtMk}}(\text{NKY-SIMEX}) &= 35,480 && \text{(Singapore settlement)} \\ P_t^{\text{MtMk}}(\text{NKY-CME}) &= 35,490 && \text{(Chicago settlement)} \\ P_t^{\text{MtMk}}(\text{NKY-OSE}) &= 35,495 && \text{(Osaka settlement).} \end{aligned}$$

The three cannot share a mark-to-market price: Singapore closes hours before Chicago, Osaka between them, and each settlement price fixes the market at a different moment. Margin must therefore use the exchange-specific price. They share one mark-to-model price because all converge to the same final settlement value, the Nikkei 225 index. A desk holding offsetting positions across the three reads net exposure from the single MtMd price while each leg settles at its own MtMk price.

Fair value adjustment. The model price admits an adjustment for the gap between theoretical and executable prices—liquidity cost, bid-ask spread, model uncertainty:

$$P_t^{\text{MtMd, adj}}(u) = P_t^{\text{MtMd}}(u) + \text{FVA}_t(u).$$

Prudent valuation under CRR Article 105 requires the adjusted price for regulatory capital; risk management uses the unadjusted price. Dual valuation carries both.

Single position set. Desk-level consistency and firm-level reconciliation coexist because both valuations read the same wallet balances. Each desk prices related positions coherently through P^{MtMd} ; the firm reads valuation discrepancy directly from $\Delta_t(u)$; internal transfers may book on either basis by policy. Position divergence between the two views is impossible—there is one position set, two price vectors.

10.4 Snapshots and Audit Trail

A snapshot is a substantiated checkpoint: a consistent cut of the move stream, carrying both pricing bases and the provenance metadata of each move. A balance query starting from the

most recent snapshot replays only the moves since it, making the query $O(k)$ in the moves since the snapshot rather than $O(n)$ in the total. A snapshot is consistent only when taken at a quiescence point—no in-flight transactions—or under multi-version concurrency control.

Conservation enforces double-entry accounting on quantities, independent of price:

$$\sum_{w \in \mathcal{W}} w_t(u) = 0 \quad \forall u, \forall t,$$

the closed-system law of Section 2 (P1). Valuation applies the price vector to the conserved quantities:

$$V_t^{\text{MtMk}} = \sum_w w_t(u) \cdot P_t^{\text{MtMk}}(u), \quad V_t^{\text{MtMd}} = \sum_w w_t(u) \cdot P_t^{\text{MtMd}}(u).$$

Quantities are conserved and shared; only the two valuations differ.

11 Implementation and Operations

The architecture is specified at the design level: data structures, algorithms, and interfaces an implementation must provide. No implementation language is assumed.

11.1 Three-Layer Architecture

The system has three layers. A CDM business event enters at the persistence layer, is decomposed into balance deltas at the computation layer, and is reflected as state at the projection layer.

1. **Move stream** (persistence). The immutable, append-only log of atomic moves: the canonical internal record. Each move carries full provenance—timestamp, source contract, metadata, external references.
2. **Aggregation engine** (computation). A pure GroupBy from a set of moves to net change per (wallet, unit) key. Having no side effects, it parallelises across independent keys.
3. **Projection** (state). Wallet balances and unit status, derived from the move stream through the aggregation engine.

The projection is a cache, not a record. Corruption of the projection—hardware fault, lost node—is repaired by replaying the move stream from the last verified snapshot. Nothing is lost: the move stream is append-only and immutable (P4).

Remark 11.1 (The projection is a rebuilt value, not a second source of truth). *UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward. The same holds for wallet balances: the projection layer is a catamorphism over the log—meaning replaying the events always rebuilds the exact balance and status that held at any chosen instant (P8).*

11.2 Balance-Update Algorithm

A transaction $\tau = \{m_1, \dots, m_n\}$ is applied in four steps.

1. **Validate**. Every move references an existing wallet and unit; τ satisfies conservation (C2, P1).
2. **Aggregate**. Net change per (wallet, unit) key by GroupBy over the moves.
3. **Apply**. Update balances atomically; on any failure, roll back the whole transaction (P2).
4. **Snapshot**. Persist the updated projection as a checkpoint for later queries.

11.3 Constant-Cost Profit and Loss

Profit and loss over $[t_0, t_1]$ loads the snapshots and prices at the endpoints, computes V_0 and V_1 , and returns $V_1 - V_0$. The cost is $O(n)$ in instruments held, not in moves, because the result depends only on the endpoint states (state-sufficiency, P10). A portfolio of 10,000 instruments and 1,000,000 moves reads 10,000 balances, not 1,000,000 moves. Valuation inputs are supplied by the market-data and valuation satellites; this section assumes prices at t_0 and t_1 as given.

11.4 Fault Tolerance

Late events. Each event carries two timestamps: economic (“what happened”) and booking (“when it was learned”). Append-only insertion admits an event after its economic time without mutation. Financial statements and regulatory returns project on the economic timestamp at the reporting date; the booking timestamp supports the as-known view.

Duplicate events. Idempotency is structural at two levels. A transaction carries a unique identifier; a re-applied identifier is rejected (P5). A lifecycle event is gated on unit status: if the status shows the event complete, the lifecycle function returns no moves and no state change (P6). CDM event-qualification functions perform this check, excluding both duplicate and out-of-sequence processing.

Contradictory external state. Virtual wallets mirror external counterparties. A disagreement between an external confirmation (ISO 20022 feedback) and the internal record is detected by comparing virtual-wallet balances to the external statement. Resolution is not automated: the discrepancy is surfaced, a human or external process determines the correct state, and that resolution is recorded as a transaction in the move stream.

Stale market data. The conservation layer—moves and balances—is independent of price feeds and operates correctly when they fail; only the valuation projection degrades. Value-dependent settlements (managed-account settlements, Section 9; margin; total-return-swap resets) produce economically incorrect moves if driven by stale prices. An implementation gates such settlements on data-quality checks and defers settlement when price quality is insufficient, rather than booking known-stale values for later correction. Staleness handling and degradation policy are specified by the market-data and deferred-settlement satellites.

Move-stream integrity. The canonical record is replicated and protected against corruption by cryptographic hash chaining (P4). Periodic verification—recomputing balances from the raw stream and comparing to the cached projection—detects silent corruption. The canonical-record design removes internal inconsistency between records but concentrates availability risk; deployment addresses this through replication, backup, and disaster recovery.

Partial failures. Atomicity is all-or-nothing (P2): a transaction that cannot complete rolls back entirely, leaving the move stream consistent. Crossing process restarts requires write-ahead logging or equivalent crash-recovery semantics.

11.5 Corrections as Events

The move stream is immutable; an error is corrected by a compensating transaction that reverses the original and books the correct version. This is event sourcing: state derives from an append-only log, and a correction is an event, not a mutation. The incorrect moves remain in the stream, preserving error history for audit.

The compensating transaction references the original through a `corrects` field in its metadata, forming an explicit correction chain. This linkage is a first-class metadata requirement: it distinguishes economic events from error corrections on replay.

A trade amendment—a change of terms, such as a notional increase—is distinct from a cancellation. An amendment is a CDM `TermsChangeEvent`: the contract’s unit status advances to the new terms, and any economic difference is captured as a settlement move. The original

terms remain in the stream. A formal correction algebra (anti-moves, amendment chains, their composition, and the interaction of corrections with snapshot consistency) is an open problem (Section 19).

11.6 From Moves to Settlement Instructions

A lifecycle event that generates moves—option exercise, coupon payment, margin call—also settles externally. The ledger move is the internal record of the economic event; the settlement instruction is the external message that transfers cash or securities between custodian, CCP, or payment-system accounts.

The mapping is a projection. Each move carries source wallet, destination wallet, unit, and quantity. The settlement layer translates wallet identifiers to external accounts (BIC, IBAN, CSD account), unit identifiers to security identifiers (ISIN, CUSIP), and quantity to settlement amount. Because the event model is CDM and CDM maps to ISO 20022, the translation is a struct-to-struct transformation, not a business computation. The information content of every settlement instruction is fully determined by the move and requires no additional capture. The settlement-layer interface and instruction timing are specified in Section 12 and the deferred-settlement satellite.

11.7 ISO 20022 Interface

Message generation is projection: the CDM object is the semantic content, the ISO 20022 message its serialisation. Every field of a `sese.023` Securities Settlement Instruction is already present in the CDM representation of the move.²

ISO 20022 field (<code>sese.023</code>)	CDM source
SecurityIdentification (ISIN)	<code>Product.security.identifier</code>
SettlementQuantity	<code>Move.quantity</code>
SettlementAmount	<code>Move.quantity</code> × price
TradeDate	<code>BusinessEvent.effectiveDate</code>
SettlementDate	<code>Transfer.settlementDate</code>
DeliveryReceiveIndicator	Move direction

Cash messages (`pacs.008`, `pacs.009`) follow the same principle: the CDM `Transfer` carries payer, payee, amount, currency, and value date; the ISO message adds only transport fields (BIC routing, message identifiers).

The traceability chain is deterministic at every step: smart contract → move (with contract reference) → ISO 20022 instruction (with `EndToEndId` and `TxId`) → external confirmation (matching identifiers) → status update in the event log. A returned confirmation (`sese.025` for securities, `camt.054` for cash) is ingested back into the move stream, closing the settlement cycle within one framework.

12 The Settlement Layer Interface

The Ledger records economic events; the settlement layer executes real-world transfers. The boundary between them is a *projection*: a pure, total function that reads one committed transaction and produces an optional settlement instruction. Everything the settlement layer needs is already present in the transaction; everything the settlement layer adds—custodian accounts, CSD connectivity, message formatting—lies outside the Ledger’s scope.

²`sese.023` is the ISO 20022 business message identifier for the Securities Settlement Instruction; under the ISO 20022 message-definition identifier scheme the full identifier `sese.023.001.NN` fixes variant and version *NN*.

12.1 The Settlement Projection

Definition 12.1 (Settlement Projection). *The settlement projection*

settleProjection : SettlementTx → Maybe SettlementInstruction

is pure, deterministic, stateless, and idempotent. It is total: defined on every settlement transaction, returning Just an instruction for the settlable classes (SETTLEMENT, COLLATERAL) and Nothing for the rest.

Its input, `SettlementTx`, is a *boundary type*, not the core transaction of Section 2. The core `Transaction` is the one atomic event recorded in the log—an ordered list of moves over ledger units, with the state delta they ride. `SettlementTx` is the settlement-layer view onto which a committed core transaction *projects*: the transaction id, its settlement class, and its moves with each move’s unit already resolved to its settlement identity (`Asset`—an ISIN or a currency). The projection from core `Transaction` to `SettlementTx` is the unit-to-asset resolution the contract performs at move generation; `settleProjection` then carries `SettlementTx` to the wire instruction. The two never share a name: there is one `Transaction` (the log event), and `SettlementTx` is what the boundary reads of it.

The projection reads only from the transaction:

- **Transaction id** — the settlement reference (`EndToEndId` in ISO 20022).
- **Each move’s source wallet, destination wallet, settlement asset, and quantity** — the economic content of the transfer.
- **The CDM event payload** (Section 13) — settlement date, counterparty identifiers (LEI, BIC), and execution venue (MIC).

It reads nothing else: not current wallet balances, not other transactions, not any external system. Purity follows: the same settlement transaction always yields the same instruction, the function is safe to re-run after a failure, and the settlement layer need not understand conservation, unit state, or lifecycle logic.

The projection, types first. A settlable move’s `smAsset` is its unit’s settlement identity—the security ISIN or the currency the unit denotes, resolved from the unit’s registered terms (Section 4) and carried into the `SettlementTx` by the contract—so the projection needs no off-transaction lookup. The settlement *type* and the *legs* are one value: each constructor of `SettlementLegs` carries exactly the legs its type names, so an instruction with no legs cannot be written and the type can never disagree with the legs.

```
-- The settlement classification of a unit: a transferable security, or a
-- currency. Every settlable move is exactly one or the other. (CashCcy, not
-- Cash, so the settlement currency case never collides with a scalar.)
data Asset = Security ISIN | CashCcy Currency

-- A settlement-layer move: it ranges over the settlement identity (Asset),
-- not the core ledger UnitId. It is what a core Move projects to once the
-- contract has resolved the unit to its settlement asset.
data SettleMove = SettleMove { smFrom, smTo :: WalletId, smAsset :: Asset, smQty
    :: Qty }

-- Settability is a field set by the contract, never inferred. Total over all
-- five classes; only SETTLEMENT and COLLATERAL settle. LIFECYCLE/CORRECTION
-- "sometimes" settle only by DECOMPOSING into separate transactions at the source
.

data TxClass = SETTLEMENT | COLLATERAL | LIFECYCLE | ACCOUNTING | CORRECTION
settles :: TxClass -> Bool
settles SETTLEMENT = True
settles COLLATERAL = True
settles LIFECYCLE  = False
```

```

settles ACCOUNTING = False
settles CORRECTION = False

data CdmPayload = CdmPayload { cdmTradeDate, cdmSettleDate :: Day
                              , cdmCounterparty :: LEI, cdmVenue :: MIC }

-- The boundary type. NOT the core Transaction (the log event of Sec.~ledger):
-- it is the settlement-layer view a committed core Transaction PROJECTS onto,
-- its moves already resolved to settlement Assets.
data SettlementTx = SettlementTx { stId :: TxId, stClass :: TxClass
                                   , stMoves :: [SettleMove], stCdm :: CdmPayload }

data SecuritiesLeg = SecuritiesLeg { slIsin :: ISIN, slQty :: Qty
                                    , slDeliver, slReceive :: WalletId }
data CashLeg = CashLeg { clCcy :: Currency, clAmount :: Qty
                        , clPayer, clReceiver :: WalletId }

-- type and legs are ONE value: "an instruction with no legs" is unrepresentable.
data SettlementLegs = DvP SecuritiesLeg CashLeg | FoP SecuritiesLeg | CashOnly
                    CashLeg

data SettlementType = DVP | FOP | CASH
settlementType :: SettlementLegs -> SettlementType -- a projection, not a stored
field
settlementType (DvP _ _) = DVP
settlementType (FoP _) = FOP
settlementType (CashOnly _) = CASH

data SettlementInstruction = SettlementInstruction -- plain data, no
behaviour
{ siRef :: TxId, siTradeDate, siSettleDate :: Day
  , siLegs :: SettlementLegs, siCounterparty :: LEI, siVenue :: MIC }

-- A security move -> securities leg (source delivers, destination receives);
-- a cash move -> cash leg (source pays, destination receives). Total over Asset.
classify :: SettleMove -> Either SecuritiesLeg CashLeg
classify (SettleMove from to (Security i) q) = Left (SecuritiesLeg i q from to)
classify (SettleMove from to (CashCcy c) q) = Right (CashLeg c q from to)

-- Nothing when there is no standard leg shape: no moves (a contract error), or a
-- non-decomposed multi-leg structure. Both give no instruction, never a partial
one.
legsOf :: [SettleMove] -> Maybe SettlementLegs
legsOf ms = case partitionEithers (map classify ms) of
  ([s],[c]) -> Just (DvP s c)
  ([s],[]) -> Just (FoP s)
  ([],[c]) -> Just (CashOnly c)
  _ -> Nothing

settleProjection :: SettlementTx -> Maybe SettlementInstruction -- pure, total
settleProjection tx
  | settles (stClass tx) = mk <$> legsOf (stMoves tx)
  | otherwise = Nothing
where mk legs = SettlementInstruction (stId tx) (cdmTradeDate (stCdm tx))
  (cdmSettleDate (stCdm tx)) legs (cdmCounterparty (stCdm tx)) (cdmVenue (
    stCdm tx))

-- Behaviour, made concrete (settlementType . siLegs <$> settleProjection tx):

```

```

-- security + cash move (a DvP equity trade) -> Just DVP
-- a securities move alone                      -> Just FOP
-- a cash move alone (a margin call)           -> Just CASH
-- an ACCOUNTING transaction                   -> Nothing -- not settleable
-- a SETTLEMENT with no moves                  -> Nothing -- contract error,
no instruction

```

Remark 12.2. *The projection is total and deterministic by construction. `settles` classifies every `TxClass`; `legsOf` is exhaustive, its wildcard absorbing both the empty and the non-decomposed multi-leg cases as `Nothing`; and `settleProjection` reads only `tx`—no clock, no balances, no external system—so the same settlement transaction always yields the same instruction and re-running after a failure is safe.*

Definition 12.3 (SettlementInstruction). *The `SettlementInstruction` is plain data with no behaviour (the `data` declaration above): instruction id, trade and settlement dates, the settlement legs, counterparty LEI, and execution venue MIC. The settlement type $\in \{DVP, FOP, CASH\}$ is a projection of which legs are present (`settlementType`), never an independent field that could disagree with them. Both legs present give DVP; securities alone give FOP (Free of Payment); cash alone gives CASH; no legs is unrepresentable, so a settleable transaction with no moves yields `Nothing`—a contract error, not an instruction.*

The standard cases—DvP equity trades, FOP transfers, cash-only settlements—carry at most one securities leg and one cash leg. Multi-leg structures decompose at the source, not in the projection: the smart contract emits one `SETTLEMENT` transaction per independently settleable leg. A repo emits two—an opening leg and a closing leg—each a standard DvP structure.

12.2 Boundary Separation

Principle 12.4 (Boundary Separation). *The Ledger provides what settles: parties, securities, quantities, dates. The settlement layer provides how: standing settlement instructions (SSIs), custodian account identifiers, CSD connectivity, ISO 20022 message generation, and exception management. Neither side reads the other’s internals.*

The `SettlementInstruction` is the sole object shared across the boundary. Wire-message generation is two steps: projection, then enrichment. The projection produces the instruction from the transaction; the settlement layer enriches it with the operational detail the Ledger does not hold—SSIs, CSD participant ids, settlement priority—before generating the ISO 20022 message (e.g. `sese.023`). This is the precise sense of “message generation is projection, not translation” (Section 11): the projection step is the Ledger’s; enrichment is the settlement layer’s.

12.3 Transaction Types and Settlability

Settlability is a first-class field on the transaction, assigned by the generating smart contract at the point of move generation, never inferred at runtime—the contract knows the business intent.

Type	Settles?	Description
SETTLEMENT	Yes	Maps to a real-world settlement instruction (e.g. equity trade $\rightarrow$ <code>sese.023</code>).
COLLATERAL	Yes	Settles through the collateral channel (e.g. margin call $\rightarrow$ cash transfer).
LIFECYCLE	Sometimes	Decomposes into settlement and accounting sub-transactions.
ACCOUNTING	No	Internal adjustment, no external transfer (e.g. PnL crystallisation, revaluation).
CORRECTION	Sometimes	Compensating transaction; settles a reversal only if the original already settled.

The projection returns `Just` for `SETTLEMENT` and `COLLATERAL`, `Nothing` for the rest. When a `LIFECYCLE` event requires both settlement and accounting, the smart contract emits both transactions: a `SETTLEMENT` for the legs that transfer externally, an `ACCOUNTING` for the internal adjustment. Physical exercise of an option, for instance, emits a `SETTLEMENT` (deliver shares, receive cash) and a separate `ACCOUNTING` (close the option position, crystallise PnL). The decomposition is the contract’s responsibility, not the projection’s.

12.4 Lifecycle-Originated Settlement

Some settlements originate from internal contract logic rather than an external instruction: an option exercise, a coupon falling due, a barrier breach. The Ledger is the originator, so there is no external instruction to match. The projection treats these identically—the transaction is tagged `SETTLEMENT` by the contract, the projection extracts the instruction, the settlement layer enriches and delivers it. The sole difference is provenance: execution-originated instructions match an external trade confirmation; lifecycle-originated instructions are generated unilaterally. The settlement layer receives the instruction with no awareness of its origin.

One lifecycle transaction may yield several instructions where it carries several legs. Physical exercise of an equity option with lot-size constraints (Section 6) carries a securities delivery (whole lots) and a cash payment (strike value plus any fractional residual); the projection classifies each move by unit type and emits the matching instruction for each.

12.5 DvP: The Two-Level Guarantee

Delivery versus Payment (DvP) is the principle that securities delivery and cash payment occur together or not at all. The framework provides it at two independent, mutually reinforcing levels.

Ledger-level. A DvP trade is one transaction holding both the securities move and the cash move. The executor commits all moves atomically—both take effect or neither does (Section 2). This is a structural consequence of the transaction primitive, not a runtime check; the smart-contract specification (Section 6) supplies the other half by requiring both legs within one transaction.

Settlement-level. The CSD or CCP provides DvP at the real-world level: securities delivery and cash payment settle simultaneously on its books. This depends on external infrastructure and lies outside the Ledger’s control.

The temporal gap. Between trade date and settlement date the Ledger shows the economic position under trade-date accounting (Section 1); the CSD confirms the real-world transfer at

settlement. The gap is not a DvP failure but the normal state under trade-date accounting, where exposure exists from execution.

Settlement failure. On failure (the counterparty cannot deliver), the Ledger records a settlement-failure event and does *not* reverse the economic position: the exposure existed from trade date regardless of settlement outcome, and persists until the failure resolves through buy-in, cancellation, or partial delivery. The discrepancy between Ledger and custodian positions is detected by virtual-wallet comparison (Section 11).

External regimes. Legal title, beneficial ownership, fail treatment, and the regulatory-capital treatment of unsettled transactions are governed by external frameworks, not by the Ledger. The Ledger supplies the position and timing data those treatments require; it does not determine the applicable regime.

Remark 12.5. *The mechanics of the trade-date-to-settlement-date interval—deferred settlement, fail lifecycle, buy-in, partial delivery, and re-instruction—are specified in the standalone deferred-settlement specification. This section fixes only the interface: what the projection produces and how settlement status returns. The satellite is out of scope here.*

12.6 Gross-to-Net Reconciliation

The Ledger records gross transactions, one per trade. The settlement layer may net: for trades in the same security with the same counterparty on the same settlement date, it produces a single net instruction. Netting is a settlement-layer optimisation; the Ledger’s gross records are authoritative and are never modified by it.

Gross and net reconcile by an algebraic identity, not a heuristic. For each (security, counterparty, settlement date) group, the algebraic sum of gross instruction quantities—deliveries positive, receipts negative—equals the net instruction quantity. The sum is a *group homomorphism*—a structure-preserving map, here meaning summing the gross legs gives the net leg—written `foldMap` from the gross legs into the additive group of quantities. A violation localises the fault: either the projection produced an incorrect instruction (a Ledger fault) or the netting computation is wrong (a settlement-layer fault).

```
-- Signed securities quantity from one party's view: delivery positive, receipt
  negative.
signedFor :: WalletId -> SecuritiesLeg -> Qty
signedFor me l | slDeliver l == me = slQty l
               | slReceive l == me = negQty (slQty l)
               | otherwise         = mempty

-- The net leg's signed quantity equals the sum of the gross legs'. An identity,
-- checked, not a reconciliation heuristic.
reconciles :: WalletId -> [SecuritiesLeg] -> SecuritiesLeg -> Bool
reconciles me gross net = foldMap (signedFor me) gross == signedFor me net
```

12.7 The Confirmation Return Path

Settlement confirmations (`sese.025` for securities, `camt.054` for cash) flow back into the Ledger as lifecycle events recording the confirmation timestamp and external reference. No moves are generated—positions were already correct from trade date. The settlement status advances:

Status	Meaning
EXECUTED	Committed in the Ledger; the settlement layer has not yet processed it.
INSTRUCTED	The settlement layer has submitted the instruction; the CSD has not confirmed.
SETTLED	The CSD confirms successful settlement; custody now matches Ledger positions.
FAILED	The CSD reports a failure. Ledger positions remain correct (economic exposure); the external transfer has not occurred. A failed trade may re-instruct (return to INSTRUCTED after buy-in or extension), partially settle (a lifecycle event), or cancel (a CORRECTION transaction reversing the original moves).

The transitions are `EXECUTED` → `INSTRUCTED` → `SETTLED`, with `INSTRUCTED` → `FAILED` as the alternate terminal. Each transition is itself a logged lifecycle event in the move stream. Settlement status is therefore a projection of those events, not a separate mutable store: replaying the events to any point rebuilds the status that held then, and the stored value is overwritten in place only as a read cache. This completes the traceability chain of Section 11: smart contract → move → settlement instruction → external confirmation → status.

13 ISDA CDM Integration

The framework needs a canonical vocabulary for products and lifecycle events: one normalised description of what is traded, one of how obligations evolve, and one deterministic rule for when a transition is permitted. The ISDA Common Domain Model, governed by the Fintech Open Source Foundation (FINOS), supplies all three. The ledger adopts it as its domain model. The correspondence is derived from the framework’s own primitives—wallet, unit, move, transaction, conservation. Each CDM record is then stored intact alongside the transaction it generates.

13.1 The CDM

The CDM is a machine-readable, machine-executable model of financial products and their lifecycle events, expressed in the Rune domain-specific language and distributed as open-source executable code. It comprises five components:

1. A *product model*: tradable contracts and instruments in normalised form (quantity, price, parties, payouts, legal agreements).
2. An *event model*: lifecycle events as state transitions built from a fixed set of primitive operators—execution, quantity change, terms change, reset, transfer.
3. A *process and workflow model*: pre- and post-trade steps—proposals, acceptances, rejections, allocations, novations.
4. A *reference-data model*: parties, legal entities, and the static data needed to interpret trades and events.
5. A *mapping layer* (synonyms): a machine-readable correspondence between CDM objects and external message formats and identifiers—FpML, FIX, ISO 20022.

A single instrument traces the abstractions: a European call on AAPL, strike \$200, expiry 2026-06-20, 100 contracts, cash-settled in USD, bought by Firm A from Firm B on 2026-03-15. The CDM describes it as an `OptionPayout` (underlier, option type, strike, expiry, settlement terms), wraps it in a `Trade` that binds counterparties and execution context, and records its exercise as a `BusinessEvent` whose `before TradeState` (ACTIVE) becomes its `after TradeState` (EXERCISED). The chain is *product* (what the contract is) → *tradable product* (who traded

it) $\rightarrow$ *business event* (what happened) $\rightarrow$ *ledger moves* (how value transfers). The full product walkthrough is in Appendix F.

13.2 Derivation: CDM Maps to the Framework Primitives

Each framework primitive has a CDM counterpart, so the integration follows from the definitions rather than from adoption.

Products as units. The wallet universe $\mathcal{U}$ instantiates directly from the CDM product model: each CDM trade or contract corresponds to a unit type or a bounded set of unit types (underlyings, legs, payouts), managed by the Unit Store (Section 3). CDM’s normalised quantity, price, and party are the primitives of wallet balances and move units.

Event model as state graph. The CDM event model already treats each lifecycle event as a functional state transition. `TradeState` and `State` are the nodes of the lifecycle state graph; the primitive operators composed in a `BusinessEvent` are its edges; the lineage between `before` and `after` states, and between `WorkflowSteps`, is the evidence chain that reconstructs any path through the graph. CDM is thus a concrete state machine for the abstract lifecycle.

Embedded logic as transition contracts. CDM embeds executable Rune logic—primitive functions, validation rules, event-qualification functions—which are the deterministic predicates a transition contract requires. A CDM function applied to a `TradeState` is a transition-contract instance; its validation and state-transition checks are the proof of execution that must pass before moves are emitted. The ledger treats CDM functions as the authoritative contracts above which wallet moves are generated.

Mapping layer as oracle interface. External messages are oracle outputs; synonym mappings are deterministic transforms from those messages into CDM trades and events; CDM functions then drive transition contracts and emit moves. Because the mapping is explicit and shared, no bilateral adapter is required per counterparty.

CDM describes *what* trades and events are; the closed ledger describes *how* value moves between wallets in response.

13.3 The Synonym Layer and Ingestion

Each external message—an FpML confirmation, a FIX execution report, an ISO 20022 settlement instruction—maps to a CDM object through a declared synonym. The mapping is deterministic, versioned, and machine-readable, and is reused across systems in place of bespoke bilateral adapters. Ingestion is a fixed pipeline:

raw message $\rightarrow$ CDM synonym mapping $\rightarrow$ CDM object $\rightarrow$ lifecycle function $\rightarrow$ moves and
state changes

The only bespoke code is at the transport layer; the semantic mapping is shared.

The move stream stores CDM objects that may originate from earlier CDM versions, so version coexistence is required. Each stored event carries its CDM version identifier. The lifecycle engine either processes an older-version event natively or applies a version migration before processing. A CDM enumeration addition—a new lifecycle intent or product type—requires a corresponding update to the lifecycle state machines and the test generators.

13.4 The Forgetful Map F

The forgetful map reads a rich CDM business event and keeps only the bare value movements the ledger acts on, holding the legal and business detail aside in storage rather than discarding it.

Let **CDM** be the collection of CDM states and business events and **Ledg** the collection of ledger states and transactions. The map $F : \mathbf{CDM} \rightarrow \mathbf{Ledg}$ sends each **BusinessEvent** to a boundary record carrying its economic content—the moves—and stores the originating CDM event as that record’s payload. The moves it carries are exactly the edges a core ledger **Transaction** records, so the boundary record projects onto the core transaction; it is not itself the core **Transaction** and keeps a distinct name. F is *forgetful*: it drops the legal and ISDA-business detail the ledger does not track and keeps only the moves. Nothing is lost, because the dropped detail survives verbatim in the payload.

F has the signature below: it consumes a CDM **BusinessEvent** and produces a boundary **CdmTransaction** that projects onto the core ledger **Transaction**.

```
-- The CDM side. A BusinessEvent carries the primitive operators F reads, the
-- lifecycle states it transitions between, and the business detail F forgets.
data Intent          = Execution | Exercise | Termination | QtyChange | ResetIntent
data LifecycleState = ActiveS    | Exercised | Terminated | SettledS
newtype TradeState  = TradeState LifecycleState

data PrimitiveInstruction
  = PiTransfer  WalletId WalletId UnitId Qty  -- value leg (cash or asset)
  | PiQtyChange WalletId WalletId UnitId Qty  -- position leg (contracts)
  | PiTerms     -- amendment: no move
  | PiReset     -- observation/reset: no move

data BusinessEvent = BusinessEvent
  { beInstructions :: [PrimitiveInstruction] -- read by F (economic content)
  , beBefore       :: TradeState             -- lifecycle before / after: read
  , beAfter        :: TradeState             -- sequencing only
  , beIntent       :: Intent                 -- FORGOTTEN: business intent
  , beLineage      :: [String]               -- FORGOTTEN: workflow lineage
  , beStructure    :: String                -- FORGOTTEN: product structure,
  , legal terms
  }

-- F emits the canonical Sec. 2 Move (mFrom, mTo, mUnit, mQty, mTime, mSource). A
-- Move names BOTH parties, so a single Move conserves BY CONSTRUCTION (a credit
-- with no offsetting debit is unrepresentable); the whole event’s conservation is
-- then the signed edge-sum of its move list -- mempty for the empty list, so a
-- move-less event (amendment, reset) is conserved for free. There is no separate
-- validation step, and none is representable.
--
-- CdmTransaction is the CDM BOUNDARY type, kept DISTINCT from the core ledger
-- Transaction (Sec. 4): its ctMoves ARE the edges a core Transaction records, so
-- it
-- PROJECTS onto the core Transaction; ctPayload keeps the originating event
-- verbatim.
data CdmTransaction = CdmTransaction
  { ctMoves      :: [Move]                -- economic content the ledger acts on (-> core
  , ctPayload    :: BusinessEvent         -- originating CDM event, kept verbatim (
  , recoverable)
  }
```



```

-- F = forget. Total over every BusinessEvent; deterministic (pure). Move
-- extraction is 'concatMap instructionMoves', a MONOID HOMOMORPHISM from the free
-- monoid of instruction lists to the free monoid of move lists --- meaning it
-- distributes over concatenation, moves (xs <> ys) = moves xs <> moves ys. The
-- restricted composition and sequencing properties below are consequences of this
-- one law, not separate guarantees.
instructionMoves :: PrimitiveInstruction -> [Move]
instructionMoves (PiTransfer a b u q) = [Move a b u q (Timestamp 0) (SourceId "
    cdm")]
instructionMoves (PiQtyChange a b u q) = [Move a b u q (Timestamp 0) (SourceId "
    cdm")]
instructionMoves PiTerms                = []                -- move-less event (FAQ Q6)
instructionMoves PiReset                 = []

```

```

forget :: BusinessEvent -> CdmTransaction
forget be = CdmTransaction (concatMap instructionMoves (beInstructions be)) be

```

A move-less event (an amendment, a reset) maps to a `CdmTransaction` with `ctMoves = []` and the full payload—`forget` is total there, it does not fail. `forget` is *forgetful* in code: `ctPayload` retains the whole event, so the detail dropped from `ctMoves` (intent, lineage, structure) is recoverable verbatim, e.g. `ctPayload (forget e) == e`.

Worked example: exercising the AAPL call. Firm A exercises the 100-contract call on 2026-06-20 at settlement price $S_T = 215$. The arriving `BusinessEvent` carries an `ExerciseInstruction` with before `TradeState` (ACTIVE, 100 contracts) and after `TradeState` (EXERCISED). F reads the primitive instruction—a cash `Transfer` of $100 \times \max(215 - 200, 0) = \text{USD } 1,500$ from seller to buyer—and emits `Move(seller[Firm B], buyer[Firm A], USD, 1500)`. Firm B’s USD balance falls by 1,500 and Firm A’s rises by 1,500; the net change is zero, so conservation holds. The option unit’s state becomes EXERCISED and accepts no further events. The full CDM `BusinessEvent`—exercise terms, counterparty identifiers, product structure, workflow lineage—is stored in the generated `CdmTransaction`’s payload for audit and reporting.

Remark 13.1. *UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward. The exercise above changes the option unit’s status to EXERCISED solely by logging its BusinessEvent; the stored EXERCISED flag is the replay of that log.*

What F preserves.

- **Composition (restricted).** For referentially independent events e_1, e_2 —neither referencing the other’s state— $F(e_2 \circ e_1) = F(e_2) \circ F(e_1)$. This is the monoid-homomorphism law of `concatMap instructionMoves` above—move extraction distributes over concatenation—so F assembles the composite’s moves from the parts without reference to neighbours. Events that carry cross-references (a novation referencing the original trade) or dependent cascades (a margin call triggered by a reset on the same unit) must first be resolved and processed in causal order. Composition then holds for the resolved sequence, provided the resolution is deterministic and alters no event’s economic content.
- **Conservation.** Every business event maps to a transaction satisfying the conservation law; F neither creates nor destroys value.

- **Sequencing.** The temporal order of CDM events is the order of the resulting move stream.
- **Idempotency.** If a CDM event is idempotent, so is its transaction. Exercising the same option twice produces no additional moves: the absorbing lifecycle guard (**Exercised** accepts no further exercise, as **Expired** is absorbing for futures, Section 8) makes the second event inert before F runs.

What F forgets. Business intent (why the trade was placed), workflow lineage (the approval and negotiation history), product structure (payout schedules and legal terms), and regulatory classification (IFRS 9 measurement category—see Section 17). Each is preserved in the CDM payload, not in ledger state.

Principle 13.2. *The ledger carries economic substance—positions, cashflows, conservation; CDM carries business meaning—intent, structure, classification. Embedding full CDM graphs in ledger state would enlarge it without strengthening any guarantee, since balance computation and conservation do not depend on business intent. The forgetfulness of F is therefore forced by the separation of the two abstractions, and the dropped detail is recoverable from the payload at all times.*

The concrete per-type translation rules that implement F —which CDM types map to ledger primitives and which are preserved as metadata—are tabulated in Appendix A.

13.5 CDM Enumerations as Generator Universe

The CDM enumerations—product types (interest rate, equity, credit, FX, commodity), event types (execution, termination, novation, exercise, reset, transfer, quantity change), party roles, date-adjustment rules, settlement types—form a closed, finite universe of valid constructs. Combined with the CDM validation rules for which event types are valid for which product types, they define the generator universe for the property-based tests of Section 16: the test framework enumerates the valid (product, event) combinations and verifies that F produces a conservation-satisfying transaction for each.

Part V

Obligations, Generalised Positions, and Securities Lending

14 Orchestration and Obligation Liveness

The framework fixes *what* must hold—conservation, atomicity, idempotency, purity—and leaves open *how* lifecycle events are scheduled, retried, and sequenced. Safety alone does not move the system forward: a coupon due on a date causes no error if it never fires. This section states the requirements an execution engine must meet to close that gap, maps the executor onto it, and then develops obligation liveness as a first-class subsystem with its own invariants (P21–P23) and proof.

14.1 Execution-engine requirements

Four requirements follow from the framework’s own primitives. They bind any execution engine; the engine is an implementation choice that satisfies them, not a result derived here.

1. **At-least-once delivery.** Idempotency (P5, transaction level; P6, lifecycle level) makes duplicate execution harmless but does not make an event fire. The engine must dispatch every scheduled event and re-dispatch on worker failure. At-least-once delivery composed with idempotent activities yields exactly-once semantics.
2. **Durable timers surviving restarts.** A coupon scheduled for a fixed future date fires on that date irrespective of process restarts, deployments, or infrastructure failures in the interval. Timer state is persisted by the engine, not held in process memory.
3. **Deterministic replay.** Time travel (Property 6) requires ledger state at any past instant to be reconstructable by replaying the move stream. The engine carries the complementary property for orchestration: workflow state at any point is reconstructable by replaying an append-only workflow history. Both derive correctness from one principle—deterministic replay of an immutable log.
4. **Single-threaded execution per instance.** Each workflow instance processes signals, timer fires, and activity completions one at a time, in deterministic order. This is the single-writer model; it eliminates concurrency races without a command queue or consensus protocol.

The reference architecture adopts Temporal.io, whose durable-execution model satisfies all four. Thereafter “the engine” denotes the adopted instance, and its guarantees enter the liveness proof as named lemmas (§14.5.7).

14.2 The executor as an activity

The executor—the single component permitted to mutate the ledger—validates conservation, checks idempotency by transaction ID (P5), and commits atomically. It runs as a retrievable engine *activity*: a unit of work the engine re-invokes on transient failure.

The mapping is exact on two points. **Retry.** The engine retries on transient failures (network partition, process crash, transient database error); the executor’s transaction-ID idempotency check makes a retried commit a no-op, upgrading at-least-once dispatch to exactly-once effect. Conservation violations, referential-integrity errors, and idempotency rejections are non-retryable—they signal logic errors, never transient ones, and a retry never succeeds. **Idempotency contract.** The activity’s idempotency key is the transaction ID. If that ID is already committed, the executor returns the prior result without re-applying moves, satisfying the engine’s requirement that an activity called repeatedly with the same input produce one observable effect.

14.3 The due-event scheduler

Every contractual obligation with a deterministic future date is mapped, at unit registration, to a durable timer: bond coupons, futures daily settlement, option expiry, IRS resets, CSA margin calls, managed-account resets (Table 7, deterministic-date rows). The timer is persisted by the engine and survives server restarts, worker deployments, version upgrades, and data-centre failovers. This resolves the liveness gap for dated obligations: the timer fires regardless of infrastructure failures, and idempotency guarantees it fires at most once.

A due event does not proceed on stale or unavailable market data. Each lifecycle workflow runs a data-quality check before invoking the lifecycle function; on failure it defers the event and retries after a fixed interval, the deferral itself durable across worker crashes. Market-data staleness thresholds and cross-source validation are specified in the market-data satellite specification; this section consumes that gate, it does not define it.

14.4 Concurrency: single-writer by construction

One lifecycle workflow is assigned per unit (or per wallet for per-wallet state such as futures accumulated cost). Because each instance processes events single-threaded in deterministic order, concurrent lifecycle events on the same unit serialise: two margin calls on one CSA cannot interleave, two coupons on one bond cannot race, and a daily settlement and an intraday trade on one futures contract process in strict causal order. The executor therefore never receives concurrent mutations for a unit from the orchestration layer; its idempotency check (P5) is a second line of defence against a duplicate reaching it through an activity retry.

Operations spanning units—corporate actions, close-out netting, CSA-level margin—are co-ordinated by a parent workflow whose deterministic execution fixes causal order across children; no shared mutable state exists between them. Serialisation is per unit, not global: distinct units execute in parallel, so throughput scales with the number of active units and the worker pool, not with any single unit’s lifecycle complexity.

14.5 Obligation liveness

The scheduler closes the gap only for obligations whose deadline is known at registration. A second class—*event-triggered* obligations—arises during the lifecycle with no timer at registration: an SBL recall creates a return-by deadline absent at loan inception; a collateral-eligibility change creates a substitution demand with a cut-off; a margin call creates a delivery obligation at T+1 from the call; a counterparty default triggers close-out netting on master-agreement deadlines. Handled only inside workflow code, such obligations are invisible to the framework, which cannot then distinguish “tracked” from “forgotten by the workflow author.” The obligation as a first-class object closes this gap.

14.5.1 The obligation type

Definition 14.1 (Obligation). *An obligation is a tuple $o = (id, type, source, t_d, D, \kappa)$:*

- *id*: a unique identifier, derived deterministically from the source event and obligation type.
- *type*: a member of the obligation-type enumeration (Table 7).
- *source*: the unit, agreement, or regulatory event that created it.
- *t_d*: the deadline by which the obligation must be discharged.
- *D*: the discharge predicate on ledger state. The obligation is discharged when a move set *M* journaled before *t_d* gives $D(\text{state_after}(M)) = \text{true}$.
- *κ*: the compensation action, a deterministic function computing compensating moves when the deadline passes undischarged.

The obligation record and its discharge predicate are expressed directly in the type system. The discharge predicate *D* and the compensation *κ* are *total* functions of ledger state—same state, same verdict—so a discharge decision is deterministic and replayable. *κ* returns **Just** moves when it can compute valid compensating moves and **Nothing** when it cannot, the latter forcing **DEFAULTED**.

```
-- D and kappa are total functions of ledger state (deterministic verdicts).
data Obligation = Obligation
  { obId      :: ObId          -- derived from source event +
    type
  , obType    :: ObType        -- taxonomy (obligation-types
    table)
  , obSource  :: Source        -- unit / agreement / regulatory
    event
  , obDeadline :: Time         -- t_d
  , obDischarge :: LedgerState -> Bool -- D : the discharge predicate
```

```

    , obCompensate :: LedgerState -> Maybe [Move] -- kappa : Just on success,
      Nothing fails
  }

```

The lifecycle state is split so that the type, not a convention, forbids leaving a terminal state: PENDING and ATTEMPTED are the only *live* states, and the handler consumes a live state alone. DISCHARGED, COMPENSATED, and DEFAULTED are terminal and have no exit edge—“an obligation leaves a terminal state” is unrepresentable, not merely unreachable.

```

data Live      = Pending | Attempted          -- non-terminal
data Terminal  = Discharged | Compensated | Defaulted -- terminal: no exit edge
data ObState   = Active Live | Settled Terminal -- stored per obligation

data Trigger   = DischargeSignal | DeadlineFired -- the two things that move
              o

-- Total over (Live x Trigger). ‘step’ takes only Live, so a Terminal can never
-- be transitioned. On DeadlineFired every arm returns Right (a terminal state):
-- that absence of a Left IS lemma L5 (handler totality), the handler half of P21.
step :: Obligation -> LedgerState -> Live -> Trigger -> Either Live Terminal
step o s _ DischargeSignal
  | obDischarge o s = Right Discharged
  | otherwise       = Left  Attempted
step o s _ DeadlineFired
  | obDischarge o s = Right Discharged
  | otherwise       = case obCompensate o s of
    Just _ms      -> Right Compensated -- kappa succeeded
    Nothing       -> Right Defaulted   -- kappa failed

```

No categorical name is introduced here: a record, a sum type split into live and terminal, and one total function discharge the obligations the type must meet; nothing further would buy a removed bug or removed code. The state-table transitions of §14.5.1 are exactly the cases of `step`. Behaviour, made concrete on the CSA call below (§14.5.8): `step csaVM` on a post-delivery state with `DischargeSignal` returns `Right Discharged`; on a no-delivery state with `DeadlineFired` it returns `Right Compensated`.

From	To	Condition
PENDING	ATTEMPTED	discharge attempted
PENDING	DISCHARGED	$D = \text{true}$ (early)
PENDING	COMPENSATED	$t > t_d$, κ fires
ATTEMPTED	DISCHARGED	$D = \text{true}$
ATTEMPTED	COMPENSATED	$t > t_d$, κ fires
PENDING	DEFAULTED	$t > t_d$, κ fails
ATTEMPTED	DEFAULTED	$t > t_d$, κ fails

14.5.2 Taxonomy

14.5.3 The obligation store

Definition 14.2 (Obligation store). *The obligation store $\mathcal{O}$ is the set of obligations recovered from the event log by filtering to obligation-typed entries. Each obligation is added atomically within the transaction that creates its triggering event; each state transition is a logged state-only transaction.*

The obligation store is a projection of the event log, not a second source of truth. Each change to an obligation’s state is a logged transaction (Definition 14.2); the stored value is a read cache, overwritten in place. Replaying the log to any instant rebuilds the value that held then,

Obligation type	Scope	Trigger	Compensation κ
<i>Deterministic-date (timer at registration)</i>			
Bond coupon	Unit	Timer at coupon date	Default event
Option expiry	Unit	Timer at termination date	N/A (auto-expire)
IRS reset	Unit	Timer at reset date	Default event
Futures daily VM	Unit	Cron at exchange EOD	Position liquidation
SBL term loan return	Unit	Timer at maturity	Buy-in (GMSLA 9.3)
<i>Event-triggered (timer at event time)</i>			
SBL recall return	Unit	Recall signal	Buy-in (GMSLA 9.3)
SBL manuf. dividend	Unit	Corp. action record date	Cost attribution
SBL collateral subst.	Unit	Demand / eligibility change	Shortfall escalation
CSA VM delivery	Agreement	Margin call computation	Close-out netting (ISDA)
CSA IM delivery	Agreement	Regulatory schedule	Close-out netting
Collateral substitution	Agreement	Schedule amendment	Close-out netting
SBL collateral top-up	Unit	MTM threshold breach	Recall / default
Close-out netting	Agreement	Default event	Waterfall / loss alloc.
<i>Regulatory</i>			
SFTR report	Unit	SBL lifecycle event	Regulatory penalty
SLATE report	Unit	SBL lifecycle event	Regulatory penalty
EMIR report	Unit	Trade/lifecycle event	Regulatory penalty
Settlement instruction	Unit	Settlement-type transaction	Failed settlement

Table 7: Obligation types by trigger and scope. Deterministic-date obligations are scheduled at unit registration; event-triggered obligations are created when the triggering event is processed; regulatory obligations are created alongside the business event.

so overwriting loses nothing, and a logged transaction is the only thing that changes the value. The value exists from the obligation’s creation onward.

14.5.4 Registration

Obligations register through two channels. At **unit registration**, the lifecycle function extracts dated obligations from the CDM **EconomicTerms** (coupon, reset, termination schedules), one per date; these correspond one-to-one with the scheduler’s timers (§14.3), making explicit what the timer alone left implicit—each has a defined discharge predicate and compensation action. At **event processing**, a lifecycle event that creates an obligation returns it alongside its moves and state deltas; the executor commits the obligation registration atomically with the triggering event.

Principle 14.3 (Obligation completeness). *A lifecycle function processing an obligation-creating event includes the obligation in its output. A function that omits a required obligation is incorrect, irrespective of whether its moves are valid.*

The CDM event-type enumeration bounds the test space, so this principle is enforceable by property-based testing: for each obligation-creating event type (Table 7), the generator produces events and the postcondition verifies a non-empty, well-formed obligation list. A typed return that mandates an obligation list for obligation-creating event types makes omission a compile-time error.

14.5.5 The obligation workflow

Each obligation is managed by one workflow—spawned per obligation, or embedded in the unit or agreement lifecycle workflow that owns its source. The pattern is uniform: a selector waits on a discharge signal channel and a deadline timer. On a discharge signal, the handler

checks D against ledger state; if satisfied, the obligation becomes DISCHARGED and the workflow completes, otherwise it becomes ATTEMPTED and waits. When the timer fires at t_d , the handler re-checks D ; if satisfied, DISCHARGED; otherwise it commits κ through the executor, reaching COMPENSATED on success and DEFAULTED (raising a default event) on failure. The workflow ID is derived deterministically from $o.id$, so the engine’s duplicate-start rejection prevents double-spawning.

14.5.6 Liveness invariants

Invariant 14.4 (P21—Obligation Liveness). *For every $o \in \mathcal{O}$ with deadline t_d :*

$$\forall t > t_d: \quad o.state \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}.$$

No obligation persists in PENDING or ATTEMPTED beyond its deadline.

Invariant 14.5 (P22—Obligation Conservation). *Every discharging move set and every compensation move set for o satisfies the conservation law $Q(u) = 0$ (P1). The obligation mechanism does not bypass the executor.*

Invariant 14.6 (P23—Obligation Idempotency). *Discharging the same obligation twice, by $o.id$, has no incremental effect: D is checked before moves are emitted, and if already satisfied no moves are produced. This composes with transaction-ID idempotency (P5) and lifecycle idempotency (P6).*

14.5.7 Proof of liveness

Theorem 14.7 (Obligation liveness). *Under the executor, engine, and obligation store, P21 (Invariant 14.4) holds: no obligation persists in PENDING or ATTEMPTED beyond its deadline.*

Proof. Five lemmas compose.

L1 (Registration completeness). Every obligation-creating event registers an obligation in $\mathcal{O}$. By Principle 14.3, registration is part of the lifecycle function’s output contract; the executor commits it atomically with the triggering event; property-based testing verifies a non-empty obligation list per obligation-creating event type.

L2 (Workflow creation). Every $o \in \mathcal{O}$ is managed by a workflow with a timer at t_d . The executor’s post-commit hook spawns it; the deterministic workflow ID makes the engine reject double-spawning.

L3 (Timer durability). The timer at t_d is persisted by the engine and survives server restarts, worker deployments, version upgrades, and data-centre failovers—a guarantee of the engine (§14.1).

L4 (Timer fires). The timer fires at t_d , by the engine’s durable-execution guarantee. *Prerequisite:* the engine cluster is available at t_d . Cluster availability is an operational prerequisite, not a framework property; multi-region replication mitigates but does not eliminate the dependency.

L5 (Handler totality). On firing, the handler transitions o out of PENDING. The handler is total: $D = \text{true}$ yields DISCHARGED; otherwise κ fires, yielding COMPENSATED on success and DEFAULTED (with a default event) on failure. Every branch terminates in a terminal state.

Composition. For o with deadline t_d : $o \in \mathcal{O}$ (L1); a workflow with a timer at t_d exists (L2); the timer is durable (L3) and fires (L4); the handler reaches a terminal state (L5). Hence for all $t > t_d$, $o.state \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}$. $\square$

P22 follows from P1 and executor exclusivity: every discharging and compensating move passes through the executor, which validates $Q(u) = 0$ before commit. P23 composes with P5 and P6: the obligation ID is part of the transaction source metadata, the discharge predicate prevents

re-emission, and the transaction ID prevents re-commitment. The composition is summarised below.

Property	Guarantee source
Every obligation fires	engine durable timer (L4)
No obligation fires twice	idempotency (P5, P6, P23)
Discharge preserves conservation	executor (P1, P22)
Handler reaches terminal state	workflow totality (L5)

Every obligation—including those created dynamically during the lifecycle—is thereby tracked, timed, and resolved, not only those with a deterministic registration-time date.

14.5.8 Worked example: CSA variation margin call

Alice and Bob hold a CSA over five IRS trades, threshold \$500,000, minimum transfer amount (MTA) \$250,000. The daily margin workflow computes aggregate mark-to-market of \$2.3M in Alice’s favour; required margin is $\max(0, |\$2.3M| - \$500K) = \$1.8M \geq \$250K$, so a call is generated. The call registers an obligation, committed atomically with the call event:

```
Obligation(
  id:          "CSA-AB-20260328-VM",
  type:        CSA_VARIATION_MARGIN,
  source:      CSA-AB,
  deadline:    2026-03-31T17:00 (T+1, next business day COB),
  discharge:   Bob.coll_post(USD, CSA-AB) increased by >= 1,800,000,
  compensation: CSA_CloseOutNetting(CSA-AB)
)
```

Discharge. Bob delivers \$1.8M before the deadline as a conservation-preserving move; the discharge signal satisfies D ; obligation $\rightarrow$ DISCHARGED. **Compensation.** Bob does not deliver; the timer fires; D fails; `CSA_CloseOutNetting` terminates all five trades, computes close-out values per the ISDA Master Agreement, and emits one net settlement move through the executor; obligation $\rightarrow$ COMPENSATED. Conservation on the discharge path:

$$\Delta w_{\text{Bob}}(\text{USD})[\text{own}] = -1,800,000, \quad \Delta w_{\text{Alice}}(\text{USD})[\text{coll_recv}] = +1,800,000, \\ Q(\text{USD}) = 0. \checkmark$$

14.5.9 Worked example: SBL collateral substitution

A lender demands substitution: bond XYZ is no longer eligible under the updated collateral schedule. The eligibility check registers an obligation (IBP references denote clauses of the ISLA Best Practice Handbook [1]):

```
Obligation(
  id:          "SBL-L123-SUBST-20260328",
  type:        SBL_COLLATERAL_SUBSTITUTION,
  source:      loan L123,
  deadline:    2026-03-28T15:30 (1h before DVP cut-off, IBP-170),
  discharge:   new eligible collateral posted AND old released,
  compensation: CollateralShortfallEscalation(L123)
)
```

Discharge. The borrower delivers German Bund as replacement in a four-move transaction, each move satisfying the Single-Coordinate Move Principle (Principle 15.1):

```
tau_subst = Transaction(type = SETTLEMENT):    -- txIntroduce = Nothing; applyTx
  is the sole door
Move 1: Borrower coll_post(XYZ) -= old_amt    -- release old
```



```

Move 2: Lender    coll_recv(XYZ)  -= old_amt
Move 3: Borrower coll_post(BUND) += new_amt    -- post new
Move 4: Lender    coll_recv(BUND) += new_amt

```

Per-unit conservation: $\Delta Q(\text{XYZ}) = -old_amt + old_amt = 0$; $\Delta Q(\text{BUND}) = new_amt - new_amt = 0$. Obligation $\rightarrow$ DISCHARGED. **Compensation.** The cut-off passes without delivery; the timer fires; `CollateralShortfallEscalation` marks L123 collateral-deficient in the SBL unit state, raises an exposure-breach notification, and—if the breach persists beyond the GMSLA grace period—triggers recall or default per GMSLA §10. Obligation $\rightarrow$ COMPENSATED.

14.5.10 Relationship to SBL invariants

P21 supplies the temporal guarantee underpinning the SBL invariants (P11–P20). It guarantees collateral top-up obligations resolve by deadline—delivery or compensation—so P13 (Collateral Sufficiency) cannot be violated by an untracked call. It guarantees each reportable event’s reporting obligation fires before its T+1 deadline or escalates, sustaining P16 (SFTR Completeness). It guarantees recall obligations resolve, so shares cannot remain on loan past a recall deadline without return or buy-in, sustaining P18 (Lender Ownership Invariance).

14.5.11 Property-based testing for obligations

Three properties extend the testing framework. **Registration completeness:** for random obligation-creating events, the output includes a non-empty obligation list with valid deadline, discharge predicate, and compensation action. **Discharge conservation:** every discharge move set satisfies $Q(u) = 0$. **Handler totality:** over random obligation states $\times$ {discharge-before-deadline, no-discharge, compensation-success, compensation-failure}, the handler reaches a terminal state in every case. The CDM event-type enumeration bounds the obligation-creating event space, making coverage structurally complete.

14.5.12 Assumptions and limitations

Theorem 14.7 depends on four assumptions, each at the framework boundary. **Engine availability at t_d :** an unavailable cluster delays the timer until recovery; liveness holds eventually, not instantaneously. **Executor availability:** an unavailable executor leaves the obligation PENDING under retry until recovery—a bounded delay, not a permanent failure. **External cooperation:** discharge conditions requiring counterparty action cannot be compelled; compensation is the contractual fallback. **Lifecycle-function correctness:** L1 assumes lifecycle functions register all required obligations, enforceable by property-based testing and, in a typed implementation, by a mandatory obligation list in the return type. These match the framework’s existing boundary: correctness within the ledger, given infrastructure availability and correct lifecycle functions; external actors and infrastructure availability lie outside it.

15 Generalised Positions and Securities Borrowing and Lending

The scalar model of Sections 2–16—one number per (wallet, unit) pair—is complete for every instrument whose ownership and possession never diverge: cash, equities, derivatives, swaps. It fails the instant they diverge: securities lending, repo, pledged collateral. This section generalises the scalar balance to a six-coordinate vector that resolves the failure, and derives Securities Borrowing and Lending (SBL) from the framework’s primitives. Repo, collateral management beyond SBL, and novation are open problems (Section 19).

15.1 Where the Scalar Model Breaks

Alice owns 1,000 VOD: $w_{\text{Alice}}(\text{VOD}) = 1,000$, a single number, and complete. She lends 400 to Bob. A scalar balance now admits only three encodings, each wrong:

1. *Transfer*: $w_{\text{Alice}} = 600$, $w_{\text{Bob}} = 400$. Conservation holds, but Alice’s PnL drops 40%—wrong: under IFRS 9 §3.2.6 the lender does not derecognise lent securities.
2. *Do not transfer*: $w_{\text{Alice}} = 1,000$, $w_{\text{Bob}} = 0$. Bob possesses 400 shares with no record of them.
3. *Credit both*: total = 1,400 against 1,000 in the external world. Conservation is violated.

A scalar balance cannot encode two operational states at once. After the loan Alice holds 600 available and 400 on loan. “On-loan” is a coordinate because “lend” is a physical action that changes operational status without changing ownership.

15.2 The Generalised Coordinate Vector

A *coordinate* is a stored quantity that passes the *physical-action test*: a distinct physical action modifies it and only it. A quantity fixed by other coordinates through an algebraic identity is a *projection*—computed on read, never stored, never written by a move.

Principle 15.1 (Single-Coordinate Move). *An atomic move modifies exactly one coordinate of one unit across the two entities it connects: one move, one coordinate, one unit, source and destination. A move is the smallest state change. Any operation requiring multiple coordinate changes is a transaction—a list of moves executed atomically.*

The scalar core is the one-coordinate case of this principle: there a move modifies one wallet of one coordinate per entity. The vector model carries the same guarantee across six coordinates.

Plainly: one number per holding becomes six. Each records a distinct operational fact about the same shares—how many the entity owns, how many it has lent out, how many it has borrowed, and how much collateral it has posted, received, and re-used. Only ownership carries economic value; the other five record where the shares physically sit. The familiar scalar balance is the degenerate case in which only the ownership number is non-zero.

Definition 15.2 (Position Vector). *For entity e and unit u , the position vector $\vec{w}_e(u) \in \mathbb{R}^6$ is*

$$\vec{w}_e(u) = (w_e^{\text{own}}(u), w_e^{\text{onloan}}(u), w_e^{\text{borr}}(u), w_e^{\text{coll-post}}(u), w_e^{\text{coll-recv}}(u), w_e^{\text{coll-rehyp}}(u)).$$

Each coordinate passes the physical-action test:

- *own*: economic ownership. Modified by buy/sell. May be negative (short). Drives PnL.
- *onloan*: securities lent out. Modified by lend/recall-return. Lender retains ownership.
- *borr*: borrowed shares received (the return obligation count). Modified by borrow-receive/return. Non-negative. Unchanged when the borrower sells.
- *coll_post*: collateral delivered to a lender or triparty agent. Modified by post/release.
- *coll_recv*: collateral held from the borrower. Modified by receive/release.
- *coll_rehyp*: non-cash collateral re-used by the taker. Modified by rehypothecate/recall. Tracked for SFTR Article 15.

Definition 15.3 (Available Inventory). *For every entity e and lendable unit u , available inventory is the read-time projection*

$$\text{avail}(e, u) \stackrel{\text{def}}{=} \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$$

—what you can trade equals what you own, less what you have lent, plus what you have borrowed. Because avail is never stored, the identity holds by definition: no stored value can drift from the coordinates that determine it.

The model's computational content is three pieces: the six-field record, `avail` as a total projection, and `applyMove` writing one coordinate.

```
-- Qty: exact minor units, the additive abelian group reused from Sec.2.
-- (One scale; own may be negative, the operational coords are guarded
-- at value level, not by type -- see below.)
newtype Qty = Qty Integer deriving (Eq, Ord, Show)
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid Qty where mempty = Qty 0
qneg :: Qty -> Qty
qneg (Qty n) = Qty (negate n)

-- The position vector: one field per coordinate (Def. position-vector).
data Position = Position
  { own      :: !Qty -- economic ownership; may be negative (short)
  , onloan   :: !Qty -- lent out; lender retains ownership
  , borrr    :: !Qty -- borrowed; the return-obligation count
  , collPost :: !Qty -- collateral delivered
  , collRecv :: !Qty -- collateral held
  , collRehyp :: !Qty -- collateral re-used
  } deriving (Eq, Show)

zeroPos :: Position -- the never-held position
zeroPos = Position mempty mempty mempty mempty mempty mempty

-- avail: a TOTAL projection, computed on read, never stored (Def. avail,
-- P20). It cannot drift because no value holds it -- it IS this function.
avail :: Position -> Qty
avail p = own p <> qneg (onloan p) <> borrr p

-- A move names exactly ONE coordinate. The Coord tag is what makes the
-- Single-Coordinate Move Principle hold by construction: there is no
-- constructor that writes two fields.
data Coord = Own | OnLoan | Borr | CollPost | CollRecv | CollRehyp
  deriving (Eq, Show)

applyMove :: Coord -> Qty -> Position -> Position
applyMove Own q p = p { own = own p <> q }
applyMove OnLoan q p = p { onloan = onloan p <> q }
applyMove Borr q p = p { borrr = borrr p <> q }
applyMove CollPost q p = p { collPost = collPost p <> q }
applyMove CollRecv q p = p { collRecv = collRecv p <> q }
applyMove CollRehyp q p = p { collRehyp = collRehyp p <> q }
```

avail is a group homomorphism—meaning the effect of any move on availability is fixed by the coordinate it writes: $\text{avail}(\text{applyMove } c \ q \ p) = \text{avail}(p) \diamond \delta_c(q)$ with $\delta_{\text{Own}} = \delta_{\text{Borr}} = q$, $\delta_{\text{OnLoan}} = \text{qneg } q$, and $\delta_c = \text{mempty}$ for the three collateral coordinates. So a *stored* availability could only ever drift; the projection cannot. After Alice lends 400 (`applyMove OnLoan (Qty 400)` on $(1000, 0, 0, 0, 0, 0)$) the record reads $(1000, 400, 0, 0, 0, 0)$ and `avail` returns `Qty 600`—ownership untouched, availability down by the loan.

Remark 15.4 (FORMALIS clearance). *The listing is FORMALIS-cleared, no CRITICAL or HIGH findings. Discharged by construction: `applyMove` is total (Coord matched exhaustively; Qty is a total group, so no input is rejected and none diverges) and deterministic; the Coord tag makes a two-coordinate move unrepresentable, discharging Principle 15.1; `avail` is a pure function of three stored fields, so P20 (Available-Inventory Identity) holds with no stored value to violate. FORMALIS noted—and the comment records—that non-negativity of `onloan`, `borrr`, and the collateral coordinates is not typed away (they share Qty with `own`, which is legitimately*

negative). It is a value-level pre-condition: the lending gate $\text{avail}(e, u) \geq Q$ of Section 15.5, enforced where the transaction is built, not here.

Remark 15.5 (Special cases).

- *Pure holder/lender* ($\text{borr} = 0$): $\text{avail} = \text{own} - \text{onloan}$.
- *Non-SBL entity* (all SBL coordinates zero): $\text{avail} = \text{own}$.
- *Borrower who sold Q of N borrowed* ($\text{own} = -Q$, $\text{borr} = N$): $\text{avail} = N - Q$.

Why own is a coordinate, not a projection. (1) PnL is a direct lookup—no summation, no regime-dependent read logic; (2) for non-lendable instruments only own is non-zero, so the vector is a scalar; (3) own is not derivable from the five operational coordinates—two states with identical operational vectors but distinct own exist whenever a borrower has sold some but not all borrowed shares; (4) buy/sell is a distinct physical action touching no other coordinate.

Proposition 15.6 (Conservation in the Generalised Model). *Each move subtracts q from one (entity, coordinate) slot and adds q to another, so the algebraic sum over all slots is preserved—the $\text{src} -= q$; $\text{dst} += q$ pattern of the core framework (Section 2). For the physical-quantity check, substitute the avail projection. For each unit u the lender’s $+\text{onloan}$ and the per-relationship virtual wallet’s $-\text{onloan}$ cancel, leaving five terms per real entity:*

$$\sum_{e \in \mathcal{E}} (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{borr}] + \vec{w}_e(u)[\text{coll_post}] + \vec{w}_e(u)[\text{coll_recv}] + \vec{w}_e(u)[\text{coll_rehyp}]) = 0,$$

$\mathcal{E}$ ranging over real entities and virtual wallets. The cancellation of onloan encodes the physical fact that lent shares are counted once—as the borrower’s borr .

Portfolio value uses own directly: $V_e(t) = \sum_u \vec{w}_e(u)[\text{own}] \cdot P_t(u)$ —the scalar formula of Section 5 with own in place of the scalar balance, no projection or conditional logic.

In-flight collateral. Trade-date accounting updates the position vector at execution; the unsettled delivery lives in the virtual wallet (custodian contra-entry). For SBL prepay collateral (IBP-177; IBP references denote clauses of the ISLA Best Practice Handbook³), instructed before the loan settles, the in-flight collateral is a virtual-wallet entry that resolves on settlement to a move writing coll_recv (lender) or coll_post (borrower). Custodian reconciliation: $\text{avail}(L, u) + \vec{w}_L(u)[\text{onloan}] + \text{inflight}(L, u) \stackrel{?}{=} \text{depot}(u)$, which on substituting avail simplifies to $\vec{w}_L(u)[\text{own}] + \vec{w}_L(u)[\text{borr}] + \text{inflight}(L, u) \stackrel{?}{=} \text{depot}(u)$.

15.3 Graceful Degeneration to the Scalar Model

Remark 15.7 (Non-lendable instruments). *For non-lendable instruments—OTC derivatives, listed options and futures, FX forwards, credit derivatives, cash—only own is non-zero: $\vec{w}_e(u) = (q, 0, 0, 0, 0, 0)$. The avail projection yields q , valuation reduces to one lookup per unit, and the scalar model of Sections 2–16 is recovered exactly. The generalisation is invisible when not needed.*

15.4 The Lending Resolution: Named Wallets

After Alice lends 400 VOD to Bob:

$$\vec{w}_{\text{Alice}}(\text{VOD}) = (1000, 400, 0, 0, 0, 0), \quad \vec{w}_{\text{Bob}}(\text{VOD}) = (0, 0, 400, 0, 0, 0).$$

Ownership is unchanged; $\text{avail}_A = 1000 - 400 + 0 = 600$, $\text{avail}_B = 0 - 0 + 400 = 400$. A per-relationship virtual wallet absorbs the delivery contra-entry, $w_{A \leftrightarrow B}^{\text{virtual}}(\text{VOD}) = -400$. Conservation by Proposition 15.6:

$$\underbrace{1000}_{\text{own}_A} + \underbrace{0}_{\text{borr}_A} + \underbrace{0}_{\text{own}_B} + \underbrace{400}_{\text{borr}_B} + \underbrace{(-400)}_{\text{virtual}} + \underbrace{(-1000)}_{\text{custodian}} = 0.$$

³International Securities Lending Association, “ISLA Best Practice Handbook,” various editions.

Alice’s PnL reads $\vec{w}_{\text{Alice}}(\text{VOD})[\text{own}] = 1,000$, correct.

Reclassification vs. transfer. A reclassification is a two-move transaction within one entity (one coordinate down, another up—e.g. rehypothecation: $\text{coll_recv} \downarrow, \text{coll_rehyp} \uparrow$). A transfer is a move between entities. The primitive is identical; the distinction is entity identity. Lending writes onloan; the drop in avail is read, not written.

15.5 Analytical Projections and Encumbrance

Definition 15.8 (Projections). *All projections are functions of the six stored coordinates; none is stored or written by a move.*

$$\begin{aligned} \text{avail}(e, u) &= \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}] && \text{tradeable inventory} \\ \text{possess}(e, u) &= \text{avail}(e, u) + \vec{w}_e(u)[\text{coll_recv}] && \text{physical custody} \\ \text{encumb}(e, u) &= \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{coll_post}] && \text{owned, not tradeable} \end{aligned}$$

An entity may sell q only if $\text{avail}(e, u) \geq q$: a scalar balance of 1,000 passes a sell of 500, but with 700 on loan only 300 are available. Available to lend: $\text{AvailableToLend}(e, u) = \text{avail}(e, u) - \text{reserved}(e, u)$.

15.6 The SBL Smart Contract

Principle 15.9 (SBL Representability). *SBL is a smart contract over existing primitives. It adds: (1) an SBL loan unit type in the Unit Store; (2) a `legal_regime` field; (3) invariants P11–P20 (Section 15.18). It changes neither the move primitive, the conservation law, nor the execution engine. Every SBL lifecycle event decomposes into moves satisfying Principle 15.1.*

Why SBL is native, not a companion system. The decisive argument is double-lending prevention. A companion system reintroduces a TOCTOU race: between querying inventory and executing the loan, another process lends the same shares. Cross-boundary conservation becomes unprovable, collateral becomes invisible (breaking P13, Section 15.18), exposure requires a two-system join, and the companion “reserved” bucket discards load-bearing information.

The loan unit’s lifecycle state is held in `UnitStatus` and read identically by every holder:

```
SBLUnitState:
  loan_id, lender, borrower, agent
  isin, quantity, original_qty, term_type, maturity_date
  fee_rate, rebate_rate
  collateral_type, margin_pct, haircut_pct, collateral_ccy
  triparty_agent, legal_regime, rehyp_consent
  lifecycle_stage, settlement_status, recall_date, recall_qty
  sftr_uti, slate_loan_id, execution_ts, trade_date
  last_mark_date, accrued_fee, fee_accrual_log
```

Remark 15.10 (Loan state is a projection of the log). *UnitStatus holds one value per unit, shared—read identically by every holder. Its value changes over time, but UnitStatus is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then. Nothing is lost by overwriting, and no path other than a logged event changes the value. A value exists from registration onward. The loan unit’s `lifecycle_stage`, `settlement_status`, and `accrual` fields are this cache; the SBL state machine (Section 15.7) is the function the log replays.*

The SBL contract is a pure function $f_{\text{SBL}} : (u_\ell, \text{state}_t(u_\ell), e, \text{data}) \rightarrow (\tau, \text{state}_{t+1}(u_\ell))$.

Loan unit valuation (bond analogy). The loan unit is priced: lender holds +1, borrower −1, conserving to zero. Its price is the accrued unpaid fee, $P_t(u_{\text{loan}}) = \text{loan_value}(t) \times \text{fee_rate} \times$

DCF(last_settle, t). Fee payment is a cash move that resets the accrual: the price drops to zero, the cash move offsets, net PnL from payment is zero—as for a bond coupon. No standalone fee-accrual term enters the PnL formula. Under IFRS 9 §3.2.6 the lender does not derecognise lent securities, because the risks and rewards of ownership are not transferred; hence lent securities remain in own.

15.7 SBL State Machine

The state machine is total: every (state, event) pair yields a valid transition or an explicit rejection. Terminal states are RETURNED, CANCELLED, DEFAULTED; $S_{\text{live}} = \{\text{ACTIVE}, \text{RECALLED}, \text{PARTIALLY_RETURNED}\}$.

From	Event	To	Moves Generated
—	new_loan	PENDING	None
PENDING	loan_settled	ACTIVE	Securities + collateral
PENDING	cancel	CANCELLED	None
ACTIVE	recall	RECALLED	None (notification)
ACTIVE/RECALLED/PART. RET.	partial_return	PART. RET.	Partial securities + collateral
ACTIVE/RECALLED/PART. RET.	full_return	RETURNED	Remaining securities + collateral
S_{live}	rate_change	S_{live}	State-only
S_{live}	mark_to_market	S_{live}	Margin call moves (if threshold)
S_{live}	collateral_sub	S_{live}	Paired collateral swap
S_{live}	corp_action	S_{live}	Manufactured payment
S_{live}	reallocation	S_{live}	State-only (lender change)
ACTIVE	maturity	RETURNED	Full securities + collateral
S_{live}	buy_in	RETURNED	Buy-in + close-out moves
any non-terminal	default	DEFAULTED	Close-out moves per GMSLA

15.8 Atomic Moves for SBL Lifecycle Events

Each lifecycle event is a move-carrying Transaction ($\text{txIntroduce} = \text{Nothing}$, $\text{txAppend} = \text{Nothing}$): it operates on units already registered, introducing no unit and appending no terms. Its txMoves are the ordered moves below, each touching one coordinate per entity; conservation holds by construction from the signed edge-sum, so there is no separate validation step. Loan value (IBP-163): $\text{LV} = \lceil Q \times P \times M\% \times \chi \rceil_{0.01}$.

Loan initiation (four moves):

```

tau_new.txMoves:
    Move 1: Lender  onloan  += Q
    Move 2: Borrower borr   += Q
    Move 3: Borrower coll_post += LV
    Move 4: Lender  coll_recv += LV
-- txIntroduce = Nothing
-- securities, u_s
-- collateral, ccy

```

Move 1 draws from $w_{L \leftrightarrow B}^{\text{virtual}}$; Move 2 adds from it; Moves 3–4 pair (cash collateral writes own instead of coll_post, the virtual wallet absorbing the non-cash contra-entry). No move writes the lender's own: P18 holds. After: $\text{avail}_L = 1000 - 400 + 0 = 600$ (down Q), $\text{avail}_B = 0 - 0 + 400 = 400$ (up Q).

Recall and return. Collateral release on partial return: $\text{LV}_{\text{new}} = \lceil (Q_{\text{out}} - Q_{\text{ret}}) \times P \times M\% \times \chi \rceil_{0.01}$, $C_{\text{release}} = C_{\text{held}} - \text{LV}_{\text{new}}$.

```

tau_return.txMoves:
    Move 1: Borrower borr   -= Q_ret
    Move 2: Lender  onloan  -= Q_ret
    Move 3: Lender  coll_recv -= C_rel
    Move 4: Borrower coll_post -= C_rel
-- txIntroduce = Nothing

```

Lender's avail rises by Q_{ret} (onloan $\downarrow$); borrower's falls by Q_{ret} (borr $\downarrow$).

Short sale. Bob sells 400 borrowed VOD to Carol—a standard buy/sell writing only own:

```
tau_sell.txMoves:                                -- txIntroduce = Nothing
  Move 1: Bob    own -= 400
  Move 2: Carol  own += 400
```

Bob's borr is untouched (it tracks the obligation, not possession): $\text{avail}_B = -400 - 0 + 400 = 0$. Bob has consumed his available inventory.

Mark-to-market margin call (one move per entity for the collateral unit):

```
tau_margin.txMoves:                             -- txIntroduce = Nothing
  Move 1: Borrower coll_post += delta_LV
  Move 2: Lender   coll_recv += delta_LV
```

Cash collateral writes the borrower's own; paired moves sum to zero.

Other events. Collateral substitution (IBP-170): four moves—release old, post new. Manufactured dividend: one cash move per entity writing own, dividend $\times Q_{\text{onloan}}$. Buy-in (GMSLA 9.3): market purchase, close-out, collateral release, cost attribution—each move satisfying Principle 15.1.

15.9 Title Transfer vs Security Interest

The legal regime determines what the collateral taker may do with received collateral:

Regime	Collateral Treatment	Rehypothecation
TITLE_TRANSFER (GMSLA 2000/2010)	Receiver owns collateral	Permitted by default
SECURITY_INTEREST (GMSLA 2018)	Pledged; poster retains ownership	Forbidden absent GMSLA Schedule consent
US_15C3_3	Regulated possession/control	Capped at 140% of customer debit

The wallet structure and the avail projection are identical under every regime. The regime affects only the PnL projection. The `legal_regime` is set at inception and immutable.

Remark 15.11 (Pledge and the own coordinate). *Under security interest the poster retains economic ownership of pledged collateral, so*

$$V_e^{\text{SI}}(t) = \sum_u (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{coll_post}]) \cdot P_t(u),$$

whereas under title transfer $V_e^{\text{TT}}(t) = \sum_u \vec{w}_e(u)[\text{own}] \cdot P_t(u)$. The own coordinate therefore does not alone capture economic ownership under pledge: ownership splits between own (lent securities) and coll_post (pledged collateral). The framework keeps own a single, regime-independent meaning (ownership from buy/sell/lend) and makes the pledge adjustment explicit in the valuation function—as the dirty-price function carries bond accrual rather than mutating the balance.

15.10 Collateral and Margin

Method	Mechanism	Ledger Representation
Cash rebate	Cash collateral; lender pays rebate	Single cash move per loan
Non-cash bilateral	Securities B→L directly	Securities to $w_L^{\text{coll-recv}}$; haircut applied
Non-cash triparty	Securities to triparty agent	RQV instruction; daily agreement (IBP-189)
Cash pool (standard)	Single pool per cpty-ccy	Aggregate marking; single margin move (IBP-322)
Cash pool (EU)	Per-loan margin tracking	Individual marks; net movement (IBP-323)
Uncollateralised	No collateral	No collateral moves (IBP-326)

Exposure (IBP-163): $LV = \lceil Q \times P_{\text{close}} \times M\% \times \chi \rceil_{0.01}$. Margin calls are conservation-preserving transfers between the borrower’s `coll_post` (or `own` for cash) and the lender’s `coll_recv`. Rehypothecation is a reclassification: two moves carrying `coll_recv` ↓, `coll_rehyp` ↑ within the taker’s position. Substitution demands and top-up obligations with deadlines are formalised as obligations under the obligation workflow (Section 14.5; worked SBL substitution at Section 14.5.9).

15.11 Short Selling and Inventory

Short selling creates a negative own coordinate—the defining feature of a short position. Alice borrows 1,000 NVDA from Bob, then sells 500:

$$\text{borrow: } \vec{w}_{\text{Bob}} = (1000, 1000, 0, 0, 0, 0), \quad \vec{w}_{\text{Alice}} = (0, 0, 1000, 0, 0, 0),$$

$$\text{after sale: } \vec{w}_{\text{Alice}} = (-500, 0, 1000, 0, 0, 0), \quad \text{avail}_{\text{Alice}} = -500 - 0 + 1000 = 500.$$

The sale writes only `own`; `borr` stays at 1,000 because the return obligation is unchanged. The negativity lives in the economic dimension alone—no operational coordinate goes negative.

Locate and available-to-lend. EU Short Selling Regulation (SSR) Article 12(1) requires a reasonable expectation of borrowing before short selling. The gate:

```
def available_to_lend(entity, security, t):
    pos = position_vector(entity, security, t)
    avail = pos.own - pos.onloan + pos.borr
    reserved = sum(loc.quantity for loc in active_locates(entity, security, t))
    regulatory_hold = regulatory_restrictions(entity, security, t)
    return max(0, avail - reserved - regulatory_hold)
```

FTD close-out: Reg SHO Rule 204 (T+3); CSDR daily cash penalties.

15.12 Cash Collateral Reinvestment

In US cash-collateral lending under Rule 15c3-3, the lender reinvests received cash collateral. Two economic positions then exist and must be tracked independently:

1. **Collateral obligation:** the duty to return cash collateral on termination—a fixed amount (mark-adjusted) in the loan unit state (`collateral_amount`), reflected in the lender’s `coll_recv`.
2. **Reinvestment asset:** the money-market fund (MMF) units, paper, or repo bought with the cash—a separate position in the lender’s `own` for the reinvestment instrument, carrying its own market risk.

Reinvestment is a standard buy/sell transaction: cash leaves `coll_recv` and the reinvestment unit enters `own`, the MMF taking the cash for its units (conservation holds per leg). The cash collateral is thereby transformed: `coll_recv(USD)` falls to zero while `own(MMF)` holds the reinvestment, and the return obligation persists fixed in the loan unit state. The separation is the point: the reinvestment asset may move in value while the obligation does not.

Reinvestment risk. If the MMF breaks the buck (NAV below \$1.00), the reinvestment position falls below the fixed obligation; the gap is uncollateralised credit risk borne by the lender. This realised in September 2008 (Reserve Primary Fund), and SEC Rule 2a-7 reforms have not eliminated it for institutional prime MMFs.

Unwind on return. Termination sells the reinvestment units back for cash, restoring `coll_recv`, which the loan-return transaction then releases. CDM: the reinvestment buy/sell is a standard **Transfer** (no gap); the obligation maps to `SecurityLendingPayout.collateral`. The full numeric walkthrough is in Appendix I (US SEC 15c3-3 worked example).

15.13 On-Lending Chains

Borrowed shares may be re-lent, forming on-lending chains of independent bilateral loans; the six-coordinate model handles them unmodified. Alice owns 1,000 XYZ, lends 1,000 to Bob (ℓ_1); Bob re-lends 500 to Charlie (ℓ_2). Each loan is its own unit with its own fee, collateral, and reporting. Columns abbreviate the coordinates: `onl` = onloan, `c_p` = `coll_post`, `c_r` = `coll_recv`, `c_rh` = `coll_rehyp`; `avail` = `own` − `onl` + `borr` is the read projection.

Entity	own	onl	borr	c_p	c_r	c_rh	avail
Alice	1,000	1,000	0	0	C_1	0	0
Bob	0	500	1,000	C_1	C_2	0	500
Charlie	0	0	500	C_2	0	0	500

Verification: $\text{avail}_{\text{Alice}} = 0$ (all lent), $\text{avail}_{\text{Bob}} = 0 - 500 + 1000 = 500$, $\text{avail}_{\text{Charlie}} = 500$; system `own` = 1,000 (physical shares unchanged).

P18 scope. P18 (Lender Ownership Invariance) fixes `own` across lending for *pure* lenders (`borr` = 0). It applies to Alice, not to Bob (`borr` = 1,000, `onloan` = 500): Bob’s exposure is through `borr`. For an intermediary re-lender (`own` = 0, `borr` > 0) the binding identity is `onloan` ≤ `borr`—an entity cannot re-lend more than it has borrowed plus owns—which follows from the lending pre-condition $\text{avail}(e, u) \geq Q_{\text{lend}}$, i.e. `own` − `onloan` + `borr` ≥ Q_{lend} .

Cascade recall. When Alice recalls 1,000 from Bob: Bob can return `avail` = 500 at once and must recall the remaining 500 from Charlie; Charlie returns 500 (Bob’s vector (0, 0, 1000, C_1 , 0, 0), `avail` = 1000); Bob returns 1,000 to Alice. The cascade carries settlement-timing risk—a downstream failure fails the upstream return, triggering CSDR penalties or Reg SHO close-out. It is orchestrated as a saga—a long-running workflow split into steps, each with a compensating action that undoes it on failure (Section 14.5): compute `avail`, return immediately what is available, recall the shortfall from downstream loans where the entity is lender (itself possibly cascading), await downstream returns under a settlement-deadline timeout, then execute the upstream return. On timeout the saga compensates by market buy-in with cost attribution per GMSLA 9.3 (IBP-328). Individual recalls and returns map to CDM **QuantityChange/Termination**; the coordinated cascade has no CDM equivalent (gap)—it is orchestration, not a single business event.

15.14 Locates

A *locate* is a pre-trade confirmation by a lender (or agent lender) to a prospective short seller that the requested securities are identified and deliverable. A locate is *not* a ledger move: no securities or collateral move, no coordinate changes, conservation is not engaged. Conservation

engages only when the locate is converted into a borrow (the loan-initiation transaction of Section 15.8).

Regulatory basis. EU SSR Article 12(1)(c): a person may short sell only where a third party has confirmed the share is located and measures taken give a reasonable expectation of settlement (ESMA 70-448-10 proposes reinforcing the locate provider’s commitment). US Reg SHO Rule 203(b)(1): before a short sale the broker-dealer must borrow, arrange to borrow, or have reasonable grounds to believe the security can be borrowed for delivery when due.

Lifecycle. Request $(u, Q) \rightarrow$ confirmation against AvailableToLend (Section 15.5), valid for a fixed period set at confirmation $\rightarrow$ conversion to a borrow or expiry.

Modelling. A locate is registered as a time-limited unit (TTL) in the Unit Store: no position-vector entries; it exists for audit and aggregate tracking and transitions to a terminal state on expiry via the due-event scheduler. This satisfies the EU SSR / ESMA 70-448-10 record-keeping obligation natively, which a purely external trading-system locate does not.

Over-location prevention. A lender must not confirm more locates than inventory supports. The gate is the same `available_to_lend` as the short-sale check, with outstanding locates in reserved:

```
def confirm_locate(lender, security, quantity, t):
    atl = available_to_lend(lender, security, t)
    outstanding = sum(loc.quantity for loc in active_locates(lender, security, t))
    return CONFIRMED if atl - outstanding >= quantity else DECLINED
```

This yields one deduction chain: owned $\rightarrow$ minus on-loan $\rightarrow$ plus borrowed $\rightarrow$ minus reserved (locates + regulatory holds) = available to lend. **P14 enforcement** is the short-sell precondition: a valid, unexpired locate covering the security and quantity must exist, else the sell is rejected. Locate has no CDM equivalent (gap, Section 15.17); a converted locate may emit a non-economic CDM `BusinessEvent` (no moves) recorded in move-stream metadata for the record-keeping obligation.

15.15 Cross-Border Regulatory Treatment

A single loan can span jurisdictions, triggering SFTR and FINRA SLATE simultaneously. A loan from BNP Paribas (EU, SFTR) to Goldman Sachs (US, SLATE) carries both `sftr_reportable/sftr_utl` and `slate_reportable/slate_loan_id` in the unit state. Both workflows fire per lifecycle event, independently: distinct schemas, identifiers (LEI for SFTR; LEI + FINRA Loan ID for SLATE), and deadlines (T+1 for SFTR; same-day before 8pm ET for SLATE).

The `legal_regime` is fixed at inception by the governing agreement:

Governing Agreement	Use	legal_regime
GMSLA 2010 (title transfer)	EU/UK bilateral	TITLE_TRANSFER
GMSLA 2018 (security interest)	EU/UK pledge	SECURITY_INTEREST
MSLA	US bilateral	US_15C3_3
PBA	US prime broker	US_15C3_3

Venue, not domicile. The locate regime is fixed by the trading venue: EU SSR Article 12(1) for EU venues, US Reg SHO Rule 203(b)(1) for US venues; a cross-border short may face both. Settlement discipline is fixed by the settlement venue: a loan settled through an EU CSD faces CSDR cash penalties; one settled through DTC faces Reg SHO Rule 204 (T+3) and FINRA Rule 4320—each independent of the counterparty’s domicile.

15.16 External Reconciliation

Participant	View	Ledger Mapping
Beneficial owner	On-loan; collateral; fee income	w_L^{onloan} , $w_L^{\text{coll-recv}}$, fee accrual
Agent lender	Pool across BOs	Aggregate own
Borrower	Borrowed; collateral posted; fee expense	w_B^{borr} , avail(B , ccy)
Custodian	Depot position	Nostro/depot wallet

Contract compare (IBP-153/155): daily at COB. Custodian reconciliation, substituting the avail projection, is $\vec{w}_L(u)[\text{own}] + \vec{w}_L(u)[\text{borr}] + \text{inflight}(L, u) \stackrel{?}{=} \text{depot}(u)$ —what you own plus what you borrowed plus what is in flight equals what the custodian holds. CSDR penalty attribution is tracked per instruction. Settlement-instruction generation branches by collateral type: non-cash bilateral yields two FOP instructions; cash rebate one FOP plus one CASH; cash pool one FOP.

15.17 CDM Alignment and Regulatory Reporting

SBL Event	CDM Type	SFTR Action
New loan	ContractFormation	NEWT
Rate change	TermsChange	MODI
Partial return	QuantityChange	MODI
Full return	Termination	ETRM
Recall	<i>No CDM equivalent (gap)</i>	—
Margin call	Transfer (collateral)	COLU
MTM	Observation + Valuation	VALU
Substitution	Transfer $\times 2$	COLU
Locate	<i>No CDM equivalent (gap)</i>	—
Rehypothecation	<i>No CDM equivalent (gap)</i>	COLU

CDM gaps: Recall, Locate, Rehypothecation. SFTR: both counterparties report, UTI per ESMA waterfall. FINRA SLATE: same-day reporting, 24 required fields. Tokenised securities posted as collateral occupy coll_post/coll_recv with the token unit (not the underlying, Section F.4); the haircut covers both the underlying’s market risk and the tokenisation platform’s operational risk. CDM has no native type for tokenised-collateral eligibility—a fourth gap.

15.18 SBL-Specific Invariants (P11–P20)

SBL adds ten invariants that narrow the valid states without relaxing any core invariant: every transaction satisfying P11–P20 also satisfies P1–P10. Worked correctness witnesses are in Appendix H (European GMSLA 2010) and Appendix I (US SEC 15c3-3); eight-scenario avail verification in Appendix G.

Invariant 15.12 (P11–P20). *11. P11—Paired Legs. Every SBL settlement transaction carries both a securities and a collateral leg (exceptions: uncollateralised, cash pool, tri-party RQV).*

12. P12—On-Loan Consistency.

$$w_e^{\text{onloan}}(u) = \sum_{\substack{u_\ell: \text{lender}(u_\ell)=e \\ \text{isin}(u_\ell)=u, \text{stage}(u_\ell) \in S_{\text{live}}}} \text{quantity}(u_\ell).$$

13. P13—Collateral Sufficiency. Held collateral covers loan value; three formulations—per-loan cash, cash pool, non-cash bilateral.

14. **P14—Locate Before Short.** Every short sale satisfies the locate pre-condition.
15. **P15—Partial Return Monotonicity.** Loan quantity is monotonically non-increasing.
16. **P16—SFTR Completeness.** Each reportable event has exactly one SFTR report.
17. **P17—Settlement State Monotonicity.** Settlement states advance monotonically.
18. **P18—Lender Ownership Invariance.** For pure lenders under title transfer, excluding buy-in, $\vec{w}_e(u)[\text{own}]$ is constant across lending operations. By construction under Principle 15.1: initiation and return write the lender's onloan, never own; MTM, substitution, and dividend write collateral or cash coordinates only. Buy-in is excluded because its market purchase writes own.
19. **P19—Rehypothecation Regime Compliance.** Rehypothecation is validated against the legal regime.
20. **P20—Available Inventory Identity.** For all e and lendable u , $\text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$. Under the six-coordinate model this is a definition, not a cross-check: avail is computed on read from three stored coordinates and cannot be violated because it is never stored.

Key Invariants and Consequences

Obligations and the six-coordinate position vector generalise the scalar core without relaxing any of P1–P10: every transaction satisfying P11–P23 also satisfies P1–P10. Each guarantee carries its canonical number in §16.

- **Conservation generalises (P1).** Each move writes exactly one coordinate of one unit across the two entities it connects (Single-Coordinate Move Principle); the signed sum over all coordinate slots is preserved, so the scalar law extends unchanged to the six-coordinate vector. (§15, Proposition 15.6.)
- **Available-inventory identity (P20).** $\text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$, computed on read from three stored coordinates and never stored, hence not violable. (§15.18, Invariant 15.12.)
- **SBL narrows without relaxing (P11–P19).** Paired legs (P11), on-loan consistency (P12), collateral sufficiency (P13), locate before short (P14), partial-return monotonicity (P15), and lender ownership invariance under title transfer (P18, by construction—lending writes *onloan*, never the lender’s *own*) narrow the valid states. (§15.18.)
- **Obligation liveness (P21).** For every obligation o with deadline t_d : $\forall t > t_d, o.\text{state} \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}$; none persists in PENDING or ATTEMPTED past its deadline. (§14.5.6, Invariant 14.4.)
- **Obligation conservation (P22).** Every discharging and every compensating move set satisfies $Q(u) = 0$; the mechanism does not bypass the executor. Follows from P1. (§14.5.6, Invariant 14.5.)
- **Obligation idempotency (P23).** Discharging the same obligation twice by $o.\text{id}$ has no incremental effect; it composes with P5 and P6. (§14.5.6, Invariant 14.6.)

Part VI

Formal Properties, Limitations, and Context

16 Invariants, Conservation Laws, and Property-Based Testing

This section is the consolidation point for the framework’s correctness claims. Properties argued elsewhere through worked examples are restated here once, as precise testable invariants, in one reconciled numbering. They serve two roles at once: the design specification—the system is correct exactly when they all hold—and the test oracle against which property-based testing checks every generated input. Executable pre/postcondition forms attach in Appendix B; this section states the prose and the cross-links.

16.1 Conservation as Central Invariant and Universal Oracle

Conservation states that nothing is created or destroyed inside the ledger: every quantity that leaves one wallet arrives in another, so across all wallets the holdings of each unit always cancel to zero. This is the one property every generated test checks.

The conservation law $Q(u) = \sum_w w_t(u) = 0$, for every unit u and time t , is the central invariant. It is an algebraic consequence of move semantics ($\text{src} \text{--} q; \text{dst} \text{+} q$) and holds for every transaction by construction (Section 2); the conserved-charge analogy is exact, the charge being the total quantity of each unit.

Conservation governs *quantities per unit*, not values. An FX trade conserves EUR and USD quantities separately; at any rate other than the trade rate the value of EUR received differs from that of USD paid. Value changes from price movements, including FX, are carried by

the valuation projection $V_t = \sum_u w_t(u) \cdot P_t(u)$, specified in the valuation/market-data spec (Section 5), not by conservation. Valuation is a linear functional on the state; dual valuation applies two price vectors to one state; PnL is the functional’s difference at two times—all derived from conservation and the additive structure of moves.

Conservation is therefore a universal oracle: for any transaction sequence, checking $Q(u) = 0$ is constant-time after the sum and detects any violation of the system’s fundamental guarantee. A conservation failure is an implementation bug, never a financial event.

16.2 The Core Invariants P1–P10

The ten core invariants are the acceptance criteria for any implementation. An implementation that violates any one is incorrect regardless of its outputs. Appendix B gives each as an executable oracle; the prose statement is fixed here.

- P1 Conservation.** $Q(u) = \sum_w w_t(u) = 0$ for all units u , all times t . (Stated as System Closure in Section 2, Theorem 2.7.)
- P2 Atomic commitment.** A transaction applies completely or not at all; no partial state is observable.
- P3 Referential integrity.** Every move references existing wallets and an existing unit.
- P4 Log monotonicity.** The event stream is append-only; no entry is retroactively rewritten. Tamper-evidence is provided by hash chaining (each entry carries the hash of its predecessor), which detects tampering but does not prevent it; prevention requires deployment controls (WORM storage, consensus, or hardware security modules). The design requires tamper-evidence as the minimum; stronger immutability depends on the environment.
- P5 Transaction idempotency.** Applying the same transaction twice (by `tx_id`) has no incremental effect.
- P6 Lifecycle idempotency.** Applying the same lifecycle event to the same unit state produces no additional moves.
- P7 Virtual/real ledger isolation.** No move has one endpoint in a virtual ledger and the other in the real ledger.
- P8 Snapshot consistency.** `clone_at(t)` reconstructs the complete view—balances *and* unit state—identical to the ledger state at time t .
- P9 Lifecycle purity.** For fixed inputs, lifecycle functions produce identical outputs and no effect beyond the return.
- P10 PnL path-independence.** $\text{PnL} = V_{t_1} - V_{t_0}$, independent of the intermediate transactions (Section 5, Theorem 5.3).

Two further oracles the same generators exercise are global rather than numbered among the ten: *totality* (every valid input returns a success or a well-typed error, never an unhandled exception) and *valid transitions only* (every product/state/event triple absent from the product’s transition table is rejected). Both are stated in executable form in Appendix B.

16.3 SBL Invariants P11–P20

Securities borrowing and lending adds ten invariants that narrow the valid-state set without relaxing any core invariant: every transaction satisfying P11–P20 also satisfies P1–P10. They are stated in full, with the position vector and Single-Coordinate Move Principle they rest on, in Section 15.18; the consolidated index follows.

P11 Paired legs

Every SBL settlement transaction carries both a securities and a collateral leg (exceptions: uncollateralised, cash pool, triparty RQV).

P12 On-loan consistency

$w_e^{\text{onloan}}(u)$ equals the summed quantity of that entity's live loans of u .

P13 Collateral sufficiency

Posted collateral covers exposure, by one of three formulations: per-loan cash, cash pool, non-cash bilateral.

P14 Locate before short

Every short sale satisfies a locate precondition.

P15 Partial-return monotonicity

Loan quantity is monotonically non-increasing.

P16 SFTR completeness

Each reportable SBL event produces exactly one SFTR report.

P17 Settlement-state monotonicity

Settlement states advance monotonically.

P18 Lender ownership invariance

A pure lender's own coordinate is constant across lending operations under title transfer (buy-in excluded). *By construction:* under the Single-Coordinate Move Principle, loan initiation and return write onloan, never the lender's own.

P19 Rehypothection regime compliance

Rehypothection is validated against the legal regime.

P20 Available-inventory identity

$\text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$. *By construction:* a definition computed on read from three stored coordinates, never stored, hence not violable (Invariant 15.12).

16.4 Obligation Liveness Invariants P21–P23

Event-triggered obligations—recalls, collateral substitutions, margin deliveries, close-out dead-lines—are first-class objects governed by three invariants, stated in full in Section 14.5.6.

P21 Obligation liveness

For every obligation o with deadline t_d : $\forall t > t_d, o.\text{state} \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}$. No obligation persists in PENDING or ATTEMPTED past its deadline (Invariant 14.4).

P22 Obligation conservation

Every discharging and every compensating move set satisfies $Q(u) = 0$; the mechanism does not bypass the executor. Follows from P1 (Invariant 14.5).

P23 Obligation idempotency

Discharging the same obligation twice (by *o.id*) has no incremental effect; the discharge predicate is checked before any move is emitted. Composes with P5 and P6 (Invariant 14.6).

16.5 The Twelve Conditions C1–C12

The three-home state model (Section 4) imposes twelve conditions, each defined once at the instrument that forces it. Each renders an illegal state unrepresentable rather than merely tested. The index below cross-links them; the discharge map (Section 16.6) gives the mechanism by which each acts.

Cond.	Content (defined in Section 4, except C7/C10 in Section 3)
C1	Position accessor returns Option; monotone carrier keeps the key set stable.
C2	Handler-level conservation per event class, vacuous base case included.
C3	Atomic Transaction across all three maps; partial application unrepresentable.
C4	Capability-scoped reads; cross-overlay reads forbidden; exports via <i>UnitStatus</i> .
C5	<i>UnitStatus</i> registration-total; product-declared defaults at registration.
C6	<i>ProductTerms</i> versioned, append-only; no in-place mutation.
C7	<i>ProductTerms</i> registration-total; first write at registration.
C8	Two-track amendment by a total fungibility predicate (Preserving / Breaking).
C9	Zero-holder handlers discharge $\sum = 0$ vacuously.
C10	Re-registration is a hard error, never a silent reset.
C11	Each <i>PositionState</i> field has a canonical writer set; an outside writer is a type error at authorship.
C12	All per-(wallet, mandate/strategy) economic state lives in <i>PositionState</i> ; the <i>W</i> -sector is empty by schema.

16.6 Which Condition Discharges Which Invariant

The highest-value content of the consolidation is the map from conditions to the guarantees they place beyond reach, and the mechanism in each case. The guarantees carry the canonical P1–P23 names this section fixes; Section 4 presents the same map under its own local labels, for which the P1–P23 names here are canonical. “Unrepresentable” is used precisely: an *abstract type* whose only constructor is a checked smart constructor makes an illegal value impossible to build, so it never reaches the operation that would act on it; a *non-empty list type* makes a versionless state untypable; a *GADT write-tag* is a per-field marker checked at compile time, then erased once writes share a stored row; a *value-level check* is a runtime guard returning an error, not a type fact. Conservation, by contrast, holds by construction: each **Move** edge names both parties, so the signed per-unit sum of any move list is the group identity **mempty**—a structural fact of the **Transaction** type, not a runtime check.

Guarantee	Cond.	Mechanism
P1 Conservation	C2, C3	A Transaction carries its flow as Move edges; each edge contributes $\text{qneg } q \triangleleft q = \text{mempty}$, so the signed per-unit sum of any move list is the group identity by construction—the empty list (registration) included. No unconserved Transaction exists to reject, hence no validate gate. C3 makes one Transaction span all three maps, so partial application is unrepresentable.
P8 Snapshot / replay	C1	Replay is the foldM of the event stream—a catamorphism, meaning replaying the events always rebuilds the exact value—hence a homomorphism $\text{replay}(xs \triangleleft ys) = \text{replay } xs \gg \text{replay } ys$, where $\gg$ runs the first error-returning step and feeds its result to the next, stopping at the first error. The monotone carrier (C1) keeps the key set stable across any checkpoint cut, so checkpoint-independence follows from the law, not a test.
P6 Lifecycle idempotency	C5	The lifecycle stage lives in $\text{UnitStatus}[u]$, u -keyed and written by replacement, so re-applying a stage write is the identity (EXPIRED over EXPIRED is EXPIRED)—structural at one key. Non-conserved <i>PositionState</i> fields share the algebra (hwm by max, entry_nav write-once); conserved fields draw replay-safety from P1.
Terms immutability (underpins P4)	C6, C7	<i>ProductTerms</i> is an append-only $\text{NonEmptyList}[\text{TermsVersion}]$, so a versionless unit is untypable; registration totality fixes the first version at registration.
Identity immutability (underpins P4)	C10, C8	Re-registration is a hard error; a breaking amendment allocates a new unit u_{new} and never rewrites u_{old} .
P7 Isolation / read scoping	C4	Isolation holds by the per-move locality predicate (both endpoints in one ledger), not by conservation; cross-overlay reads are forbidden at the capability layer wrapping the accessors, not in the data reference.
Handler-field canon (underpins P9)	C11	The canonical writer set per field is a FieldWrite GADT: a write by a field-writer outside the set is a type error at authorship, erased once writes share a delta row.

The lifecycle-idempotency and snapshot guarantees rest on the status discipline:

Remark 16.1 (*UnitStatus* is a projection, not a source of truth). *UnitStatus* holds one value per unit, shared—read identically by every holder. Its value changes over time, but *UnitStatus* is not a separate source of truth: the immutable event log is. Every change is caused by a logged

event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward.

Each mechanism above is a type or a value-level gate carrying a theorem; the Haskell rendering makes the correspondence explicit.

Types as theorems. The five anchors below render the discharge mechanisms as Haskell, one type or function per guarantee. They are verbatim excerpts of the FORMALIS-cleared reference (`reference/Ledger.hs`, the three-home model of Section 4; see Appendix B); each carries that clearance. Quantities are exact `Integer` minor units forming an additive abelian group (`Qty`, `<>` = addition, `mempty` = zero); floating point is excluded because conservation and replay are arithmetic facts it would forfeit.

(1) P1 conservation — Move edges / unitDelta (C2). Conservation is a property of the `Transaction` type, not a checked side-condition. Each `Move` names both parties, so its per-unit contribution is `qneg q <> q = mempty`. Summing those contributions over any move list is a *monoid homomorphism* from the free monoid of moves to `Qty`; per unit, the column sum over wallets is the *group homomorphism*—the total over all wallets equals the running sum taken edge by edge. The composite is identically `mempty` by the monoid laws. The empty list (a move-less registration or amendment) is `mempty`, so a zero-holder event (C9) conserves with no special case; summing edges, never dividing by a holder count, excludes the divide-by-zero-holders bug class. There is no unconserved `Transaction` to reject, hence no validate gate.

```
data Move = Move
  { mFrom :: !WalletId, mTo :: !WalletId, mUnit :: !UnitId
  , mQty   :: !Qty, ... }           -- positive magnitude; from -= q, to +=
    q

-- Conservation witness: the signed per-unit sum over the edges. By construction:
unitDelta :: UnitId -> Transaction -> Qty
unitDelta u tx =
  foldMap (\m -> if mUnit m == u then qneg (mQty m) <> mQty m else mempty) (
    txMoves tx)
-- law: unitDelta u tau == mempty   for every unit u and transaction tau

netDelta :: Transaction -> Qty
netDelta tx = foldMap (\m -> qneg (mQty m) <> mQty m) (txMoves tx)
-- law: netDelta tau == mempty   for every tau (the empty/registration case too)
```

A one-edge transfer shows there is nothing to reject: a `Move` names both endpoints, so a half-edge is unrepresentable.

```
-- tx = Transaction u [Move a b u 1000 t s] mempty [] Nothing Nothing
netDelta tx == mempty           -- by construction; a one-sided delta cannot be
    built
```

(2) P4 terms immutability — NonEmpty (C6/C7). `ProductTerms` wraps a *non-empty* list, so a “registered but versionless” unit is untypable; `currentTerms` is total without a `Maybe`. No setter exists; the only growth operations are `register` (singleton) and `appendVersion` (append, never rewrite).

```
newtype ProductTerms = ProductTerms (NonEmpty TermsVersion)   -- abstract

currentTerms :: ProductTerms -> TermsVersion                    -- total: no Maybe
currentTerms (ProductTerms vs) = NE.last vs
```

```

appendVersion :: TermsVersion -> ProductTerms -> ProductTerms -- append-only (C6)
appendVersion tv (ProductTerms vs) = ProductTerms (vs <> (tv :| []))

```

(3) **P8 monotone carrier — the Option accessor (C1).** `position` returns `Maybe`: `Nothing` is *never held*, `Just zeroP` is *held and currently flat*—distinct states the spec reads apart. The carrier is monotone by absence: `Ledger` is abstract, `applyTx` only inserts or updates a row, and no row-deleter is exported, so the key set never shrinks and a checkpoint cut cannot lose a row.

```

position :: Ledger -> WalletId -> UnitId -> Maybe PositionState
position l w u = Map.lookup (w, u) (ledgerPS l)
-- Nothing = wallet has NEVER held this unit
-- Just zeroP = held once, currently flat (the two cannot be collapsed)

```

(4) **P8 replay homomorphism — Kleisli $\gg$.** Replay is an error-stopping fold (a *cata-morphism*—meaning replaying the events always rebuilds the exact value). Determinism and checkpoint-independence are the homomorphism law `replay(xs <> ys) = replay xs  $\gg$  replay ys`, not a test: replaying two batches in sequence equals replaying them together. With the stable key set from (3), where a cut falls cannot change the result.

```

-- ( $\gg$ ) runs the first Either-returning step, feeds its result to the next,
-- and stops at the first error.
replay :: [Transaction] -> Ledger -> Either LedgerError Ledger
replay ds l0 = foldM (\l d -> applyTx d l) l0 ds

```

(5) **Handler-field canon — FieldWrite GADT (C11, underpins P9).** The GADT constructors *are* the field-to-writer table: a write by a handler outside a field’s writer set is a type error at authorship. A handler is typed by the `Handler` it speaks for, so the phantom index constrains at a live call site, not merely in an alias; the index is erased to `SomeWrite` once authored, the check having already happened.

```

data Handler = Settle | Trade | Transfer | FeeCrystallise | Subscribe

-- The balance field is NOT in this table: its sole canonical writer is the Move
-- edge (handler 'Transfer'), applied in applyTx -- so a balance change without a
-- conserving move is unrepresentable. The rest are non-balance per-position rows.
data FieldWrite (h :: Handler) where
  WAc      :: Qty -> FieldWrite 'Settle
  WAcTrade :: Qty -> FieldWrite 'Trade
  WHwm     :: Qty -> FieldWrite 'FeeCrystallise
  WEntryNav :: Qty -> FieldWrite 'Subscribe

settleHandler :: [(WalletId, Qty)] -> Map WalletId [FieldWrite 'Settle]
settleHandler legs = Map.fromList [ (w, [WAc q]) | (w, q) <- legs ]
-- A settle body bumping hwm would emit WHwm :: FieldWrite 'FeeCrystallise and
-- fail to typecheck against the declared 'Settle output -- C11 IS a type error.

```

16.7 Invariants as Specification and Oracle

The invariants define correctness independently of any implementation: the system is correct if and only if all hold, and any property-based test checking an invariant thereby tests the specification. An example-based test verifies one input/output pair; an invariant-based test verifies a property over a whole input class.

A bond coupon payment illustrates the dual role. P1 requires $Q(\text{USD}) = 0$ before and after the coupon move; P6 requires that paying the same coupon twice produces no additional moves. Applied to randomly generated bonds with random coupon schedules, these two checks exercise scenarios no hand-written suite would enumerate. The equation $Q(u) = 0$ governs both the ledger and its test suite.

16.8 Property-Based Testing over the CDM Generator Universe

Each property has a *generator* (the random-input space) and a *postcondition* (the invariant). The framework draws inputs, applies the operation, and checks the postcondition; failure yields a minimal counterexample. The executable pre/postcondition for each of P1–P10, together with the totality and valid-transition oracles, is given once in Appendix B.

The generators draw from the CDM enumerations—product types, event types, party roles, date rules (Section 13). Because these enumerations are closed and finite, the state space of any product’s lifecycle is bounded, and coverage is a checklist rather than a probability: where a random fuzzer might miss a rare combination after millions of runs, the enum-driven generator covers every combination in its first pass. The strategy is compositional: draw a CDM product, draw a sequence of valid lifecycle events, map through F to ledger transactions, and check every invariant.

Completeness by enumeration. The CDM `EventIntentEnum` is the closed set of lifecycle intents. For an equity option the applicable intents map one-to-one to unit-state transitions:

EventIntentEnum	Unit-state transition	Status
OPEN	$\emptyset \rightarrow \text{ACTIVE}$	Covered
EXERCISE	$\text{ACTIVE} \rightarrow \text{EXERCISED}$	Covered
EXPIRE	$\text{ACTIVE} \rightarrow \text{EXPIRED}$	Covered
ASSIGN	$\text{ACTIVE} \rightarrow \text{ASSIGNED}$	Covered
NOVATION	$\text{ACTIVE} \rightarrow \text{NOVATED}$	Covered
PARTIAL_TERMINATION	—	Not applicable (option is all-or-nothing)

Because the enum is closed, the generator drawing intents uniformly from it achieves full intent coverage by construction. Incompleteness is surfaced, not hidden: a new enum value with no transition (say `RESTRUCTURING`) is produced by the generator, finds no matching rule, and is flagged by the valid-transitions oracle. A missing rejection is as much a bug as a missing acceptance. The `OptionTypeEnum` $\{\text{CALL}, \text{PUT}, \text{PAYER}, \text{RECEIVER}, \text{STRADDLE}\}$ bounds payout directionality the same way: every variant must have payout logic, and a missing case is a compile-time or test-time error, not a runtime surprise.

16.9 Specification-First Development

The properties are written before the implementation: typed state schemas make illegal field combinations unrepresentable; the core properties are executable specifications; the simplest lifecycle case anchors the simulation harness; the full product universe is reached using CDM enumerations as the generator source. Correctness is thereby defined independently of language, storage backend, or deployment, and the same properties hold from single-process prototype to distributed system. The approach is falsifiable: a property satisfiable by no reasonable implementation reveals a design contradiction before implementation begins.

17 Regulatory Obligations and the Direction of Travel

Post-trade regulatory reporting is the dominant compliance cost for derivatives market participants. The same economic event must be reported to multiple trade repositories in jurisdiction-specific formats: EMIR Refit (EU 2024/1856, 203 reportable fields), MiFIR (RTS 22, 65 fields), SFTR (Regulation EU 2015/2365, 155 fields), and CFTC Parts 43 and 45 (rewritten 2022–2024). Dual-sided reporting under EMIR and SFTR makes the reconciliation burden structural: two counterparties independently produce matching reports for every lifecycle event, and mismatches trigger regulatory queries. Firms therefore maintain parallel pipelines per regime, each with bespoke field mappings, validation, and exception management.

CDM and Digital Regulatory Reporting. The ISDA Common Domain Model, governed by FINOS, supplies a single machine-readable vocabulary for trade description. ISDA’s Digital Regulatory Reporting (DRR) encodes each regime’s reporting rules as executable code over the CDM, so the interpretation of a regulation is a shared, testable artefact rather than a firm-specific implementation. DRR is in production, not sandbox: as of the source below, four institutions file DRR-generated reports with live trade repositories, with further institutions in proof-of-concept, at reported acknowledgement rates above 98% under EMIR Refit.⁴ The convergence point is a unified post-trade stack: a trade described once in CDM, reports generated mechanically by DRR, settlement executed by CDM-native contracts.

Framework alignment. The architecture derives this alignment from its own primitives rather than adopting CDM because a standard mandates it. Transactions are expressed in CDM (Section 13); instruments are deterministic move-generating contracts (Section 6); the canonical internal record is the immutable event log (Section 2). The economic content of a DRR report—the lifecycle event, its quantities, and its timing—is therefore already present in the move stream and is generated from it by projection, with no separate trade-data pipeline.

Two boundaries bound this claim and are stated once. First, a complete report (e.g. EMIR Refit’s 203 fields) also requires reference-data enrichment—UTI generation, LEI validation, counterparty classification, collateral detail—that does not reside in the move stream; this enrichment is supplied by the reference-data authority at the ledger boundary, specified in the standalone data spec (pointer per Section 18), not by this framework. Second, the framework supplies technical substrate, not organisational compliance.

Conformance checkpoints. The event log satisfies the audit-trail requirements of BCBS 239 Principle 6 (accuracy and integrity) and Principle 11 (distribution): every report projects from one immutable record. The property-based testing strategy (Section 16), deterministic lifecycle functions, and immutable audit trail are the technical substrate for the testing and traceability requirements of the EU Digital Operational Resilience Act (DORA, Regulation EU 2022/2554), Article 8 (ICT risk management) and Chapter IV (digital operational resilience testing). Full DORA conformance requires operational and organisational measures outside the scope of a design specification.

Mandate issuance reporting. Representing a managed-account mandate as a unit creates an SFTR/EMIR reporting surface for mandate issuance. The framework supplies the mechanism (flag F5): the `reportable` predicate on `ProductTerms` gates whether a unit’s issuance requires a UTI/LEI pair and which units project into the reporting layer; the managed-account treatment (Section 9) carries the construction. F5 remains flagged pending external regulatory sign-off (Section 19).

⁴ISDA, “Digital Regulatory Reporting: 2025 Year in Review,” January 2026. Figures are pinned to that publication and not restated as current.

18 Scope and Limitations

The framework’s guarantees hold for activity recorded within the ledger boundary fixed in §1. This section states the boundary in full: what is inside, what is outside, and what the framework does not do. It establishes no new result; it bounds the claims of the preceding sections.

18.1 Inside the Boundary

The following are governed by the framework’s invariants.

- Positions: wallet balances, the generalised position vector, and the conservation law.
- Moves: atomic transfers of quantity between wallets, with full provenance.
- Conservation: $\sum_w w_t(u) = 0$ for every unit at all times.
- Lifecycle: unit state transitions, recorded as transactions with CDM event metadata.
- Valuation: portfolio value $V_t = \sum_u w_t(u) \cdot P_t(u)$ given an externally supplied price vector P_t (valuation mechanics are specified in the valuation specification).
- Time travel: reconstruction of historical state by replay of the immutable event log.
- Internal reconciliation: the trading-system, general-ledger, and balance-sheet views are projections of one record.

18.2 Outside the Boundary

The following depend on external systems and are not governed by the framework.

- Price formation: P_t is an external input. The framework does not compute, validate, or guarantee executable prices.
- Legal agreements: ISDA Master Agreements, GMSLAs, GMRAAs, CSAs, and custody agreements determine legal rights. The ledger records economic positions; it does not determine legal ownership or enforceability.
- Settlement infrastructure: CSD connectivity, payment-system access, settlement finality, and real-world DvP depend on external infrastructure (DTC, Euroclear, Clearstream, CLS, TARGET2).
- Regulatory gateways: trade reporting (EMIR, MiFIR, SFTR, Dodd–Frank), transaction reporting (MiFID II RTS 25), and prudential reporting (CRR, FRTB) require submission infrastructure beyond the ledger.
- Reference-data authority: instrument master data (ISINs, contract specifications, corporate-action terms) is consumed from external sources, not created by the ledger.
- Model governance: pricing models, risk models, and their validation are external.
- Accounting policy: IFRS 9 classification (FVTPL, FVOCI, amortised cost), hedge-accounting designation, and impairment assessment are accounting decisions outside the ledger’s scope.

18.3 What the Framework Does Not Do

The following limitations are inherent in the design.

1. **Does not guarantee executable prices or replace pricing models.** P_t is an external input; the framework is agnostic to pricing methodology and requires only that prices be supplied. Under stress, P_t may be stale, unavailable, or detached from achievable liquidation values.
2. **Does not replace legal documentation or determine legal ownership.** Legal ownership depends on custodial arrangements, nominee structures, registrar entries, and governing law, all outside the boundary.

3. **Does not guarantee settlement finality.** The ledger provides ledger-level atomicity: all moves in a transaction commit or none do. Real-world finality is a property of the external settlement system and its governing law (e.g. the EU Settlement Finality Directive).
4. **Does not determine accounting policy.** IFRS 9 classification, hedge-accounting designation under IAS 39/IFRS 9, expected-credit-loss provisioning under IFRS 9, and fair-value-hierarchy classification under IFRS 13 require judgment beyond the ledger's data. The framework supplies the position and valuation data that support these decisions; it does not make them.
5. **Does not eliminate market, credit, or operational risk from external systems.** The framework eliminates classes of internal reconciliation failure. It does not eliminate price risk, counterparty-default risk, liquidity risk, or operational failure in custodians, payment systems, or exchanges.
6. **Does not replace model governance or validation.** Pricing and risk models and their independent validation (CRR Article 368, SR 11-7) are external. The framework records model outputs; it does not validate the models.
7. **Does not eliminate external reconciliation at the boundary.** Virtual wallet balances reconcile against custodian statements, counterparty confirmations, and CSD records. The framework makes this reconciliation narrowly scoped and detectable but cannot eliminate it: external systems are independent actors.
8. **Does not replace regulatory submission infrastructure.** The framework supplies the economic content of regulatory reports. Formatting, submission to trade repositories, acknowledgement handling, and query response require infrastructure beyond the ledger.
9. **Does not guarantee reproducibility without dependency versioning.** Replay reproduces results identically only under identical versions of market-data snapshots, reference data, and pricing-model parameters. An implementation must version-pin these dependencies; the framework specifies the requirement, not the versioning infrastructure.

18.4 Theorems and Modelling Choices

Several results stated as theorems are consequences of the framework's modelling choices, not independent mathematical discoveries. The conservation law ($Q(u) = 0$) is an algebraic identity forced by the move semantics ($\text{src} \text{--} = q; \text{dst} \text{+} = q$). Path-independent valuation follows from defining portfolio value as a function of current state. State-sufficiency follows from making unit state carry all economically relevant information. Their formal value is that any implementation that violates one exhibits a bug, and any extension that breaks an invariant must justify the violation; they serve as executable specifications for property-based testing.

18.5 Architecture and Operational Reality

The framework specifies a target architecture, not an operational system. The architecture reduces the structural complexity of post-trade processing; it does not remove the need for operational controls. Four-eyes trade approval, exception management, manual intervention for settlement failures, regulatory query response, and business-continuity planning remain necessary. The architecture makes these controls more effective by supplying better data and removing certain failure modes; it does not replace them.

19 Conclusion and Open Problems

19.1 Summary

All economic activity within scope uses one primitive: the atomic wallet-to-wallet move. Every instrument—equity, bond, derivative, swap, managed account, QIS strategy—is a deterministic contract that emits moves. One law (conservation) governs the system; one immutable event log records it; pure lifecycle functions extend it, giving idempotent processing and replay to any past state.

The scalar position—one number per (wallet, unit) pair—suffices for instruments where ownership and possession coincide. Where they diverge (securities lending), the generalised position vector of §15 extends the balance to six stored coordinates, with own giving economic ownership and avail = own – onloan + borrr computed on read; the non-lendable case degenerates to the scalar. The Single-Coordinate Move Principle governs every state change: one move, one coordinate, one unit, two entities; multi-coordinate operations are transactions.

The architecture rests on five concepts—wallet, unit, move, transaction, contract—and one invariant. Every financial event is a set of quantity transfers between wallets; the total quantity of each unit is conserved. PnL, valuation, dual pricing, time travel, lifecycle management, CDM integration, property-based testing, and securities lending all follow. Product complexity is isolated in contracts; the ledger engine is invariant under what is traded.

19.2 The Four Design Goals

Auditability. Every balance change carries a provenance chain. The move stream is the complete audit trail; balance-sheet substantiation at any date is a deterministic computation.

Reproducibility. Pure lifecycle functions and the immutable log reconstruct any historical state and re-execute any lifecycle logic to identical results, under identical versions of market data, reference data, and pricing models. Reproducibility requires version-pinning of these external dependencies (§18).

Time travel. `clone_at(t)` reconstructs the ledger at any past date. With pure lifecycle functions, this is a strict notion of historical truth.

Encoded correctness. Conservation is algebraic, not checked. Typed unit state makes illegal lifecycle transitions structurally unreachable. Invariants are the specification language; property-based tests verify them.

19.3 Self-Consistency as the Standard

Internal reconciliation—comparing two independent records of one reality within the firm—is unnecessary for activity in the ledger. Balance sheet, PnL, position reports, and regulatory filings are projections of one canonical record. The most damaging failures—trading/GL divergence, PnL/BS mismatch, inter-entity breaks—are structurally unreachable within scope.

The standard is self-consistency: internally inconsistent states are unreachable by construction, not detectable after the fact. Where a failure depends on an external actor and cannot be eliminated, it is made detectable and narrowly scoped. The reconciliation-failure taxonomy in Appendix C records this assessment per failure mode. External reconciliation—against custodians, counterparties, and regulatory authorities—remains necessary at the ledger boundary.

19.4 Resolved Design Problems

Three problems intrinsic to an event-sourced post-trade ledger are resolved; they are stated here only to close them.

- Concurrency.** Resolved by the single-writer workflow model (§14). Each lifecycle workflow instance serialises events for its unit; cross-unit coordination is handled by parent workflows; the executor never receives concurrent mutations for one unit.
- Liveness.** Resolved in two stages (§14): deterministic-date obligations by durable timers, event-triggered obligations (SBL recalls, collateral substitution, CSA margin, close-out deadlines) by the obligation as a first-class object with invariants P21–P23. The guarantee covers all obligation types. Durable timers (liveness) composed with the idempotency chain (safety) yield exactly-once lifecycle processing.
- State attachment.** The unit-versus-wallet question of where unit state lives is resolved by the three-home state model (§4): unit state is a materialised projection of the immutable event log—a read cache the log always rebuilds—not an authoritative mutable store.

19.5 Open Problems

The following are design-level questions, not implementation details, and must be resolved before production. Repos and access control can be addressed incrementally.

- Netting and close-out.** Atomic moves are gross. Netting is a higher-level operation that inspects pending gross moves, computes net amounts per (counterparty, unit) pair, and emits reduced net moves; conservation holds by construction, the net moves forming a transaction whose signed edge-sum is zero. The algebra of netting—commutativity, associativity, interaction with CCP default management—is unformalised, particularly for multilateral netting through CCPs. Close-out netting under the ISDA Master Agreement is a mass lifecycle event over many contracts; the per-contract model requires a cross-contract close-out mechanism that terminates multiple contracts, computes close-out values, and emits one net settlement move.
- Correction algebra.** Compensating transactions are described informally. A formal algebra of corrections (anti-moves, amendment chains, composition rules) would make corrections first-class objects.
- Multi-entity federation.** The design assumes one shared ledger. Multi-entity deployments require a federation protocol preserving conservation across ledger boundaries; inter-company eliminations for consolidated reporting are unaddressed.
- XVA.** CVA, DVA, and FVA are non-transfer value changes that do not decompose into wallet-to-wallet moves and may require a dedicated adjustment layer (valuation mechanics: valuation specification).
- Bitemporal semantics.** Economic and booking timestamps are distinguished informally. A full bitemporal model would handle market-data restatements without corrupting the historical record (market-data: market-data specification).
- Tax-lot tracking.** Wallets store net quantities. US wash-sale and specific-identification reporting require per-lot tracking (date, quantity, cost basis) as move-stream metadata, with a lot-selection method (FIFO, specific identification). Its interaction with the futures accumulated-cost method (§8), which deliberately avoids per-trade tracking, is unresolved.
- Impairment and ECL.** IFRS 9 expected credit losses and IAS 36 impairment reduce carrying value without a counterparty transfer; these may require a virtual impairment-reserve wallet or an extension of the move primitive.
- Data privacy and GDPR.** The append-only log conflicts with GDPR Article 17 (erasure) when moves carry personal data. Candidate resolutions: pseudonymous identifiers linked to a separate mutable store; crypto-shredding; or a schema in which personal data never enters the log. Crypto-shredding trades against time travel—`clone_at(t)` returns anonymised counterparty references once keys are destroyed, acceptable for financial replay, incomplete for audit requiring counterparty identification. The choice fixes the move schema and must be made at design time (data: data specification).
- Access control and actor attribution.** The move primitive carries no authenticated actor

identity. Production requires per-move attribution to an authorised actor, maker-checker workflows, cryptographic non-repudiation, and wallet-level read/write permissions scoped to role and organisational boundary.

Repos. Repurchase agreements (GMRA) share collateral and settlement primitives with SBL (§15) but differ in title-transfer semantics. Extension requires the GMRA-specific lifecycle events (open leg, close leg, repricing, substitution, classic repo versus buy-sell back).

Default management. Counterparty or clearing-member default requires close-out, portfolio auction, and loss allocation. Conservation must accommodate loss mutualisation—transfer of losses to surviving participants—which needs governance and waterfall rules the framework does not provide.

Pre-trade validation. The framework is silent on pre-trade checks (credit limits, position limits, margin sufficiency, regulatory constraints). A pre-condition gate on the move primitive is a natural extension point; its interaction with the conservation law is unspecified.

Migration from legacy architectures. Adoption requires migrating from separate trading systems, risk engines, and general ledgers—parallel running with reconciliation during transition, progressive routing of new trades, back-loading of historical positions. The political economy of the transition is as consequential as the technical path.

Tokenised securities. Appendix F establishes the tokenised-equity model, the double-counting resolution, and the custodian-is-flat principle, and identifies four CDM v6.0.0 gaps. Open: cross-chain interoperability; partial backing and proof-of-reserves; regulatory classification under MiFID II and SEC frameworks; tokenised-collateral eligibility; and the interaction of on-chain governance with custodian-mediated shareholder voting.

19.6 Flagged Open-Items Register

The items below are honest open items—the next-version agenda—kept distinct from the proven quantity algebra. They are migration, governance, operational, and conformance risks; none invalidates the design on correctness grounds. Each escalation is also cited at its home section: futures in §8, managed-account in §9.

19.6.1 State-Model Adoption Risk Register (F1–F8)

These accompany the three-home state model (§4). F2, F5, F6 require external stakeholders before any code ships; F1, F3, F4, F7, F8 are internal engineering risks manageable within the delivery programme.

	Risk	Mitigation
F1	Migration scope of retrofitting every downstream consumer that reads unit state as a single combined field.	Ship a read-compatibility accessor exposing the composed single-field view; split additively before cutover; parallel-run $\geq$ one quarter with reconciliation.
F2	Ownership of the C8 fungibility predicate is ungoverned.	Per-product RACI in the Unit Store registry; Legal/Product/Risk sign-off for predicate changes; version the predicate.
F3	Monotone carrier key-space growth at $10^7 \times 10^6$ scale is unbenchmarked.	Benchmark Option-accessor cost and cache behaviour before merge; tombstone-compaction policy for rows Some(zero) beyond the retention horizon.

	Risk	Mitigation
F4	C1–C12 is a multi-sprint test programme, not a release.	Land the schema with the type-enforced conditions first; stage C2/C3/C8/C11 over later releases; never merge below C1+C2+C3.
F5	Mandate-as-unit creates an SFTR/EMIR reporting surface for mandate issuance.	Pre-flight with Regulatory; determine whether u_{MA} issuance needs a UTI/LEI pair; possible reportable opt-out flag.
F6	CDM alignment (TradeState per Trade vs PositionState $[w, u]$) is asserted, not verified.	Rerun the Rosetta NS1–7 mapping against the three-map schema before any CDM adapter work; publish a delta document.
F7	Onboarding cost of three maps, Option/Monotone carriers, and twelve conditions is real.	Ship a decision tree, runbook, and cheat sheet; mandatory training before handler work.
F8	Forward migration is irreversible once the pre-migration combined wallet-state representation is collapsed into the three maps.	Keep a read-only mirror of that pre-migration representation $\geq$ four quarters post-cutover; document the inverse mapping explicitly.

19.6.2 Testing Commitments (Mutation Score)

Concrete targets, to be met before production. The generator universe—CDM enum $\times$ product-type cross-product—is finite and enumerable, so coverage is structurally bounded.

- Event handlers (decimal arithmetic): 85–90 %. Conservation kills most sign and coefficient mutants.
- Lifecycle guards: 70–80 %. Boundary-comparison mutants ($>$ versus $\geq$) require targeted tests.
- Overall core: ≥ 80 % (the Feathers change-safety threshold).
- State-machine model at $|W| = 3$, $|U| = 2$, depth ≤ 6 : 10^5 – 10^6 reachable states (TLC-tractable); conservation-violating handlers are caught at depth 1.

19.6.3 Managed-Account Escalations (ME1–ME5)

Open items from §9, distinct from the segregation theorem ($\text{CONS} \wedge \text{LOC} \wedge \text{C4}$) proved there.

Open item
ME1 Store-versus-derive for account-level scalars (NAV, high-water mark): which are stored and which recomputed on read is unsettled.
ME2 The observation surface is unattested (Nazarov attestation-envelope finding): the external observations driving mandate evaluation carry no cryptographic attestation; an attestation envelope is required.
ME3 Counterparty legal identity: the mapping from client to a governed LEI is outside the ledger and must be bound.
ME4 Localisation condition C4 is asserted in the segregation argument, not yet discharged by a type.
ME5 No in-ledger solvency liveness: continuous solvency monitoring of the managed account is not a framework guarantee.

19.6.4 Futures Escalations (FE1–FE2)

Open items from §8, distinct from the futures lifecycle and its closing identity (cumulative VM = economic PnL) proved there.

Open item
FE1 Variation-margin settlement fan-out cost at scale is unbenchmarked.
FE2 The derived-consequence alternative to storing accumulated cost per position (C11) was declined; the trade-off is recorded, not closed.

Concurrency, liveness, and state attachment are resolved (§14, §4). The items in this register and in Open Problems are not. The quantity algebra is sound within the boundary of §18; the open items bound what the framework does not yet establish.

20 Frequently Asked Questions

Each answer stands on the framework’s primitives; cross-references point to the section that establishes the result, never restate it.

Q1: Are move-less lifecycle events valid, and must the executor accept them?

Yes. A lifecycle event may produce a state change with an empty move list. The transaction is recorded in the move stream as a state-change entry carrying the full CDM event payload; the audit trail and time travel are preserved. Conservation holds trivially: the empty sum is zero for every unit in every wallet.

Such events are not optional records. A barrier knock-out on a product where the owner holds no position transitions the unit from **ACTIVE** to **TERMINATED** and emits no moves. Without the recorded state change, `clone_at(t)` for t after the knock-out cannot distinguish “terminated by barrier breach” from “active with zero holders”—economically distinct states, one carrying zero contingent liability and the other a potentially large one. Under IFRS 9 the fair-value liability drops to zero on termination, not on payment; the recorded state change is the record of *why* it disappeared.

Further instances: out-of-the-money expiry whose position-closing move has zero economic effect, suspension and delisting events, and intraday futures trades that update unit state without cash moves (Section 8). The futures lifecycle already establishes state-only transactions; the principle applies to all product types, and the executor must not reject them.

Remark 20.1. *The state a lifecycle event changes is held in `UnitStatus`. `UnitStatus` holds one value per unit, shared—read identically by every holder. Its value changes over time, but `UnitStatus` is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward. “Move-less” therefore names a transaction that overwrites this cache without altering any wallet balance.*

Q2: Does the Ledger guarantee Delivery-versus-Payment atomicity?

Internally, yes, by construction. A DvP trade is a single transaction containing both the securities move and the cash move; the executor commits all moves of a transaction together or none. This follows from the transaction primitive (Definition 2.4, Section 2), not from a runtime check.

Conservation alone does not deliver DvP: a single-leg transfer balances quantities across wallets and is conservation-preserving. The guarantee is the conjunction of two properties:

1. **Transaction atomicity** (executor): all moves in a transaction commit together or not at all.
2. **Contract correctness** (smart contract): the trade contract emits both legs—cash and securities—in the same transaction (Section 6).

External settlement atomicity is provided independently by the CSD. The framework guarantees internal consistency; the settlement infrastructure guarantees external settlement (deferred-settlement spec; Section 12.5).

Q3: How does the framework settle physical delivery under lot-size constraints?

Physical delivery of equities requires whole multiples of the board lot L . When exercise produces a non-integer lot quantity, the smart contract delivers $N_{\text{deliver}} = L \lfloor N/L \rfloor$ shares against strike cash and cash-settles the residual at intrinsic value. This is one atomic transaction with two

move types: share delivery against strike cash, and a cash payment on the undeliverable residual. Conservation holds because the put payoff $(K - S_T) \times N$ is split between physical and cash legs, not altered.

The lot size is a product term—a property of the underlying security recorded in unit reference data (Section 3), not a settlement concern. The contract computes delivery as a pure function of (K, N, S_T, L) , making the logic property-testable for conservation, lot compliance ($N_{\text{deliver}} \bmod L = 0$), and economic equivalence (total value transferred equals the payoff for any lot split).

Boundary cases follow from the formula. Exercise below one lot ($N < L$) delivers zero and cash-settles entirely. Exercise an exact multiple of L leaves zero residual and no cash adjustment. Odd-lot markets set $L = 1$, so the residual is always zero. The settlement layer projects the single transaction into a securities delivery for the whole-lot quantity and a cash payment for the combined amount; DvP (Q2) applies at both Ledger and CSD level. The lot-adjustment function is shared across physically settled instruments—convertible conversion, warrant exercise, equity-forward delivery; only the payoff formula differs (Section 6).

Q4: Why is securities lending inside the Ledger rather than in a companion system?

Five reasons, each a failure of the companion design:

1. A companion system reintroduces the TOCTOU race, enabling double-lending.
2. Conservation across the system boundary becomes unprovable.
3. Collateral becomes invisible, breaking P13 (collateral sufficiency, Section 15.18).
4. Exposure calculation requires joining two systems.
5. The “reserved” bucket in a companion system discards load-bearing information.

The full argument is in Section 15.6.

Q5: Why is `own` a stored coordinate while `avail` is a projection?

`own` passes the physical-action test: buy and sell modify `own` and only `own`. It is not derivable from the five operational coordinates—two states with identical operational vectors but different `own` exist whenever a borrower has sold some but not all borrowed shares. `avail` fails the test: every action that changes available inventory also changes `onloan`, `borr`, or `own`. The identity `avail = own - onloan + borr` fixes `avail` from stored coordinates, so storing it would be a cache, not a fact (Section 15.2).

Q6: Is a scalar balance sufficient for cash, including cash collateral in securities lending?

The scalar model is sufficient for all cash units. The six-coordinate position vector (`own`, `onloan`, `borr`, `coll_post`, `coll_recv`, `coll_rehyp`) of Section 15.2 degenerates to $(q, 0, 0, 0, 0, 0)$ for any cash unit—the scalar q . Cash is not a lendable security: ownership and possession never diverge, so no system tracks `onloan` for cash. The lending-specific coordinates (`onloan`, `borr`, `coll_post`, `coll_rehyp`) are never populated for cash; only `own` and `coll_recv` are engaged, and `coll_recv` is itself scalar per cash unit.

Cash collateral in securities lending uses only those two scalar coordinates and the SBL loan unit state:

1. **Receipt.** Borrower-posted cash collateral enters the lender’s `coll_recv(USD)` as a scalar amount earmarked as collateral (Step 1, Appendix I). The collateral relationship is recorded in the SBL loan unit state, not in the lending coordinates of the position vector.
2. **Reinvestment under title transfer (GMSLA 2010).** Title passes outright; the lender owns the cash (Section 15.9) and reinvests by ordinary atomic moves—cash leaves

coll_recv(USD), the instrument enters own(MMF), both scalar. The obligation to return equivalent cash is held in the loan unit state as `collateral_amount` and enforced by the SBL contract’s termination pre-conditions (P18, Section 15.18; Section 15.12).

3. **Reinvestment under pledge (US Rule 15c3-3 / MSLA).** The lender holds the cash subject to possession-or-control and reserve-formula requirements; reinvestment is permitted but constrained to approved instruments. The framework models this identically: cash in coll_recv, reinvestment in own(MMF), return obligation in the loan unit state. The rehypothecation cap (140% of customer debit balance, P19, Section 15.18) constrains *securities* rehypothecation, not cash reinvestment (Section 15.9).
4. **Return on termination.** The reinvestment is unwound—MMF units sold, cash returns to coll_recv—and the collateral is released to the borrower’s own (Section 15.12). Every move is scalar; conservation holds at each step.

The scalar cash balance, the scalar coll_recv coordinate, and the SBL loan unit state jointly represent all cash-collateral operations under both GMSLA 2010 title transfer and US Rule 15c3-3 / MSLA pledge. No extension to the cash model is required.

Part VII

Appendices

A CDM Type Mapping Tables

The mapping F of Section 13 is a set of translation rules, one per CDM type. Types carrying economic substance — products, events, transfers — map to ledger primitives. Types carrying business context — workflow steps, lineage — are preserved as event-log metadata.

CDM Type	Ledger Concept	Notes
Product	Unit type ($u \in \mathcal{U}$)	One product $\rightarrow$ one or more units
TradeState	Unit state ($\text{state}_t(u)$)	Lifecycle position and flags
BusinessEvent	Transaction (τ)	One or more moves
PrimitiveEvent	Move (m)	Atomic transfer
Transfer	Move (m)	Cash or securities movement
QuantityChange	Move (m)	Position adjustment
Party	Wallet (w)	One party $\rightarrow$ one or more wallets
Account	Wallet (w)	Direct correspondence
WorkflowStep	Transaction metadata	Event-log payload
Lineage	Transaction metadata	Event-log payload

`TradeState` maps to unit state $\text{state}_t(u)$, a materialised projection of the immutable event log: its value at t is rebuilt by replaying the events up to t , never an authoritative store written off-log.

B Property Test Catalogue

The ten core invariants of Section 16 restated as executable oracles. Each is a precondition that constrains the generated input and a postcondition the operation under test must satisfy. The CDM enumeration universe (Section 13) supplies the generators: the framework draws random inputs, applies the operation, and checks the postcondition. A failure yields a minimal counterexample. The invariant prose is stated once in Section 16 and not repeated here; this catalogue gives the test forms.

Invariant	Precondition	Postcondition (oracle)
P1 Conservation	τ applied to state s , accepted	$\forall u : Q_{s'}(u) = \sum_w w(u) = 0$
P2 Atomic commitment	$\tau = \langle m_1, \dots, m_n \rangle$; move m_k fails validation, $1 \leq k \leq n$	$\text{execute}(s, \tau) = s$; no partial state observable
P3 Referential integrity	m a move in an accepted τ	$m.\text{src} \in \mathcal{W} \wedge m.\text{dst} \in \mathcal{W} \wedge m.\text{unit} \in \mathcal{U}$
P4 Log monotonicity	log $L = \langle e_1, \dots, e_n \rangle$ after any operation sequence	$\forall i > 1 : e_i.\text{prevHash} = H(e_{i-1})$; no $e_j, j < n$, rewritten (append-only, hash-chained)
P5 Transaction idempotency	state s , transaction τ	$\text{execute}(\text{execute}(s, \tau), \tau) = \text{execute}(s, \tau)$ (dedup by <code>tx_id</code>)
P6 Lifecycle idempotency	$f(u, st, d) = (\text{moves}, st')$; st' already reflects the event	$f(u, st', d) = (\emptyset, st')$; second call emits no moves
P7 Ledger isolation	m a move in virtual ledger $\mathcal{L}_v$	$m.\text{src} \in \mathcal{L}_v \wedge m.\text{dst} \in \mathcal{L}_v$; no move crosses the virtual/real boundary
P8 Snapshot consistency	time t in the log; s_t the state at t	$\forall w, u : \text{clone_at}(t).\text{balance}(w, u) = s_t.\text{balance}(w, u) \wedge \text{clone_at}(t).\text{unit_state}(u) = s_t.\text{unit_state}(u)$
P9 Lifecycle purity	lifecycle function f , fixed inputs (u, st, d)	repeated evaluation yields identical (moves, st') ; no effect beyond the return
P10 PnL path-independence	price vectors P_{t_0}, P_{t_1} ; any transaction sequence on $[t_0, t_1]$	$\text{PnL}_{\text{price}} + \text{PnL}_{\text{flow}} = V_{t_1} - V_{t_0}$, independent of the intermediate path

Remark B.1 (P8 and the projection discipline). *The P8 oracle reconstructs both balances and unit state from the log. `unit_state` is read from `UnitStatus`, a materialised projection of the immutable event log—a read cache the log always rebuilds, never a separate source of truth (Section 7). The oracle is the executable form of that discipline: a catamorphism—replaying the events up to t rebuilds the exact value that held then—so $\text{clone_at}(t).\text{unit_state}(u) = s_t.\text{unit_state}(u)$ holds by construction, and a failure exposes any drift between the cache and the log.*

Totality oracles. Two global oracles the same generator exercises, not numbered among the core ten:

- *Totality.* For every generated input i , $\text{execute}(s, i)$ returns success or a well-typed error—never an unhandled exception.

- *Valid transitions only.* For every (product type, state, event) triple absent from the product’s transition table, the lifecycle function returns an error. This is the executable witness that illegal lifecycle transitions are unrepresentable (Section 7).

PnL inputs. V_t and the price vectors $P_t(u)$ that drive P10 are produced by the valuation projection; their construction and the state-aware price function are specified in the valuation/market-data spec, not duplicated here.

SBL and obligation extensions. Section 15.18 adds ten invariants P11–P20 specific to Securities Borrowing and Lending (collateralisation, lender ownership invariance, partial-return and settlement monotonicity), and Section 14.5.6 adds P21–P23 for obligation liveness. These never relax a core invariant: every transaction satisfying P11–P23 also satisfies P1–P10. Their executable forms follow the same precondition/postcondition shape.

Typed oracles. Each oracle above has a typed counterpart: a precondition that constrains the generated input and a postcondition the operation under test must satisfy. Both are total, so the generator and the checker share one type. The operation under test returns a typed **Outcome**; **Accepted** is the *only* constructor that carries a ledger, so a partially applied (“half-committed”) ledger is not representable. That absence is the *Totality* oracle and the “no partial state” half of P2 made structural rather than tested.

```
-- execute = the canonical (Sec. 4) three-home transaction-apply, lifted to
-- return Outcome; supplied by the implementation, not a symbol defined here.
data Outcome = Accepted Ledger | Rejected TxError -- no third "crashed" case

-- A property: a precondition constraining the generated input and a
-- postcondition over the outcome of running the operation on it. Both total.
data Property i o = Property
  { pre  :: Ledger -> i -> Bool
  , post :: Ledger -> i -> o -> Bool }

data TxInput = TxInput { tiTx :: OracleTx, tiScope :: Scope }
data Scope   = Scope   { scWallets :: [WalletId], scUnits :: [UnitId] }

registered :: Ledger -> UnitId -> Bool
registered l u = maybe False (const True) (productTerms l u)

-- Sum a wallet's holding of u across a wallet set. Summation is a monoid
-- homomorphism into the Qty group -- the empty set sums to mempty (=0) with no
-- special case, so a zero-holder unit conserves vacuously (no divide-by-holders).
unitTotal :: Ledger -> [WalletId] -> UnitId -> Qty
unitTotal l ws u = foldMap (\w -> maybe mempty psBalance (position l w u)) ws

-- Equality of ledgers BY OBSERVATION over the scope -- the only equality the
-- oracles need; the abstract Ledger exports no structural Eq.
sameLedger :: Scope -> Ledger -> Ledger -> Bool
sameLedger sc a b =
  and [ position a w u == position b w u | w <- scWallets sc, u <- scUnits sc ]
  && and [ unitStatus a u == unitStatus b u | u <- scUnits sc ]
```

Five oracles test the same operation, **execute**, and share the one **Property TxInput Outcome** shape. Each postcondition that asserts a positive fact is guarded on **Accepted**, so it claims nothing about a rejected transaction.

```
-- P1 Conservation. On any accepted transaction every unit's holdings sum to
-- zero across the scope (closed system); a rejected one asserts nothing.
p1 :: Property TxInput Outcome
p1 = Property (\_ _ -> True) $ \_ (TxInput _ sc) o -> case o of
  Rejected _ -> True
```

```

Accepted l -> all (\u -> unitTotal l (scWallets sc) u == mempty) (scUnits sc)

-- P2 Atomic commitment. Any move failing validation rejects the whole txn;
-- no partial ledger exists (Outcome cannot carry one).
p2 :: Property TxInput Outcome
p2 = Property (\l (TxInput tx _) -> any (invalidMove l) (otMoves tx))
      (\_ _ o -> isRejected o)

-- P3 Referential integrity. Every move in an accepted txn names a registered
-- unit and in-scope wallets.
p3 :: Property TxInput Outcome
p3 = Property (\_ _ -> True) $ \_ (TxInput tx sc) o -> case o of
  Rejected _ -> True
  Accepted l -> all (\m -> registered l (mUnit m)
                    && inScope sc (mFrom m) && inScope sc (mTo m)) (otMoves tx)
    )

-- P7 Ledger isolation. With Scope = the virtual ledger's wallets, no move
-- crosses its boundary -- a property of the moves, independent of the outcome.
p7 :: Property TxInput Outcome
p7 = Property (\_ (TxInput tx _) -> not (null (otMoves tx)))
      $ \_ (TxInput tx sc) _ ->
      all (\m -> inScope sc (mFrom m) && inScope sc (mTo m)) (otMoves tx)

-- P5 Transaction idempotency (dedup by tx_id). Re-running an accepted txn is a
-- no-op; this oracle re-runs the operation, so it takes it directly.
idempotentTx :: (Ledger -> OracleTx -> Outcome)
              -> Scope -> Ledger -> OracleTx -> Bool
idempotentTx exec sc l tx = case exec l tx of
  Rejected _ -> True
  Accepted l' -> case exec l' tx of Accepted l'' -> sameLedger sc l' l''
                                Rejected _ -> False

```

The remaining five test different operations – the log append, the snapshot rebuild, the pure lifecycle handler, the PnL decomposition. Forcing them into the `Property` shape would buy nothing, so each is a bespoke total predicate (the restraint rule: stop at the cheaper guarantee).

```

-- P4 Log monotonicity. The log is append-only and hash-chained BY CONSTRUCTION
-- (built only by 'append', which stamps the predecessor's hash; no rewrite or
-- delete is exported). The oracle re-checks the link; a break needs API-tamper.
data Hashed = Hashed { hPrev :: Hash, hPayload :: Transaction, hSelf :: Hash }
chainOK :: [Hashed] -> Bool
chainOK es = and (zipWith (\a b -> hPrev b == hSelf a) es (drop 1 es))

-- P8 Snapshot consistency. cloneAt t rebuilds the state at t by replaying the
-- events up to t (a catamorphism -- replaying the log always rebuilds the exact
-- value that held then), so snapshot must equal s_t by observation (both
-- balances and unit state -- sameLedger covers both).
p8 :: Scope -> Ledger -> Ledger -> Bool          -- cloneAt t vs s_t
p8 = sameLedger

-- A lifecycle handler is a PURE TOTAL function. Purity (no IO in the type) IS
-- P9: re-evaluation cannot differ and there is no effect to leak.
type Lifecycle st ev = UnitId -> st -> ev -> ([Move], st)

-- P6 Lifecycle idempotency. Feeding the handler its own output state emits no
-- further moves and fixes the state.
idempotentL :: Eq st => Lifecycle st ev -> UnitId -> st -> ev -> Bool
idempotentL f u st ev = let (_, st') = f u st ev in f u st' ev == ([], st')

```

```

-- Valid-transitions-only. For every (product, state, event) absent from the
-- transition table, the guarded handler returns an error: not in table => Left.
-- An illegal transition is thus not representable as success.
validTransitionsOnly
  :: (p -> st -> ev -> Bool)                -- in the table?
  -> (p -> st -> ev -> Either TxEError ([Move], st)) -- the guarded handler
  -> p -> st -> ev -> Bool
validTransitionsOnly inTable h p st ev
  | inTable p st ev = True
  | otherwise       = case h p st ev of Left _ -> True; Right _ -> False

-- P10 PnL path-independence. V_t and the price vectors P_t(u) come from the
-- valuation projection (valuation/market-data spec); the decomposition closes
-- regardless of the intermediate transaction path.
p10 :: Cash -> Cash -> Cash -> Cash -> Bool      -- pnlPrice pnlFlow V_t0 V_t1
p10 pnlPrice pnlFlow v0 v1 = pnlPrice <> pnlFlow == v1 <> cashNeg v0

```

The full self-contained listing (helpers `inScope`, `isRejected`, `invalidMove`, the `Move/OracleTx` types, and the `total` coverage witness) reuses the `FORMALIS-cleared Ledger` library and its total accessors, consolidated in `reference/Ledger.hs`.

C Reconciliation Failure Mode Taxonomy

Each class of internal reconciliation break is classified against the architecture as *structurally unreachable*, *easier to detect*, or *addressed by design*, for activity recorded in the ledger. Three modes are structurally unreachable: the architecture makes the divergent state unrepresentable. Six are easier to detect but not eliminable, because they depend on an external actor the ledger only mirrors. Securities-lending breaks are eliminated by design. Operational failures outside the architecture—incomplete booking, duplicate booking through external channels, stale-data ingestion—remain possible and require operational controls.

Failure Mode	Assessment	Rationale
Trading / GL divergence	Structurally unreachable	Trade and accounting entry are one object
PnL / balance-sheet mismatch	Structurally unreachable	Both project the same position set and price vector
Inter-entity breaks (shared ledger)	Structurally unreachable	An inter-entity move is one atomic event
Custodian / depot breaks	Easier to detect	The virtual wallet is an independent comparison point against the external system
Nostro / vostro breaks	Easier to detect	The virtual wallet mirrors the external account; ISO 20022 feedback (deferred-settlement spec) narrows the window
Corporate action mismatches	Easier to detect	Lifecycle events are transactions; idempotence prevents double-processing

Failure Mode	Assessment	Rationale
Collateral disputes	Easier to detect	Internal valuation is recorded; the counterparty may disagree. Posting and return are moves; eligibility, haircuts, and substitution under UMR are deferred to further specification
FX rate discrepancies	Easier to detect	The dual-valuation framework (valuation spec) surfaces the differential
Securities-lending breaks	Addressed by design	SBL integration (§15) removes the split-system break

D Coordination Between Pricing, Contract State, and Market Data

A unit's valuation and the cash recorded in wallets are consistent at every instant. Where they diverge, value is duplicated or lost. The atomicity of the lifecycle event (C3, 2) makes that divergence structurally unreachable; this appendix exhibits the mechanism on coupons and dividends.

D.1 The coupon

A bond pays a stream of coupons. At each coupon date two facts must hold together:

1. the cash flow is recorded — a move from issuer wallet to holder wallet;
2. the bond goes ex-coupon — subsequent pricing and accrual reflect that this coupon is paid and no longer part of the remaining promise.

Decoupling them admits exactly two corruptions:

- the coupon cash is paid but pricing still treats the bond as cum-coupon — the coupon is counted twice, once as cash and once embedded in the dirty price;
- the bond is moved ex-coupon but no cash flow is recorded — the coupon vanishes from the economic total.

Either way the total value, cash plus mark-to-market, is wrong.

D.2 State-aware pricing

The pricing of a unit u depends on its lifecycle state, which records the events that have occurred:

$$P_t(u) = P(u, \text{state}_t(u), \text{market data at } t).$$

The signature below makes this dependence explicit. The pricing-relevant slice of lifecycle state distinguishes only whether the distribution is still embedded (**Cum**) or has detached and been paid (**Ex amount**); the price is a total, pure — hence deterministic — function of that state *and* the quote, and the type admits no pricer that reads the quote alone.

```

--  $P_t(u) = P(u, \text{state}_t(u), \text{market data at } t)$ . Types first.
newtype Quote = Quote Integer    -- raw disseminated spot, minor units
newtype Price = Price Integer    -- the internally-used price; NOT a Quote
newtype Cash  = Cash  Integer    -- a paid distribution amount, minor units

-- The pricing-relevant slice of state. A coupon or dividend is either still
-- embedded (Cum) or has detached and been paid (Ex amount). Carrying the paid
-- amount on Ex makes the adjustment TOTAL: an ex-unit always knows what was
-- removed, so no failable lookup is needed. The Cum/Ex fuse also makes the
-- nonsense state "ex but amount unknown" unrepresentable.
data Distribution = Cum | Ex Cash

-- Market data the pricer may read. mQuoteEx records whether the EXTERNAL quote
-- has itself gone ex; it may LAG the ledger's own ex transition (next-open lag).
data Market = Market { mQuote :: Quote, mQuoteEx :: Bool }

--  $P_t(u) = P(u, \text{state}, \text{market})$ . Total over both Distribution cases and every
-- Market; pure, hence deterministic. The quote is consulted THROUGH the state,
-- never alone -- so the double-count is not a check but a missing function.
priceOf :: UnitId -> Distribution -> Market -> Price
priceOf _ Cum          (Market (Quote q) _ ) = Price q          -- cum: quote
      as is
priceOf _ (Ex _)       (Market (Quote q) True ) = Price q       -- quote
      already ex
priceOf _ (Ex (Cash d)) (Market (Quote q) False) = Price (q - d) -- quote lags
      : remove once

```

The type is the argument: a function from Market to Price could not see the state and would double-count by construction; priceOf cannot, because it must consume the Distribution before it can return a Price. Because Ex carries the paid amount, the adjustment branch is total — there is no ex-state in which the amount to remove is unknown.

A coupon date is one atomic lifecycle event (C3): it generates the cash-flow move and advances $\text{state}_t(u)$ to ex-coupon as a single indivisible Transaction. No intermediate configuration exists in which the cash has appeared without the state advancing, or the reverse. Pricing always evaluates $P_t(u)$ against a $\text{state}_t(u)$ consistent with the move stream, so the price jump at the coupon date exactly offsets the coupon cash entering the wallet, and total value (cash plus bond) is continuous through the event.

D.3 Ex-dividend: the external-quote lag

Equity dividends raise the same coordination problem with one added constraint: the external quote need not switch state at the ledger event. On the ex-dividend date the ledger records the dividend move from issuer wallet to holder wallet and advances the equity state from cum-dividend to ex-dividend, atomically as above. The raw spot disseminated by an exchange, however, is not guaranteed to go ex-dividend at that instant; the first ex-dividend quote may appear only at the next market open, and until then the published spot may still carry the cum-dividend value.

Consuming that raw spot while the ledger has already paid the dividend and marked the unit ex-dividend counts the dividend twice — once as cash, once embedded in the equity price. This is the same $P_t(u)$ as the coupon: the ex-dividend lag is exactly the `priceOf u (Ex d) (Market q False)` branch above, which removes the paid amount once and so makes total value continuous through the event:

```

-- ghci> priceOf u Cum          (Market (Quote 102) False) -- cum-distribution
-- Price 102
-- ghci> priceOf u (Ex (Cash 2)) (Market (Quote 102) False) -- ledger ex, quote

```

```

LAGS
-- Price 100    -- the 2 paid as cash is removed from the mark: counted ONCE
-- ghci> priceOf u (Ex (Cash 2)) (Market (Quote 100) True)    -- quote caught up
-- Price 100    -- no adjustment; the quote already excludes the distribution

```

Holder value before the event is the mark 102; at the event the cash leg adds +2 and the state-aware mark falls to 100, so cash plus mark is unchanged. The two reachable ex-cases (quote lagging, quote caught up) agree on the price; the unreachable third reading — quote alone, ignoring state — is not a case the type can express.

D.4 Scope

External prices are processed in light of instrument state, never consumed blindly. The distinction between “primary” and “derived” market data carries no content here; what matters is that valuation prices cohere with the ledger’s recorded lifecycle events, which C3 guarantees by making cash flow and state transition indivisible. The self-inconsistency conventional systems detect and repair is thus unreachable within the ledger’s scope.

The construction and provenance of the external quote feeding $P_t(u)$ — its sourcing, snapshotting, and curve construction — are the market-data satellite specification’s concern, out of scope here; see the standalone market-data spec.

Remark D.1 (FORMALIS clearance). *The `priceOf` listing is new code; FORMALIS reviewed and cleared it. Findings resolved: totality (the equations cover `Cum` and both `Ex` Boolean cases; `Ex` carries the paid amount so the adjustment branch never wants a failable lookup); determinism (a pure function of `(Distribution, Market)`, no effect, no ambient read); and illegal-state exclusion (a quote-only pricer is not expressible at this type, and “ex with unknown amount” is unrepresentable by the `Ex Cash` fuse). The `Quote/Price/Cash` separation reuses the already-cleared dimension discipline of the futures lifecycle section, where `Price` deliberately carries no `Monoid` instance so a price cannot be summed into a balance.*

E Glossary

Terms carry these meanings throughout. A defined term is used thereafter without re-explanation. Property identifiers P1–P23 and condition identifiers C1–C12 are stated where they are established and referenced by identifier elsewhere.

Accumulated cost

Per-wallet signed sum $\sum_i (-\text{qty}_i \times \text{price}_i \times \text{mult})$ over the trades of a futures position: negative net-long, positive net-short, zero when flat. A conserved field.

Atomic move

The indivisible primitive: transfer of a positive quantity of one unit from a source wallet to a destination—the edge presentation of a conserved flow (§2).

CDM

ISDA Common Domain Model: a standardised, machine-executable model of financial products and lifecycle events.

Clone_{at}(t)

Reconstruction of ledger state as of historical time t by replaying the move stream and the logged state changes up to t .

Conservation law

The total quantity of each unit across all wallets is zero at all times: $Q(u) = 0$ (P1).

Conserved field

A **PositionState** field whose per-event-class deltas, $\sum_w \Delta f(w, u) = 0$, sum to zero across wallets, established per handler with a vacuous zero-holder base case (C2, C9). Net quantity, accumulated cost, and variation margin are conserved fields.

Coordinate

A component of the position vector; a position type in the wallet model. Each coordinate passes the physical-action test: a distinct physical action modifies it and only it.

Dual valuation

Two price functions—mark-to-market and mark-to-model—over the same position set.

Encumbrance

Owned but not freely deployable: $\text{encumb} = \text{onloan} + \text{coll_post}$.

Event sourcing

All state is a projection of an append-only log of events (moves).

Forgetful mapping

The CDM-to-ledger map F , which preserves economic substance while forgetting business intent and workflow lineage.

Idempotency

Applying the same operation twice has no incremental effect beyond the first application.

Lifecycle event

A state transition on a unit (coupon, margin call, exercise, reset) expressed as a transaction.

Managed account

An internal book treated as a virtual portfolio with periodic cash settlement of accumulated PnL.

Monotone carrier

A state carrier whose accessor commutes with replay—meaning the value rebuilt from the first k events, then advanced by the rest, equals the value rebuilt from all events. Replay is therefore a single fold with no order dependence (C1; discharges P8).

Move stream

The immutable, append-only log of all atomic moves: the canonical internal record.

Obligation

A recorded commitment to perform a move by a stated time. Discharged when a matching committed transaction appears; liveness is read by a total discharge predicate (§14).

Option accessor

The position read $\text{position}(w, u)$ returns an optional value: *None* until the first event touches (w, u) , a value thereafter. No row is fabricated on read (C1).

Own

First coordinate of the position vector, $\vec{w}_e(u)[\text{own}]$: economic ownership, maintained by buy/sell, determines PnL.

Ownership consistency

$\text{avail} = \text{own} - \text{onloan} + \text{borr}$ for lendable securities. A definitional identity under the six-coordinate model (P20).

PnL

Profit and loss, $V_{t_1} - V_{t_0}$: depends only on initial and final portfolio states (P10).

Position vector

$\vec{w}_e(u) \in \mathbb{R}^6$: (own, onloan, borrr, coll_post, coll_recv, coll_rehyp). Available inventory avail = own – onloan + borrr is a projection computed on read. For non-lendable instruments the vector degenerates to a scalar (§15).

PositionState

Per-(wallet, unit) economic state: the third state home, holding conserved fields (accumulated cost, high-water mark, entry NAV). All per-(w , mandate) economic state lives here; no flat per-wallet scalars (C12). Read through the **Option accessor** (§4).

PosQty

The position-side quantity carried in a **PositionState**, a type distinct from trade **Qty**, cash, and price so the four cannot be combined by mistake; converted only at the futures-lifecycle boundary (§8).

ProductTerms

The first state home: a unit’s contractual terms, immutable, versioned, and append-only—a non-empty list of term versions, written first at registration (C6, C7) and never mutated in place. Amendment is two-track (C8).

Property-based testing

Checking invariants against randomly generated inputs.

Purity

Lifecycle functions produce identical outputs for identical inputs, with no side effects (P9).

$Q(u)$

Total quantity of unit u across all wallets, $Q(u) = \sum_{w \in \mathcal{W}} w_t(u)$. Zero by the conservation law, the framework being a closed double-entry system.

Real wallet

A wallet representing an actual portfolio position; changes affect PnL.

Reclassification

A move between wallets of the same entity with different position types (e.g. rehypothecation, coll_recv $\rightarrow$ coll_rehyp).

Registration

Introduction of a unit, recorded as a move-less **Transaction** that writes the first **ProductTerms** version and the **UnitStatus** default together (C5, C7). Vacuously conserved, its move list being empty; re-registration of an existing identifier is a hard error (C10). It is the move-less instance of the one event type, not an operation standing outside it (§4).

Self-consistency

Internally inconsistent states are structurally unreachable within the ledger’s scope, not merely detectable.

Single-Coordinate Move Principle

An atomic move modifies exactly one coordinate of one unit in the position vectors of the entities it connects.

Smart contract

A deterministic program mapping inputs and state to sequences of atomic moves.

State-sufficiency

Portfolio value depends only on current balances and prices, not on transaction history.

Time travel

Reconstruction of the ledger at any historical date and re-execution of lifecycle logic; the read side of **Clone_at**.

Transaction

The one atomic event recorded in the log: an ordered list of moves together with the state delta they carry—per-wallet bookkeeping (accumulated cost, high-water mark, entry NAV), shared **UnitStatus** writes, and either a **ProductTerms** introduction (registration) or an append (preserving amendment). Conservation holds by construction from the signed edge-sum of the moves; the empty move list is conserved for free, so a move-less event needs no check (P1). Applied whole or rejected whole through the single operation **applyTx** (P2, C3); there is no separate validation step and no unconserved transaction to reject.

Transfer

A move between wallets of different entities.

TRS

Total return swap: a contract converting virtual ledger performance into real cash flows.

Unit

Any tradable or bookable object: currency, security, derivative, contract.

UnitStatus

The second state home: one value per unit, shared—read identically by every holder. Its value changes over time, but **UnitStatus** is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward, product-declared defaults applied there (C5; §4). It is the sole home of unit-level shared state.

Virtual ledger

A separate instance of the same framework in which every wallet is virtual, used for strategy simulation.

Virtual wallet

A wallet representing an external counterparty; exists to close the system for conservation.

The twelve conditions. Each renders a class of illegal state unrepresentable; established in §4 (C7 and C10 in §3) and cross-referenced from the invariant hub (§16).

C1 Option accessor and monotone carrier, both.

C2 Handler-level conservation, per event class, with a vacuous base case.

C3 Atomic **Transaction** across the three state homes; partial application rejected.

C4 Capability-scoped reads; cross- (w, u) overlay reads forbidden; strategy exports flow only through **UnitStatus**.

C5 **UnitStatus** registration-total; product-declared defaults applied at Unit Store registration.

- C6** **ProductTerms** versioned, append-only; no in-place mutation.
- C7** **ProductTerms** registration-total; first write at registration.
- C8** Amendment two-track: a product-declared fungibility predicate decides *Preserving* (append a term version) versus *Breaking* (allocate a fresh unit, with **SupersededBy**).
- C9** Handlers on zero-holder units discharge $\sum = 0$ vacuously; the per-class proof obligation includes the empty case.
- C10** Re-registration of a unit identifier is a hard error, never a silent reset.
- C11** Each **PositionState** field is tagged with the canonical set of writers permitted to write it.
- C12** All per-(wallet, mandate) economic state lives in **PositionState**; no flat per-wallet scalars. Enforced by schema.

F The CDM Product Model—A Developer’s Guide

The ISDA Common Domain Model defines a layered object hierarchy for financial products. Each layer maps to one ledger construct, and the map crystallises a single result: the ledger unit is the CDM **Trade**, not the product. This appendix establishes that result through the six layers, then exercises three boundaries the basic layers do not reach—payout composition, tokenised custody, and date resolution—with one worked instrument per result. Throughout, “CDM” denotes Common Domain Model v6.0.0 (FINOS-governed); field names follow the Rosetta/Rune schema.

F.1 The Six-Layer Hierarchy

The layers narrow from shared to unique. Five are reusable infrastructure; only the Payout is product-specific, and only the Trade is a unit.

Layer 1 — Observable. The market quantity a contract references (an equity index, a rate). It is identical across product type, direction, and venue: an option and a future on the same index share one Observable. It is a market-data input to pricing and lifecycle observation—not a unit, never held or transferred.

Layer 2 — Payout. A choice type, and the only truly product-specific layer. An option is one **OptionPayout**; a future is a **PerformancePayout** paired with a **FixedPricePayout**, mirroring an equity swap. CDM does not distinguish listed from OTC here—the economic logic of a listed and an OTC option is the same payout with different strike, expiry, and settlement currency. A put flips **optionType**; a short reverses direction (**buyerSeller** for options, **payerReceiver** for swap-like legs); nothing else changes. The Payout is the move-generating logic of the smart contract: it defines what transfers occur under what conditions, and carries *no* counterparty identity and *no* collateral reference.

Layer 3 — EconomicTerms. Wraps the Payout with **effectiveDate**, **terminationDate**, and calculation-agent metadata. It drives the lifecycle state machine: the effective date triggers **ACTIVE**, the termination date triggers expiry, exercise, or settlement observation. Its **collateral** field is product-intrinsic (repo haircuts, securities-lending collateral)—not the variation-margin CSA, which attaches at Trade.

Layer 4 — NonTransferableProduct. Wraps EconomicTerms with identifiers and ISDA taxonomy. It is the contract *template*—the code, not the instance—shared by every trade with identical terms. CDM offers two product types: **TransferableProduct** (extends **Asset**; for instruments whose economics follow from ownership—equities, plain bonds) and **NonTransferableProduct** (wraps EconomicTerms; for contracts whose payout must be explicitly modelled). Listed options and futures are **NonTransferableProduct**: exchange-traded and fungible, but contracts with explicit payout logic, not assets transferred by ownership. Their exchange fungibility is captured by a shared template—all trades in one contract specification reference the same NonTransferableProduct.

Layer 5 — TradableProduct. Binds a template to a **TradeLot** (agreed price and quantity) and counterparty roles. It is the instance before collateral attachment: counterparties and economics, no collateral terms. A product quoted at many prices to many parties is many TradableProducts; confirmation makes one a Trade.

Layer 6 — Trade. Extends TradableProduct with **tradeIdentifier** (UTI), **tradeDate**, resolved parties (LEIs), **executionDetails**, and **collateral**. The instruments share everything up to here and diverge completely at Trade: counterparty (named bilateral vs. CCP), collateral source (bilateral CSA vs. CCP rulebook), and venue.

CDM layer	Ledger construct	Varies across instruments?
Observable	Market-data input	No
Payout	Move-generating logic	Yes—the only product-specific layer
EconomicTerms	Contract definition	No (structural wrapper)
NonTransferableProduct	Contract template	Shared if terms identical
TradableProduct	Priced instance	Trade-specific
Trade	Unit	Always

F.2 Unit Identity: Trade Is the Unit

Principle F.1 (Trade = Unit). *The ledger unit is the full CDM Trade object, including its Collateral field. Product identity is not unit identity.*

The unit key is the Trade identifier (UTI for OTC; contract specification for listed). The forgetful map $U : \mathbf{Trade} \rightarrow \mathbf{NonTransferableProduct}$ that strips counterparty and collateral is many-to-one—meaning many distinct trades collapse to one product. The ledger operates on the source **Trade**, never on the quotient, and the Unit Store (Section 3) enforces injectivity at registration: distinct Trades map to distinct units.

The dual-CSA test. Two OTC trades reference the same template—a European call on the same index at the same strike—but one carries a USD CSA (SOFR discounting, New York law) and the other a EUR CSA (ESTR discounting, English law). Because **Collateral** differs, the Trade objects differ, and the units differ. Two otherwise-identical options collateralised in different currencies are **different units**: different discount curves, margin flows, and settlement currencies.

Listed and cleared. For listed instruments the CCP novates itself in as sole counterparty, so all positions in one contract specification are fungible regardless of original counterparty. Listed-vs-OTC is therefore a Trade-level distinction—counterparty becomes the CCP, collateral becomes the CCP rulebook, venue becomes the exchange—not a product-level one. For futures, daily settlement is itself the PnL realisation rather than collateral against it; the cash mechanics are in the futures lifecycle (Section 8).

CCP margin is a CDM gap. CDM v6.0.0 has no type for CCP margin rules distinct from bilateral collateral provisions; the `Collateral` type on `Trade` was built for bilateral CSAs. For cleared trades the margin framework is an external reference to the CCP rulebook. The two-contract design (payoff contract plus margin contract) handles both regimes; CDM’s representation of the cleared case is the less mature.

F.3 Payout Composition and the TransferableProduct Boundary

A structured equity note tests payout composition and the product-type boundary at once. BankCo issues a 1Y zero-coupon note on an equity index, EUR 1,000,000 notional, to FundCo. Redemption is

$$R = N \times \min\left(1, \frac{S_T}{S_0}\right),$$

par if the index is flat or up, full downside if it falls. Economically the note is a zero-coupon bond *plus* a short put struck at par. It has an ISIN, lists on an exchange, and settles through a CSD: a transferable security in law and market infrastructure.

Result 1: economic decomposition is not CDM decomposition. The correct representation is a single `PerformancePayout` with a return capped at 0%—not a `FixedPricePayout` (par bond) plus an `OptionPayout` (short put). The decomposition is economically correct yet wrong as a CDM model for three reasons: the embedded put has no independent exercise right (the redemption formula applies automatically, so there is no `OptionPayout`); the note has one contractual cash flow at maturity, not two; and CDM qualification recognises a capped `PerformancePayout` on an equity index as a known structure, but not a premium-less `OptionPayout`. Risk systems decompose into bond + put for Greeks; CDM models the contractual cash flow.

Result 2: TransferableProduct is narrower than “transferable.” The note is a `NonTransferableProduct` despite being a transferable security. The criterion is not legal transferability but modelling complexity: `TransferableProduct` is for instruments whose cash flows follow mechanically from ownership (a plain bond’s coupon and par); any instrument whose cash flows depend on an observable—requiring an explicit Payout tree—is `NonTransferableProduct`. The ISIN lives in the `NonTransferableProduct identifier` field; `TransferableProduct` is not required to carry it. This parallels the IFRS 9 SPPI test: the note fails SPPI because of the equity-linked redemption and is measured at fair value, exactly the signal—non-trivial payout—that forces `NonTransferableProduct` and trading-book scope (Section 1).

Result 3: one unit, three settlement paradigms. The note is one security, one ISIN, one Trade, one ledger unit; the embedded put is not separately tradable, exercisable, or settleable. Issuance and redemption are two-move transactions that conserve per unit. In the index-down-20% scenario redemption is EUR 800,000:

$$\text{Move}(\text{BankCo} \rightarrow \text{FundCo}, \text{EUR}, 800,000), \quad \text{Move}(\text{FundCo} \rightarrow \text{BankCo}, \text{note}, 1).$$

The “missing” EUR 200,000 is the put payoff, already recognised through mark-to-market: before maturity FundCo holds one note valued at EUR 800,000, after maturity EUR 800,000 cash, and the value jump at redemption is zero—lifecycle value invariance (Section 1) holds. The note also exposes the settlement spectrum: bilateral CSA (well modelled), CCP margin (partially modelled, Section F.2), and CSD settlement (essentially unmodelled—CDM v6.0.0 has no `CSDSettlement` type, so settlement through Euroclear or Clearstream is carried in generic `executionDetails`). CDM support decreases left to right, reflecting its OTC-derivative origin.

F.4 Tokenised Securities: Double-Counting and the Flat Custodian

Where the structured note tested product-type, a tokenised equity tests custody. In the mirror/backed model a regulated custodian holds real shares in segregation and issues on-chain tokens 1:1; each token is a beneficial claim on one share, held against the custodian. The underlying exposure, dividends, and corporate actions are unchanged; the identifier scheme (contract address, chain ID, token standard), custodial layer, settlement (on-chain atomic vs. T+1), and fractional divisibility are new.

Two units, not one. The share (NVDA) and the token (NVDA_TOKEN) are distinct units in the Unit Store: different settlement mechanisms, different identifiers, and different risk profiles—the token carries custodian credit risk the share does not. Treating them as one unit would erase the distinction between a holder of real shares and a holder of tokenised claims.

The double-counting problem and its resolution. Distinct units invite a naïve aggregation of NVDA exposure that double-counts: 1,000,000 real shares in custody plus 1,000,000 tokens held by investors sums to 2,000,000, though only 1,000,000 shares of exposure exist.

Entity	own(NVDA)	own(NVDA_TOKEN)	Net NVDA exposure
Custodian	+1,000,000	−1,000,000	0
Holders (all)	0	+1,000,000	+1,000,000
System	+1,000,000	0	+1,000,000

Principle F.2 (The custodian is flat). *A backing custodian’s net economic exposure is zero by conservation, not by choice. It is long the underlying and short the tokens it issued in equal measure: $\text{own}(\text{NVDA}) = +1,000,000$ and $\text{own}(\text{NVDA_TOKEN}) = -1,000,000$, so a unit price move changes asset and liability identically.*

Economic exposure is the read-time projection $\text{own}(\text{NVDA}) + \text{own}(\text{NVDA_TOKEN})$ for any entity: zero for the custodian, +100 for a holder of 100 tokens. The token balance conserves to zero across issuer and holders; the share balance conserves against the virtual issuance wallet. If the custodian issued tokens without holding shares, the shortfall would surface as a conservation violation—the resolution is a consequence of system closure, not a modelling convention. The structure is the securities-lending custody problem (Section 15): possession is not ownership. SBL distinguishes them by the position vector’s **own** and **onloan** coordinates; tokenisation distinguishes them by liability netting; the principle is identical.

Variants. Two cases avoid the problem because no backing inventory exists: a *synthetic* token is a delta-one derivative on the underlier (**NonTransferableProduct** with a **PerformancePayout**, no custodian); a *native-issuance* token is the share on chain (**TransferableProduct** with blockchain-native identifiers). The mirror token is the first **TransferableProduct** in the walkthrough and the only one with a non-trivial custody layer.

Four CDM v6.0.0 gaps for tokenised securities.

1. **No blockchain identifier type.** **ProductIdentifier/ProductIdSourceEnum** omit contract address, chain ID, and token standard. Workaround: **Other** with a scheme convention. Significant—every token needs an ad-hoc convention.
2. **No custodial backing model.** No type relates a token to the underlying held by a custodian; backing ratio, custodian identity, and segregation are not representable. Workaround: trade-level **contractDetails**. Significant—the custodial relationship is the defining feature.

3. **DigitalAsset underspecified.** The branch exists but neither extends `TransferableProduct` nor carries `EconomicTerms`; built for crypto-native assets, not tokenised securities. Moderate.
4. **No on-chain settlement type.** `SettlementTypeEnum` has no on-chain atomic value and no structure for chain, confirmation depth, or gas. Workaround: `Other`. Moderate.

The gaps are incremental extensions, consistent with CDM’s trajectory from bilateral OTC to cleared to tokenised; none requires an architectural change. On-chain delivery-versus-payment, when both legs settle on chain, is the strongest DvP—atomic within one transaction, eliminating settlement risk; mixed on/off-chain settlement degrades to conventional DvP risk, a settlement-layer concern (Section 12), not a ledger-layer one.

F.5 Date Handling: Types, Resolution, and Gotchas

A contract date is never a bare date: it carries adjustment rules, calendar references, and resolution logic that fix the operational date. CDM composes these from small precise types in the `base.datetime` namespace. The types feed every dated computation in this document—the IRS move schedule and day-count fractions (Section 6), coupon adjustment (Section 14.3), and settlement-date adjustment (Section 12). A wrong calendar, convention, or day count is a valuation error, not a rounding error.

F.5.1 The Type Taxonomy

Type / enum	Role
<code>date</code>	Resolved ISO 8601 calendar date; the output, not the input.
<code>Period</code>	Multiplier + unit (D/W/M/Y); a zero-length period must be in days.
<code>Offset</code>	<code>Period</code> + <code>dayType</code> (Business/Calendar/Exchange...); <code>dayType</code> present only when in days.
<code>BusinessDayConventionEnum</code>	FOLLOWING, MODFOLLOWING, PRECEDING, NEAREST, NONE...
<code>BusinessCenters</code>	Holiday calendars (USNY, GBLO, JPTO...); a date is good only if good in <i>all</i> listed.
<code>BusinessDayAdjustments</code>	Convention + business centres: the reusable adjustment recipe.
<code>AdjustableDate</code>	<code>unadjustedDate</code> (contractual) + adjustments → <code>adjustedDate</code> (operational).
<code>RelativeDateOffset</code>	“ <i>n</i> days/months from a reference date, adjusted by <i>C</i> against <i>B</i> .”
<code>AdjustableOrRelativeDate</code>	Required-choice union of the two above; used throughout the product model.
<code>CalculationPeriodFrequency</code>	Frequency + <code>rollConvention</code> (EOM, IMM, _1.._30); drives schedule generation.
<code>DayCountFractionEnum</code>	ACT/360, ACT/365F, 30/360, ACT/ACT (ISDA/ICMA)...; each a dispatched <code>YearFraction</code> .

MODFOLLOWING dominates OTC derivatives: it advances to the next business day unless that crosses a month boundary, then retreats, keeping the payment date aligned with its calculation period. Two day-count formulas recur: ACT/360 is actual days/360; 30/360 (Bond Basis) is

$$\frac{360(Y_2 - Y_1) + 30(M_2 - M_1) + (D_2 - D_1)}{360}, \quad D_1 = \min(d_1, 30), D_2 = \min(d_2, 30) \text{ if } D_1 > 29.$$

F.5.2 The Resolution Chain

An `AdjustableOrRelativeDate` resolves deterministically in three steps; `ResolveAdjustableDate` encapsulates them.

1. **Resolve the reference.** If relative, offset from the anchor (`dateRelativeTo`): Add `BusinessDays` when `dayType = Business`, calendar arithmetic otherwise. Yields the unadjusted derived date.
2. **Apply business-day adjustment.** Test the unadjusted date against the centres; if not a good business day, shift by the convention.
3. **The adjusted date is final**—the operational date for the cash flow, exercise, or settlement.

A 5Y USD SOFR swap rolling on the 3rd under `MODFOLLOWING/USNY` illustrates the chain: 2026-10-03 (Sat) → Mon Oct 5; 2027-10-03 (Sun) → `FOLLOWING` within month → Mon Oct 4; weekday roll dates pass unadjusted. The first fixed period (2026-04-03 to 2026-10-05, 30/360) gives $\delta = (30 \cdot 6 + 2)/360 = 182/360 \approx 0.5056$; the first floating period (2026-04-03 to 2026-07-03, `ACT/360`) gives $91/360 \approx 0.2528$. These adjusted dates and fractions are exactly the reset dates t_k and δ_k the IRS contract consumes (Section 6).

F.5.3 Gotchas

Each is a bug class the type system prevents only when the types are used correctly.

1. **Omitted business centres.** A convention with no centres adjusts against no calendar—every day is a business day and the convention is inert. Validate that any non-`NONE` convention has at least one centre.
2. **Calendar vs. business days.** “T+2” from a Friday is Sunday under `Calendar`, Tuesday under `Business`. `dayType` is load-bearing, not decoration.
3. **EOM vs. fixed-day roll.** EOM rolls to each month’s last day (Feb 28/29, Mar 31); `_28` always rolls to the 28th. Confusing them shifts payments by up to three days.
4. **Joint-holiday intersection.** Multiple centres take the *intersection* of business days: good in New York but a London holiday is not good for `USNY+GBLO`. More dates are non-business under joint calendars than either alone.
5. **Stub periods.** An effective date off the roll cycle produces a short or long first period; `balanceOfFirstPeriod` places the stub. Ignoring it gives a wrong year fraction for that period.
6. **Day-count mismatch.** For a 182-day semi-annual period, `ACT/360` gives 0.5056 and `30/360` gives 0.5000. On EUR/USD 50,000,000 at 4%, the difference is $0.005556 \times 0.04 \times 50,000,000 \approx \$11,111$ —a valuation error, not a rounding error.

G Available Inventory Identity: Eight-Scenario Verification

The available-inventory identity (P20, 15.12)

$$\text{avail} = \text{own} - \text{onloan} + \text{borr}$$

is verified across eight scenarios, each exercising a distinct combination of the six stored coordinates `own`, `onl`, `borr`, `c_p` (`coll_post`), `c_r` (`coll_recv`), `c_rh` (`coll_rehyp`). Available inventory is never stored: it is computed on read from three of those coordinates (`own`, `onl`, `borr`), so the identity cannot be violated by construction.

Entity (unit)	own	onl	borr	c_p	c_r	c_rh	avail = own − onl + borr
---------------	-----	-----	------	-----	-----	------	--------------------------

A. Alice buys 1,000 AAPL outright.

Entity (unit)	own	onl	borr	c_p	c_r	c_rh	avail = own - onl + borr
Alice (AAPL)	1000	0	0	0	0	0	$1000 - 0 + 0 = 1000$ ✓
Virtual	-1000	—	—	—	—	—	(issuance counterpart)
<i>B. Alice owns 1,000 VOD, lends 400 to Bob.</i>							
Alice (VOD)	1000	400	0	0	0	0	$1000 - 400 + 0 = 600$ ✓
Bob (VOD)	0	0	400	0	0	0	$0 - 0 + 400 = 400$ ✓
<i>C. Bob lends 1,000 NVDA to Alice, Alice sells 500.</i>							
Bob (NVDA)	1000	1000	0	0	0	0	$1000 - 1000 + 0 = 0$ ✓
Alice (NVDA)	-500	0	1000	0	0	0	$-500 - 0 + 1000 = 500$ ✓
<i>D. Swap desk holds 5Y EUR IRS (non-lendable).</i>							
Desk (IRS)	+1	0	0	0	0	0	N/A (non-lendable; own = 1)
<i>E. Bob lends 1,000 NVDA, Alice sells all 1,000.</i>							
Bob (NVDA)	1000	1000	0	0	0	0	$1000 - 1000 + 0 = 0$ ✓
Alice (NVDA)	-1000	0	1000	0	0	0	$-1000 - 0 + 1000 = 0$ ✓
<i>F. Alice holds 1,000 AAPL, lends 400 to Bob, Bob sells 200.</i>							
Alice (AAPL)	1000	400	0	0	0	0	$1000 - 400 + 0 = 600$ ✓
Bob (AAPL)	-200	0	400	0	0	0	$-200 - 0 + 400 = 200$ ✓
<i>G. Rehypotheication chain: Alice lends 600 MSFT to Bob, Bob re-lends 300 to Charlie.</i>							
Alice (MSFT)	1000	600	0	0	0	0	$1000 - 600 + 0 = 400$ ✓
Bob (MSFT)	0	300	600	0	0	0	$0 - 300 + 600 = 300$ ✓
Charlie (MSFT)	0	0	300	0	0	0	$0 - 0 + 300 = 300$ ✓
<i>H. Alice lends 400 AAPL to Bob under security interest; Bob posts 200 MSFT as pledge.</i>							
Bob (MSFT)	0	0	0	200	0	0	$0 - 0 + 0 = 0$ ✓
Alice (MSFT)	0	0	0	0	200	0	$0 - 0 + 0 = 0$ ✓

Remark G.1 (Rehypotheication defeats the naive identity). *For intermediary re-lender Bob (G), $\text{own} = 0 \neq \text{avail} + \text{onl} = 300 + 300$: the identity $\text{own} = \text{avail} + \text{onloan}$ fails. The correct projection $\text{avail} = \text{own} - \text{onl} + \text{borr}$ holds because it credits borrowed shares.*

Remark G.2 (Pledge preserves economic ownership). *Under pledge (H), Bob retains economic ownership of posted collateral; the regime difference surfaces in PnL projection, where Bob's MSFT PnL is $c_p(200) \times P$, not $\text{own}(0) \times P$.*

H European SBL Worked Example: GMSLA 2010 Title Transfer with Rehypotheication

A complete European securities loan under GMSLA 2010 (title transfer): non-cash collateral, rehypotheication, daily mark-to-market, a manufactured dividend, a partial recall, and full return. Every move is a single-coordinate write of the six-coordinate position vector (Section 15, Principle 15.1); conservation is verified at each step. This is the title-transfer correctness witness; the pledge counterpart is Appendix I.

H.1 Participants and Setup

Entity	Role	Description
CALPERS_EU	Lender (beneficial owner)	Holds 500,000 TotalEnergies (ISIN FR0000120271). Agent lender BNY Mellon.
GOLDMAN	Borrower	Prime broker; borrows TTE to cover a client short.
HEDGE_FUND	Goldman's client	Short seller; sells the borrowed TTE to market.
MARKET	Market buyer	Virtual wallet for the open market.
COUNTERPARTY_C	Third party	Receives rehypothecated Bund collateral in a separate repo.

Loan terms. 500,000 TotalEnergies (TTE), open loan, fee 25 bps p.a. (ACT/360), non-cash collateral (German Bund, ISIN DE0001102580), margin 102%, GMSLA 2010 title transfer. Under title transfer Goldman takes ownership of the Bund collateral and may rehypothecate it; SFTR Article 15 documentation is required.

Units. TTE (equity), BUND (German Bund), EUR (cash). Position vector (own, onloan, borr, coll_post, coll_recv, coll_rehyp), abbreviated own, onl, borr, c_p, c_r, c_rh.

Remark H.1. *Loan, collateral, and exposure values follow the valuation rules of the valuation satellite specification (IBP-163, IBP-129, IBP-191); the figures are stated here as inputs, not re-derived. Intermediate quantities are computed in exact integer minor units; round-half-to-even (IEEE 754) applies only at the settlement-instruction projection boundary.*

H.2 Step 1: Loan Initiation (Day 0)

TTE €58.50; Bund €98.50 per €100 nominal.

$$LV = 500,000 \times 58.50 = \text{€}29,250,000, \quad RCV = LV \times 102\% = \text{€}29,835,000.$$

$$Q_{\text{Bund}} = \left\lceil \frac{29,835,000}{98.50} \right\rceil = 302,894 \text{ (units of €100 nominal),}$$

market value $302,894 \times 98.50 = \text{€}29,835,059$ (slight over-collateralisation from rounding up).

```
tau_1 = Transaction(type = SETTLEMENT, day = 0):
  -- Securities leg (TTE)
  Move 1: CALPERS_EU onloan += 500,000 [onloan]
  Move 2: GOLDMAN      borr  += 500,000 [borr]
  -- Collateral leg (BUND) -- title transfer
  Move 3: GOLDMAN      own   -= 302,894 [own]
  Move 4: CALPERS_EU   coll_recv += 302,894 [coll_recv]
```

Move 3 writes own (not coll_post): under title transfer the poster transfers *ownership*, so Goldman's Bund ownership decreases and CalPERS holds the Bunds as received collateral. This is the defining distinction from the pledge regime (Appendix I). Virtual wallets absorb the contra-entries for both legs.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	500,000	0	0	0	0	0
CALPERS_EU	BUND	0	0	0	0	302,894	0	0
GOLDMAN	TTE	0	0	500,000	0	0	0	500,000
GOLDMAN	BUND	-302,894	0	0	0	0	0	-302,894
Virtual _{C↔G}	TTE	-500,000	(securities delivery contra)					
Virtual _{mkt}	BUND	+302,894	(Bund delivery contra)					

Conservation (TTE). $500,000_C + 500,000_{G,\text{borr}} - 500,000_{\text{Virt}} - 500,000_{\text{Cust}} = 0$. ✓ **P18.** CalPERS own(TTE) = 500,000, unchanged. ✓ **SFTR.** NEWT.

H.3 Step 2: Delivery to Hedge Fund and Short Sale (Day 0)

Locate (EU SSR Article 12(1)). Before the short sale HEDGE_FUND satisfies the locate obligation; the borrow from CalPERS meets Article 12(1)(a) (the person “has borrowed the share”). Locate is a pre-trade validation the executor checks before τ_3 ; it generates no moves and changes no coordinate.

```
tau_2 = Transaction(type = SETTLEMENT, day = 0):  -- Goldman -> Hedge Fund
    Move 1: GOLDMAN      own -= 500,000      [own]
    Move 2: HEDGE_FUND   own += 500,000      [own]

tau_3 = Transaction(type = SETTLEMENT, day = 0):  -- Hedge Fund short sale to
    market
    Move 1: HEDGE_FUND   own -= 500,000      [own]
    Move 2: MARKET      own += 500,000      [own]
    Move 3: MARKET      own -= 29,250,000    [own, EUR]
    Move 4: HEDGE_FUND   own += 29,250,000    [own, EUR]
```

Goldman’s borr is untouched: it tracks the return obligation, not possession.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	500,000	0	0	0	0	0
GOLDMAN	TTE	−500,000	0	500,000	0	0	0	0
HEDGE_FUND	TTE	0	0	0	0	0	0	0
MARKET	TTE	500,000	0	0	0	0	0	500,000
HEDGE_FUND	EUR	29,250,000	0	0	0	0	0	29,250,000

Goldman is flat in available shares ($-500,000 + 500,000_{\text{borr}} = 0$) but short in ownership: the economic short passed through from the hedge fund.

Conservation (TTE). $500,000_C - 500,000_G + 500,000_M - 500,000_{\text{Virt}} = 0$. ✓

H.4 Step 3: Rehypothecation of Bund Collateral (Day 1)

Goldman, owning the Bunds under title transfer, re-uses 302,894 of them as collateral in a separate repo with COUNTERPARTY_C. P19 (rehypothecation regime compliance) requires SFTR Article 15 re-use documentation: an operational compliance flag, not a move.

```
tau_4a = Transaction(type = RECLASSIFICATION, day = 1):  -- within CalPERS vector
    Move 1: CALPERS_EU   coll_recv -= 302,894    [coll_recv, BUND]
    Move 2: CALPERS_EU   coll_rehyp += 302,894    [coll_rehyp, BUND]

tau_4b = Transaction(type = SETTLEMENT, day = 1):  -- onward delivery to
    COUNTERPARTY_C
    Move 1: GOLDMAN      own -= 302,894          [own, BUND]
    Move 2: COUNTERPARTY_C own += 302,894          [own, BUND]
    Move 3: COUNTERPARTY_C own -= 29,835,059      [own, EUR]
    Move 4: GOLDMAN      own += 29,835,059        [own, EUR]
```

Goldman’s Bund own moves $-302,894 \rightarrow -605,788$: doubly short, having delivered Bunds to CalPERS and to COUNTERPARTY_C, and owing both. The reclassification records the re-use inside CalPERS’s vector without changing CalPERS’s total Bund holding.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	BUND	0	0	0	0	0	302,894	0
GOLDMAN	BUND	−605,788	0	0	0	0	0	−605,788
COUNTERPARTY_C	BUND	302,894	0	0	0	0	0	302,894
GOLDMAN	EUR	29,835,059	0	0	0	0	0	29,835,059
COUNTERPARTY_C	EUR	−29,835,059	0	0	0	0	0	−29,835,059

Conservation (BUND). $302,894_C - 605,788_G + 302,894_{CC} + 302,894_{\text{Virt}} - 302,894_{\text{Cust}} = 0$.
✓ **P19.** Regime TITLE_TRANSFER permits rehypothecation; Art. 15 flag set. ✓

H.5 Step 4: Mark-to-Market (Day 3)

TTE €60.25; Bund €98.20.

$LV_3 = 500,000 \times 60.25 \times 102\% = €30,727,500$, collateral = $302,894 \times 98.20 = €29,744,189$.

Exposure = $30,727,500 - 29,744,189 = €983,311$, $Q_{\text{add}} = \left\lceil \frac{983,311}{98.20} \right\rceil = 10,014$ units.

The lender is under-collateralised; a margin call delivers additional Bunds.

```
tau_5 = Transaction(type = MARGIN_CALL, day = 3):
    Move 1: GOLDMAN      own      -= 10,014    [own, BUND]
    Move 2: CALPERS_EU   coll_rcv += 10,014    [coll_rcv, BUND]
```

The addition enters coll_rcv (not coll_rehyp): it is not yet re-used.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh
CALPERS_EU	BUND	0	0	0	0	10,014	302,894
GOLDMAN	BUND	-615,802	0	0	0	0	0
COUNTERPARTY_C	BUND	302,894	0	0	0	0	0

P13. CalPERS holds $312,908 \text{ Bunds} \times 98.20 = €30,727,566 \geq €30,727,500$ required. ✓ **Fee accrual (Days 0–3).** Day 0 base $29,250,000 \times 0.0025/360 = €203.13/\text{day}$; from Day 3, $30,125,000 \times 0.0025/360 = €209.20/\text{day}$. Running total €818.58. **SFTR.** VALU.

H.6 Step 5: Manufactured Dividend (Day 7)

TotalEnergies pays €1.25 per share. CalPERS retains economic ownership; Goldman pays a manufactured dividend of $500,000 \times 1.25 = €625,000$. The manufactured payment is a contractual GMSLA payment, not a distribution, and its net amount may differ from the gross dividend under the applicable withholding-tax treaty.

```
tau_6 = Transaction(type = CORP_ACTION, day = 7):
    Move 1: GOLDMAN      own -= 625,000    [own, EUR]
    Move 2: CALPERS_EU   own += 625,000    [own, EUR]
```

Entity (EUR)	own	onl	borr	c_p	c_r	c_rh
CALPERS_EU	625,000	0	0	0	0	0
GOLDMAN	29,210,059	0	0	0	0	0

Conservation (EUR). $625,000_C + 29,210,059_G + 29,250,000_{HF} - 29,835,059_{CC} + \text{virtual offsets} = 0$. ✓

H.7 Step 6: Partial Recall and Return (Days 10–12)

CalPERS sells 200,000 TTE; BNY Mellon recalls 200,000 shares. Recall notice (IBP-328) reaches Goldman on Day 10; settlement at $T + 2$ (Euronext Paris) on Day 12.

```
tau_7 = Transaction(type = RECALL, day = 10):
    -- Move-less: empty move list (vacuously conserved); carries only a state delta.
    -- SBL state ACTIVE -> RECALLED; recall_qty = 200,000; recall_date = Day 10
```

Day 12. TTE €59.80; Bund €98.30. Remaining loan 300,000:

$$LV_{12} = 300,000 \times 59.80 \times 102\% = €18,298,800.$$

Collateral held $312,908 \times 98.30 = €30,758,856$; excess €12,460,056:

$$Q_{\text{release}} = \left\lfloor \frac{12,460,056}{98.30} \right\rfloor = 126,755 \text{ Bund units.}$$

Remaining $312,908 - 126,755 = 186,153$ Bunds, value $186,153 \times 98.30 = €18,298,835 \geq LV_{12}$ (P13 ✓). Release draws from coll_recv first (10,014), then coll_rehyp (116,741).

```
tau_8 = Transaction(type = PARTIAL_RETURN, day = 12):
    -- Securities return (TTE)
    Move 1: GOLDMAN      borr      -= 200,000    [borr]
    Move 2: CALPERS_EU   onloan     -= 200,000    [onloan]
    -- Collateral release (BUND): coll_recv then coll_rehyp
    Move 3: CALPERS_EU   coll_recv  -= 10,014     [coll_recv]
    Move 4: GOLDMAN      own        += 10,014     [own]
    Move 5: CALPERS_EU   coll_rehyp -= 116,741    [coll_rehyp]
    Move 6: GOLDMAN      own        += 116,741    [own]

tau_8b = Transaction(type = SETTLEMENT, day = 12): -- Goldman unwinds repo leg
    Move 1: COUNTERPARTY_C own      -= 116,741    [own, BUND]
    Move 2: GOLDMAN      own      += 116,741    [own, BUND]
    Move 3: GOLDMAN      own      -= 11,473,439   [own, EUR]
    Move 4: COUNTERPARTY_C own      += 11,473,439   [own, EUR]
```

Releasing 116,741 from coll_rehyp forces Goldman to recall those Bunds from COUNTERPARTY_C first (τ_{8b}).

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	300,000	0	0	0	0	200,000
GOLDMAN	TTE	-500,000	0	300,000	0	0	0	-200,000
CALPERS_EU	BUND	0	0	0	0	0	186,153	0
GOLDMAN	BUND	-372,306	0	0	0	0	0	-372,306
COUNTERPARTY_C	BUND	186,153	0	0	0	0	0	186,153

P18. CalPERS own(TTE) = 500,000, unchanged. ✓ **P15.** Loan quantity $500,000 \rightarrow 300,000$, monotonically non-increasing. ✓ **SFTR.** MODI (not ETRM: partial, not full return).

H.8 Step 7: Full Return and Settlement (Day 20)

Remaining 300,000 TTE returned. TTE €61.00; Bund €98.40.

```
tau_9 = Transaction(type = FULL_RETURN, day = 20):
    Move 1: GOLDMAN      borr      -= 300,000    [borr]
    Move 2: CALPERS_EU   onloan     -= 300,000    [onloan]
    Move 3: CALPERS_EU   coll_rehyp -= 186,153    [coll_rehyp]
    Move 4: GOLDMAN      own        += 186,153    [own]

tau_9b = Transaction(type = SETTLEMENT, day = 20): -- unwind remaining repo
    Move 1: COUNTERPARTY_C own      -= 186,153    [own, BUND]
    Move 2: GOLDMAN      own      += 186,153    [own, BUND]
    Move 3: GOLDMAN      own      -= 18,308,845   [own, EUR]
    Move 4: COUNTERPARTY_C own      += 18,308,845   [own, EUR]
```

Fee. Accrued daily on the mark-to-market loan value over 20 days:

Period	Qty	Loan value (no margin)	Daily fee
Days 0–2	500,000	€29,250,000	€203.13
Days 3–6	500,000	€30,125,000	€209.20
Days 7–11	500,000	various	≈€207.64 avg
Days 12–19	300,000	≈€18,000,000	≈€125.00 avg

Total accrued fee €3,548.40 (exact amount depends on daily closing prices).

```
tau_10 = Transaction(type = FEE_SETTLEMENT, day = 20):
  Move 1: GOLDMAN      own -= 3,548.40    [own, EUR]
  Move 2: CALPERS_EU   own += 3,548.40    [own, EUR]
```

Loan disposal. $\vec{w}_{\text{CALPERS}}(u_{\text{loan}})[\text{own}]: +1 \rightarrow 0$; $\vec{w}_{\text{GOLDMAN}}(u_{\text{loan}})[\text{own}]: -1 \rightarrow 0$. SBL state RETURNED (terminal); SFTR ETRM.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	0	0	0	0	0	500,000
CALPERS_EU	BUND	0	0	0	0	0	0	0
CALPERS_EU	EUR	628,548	0	0	0	0	0	628,548
GOLDMAN	TTE	−500,000	0	0	0	0	0	−500,000
GOLDMAN	BUND	0	0	0	0	0	0	0
GOLDMAN	EUR	−575,773	0	0	0	0	0	−575,773
COUNTERPARTY_C	EUR	−52,775	0	0	0	0	0	−52,775
HEDGE_FUND	EUR	29,250,000	0	0	0	0	0	29,250,000
MARKET	TTE	500,000	0	0	0	0	0	500,000

Goldman retains $\text{own}(\text{TTE}) = -500,000$ from the delivery to the hedge fund (a separate relationship); the CalPERS loan is fully closed, all 500,000 TTE returned, 0 on loan.

Final conservation. TTE: $500,000_C - 500,000_G + 500,000_M - 500,000_{\text{Virt}} = 0 \checkmark$. BUND: all zero $\checkmark$. EUR: sums to zero across all entities including virtual wallets $\checkmark$.

PnL.

- **CalPERS:** €3,548.40 fee + €625,000 manufactured dividend; TTE position unchanged, unrealised $500,000 \times (61.00 - 58.50) = €1,250,000$.
- **Goldman:** €3,548.40 fee + €625,000 manufactured dividend paid; earned the repo spread on rehypothecated Bunds.
- **Hedge Fund:** short PnL $(58.50 - 61.00) \times 500,000 = -€1,250,000$; must buy back at €61.00 to close.

I US SBL Worked Example: SEC Rule 15c3-3 with Cash Collateral

This appendix is a regulatory correctness witness for the US securities-lending regime: cash collateral, rebate mechanics, and the rehypothecation cap of SEC Rule 15c3-3 [2], with FINRA SLATE reporting mapped at each step. It is the counterpart to the European title-transfer example (App. H). Title transfer is not pledge; the two regimes exercise the same Ledger primitives under different law, and both witnesses are retained.

I.1 Participants and Setup

Entity	Role	Description
FIDELITY	Lender (beneficial owner)	US mutual fund. Holds 1,000,000 shares of NVIDIA (NVDA, ISIN US67066G1040). Agent lender: State Street.
MORGAN_STANLEY	Borrower	US broker-dealer. Borrows NVDA to cover a client short. Subject to Rule 15c3-3.
STATE_STREET	Agent lender	Acts for FIDELITY; negotiates and manages the loan.

Loan terms. 1,000,000 NVDA, open loan, cash collateral (USD), margin 102%. Cash rebate: Fed Funds Effective Rate minus 25 bps. Take Fed Funds = 5.25%, so rebate rate = 5.00%. The lender reinvests the cash collateral and earns the spread between reinvestment return and rebate paid. As in App. H, intermediate quantities are computed in exact integer minor units; the fractional cash figures shown below (rebates, spreads) are display roundings, with round-half-to-even (IEEE 754) applied only at the settlement-instruction projection boundary.

Legal regime US_15C3_3.

- Cash collateral transfers from Morgan Stanley to Fidelity/State Street, who reinvest it.
- Rehypothecation of customer securities is capped at 140% of the customer’s debit balance (Rule 15c3-3(b)(3)).
- The poster’s PnL includes coll_post: $V_e^{15c3} = \sum_u (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{coll_post}]) \cdot P_t(u)$.

Locate precondition (Reg SHO Rule 203(b)(1)) [3]. Before the client’s short sale executes, Morgan Stanley satisfies the locate obligation, either by holding the borrow or by a locate confirmation against inventory. The locate is pre-trade validation: no moves occur, no coordinate changes, and conservation is not engaged until the borrow executes.

I.2 Step 1: Loan Initiation (Day 0)

NVDA price \$875.00. Loan value and required collateral:

$$\text{LV} = 1,000,000 \times 875 = \$875,000,000, \quad \text{RCV} = 875,000,000 \times 102\% = \$892,500,000.$$

Cash is always owned; the collateral leg writes own on the poster and coll_recv on the receiver, the earmark that must be returned at close.

Each τ below is a single Transaction carrying moves, with **txIntroduce** = NOTHING and **txAppend** = NOTHING: neither a registration nor an amendment. Its move list is the conserved flow, so conservation holds by construction from the signed edge-sum; the trailing comment names the economic role, not a type field on the core Transaction.

```
tau_1 = Transaction(day 0):  -- securities-loan settlement
  -- Securities leg (unit: NVDA)
  Move 1: FIDELITY          onloan    += 1,000,000    [onloan]
  Move 2: MORGAN_STANLEY    borr      += 1,000,000    [borr]
  -- Cash collateral leg (unit: USD)
  Move 3: MORGAN_STANLEY    own       -= 892,500,000  [own]
  Move 4: FIDELITY          coll_recv += 892,500,000  [coll_recv]
```

Units. NVDA (equity), USD (cash). Position vector

$$(\text{own}, \text{onloan}, \text{borr}, \text{coll_post}, \text{coll_recv}, \text{coll_rehyp}),$$

abbreviated own, onl, borr, c_p, c_r, c_rh.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
FIDELITY	NVDA	1,000,000	1,000,000	0	0	0	0	0
FIDELITY	USD	0	0	0	0	892,500,000	0	0
MORGAN_STANLEY	NVDA	0	0	1,000,000	0	0	0	1,000,000
MORGAN_STANLEY	USD	-892,500,000	0	0	0	0	0	-892,500,000

P18 check. Fidelity $\vec{w}(\text{NVDA})[\text{own}] = 1,000,000$, unchanged. ✓ **SLATE.** New Loan Event: FINRA Loan ID, Client Loan ID, execution timestamp, ISIN/CUSIP, quantity, collateral type (Cash), collateral amount, rebate rate, term type (Open), borrower/lender LEI.

I.3 Step 2: Rebate Mechanics and Cash Reinvestment

Fidelity reinvests the \$892,500,000 in approved money-market instruments, earning the money-market yield (take = 5.25%) and paying Morgan Stanley the rebate (5.00%). The reinvestment transforms the cash earmark into MMF units (see §15.12):

```
tau_reinvest = Transaction(day 0):  -- cash reinvestment
    -- Deploys the cash into MMF units; the USD coll_recv tables below
    -- carry the unchanged return obligation, with the MMF overlay separate.
Move 1: FIDELITY coll_recv -= 892,500,000 [coll_recv, USD]
Move 2: FIDELITY own      += 892,500      [own, MMF_UNIT]
Move 3: MMF_FUND own      -= 892,500      [own, MMF_UNIT]
Move 4: MMF_FUND own      += 892,500,000 [own, USD]
```

The reinvestment deploys the cash, but the \$892.5M return obligation persists in the loan unit state (`collateral_amount`): the collateral must be returned in full at close. The position-vector tables that follow therefore carry `coll_recv` (USD) at the collateral *obligation* amount throughout—the cash-equivalent Fidelity owes back—while the 892,500 MMF units are held in parallel, in a separate unit, and tracked separately. The margin-call (Step 4) and release (Steps 6–7) moves accordingly act on the obligation amount shown. Fidelity now bears reinvestment risk; conservation holds in each unit independently.

Daily economics.

$$\text{Reinvestment income} = 892,500,000 \times 5.25\% / 360 = \$130,104.17$$

$$\text{Rebate payable} = 892,500,000 \times 5.00\% / 360 = \$123,958.33$$

$$\text{Lending spread} = 130,104.17 - 123,958.33 = \$6,145.83$$

Billing here is monthly. Rebate at Day 30: $892,500,000 \times 5.00\% \times 30 / 360 = \$3,718,750$.

```
tau_rebate = Transaction(day 30):  -- rebate (fee settlement)
Move 1: FIDELITY      own -= 3,718,750 [own, USD]
Move 2: MORGAN_STANLEY own += 3,718,750 [own, USD]
```

Reinvestment over the month = \$3,903,906; rebate paid = \$3,718,750; net = \$185,156, the lending spread.

I.4 Step 3: Rehypothecation Constraints under Rule 15c3-3

Rule 15c3-3(b)(3) permits Morgan Stanley to use customer securities only up to 140% of the customer’s aggregate debit balance. This contrasts with the title-transfer regime of App. H:

	European (GMSLA 2010 TT)	US (Rule 15c3-3)
Rehypothecation	Receiver rehypothecates collateral freely	Borrower faces a regulatory cap
Legal basis	Title transfer: receiver <i>owns</i> collateral	Pledge/control: customer retains ownership interest
Cap	None (SFTR Art. 15 documentation only)	140% of customer debit balance

Pre-condition enforcement (P19). The SBL contract validates a rehypothecation request before any move commits; the `legal_regime` field selects the path:

```

validate_rehypothecation(entity, unit, quantity, regime):
    if regime == US_15C3_3:
        max_rehyp = customer_debit_balance(entity) * 1.40
        current_rehyp = sum(entity.coll_rehyp[u] for u in units)
        if current_rehyp + quantity * price(unit) > max_rehyp:
            REJECT("15c3-3 rehypothecation cap exceeded")
    elif regime == TITLE_TRANSFER:
        if not sftr_art15_documented(entity): REJECT("SFTR Art 15 missing")
        PERMIT()

```

A request exceeding the cap is rejected, never committed: P19 is a pre-condition, so the illegal state is unreachable rather than reached and rolled back.

I.5 Step 4: Daily Mark-to-Market (Day 5)

NVDA rises to \$910.00.

$$RCV_5 = 1,000,000 \times 910 \times 102\% = \$928,200,000,$$

$$\Delta C = 928,200,000 - 892,500,000 = \$35,700,000.$$

```

tau_2 = Transaction(day 5):  -- margin call
    Move 1: MORGAN_STANLEY  own      -= 35,700,000  [own, USD]
    Move 2: FIDELITY        coll_recv += 35,700,000  [coll_recv, USD]

```

Entity	Unit	own	onl	borr	c_p	c_r	c_rh
FIDELITY	NVDA	1,000,000	1,000,000	0	0	0	0
FIDELITY	USD	0	0	0	0	928,200,000	0
MORGAN_STANLEY	NVDA	0	0	1,000,000	0	0	0
MORGAN_STANLEY	USD	-928,200,000	0	0	0	0	0

P13 check. Collateral held = \$928,200,000 = required. ✓ **SLATE.** Modification: collateral amount and value updated.

I.6 Step 5: FINRA SLATE Reporting

Rule 10c-1a [4] and the FINRA Rule 6500 Series [5] require every covered securities loan to be reported to SLATE. The lifecycle maps as follows.

Lifecycle Event	SLATE Event	Key Data Elements
Loan initiation (Day 0)	New Loan	FINRA Loan ID, Client Loan ID, timestamp, ISIN US67066G1040, quantity 1,000,000, collateral type Cash, amount \$892.5M, rebate 5.00%, term Open, borrower/lender LEI
Margin call (Day 5)	Modification	Collateral amount $\rightarrow$ \$928.2M; timestamp
Partial return (Day 15)	Modification	Quantity $\rightarrow$ 500,000; collateral reduced
Full return (Day 25)	Modification	Quantity $\rightarrow$ 0; loan closed

Deadlines. Rule 10c-1a (adopted January 2024) and the FINRA Rule 6500 Series (effective 2025) set: loans effected on a business day from 12:00am ET through 7:45pm ET report same day before 8:00pm ET; loans after 7:45pm ET report next business day before 8:00pm ET.

Dual identifiers. Each loan carries a FINRA Loan ID (assigned by SLATE) and a Client Loan ID (the firm’s loan_id in the SBL unit state).

Unsettled Loan Flag (Field #44). Set when the loan has not settled on the reporting day. The SBL settlement status (§15.7) maps directly: PENDING $\rightarrow$ Unsettled, ACTIVE $\rightarrow$ Settled.

Remark I.1. *The SBL settlement status is a materialised projection of the immutable event log—a read cache the log rebuilds—not an authoritative mutable store. Every status change is caused by a logged lifecycle event; replaying events to any point reconstructs the status that held then. The lifecycle authority is §7.*

I.7 Step 6: Partial Recall and Return (Day 15)

Fidelity recalls 500,000 shares; NVDA \$920.00.

$$\text{RCV}_{15} = 500,000 \times 920 \times 102\% = \$469,200,000,$$

$$C_{\text{release}} = 928,200,000 - 469,200,000 = \$459,000,000.$$

```
tau_3 = Transaction(day 15): -- partial return
-- Securities return (unit: NVDA)
Move 1: MORGAN_STANLEY  borrr  -= 500,000      [borrr]
Move 2: FIDELITY         onloan -= 500,000      [onloan]
-- Cash collateral release (unit: USD)
Move 3: FIDELITY         coll_recv -= 459,000,000 [coll_recv]
Move 4: MORGAN_STANLEY  own      += 459,000,000 [own]
```

Entity	Unit	own	onl	borrr	c_p	c_r	c_rh	avail
FIDELITY	NVDA	1,000,000	500,000	0	0	0	0	500,000
FIDELITY	USD	0	0	0	0	469,200,000	0	0
MORGAN_STANLEY	NVDA	0	0	500,000	0	0	0	500,000
MORGAN_STANLEY	USD	−469,200,000	0	0	0	0	0	−469,200,000

P15 check. Loan quantity $1,000,000 \rightarrow 500,000$, monotonically non-increasing. ✓ **P18 check.** Fidelity $\vec{w}(\text{NVDA})[\text{own}] = 1,000,000$, unchanged. ✓

Pro-rated rebate on the returned portion: $459,000,000 \times 5.00\% \times 15/360 = \$956,250$.

```
tau_3b = Transaction(day 15): -- pro-rated rebate (fee settlement)
Move 1: FIDELITY         own -= 956,250      [own, USD]
Move 2: MORGAN_STANLEY  own += 956,250      [own, USD]
```

I.8 Step 7: Full Return and Close-Out (Day 25)

Remaining 500,000 NVDA returned; NVDA \$905.00.

```
tau_4 = Transaction(day 25): -- full return
-- Securities return (unit: NVDA)
Move 1: MORGAN_STANLEY  borr    -= 500,000      [borr]
Move 2: FIDELITY        onloan  -= 500,000      [onloan]
-- Cash collateral release (unit: USD)
Move 3: FIDELITY        coll_recv -= 469,200,000 [coll_recv]
Move 4: MORGAN_STANLEY  own      += 469,200,000 [own]
```

Final rebate: $469,200,000 \times 5.00\% \times 10/360 = \$651,666.67$.

```
tau_4b = Transaction(day 25): -- final rebate (fee settlement)
Move 1: FIDELITY          own    -= 651,666.67   [own, USD]
Move 2: MORGAN_STANLEY    own    += 651,666.67   [own, USD]
```

The loan unit is disposed: $\vec{w}_F(u_{\text{loan}})[\text{own}] = \vec{w}_M(u_{\text{loan}})[\text{own}] = 0$; SBL state RETURNED.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
FIDELITY	NVDA	1,000,000	0	0	0	0	0	1,000,000
FIDELITY	USD	-1,607,917	0	0	0	0	0	-1,607,917
MORGAN_STANLEY	NVDA	0	0	0	0	0	0	0
MORGAN_STANLEY	USD	+1,607,917	0	0	0	0	0	+1,607,917

Fidelity's negative USD is the total rebate paid ($956,250 + 651,666.67 = \$1,607,917$). Reinvestment income (§I.3) offsets it but accrues in MMF units, a separate unit, not the USD position shown; Fidelity's net loan income is reinvestment return minus rebate paid.

Conservation.

- NVDA: $1,000,000_F + 0_M + \text{virtual}(-1,000,000) = 0$. ✓
- USD: $(-1,607,917)_F + 1,607,917_M = 0$ (plus virtual offsets for the original collateral movements). ✓

SLATE. Final Modification, quantity = 0, closing the loan.

I.9 Cross-Jurisdictional Comparison

The two worked examples differ only in three places: the `legal_regime` field of the SBL unit state, the smart-contract pre-conditions (P19), and the PnL projection. The six-coordinate position vector, the Single-Coordinate Move Principle, and the conservation law are identical across regimes.

Aspect	European (GMSLA 2010 TT)	US (Rule 15c3-3)
Collateral ownership	Title transfers to receiver, who <i>owns</i> collateral.	Pledge: poster retains economic ownership; receiver holds under control.
Collateral type	Non-cash (German Bunds); haircut/margin on collateral market value.	Cash (USD); 102% of loan value posted.
Rehypothecation	Unrestricted under title transfer; SFTR Art. 15 documentation required.	Capped at 140% of customer debit balance (Rule 15c3-3(b)(3)); contract blocks excess.

Aspect	European (GMSLA 2010 TT)	US (Rule 15c3-3)
Fee structure	Lending fee (25 bps on loan value); borrower pays lender.	Rebate (Fed Funds - 25 bps); lender pays borrower on cash, earns reinvestment spread.
Collateral coordinate	own for poster, coll_recv for receiver.	own for poster, coll_recv for receiver.
PnL formula	$V_e = \sum_u \vec{w}_e(u)[\text{own}] \cdot P_t(u)$	$V_e = \sum_u (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{coll_post}]) \cdot P_t(u)$
Regulatory reporting	SFTR (NEWT, MODI, VALU, ETRM); dual-sided; UTI per ESMA waterfall.	FINRA SLATE (New Loan, Modification); single-sided (lender reports).
Manufactured dividends	French withholding-tax treatment.	US tax-treaty provisions (IRC §871(m)).
Settlement discipline	CSDR: daily cash penalties for fails; auto-partial (IBP-125).	Reg SHO Rule 204: close-out by T+3; FINRA Rule 4320.
Governing agreement	GMSLA 2010 (title transfer); ISLA Best Practice Handbook.	Master Securities Loan Agreement; SEC/FINRA rules.
Position vector	Identical six coordinates.	Identical six coordinates.
Conservation law	$\sum_e \vec{w}_e(u)[\text{coord}] = 0.$	Identical.
Smart contract	Same SBL contract; legal_regime selects validation.	Same SBL contract; legal_regime selects validation.

The Ledger engine, conservation law, and move primitive are jurisdiction-neutral. Title transfer versus pledge, unrestricted versus capped rehypothecation, and lending fee versus rebate are encoded as SBL unit-state parameters and enforced by P19 pre-conditions; only the PnL projection and the pre-condition logic differ, as derived in §15.9.

References

- [1] International Securities Lending Association, “ISLA Best Practice Handbook,” various editions.
- [2] U.S. Securities and Exchange Commission, “Rule 15c3-3: Customer Protection—Reserves and Custody of Securities,” 17 CFR §240.15c3-3.
- [3] U.S. Securities and Exchange Commission, “Regulation SHO, Rule 203(b)(1): Locate Requirement,” 17 CFR §242.203(b)(1).
- [4] U.S. Securities and Exchange Commission, “Rule 10c-1a: Reporting of Securities Loans,” 17 CFR §240.10c-1a, adopted January 2024.
- [5] Financial Industry Regulatory Authority, “FINRA Rule 6500 Series: Securities Lending and Transparency Engine (SLATE),” 2024.